tclutils

Pure-Tcl Engines, 0.69.0

2026-10-27

Contents

1 Understanding tclutils **11**
- 1.1 Introduction 11
- 1.2 Dependencies 11
- 1.3 Role 11
- 1.4 Scope 11
- 1.5 Structure 12
- 1.6 In short 12

2 Installation **12**
- 2.1 Adding a directory 12
- 2.2 setup.tcl 12
- 2.3 Which file is loaded 13
- 2.4 In short 13

3 Umbrella and Modules **13**
- 3.1 What the umbrella loads 13
- 3.2 File, docs, test 14
- 3.3 common 15
- 3.4 Error codes 15
- 3.5 In short 15

4 Text and Files **16**
- 4.1 Head and Tail 16
- 4.2 Searching: tugrep 16
- 4.3 Sorting and Duplicates 17
- 4.4 Fields and Replacing 18
- 4.5 File Type and Searching the Tree 19
- 4.6 ZIP 20
- 4.7 In short 20

5 More Line Filters **21**
- 5.1 Numbering lines: tunl 21
- 5.2 Wrapping: tufold and tufmt 22
- 5.3 Tabs: tuexpand 23
- 5.4 Columns: tucolumn 24
- 5.5 Tables: tutable 24
- 5.6 Reversing and shuffling: tutac, turev, tushuf 25
- 5.7 Number sequences: tuseq 26
- 5.8 Replacing characters: tutr 27
- 5.9 Counting: tuwc 28
- 5.10 Pages: tupr 29

6 Joining, Splitting, Pipelines **29**
- 6.1 Columns side by side: tupaste 30
- 6.2 Joining on a key: tujoin 30
- 6.3 Comparing two sets: tucomm 32
- 6.4 Splitting at patterns: tucsplit 33
- 6.5 Splitting by size: tusplit 34
- 6.6 Writing to several targets: tutee 35
- 6.7 Processing lists in batches: tuxargs 36

6.8 Ordering dependencies: tutsort 37
6.9 Fields and rules: tuawk 38
6.9.1 Rules 38
6.9.2 Fields and variables 38
6.9.3 Differences from awk 39
6.9.4 Examples 39
6.10 Approximate comparison: tufuzzy 40
6.11 Approximate search: tuagrep 41
7 Characters, Numbers, Dates and Paths 43
7.1 tustr 43
7.1.1 Visible Characters 44
7.2 tupath 44
7.3 tudate 45
7.4 tunum 47
7.5 In short 47
8 Data Formats 48
8.1 CSV: tucsv 48
8.2 JSON: tujson 49
8.3 INI: tuini 50
8.4 XML: tuxml 51
8.5 SQLite: tusqlite 52
8.6 LDIF: tuldif 53
8.7 Bookmarks: tubookmark 54
8.8 What Has No Section of Its Own Here 55
8.9 In short 56
9 TDBC and MaxMind 56
9.1 TDBC: tutdbc 56
9.2 MaxMind: tummdb 57
9.3 In short 57
10 Encoding, Hashing and Comparing 57
10.1 Base64: tubase64 58
10.2 Base32: tubase32 58
10.3 CRC: tucrc 59
10.4 Digests: tuhash 60
10.5 xxHash32: tuxxhash 60
10.6 Changing the Encoding: tuiconv 61
10.7 UUID: tuuuid 62
10.8 Query Strings: tuurl 62
10.9 Nested Dicts: tudict 63
10.10 Line Comparison: tudiff 63
10.11 Applying a Diff: tupatch 64
10.12 In short 65
11 Binary Data and File Information 65
11.1 Hexdump: tuhexdump 66
11.2 Octal, Decimal, Characters: tuod 67
11.3 Readable Strings: tustrings 68
11.4 Reading, Searching, Patching: tuhexedit 69

11.5 Comparing Byte by Byte: tucmp 70
11.6 Sizes: tusize 71
11.7 Metadata: tustat 73
11.8 Character Tables: tucode 74
11.9 In short 76
12 Notes and Calendars 76
12.1 Markdown: tumd 76
12.2 iCalendar: tuical 77
12.3 Recurrences: turrule 78
12.4 vCard: tuvcard 80
12.5 Note Tree: tunotes 81
12.6 Notes in SQLite: tunotesdb 81
12.7 Monthly Calendar: tucal 83
12.8 Public Holidays: tuholiday 84
12.9 Calendar Views: tical 85
12.10 In short 85
13 Text from PDF Files 86
13.1 Commands 86
13.2 Errors 87
13.3 What Comes Through 87
13.4 What It Does Not Do 88
13.5 Command Line 88
13.6 In short 88
14 ODF and PDF Structure 88
14.1 PDF: tupdf 88
14.2 ODF: tuodf 89
14.3 Word: tudocx 90
14.4 Excel: tuxlsx 91
14.5 In short 91
15 Diagrams 91
15.1 From Text to File: tuflow 93
15.2 Model: tudiagram 94
15.3 Flowchart: tuflow parse 96
15.4 State Diagram: tustate 99
15.5 Class Diagram: tuclass 99
15.6 ER Diagram: tuer 100
15.7 Requirements: turequirement 101
15.8 Mind Map: tumindmap 102
15.9 C4: tuc4 103
15.10 Blocks: tublock 104
15.11 Git History: tugit 105
15.12 Architecture: tuarchitecture 105
15.13 Sequence Diagram: tusequence 106
15.14 Gantt: tugantt 108
15.15 Pie Chart: tupie 111
15.16 XY Chart: tuxychart 112
15.17 Quadrant: tuquadrant 113

15.18 Journey: tujourney 115
15.19 Timeline: tutimeline 116
15.20 Sankey: tusankey 117
15.21 Radar: turadar 118
15.22 Treemap: tutreemap 119
15.23 Bit Fields: tupacket 120
15.24 Kanban: tukanban 121
15.25 PNG and Fonts 122
15.26 In short 122
16 Images without Tk: PNG, SVG, Colors 122
16.1 Colors: tucolor 124
16.2 Encoding and Decoding PNG: tupng 124
16.3 Drawing: tupngdraw 127
16.4 Table as an Image: tutablepng 131
16.5 Character Table as an Image: tucodepng 132
16.6 Month Sheet: tumonthpng 133
16.7 Uniform Size: tupngpad 136
16.8 SVG: tusvg 137
16.8.1 Surface 138
16.8.2 Document as a Dict 139
16.8.3 Icons 140
16.9 Detecting the Image Format: tuimage 140
16.10 Windows Icons: tuico 141
16.11 Similar Images: tudhash 143
17 Providers 144
17.1 One API, several backends 144
17.2 Caps per scheme 145
17.2.1 Entry 145
17.2.2 Writes the caps forbid 146
17.3 Apps on top 146
17.4 Summary 146
18 Building Blocks for Applications 146
18.1 tulog 146
18.2 tuevent 148
18.3 turegistry 149
18.4 tuvalidate 149
18.5 tulist 150
18.6 tumath 152
18.7 tunumany 152
18.8 tuappinfo 153
18.9 tusysinfo 155
18.10 tuedid 157
18.11 tudeploy 158
18.11.1 roots and addModulePaths 158
18.11.2 require 159
18.11.3 resourceDirs 159
18.11.4 sourceModule 160

18.12 tuexe 160
18.13 tuopen 161
18.14 tuzipfs 163
18.14.1 Reading an archive 164
18.14.2 Building a program 165
18.15 tuterm 166
18.16 tupagespec 167
18.17 tulayout 169
18.17.1 Page and scale 169
18.17.2 Blocks 169
19 Network and HTTPS 171
19.1 tuhttps 171
19.2 tls 1.x and tls 2.0 172
19.3 What socketCmd returns 172
19.4 tufetch 172
19.5 tusparql 174
19.6 Summary 175
20 tusettings 175
20.1 Purpose 175
20.2 API (short) 175
20.3 Directories per platform 176
20.4 Recent list 176
20.5 Limits and errors 176
20.6 Summary 177
21 Tasks with tutask 177
21.1 Purpose 177
21.2 The contract: starters 178
21.3 Tasks 178
21.4 Waiting: await and sleep 179
21.5 Cancelling with cleanup 179
21.6 Operations 180
21.7 Groups 180
21.8 Thread pool 181
21.9 Sandbox 181
21.10 Leaks and forgotten errors 182
21.11 In Tk programs 182
21.12 Error codes 182
21.13 Summary 183
22 Tests and tutestrun 183
22.1 Suite 183
22.2 tutestrun 184
22.3 Checking the origin: tupkgfinder 0.2 185
22.4 Checking Rules: tulint 185
22.5 Windows 186
22.6 Summary 186
23 Limits 186
23.1 Windows notes 187

23.2 Summary 188
24 Pitfalls 188
24.1 Fixed in 0.66.0 190
24.2 Summary 190
25 Modules (Orientation) 190
25.1 Categories 190
25.2 New in 0.69.0 191
25.3 New in 0.68.0 195
25.4 New in 0.67.0 199
25.5 New in 0.66.0 199
25.6 New in 0.65.0 199
25.7 New and Changed in 0.64.0 199
25.8 Summary 200
26 Glossary 200
26.1 Summary 201
27 Error Messages 201
27.1 Summary 207
28 API Catalog: tclutils Modules 207
28.1 common 207
28.2 tuagrep 207
28.3 tuappinfo 207
28.4 tuarchitecture 207
28.5 tuawk 208
28.6 tubase32 208
28.7 tubase64 208
28.8 tubin 208
28.9 tublock 209
28.10 tubookmark 209
28.11 tuc4 209
28.12 tucal 210
28.13 tucat 210
28.14 tuclass 210
28.15 tucmp 210
28.16 tucode 210
28.17 tucodepng 210
28.18 tucolor 211
28.19 tucolumn 211
28.20 tucomm 211
28.21 tucrc 211
28.22 tucsplit 211
28.23 tucsv 212
28.24 tucut 212
28.25 tudate 212
28.26 tudav 212
28.27 tudeploy 213
28.28 tudhash 213
28.29 tudiagram 213

28.30 tudict 213
28.31 tudiff 213
28.32 tudocx 214
28.33 tuedid 214
28.34 tuer 214
28.35 tuevent 214
28.36 tuexe 214
28.37 tuexpand 215
28.38 tufetch 215
28.39 tufile 215
28.40 tufind 215
28.41 tuflow 215
28.42 tufmt 215
28.43 tufold 215
28.44 tufuzzy 216
28.45 tugantt 216
28.46 tugit 216
28.47 tugrep 216
28.48 tuhash 216
28.49 tuhead 217
28.50 tuhexdump 217
28.51 tuhexedit 217
28.52 tuholiday 217
28.53 tuhttps 217
28.54 tuical 217
28.55 tuico 218
28.56 tuiconv 218
28.57 tuimage 218
28.58 tuini 218
28.59 tujoin 219
28.60 tujourney 219
28.61 tujson 219
28.62 tukanban 219
28.63 tulayout 220
28.64 tuldif 220
28.65 tulint 220
28.66 tulist 220
28.67 tulog 221
28.68 tumath 221
28.69 tumd 221
28.70 tumindmap 221
28.71 tummdb 221
28.72 tumonthpng 221
28.73 tunl 222
28.74 tunotes 222
28.75 tunotesdb 222
28.76 tunum 222

28.77 tunumany 223
28.78 tunumfmt 223
28.79 tuod 223
28.80 tuodf 223
28.81 tuopen 223
28.82 tupacket 223
28.83 tupagespec 224
28.84 tupaste 224
28.85 tupatch 224
28.86 tupath 224
28.87 tupdf 224
28.88 tupdftext 225
28.89 tupie 225
28.90 tupkgfinder 225
28.91 tupng 225
28.92 tupngdraw 225
28.93 tupngpad 225
28.94 tupostgrest 226
28.95 tupr 226
28.96 tuprovider 226
28.97 tuquadrant 226
28.98 turadar 226
28.99 turegistry 226
28.100 turequirement 227
28.101 turev 227
28.102 turrule 227
28.103 tusankey 227
28.104 tused 227
28.105 tuseq 227
28.106 tusequence 228
28.107 tusettings 228
28.108 tushuf 228
28.109 tusize 228
28.110 tusort 228
28.111 tusparql 228
28.112 tusplit 229
28.113 tusqlite 229
28.114 tustat 229
28.115 tustate 229
28.116 tustr 229
28.117 tustrings 229
28.118 tusvg 230
28.119 tusysinfo 230
28.120 tutable 230
28.121 tutablepng 230
28.122 tutac 230
28.123 tutail 230

28.124 tutask 230
28.125 tutdbc 231
28.126 tutee 231
28.127 tuterm 231
28.128 tutestrun 231
28.129 tutimeline 231
28.130 tutr 232
28.131 tutreemap 232
28.132 tutsort 232
28.133 tuuniq 232
28.134 tuurl 232
28.135 tuuuid 233
28.136 tuvalidate 233
28.137 tuvcard 233
28.138 tuwc 233
28.139 tuxargs 234
28.140 tuxlsx 234
28.141 tuxml 234
28.142 tuxxhash 234
28.143 tuxychart 234
28.144 tuzip 234
28.145 tuzipfs 235
29 Practice and Failure Cases 235
29.1 Processing CSV Lines 235
29.2 Handling Error Cases Specifically 235
29.3 Dictionaries with tudict 236
29.4 File Paths and Portability 236
30 License 236

1 Understanding tclutils

1.1 Introduction

tclutils is a collection of small pure-Tcl modules without Tk: text filters, data formats, providers, HTTPS policy, settings, background tasks (`tutask`) and a test runner. The core needs only Tcl itself (`zlib` for ZIP is part of Tcl). The modules require `package require Tcl 8.6-` and thus run under Tcl **8.6** and **9.x**.

This manual describes tclutils **0.69.0**. It is meant for server, CLI and CI programs without Tk and for applications that use the engines beneath tkutils and ctrlutils.

1.2 Dependencies

Some optional modules need more, and only when called:

Module	needs
tufetch	tls or a curl/wget binary
tuhttps, tudav, tupostgrest	tls for HTTPS
tuprovider::ftp	tcllib package ftp (only on open ftp …)
tuprovider::sftp	OpenSSH sftp, its own package require
tusqlite / tutdbc	an sqlite3 or TDBC handle opened by the caller
tutask (pool only)	package Thread (only on the first pool)

`package require tclutils` loads even without tls and Thread. `tuprovider::ftp` requires tcllib `ftp` only on `open ftp …`, not when the umbrella is loaded.

1.3 Role

Part	Role
Prefix	tu* / namespace tclutils::…
Layer	Model — logic without GUI
Siblings	tkutils (View), ctrlutils (Controller)
Loading	package require tclutils or a single module

```
tclutils = Model · tkutils = View · ctrlutils = Controller
```

The GUI layer and the controllers are described in the books *tkutils* and *Utils-Stack*. This book covers engines, paths, providers, settings, tasks and tests.

1.4 Scope

- Not a copy of all `docs/*.md`; the detailed API is there, in the tclutils tree.
- Not a replacement for *Packages and Modules* (`.tm` mechanics in general).

- Not a file manager: file view and explorer are described in *Utils-Stack*.

1.5 Structure

Chapters 2 and 3 cover installation and modules. They are followed by text and files, characters and paths, data formats, TDBC and MaxMind, encoding and hashing, notes and calendars, PDF text, ODF and PDF structure, diagrams, providers, network and HTTPS, settings, `tutask`, the tests and the limits. The appendices contain pitfalls (A), the module list (B), the glossary (C), error messages (D), the API catalog (E) and practice examples (F).

1.6 In short

- Pure Tcl, no Tk, version 0.69.0.
- Core without third-party packages; tls, tcllib `ftp`, curl/wget, Thread only for optional modules or the `tutask` pool.
- Pairing according to the CHANGELOG of tkutils 0.49.0: tclutils 0.69.0
  - tkutils 0.49.0 + ctrlutils 0.5. tkutils 0.49.0 needs at least tclutils 0.64.0.

Next chapter: Installation

2 Installation

tclutils is a tree of Tcl modules under `lib/tm/`. Tcl loads a `.tm` file only if its directory is on the module path. `lappend auto_path` is not enough for that.

2.1 Adding a directory

```
::tcl::tm::path add /pfad/zu/tclutils-0.69.0/lib/tm
package require tclutils                  ;# -> 0.69.0
package require tclutils::tucsv           ;# ein Modul, der Umbrella
    ist nicht nötig
```

`package require` returns the package version. If the package is missing, it aborts with `can't find package tclutils::gibt-es-nicht`. A directory that does not exist is not added by `setup.tcl`. The error only appears at `package require`.

2.2 setup.tcl

`tools/setup.tcl` is identical in tclutils and in tkutils. Sourcing it once adds the existing `lib/tm` of both libraries:

```
source /pfad/zu/tclutils/tools/setup.tcl
package require tclutils::tubin
```

The candidates are listed in `tools/setup.tcl` in this order. Only directories that exist and are not yet on the module path are added. They are added from back to front, so that the first existing candidate ends up in front and wins:

#	Candidate
1	TCLUTILS_TM or TKUTILS_TM, if the variable is set and not empty
2	lib/tm of the tree that setup.tcl comes from
3	next to the Tcl library: [file dirname [info library]]/<paket>/lib/tm
4	/usr/local/share/tcltk/<paket>/lib/tm
5	$XDG_DATA_HOME/tcltk/<paket>/lib/tm, otherwise ~/.local/share/tcltk/<paket>/lib/tm
6	sibling folder <paket>/lib/tm next to the tree
7	<paket>-*/lib/tm, the highest version that has a lib/tm

If `TCLUTILS_TM` is set, `package ifneeded tclutils 0.69.0` comes from that directory, even if the source tree lies next to it. As soon as installation and tree diverge, the variable decides, not the proximity to the script.

Details of the search: `tools/setup.md`. What `.tm` is in general: the book *Packages and Modules*.

2.3 Which file is loaded

```
export TCLUTILS_TM=/pfad/zu/tclutils/lib/tm
tclsh8.6 tests/all.tcl
```

The runner writes into its banner which umbrella file `tclutils` resolves to. It looks up the path; it does not load it at this point.

```
Modules resolve to:
  tclutils   0.69.0   …/tclutils/lib/tm/tclutils-0.69.0.tm
```

If the same version lies once more further back on the module path, the front one wins. Which copies exist is shown by `tupkgfinder` in the chapter Tests and tutestrun.

2.4 In short

- Module path: `tcl::tm::path add …/lib/tm`, or `setup.tcl` once.
- `TCLUTILS_TM` comes before the tree that `setup.tcl` comes from.
- `package require tclutils` loads the umbrella. A single module works without it.
- A missing package is reported only by `package require`.

Next chapter: Umbrella and Modules

3 Umbrella and Modules

3.1 What the umbrella loads

```
package require tclutils    ;# -> 0.69.0
```

The file is `lib/tm/tclutils-0.69.0.tm`. It requires 121 modules with `package require tclutils::…`, among them `common` 0.2, `tutask` 0.2, `tusettings` 0.1, `tupdftext` 0.1, `tufetch` 0.4, `tuhttps` 0.1 and `tuflow` 0.4. `tuflow` requires `tudiagram` when it loads. Afterwards 122 packages `tclutils::*` are loaded.

24 module files in the tree stay outside until someone requires them individually:

Group	Packages
only with their own package require	tutestrun, tutdbc, tuprovider::sftp
diagram, tuflow loads them on the first call of the respective kind	tustate, turequirement, tuer, tuclass, tumindmap, tuc4, tublock, tugit, tuarchitecture, tupie, tuxychart, tuquadrant, tujourney, tutimeline, tusankey, tugantt, tusequence, turadar, tutreemap, tupacket, tukanban

`tudiagram` is not in the list of the 121, but is present after the umbrella, because `tuflow` loads it immediately. The other diagram modules are not. 25 files are not in the umbrella's require list.

```
::tcl::tm::path add /pfad/zu/tclutils-0.69.0/lib/tm
package require tclutils                        ;# -> 0.69.0
package present tclutils::tutask                ;# -> 0.2
package present tclutils::tusettings            ;# -> 0.1
package present tclutils::tupdftext             ;# -> 0.1
catch {package present tclutils::tutestrun}     ;# -> 1
```

`catch` returns 1 because `tutestrun` is not loaded. The package exists in the tree. `package require tclutils::tutestrun` loads it without the umbrella.

In 0.69.0 came `tucsv` 0.2 (Excel profiles, BOM, encoding) and `tusqlite` 0.2 (parameter dict, `update`, `delete`), chapter Data Formats. In 0.68.0, 76 modules received fixes and a new version; the list is in Appendix B. New in 0.67.0 is `tupdftext` 0.1, chapter Text from PDF Files. In 0.66.0 `tutask` became 0.2, chapter Tasks with tutask. In 0.65.0 `tutask` 0.1 was added to the umbrella. From 0.64.0 remain `tusettings` 0.1, `tunum` 0.4 and `tupkgfinder` 0.2; `tustr` and `tudate` are at 0.3 since 0.69.0.

FTP is in the umbrella (`tuprovider::ftp` 0.2) and requires tcllib `ftp` only on `open`. SFTP is not. TDBC is not.

3.2 File, docs, test

Part	Example
Package	tclutils::tujson
File	lib/tm/tclutils/tujson-0.1.tm
Commands	namespace ::tclutils::tujson
Docs	docs/tujson.md

Part	Example
Test	tests/tujson.test
man	man/mann/tujson.n

The version in the file name is that of the module, not that of the umbrella. `tujson-0.1.tm` stays 0.1 while the umbrella is at 0.69.0.

```
tclsh tools/check-modules.tcl -require test,doc,man
```

148 modules, none without test, doc or man page.

3.3 common

`tclutils::common` 0.2 is the shared helper, not a module for the application. The option errors of the other modules go through it.

Command	Effect
parseOptions defaults args	an unknown option becomes {TCLUTILS COMMON OPTION name}
ensureBoolean wert option	a non-boolean becomes {TCLUTILS COMMON BOOLEAN …}
ensurePositiveInteger wert was	{TCLUTILS COMMON INTEGER …}
ensureOneOf wert liste was	a value not in the list becomes {TCLUTILS COMMON VALUE …}
readFile, writeFile	text; the channel is closed even on error
readBinaryFile	bytes, channel iso8859-1, so that Tcl 9 works without the old encoding binary

```
::tclutils::tudate::iso 0 -nope 1
```

This throws `{TCLUTILS COMMON OPTION -nope}` and names the known options, here `-time`.

3.4 Error codes

A module's own error has the form `{TCLUTILS <MODUL> <GRUND> …}`. `try`/`trap` compares from the front. Catching `{TCLUTILS TUFETCH}` gets every error from `tufetch`. The list of codes this book has tried is in the appendix Error Messages.

3.5 In short

- `package require tclutils` loads 122 packages, not the 146 files in the tree.
- `tutestrun`, `tutdbc` and `tuprovider::sftp` stay outside.
- `tuflow` loads the diagram kinds other than `tudiagram` only when called. Commands: chapter Diagrams.

- Unknown options arrive as `{TCLUTILS COMMON OPTION …}`.

Next chapter: Text and Files

4 Text and Files

The text filters mirror Unix tools as a Tcl API. A string or a list goes in, a string or a list comes out. The filters themselves start no program. The mapping to the Unix names is in `docs/guide/coreutils-mapping.md`. This chapter describes the commands used to build a pipeline from filters, file type and ZIP. It does not list every filter module from the mapping.

Common pattern, where the module has it: `text` works on a string, `file` on a path, `files` on a list of paths.

4.1 Head and Tail

Command	Effect
tucat::text, file, files	text unchanged, or with line numbers
tuhead::text, file	the first n lines; default 10
tutail::text, file	the last n lines; default 10

With `-number 1`, `tucat` numbers every line, including an empty one, as six digits, tab, text (`format %6d`). `-nonblank 1` numbers only lines with content. As with `cat -n`, the rest after the last `\n` is not a line and gets no number, and `files` numbers the files one after another in one run. A file without a final `\n` runs into the next one. An unknown option throws `{TCLUTILS COMMON OPTION -name}` (since 0.2; up to 0.1 the empty last line got a number, every file started at 1, and unknown options were silently ignored).

```
package require tclutils::tucat
package require tclutils::tuhead
package require tclutils::tutail
set text "alpha\nbeta\nbeta\nalpha"
::tclutils::tuhead::text $text 2          ;# -> alpha\nbeta\n
::tclutils::tutail::text $text 1          ;# -> alpha
::tclutils::tucat::text "alpha\nbeta" -number 1
                                          ;# -> "     1\talpha\n
    2\tbeta"
```

As with `head -n` and `tail -n`, the result keeps the line ends: every line ends with `\n`, except the last line of the input when it has none. `text` and `file` return the same; `file` reads UTF-8, other encodings with `-encoding`. Up to 0.1 `tutail::text "eins\nJörg\n" 1` was empty, and `file` dropped every line end. So print with `puts -nonewline`. `tuhead` and `tutail` do not follow files. A negative count throws `{TCLUTILS TUHEAD COUNT}` or `{TCLUTILS TUTAIL COUNT}`.

4.2 Searching: tugrep

```
::tclutils::tugrep::search text muster ?option wert ...?
::tclutils::tugrep::match  zeile muster ?option wert ...?
::tclutils::tugrep::file   pfad muster ?option wert ...?
::tclutils::tugrep::files  pfade muster ?option wert ...?
```

The pattern is a regular expression. `-fixed 1` searches for the string literally.

Option	Default	Effect
-nocase	0	ignore case
-linenumbers	0	each line as {nummer text}, number from 1
-invert	0	the lines without a match
-fixed	0	no regex
-word	0	whole word, like grep -w
-wholeline	0	the whole line, like grep -x
-max	0	stop after N matches; 0 means no limit
-count	0	only the count
-fileswithmatches	0	only the paths with a match
-filenames	0	matches prefixed with the file name

```
package require tclutils::tugrep
set text "alpha\nbeta\nERROR eins\nerror zwei\nERRORCODE"
::tclutils::tugrep::search $text ERROR
                                          ;# -> {ERROR eins} ERRORCODE
::tclutils::tugrep::search $text error -nocase 1 -linenumbers 1
                                          ;# -> {3 {ERROR eins}} {4
    {error zwei}} {5 ERRORCODE}
::tclutils::tugrep::search "ERROR\nERRORCODE" ERROR -word 1
                                          ;# -> ERROR
::tclutils::tugrep::search "a\na\na" a -max 1
                                          ;# -> a
::tclutils::tugrep::search $text error -nocase 1 -count 1
                                          ;# -> 3
```

`-word` matches `ERROR` and not `ERRORCODE`. `-max` counts lines, not matches within a line. The rest after a final `\n` is not a line: `.*` matches two lines in `"eins\nJörg\n"` (since 0.2; up to 0.1.3 three, and `file` dropped empty lines at the end of the file).

4.3 Sorting and Duplicates

`tusort` sorts lines or a list. `tuuniq` removes only adjacent duplicates, like Unix `uniq`. To remove all duplicates, sort first.

Option tusort	Effect
-numeric, -integer, -real	numeric comparison
-dictionary	like lsort -dictionary
-nocase	ignore case
-reverse	descending
-unique	drop duplicates while sorting

`tuuniq` knows `-nocase`. `adjacentCount` returns `{anzahl wert}` per group of neighbors. `count` counts over the whole list.

```
package require tclutils::tusort
package require tclutils::tuuniq
::tclutils::tusort::text "10\n2\n10" -numeric 1 -unique 1
                                          ;# -> 2\n10
::tclutils::tuuniq::uniqList {a b b a}    ;# -> a b a
::tclutils::tuuniq::uniqList [::tclutils::tusort::sortList {a b b a}]
                                          ;# -> a b
::tclutils::tuuniq::adjacentCount {a a b} ;# -> {2 a} {1 b}
```

4.4 Fields and Replacing

`tucut` cuts fields from a line. This is not CSV: quotes and commas inside a field belong to `tucsv`, chapter Data Formats. Field numbers count from 1. Ranges are written `2-4`. `-chars` cuts characters, also from 1, also open-ended as `4-` or `-3`.

```
package require tclutils::tucut
::tclutils::tucut::field "a;b;c" ";" 2         ;# -> b
::tclutils::tucut::fields "a;b;c;d" ";" {1 3}  ;# -> a;c
::tclutils::tucut::fields "a;b;c;d" ";" 2-3    ;# -> b;c
```

`text` and `file` take `-delimiter`, `-fields` and `-chars`. As with `tusort::text` and `tuuniq::text`, the result ends with `\n` exactly when the input does; up to tclutils 0.68.0 it was dropped.

`tused` replaces and deletes line by line. Without `-all`, the first match in the line applies. `-fixed 1` searches literally, otherwise as a regex. `-nocase 1` ignores case. `delete` drops the matching lines. `process` takes a list of rules (`s`, `d`).

```
package require tclutils::tused
::tclutils::tused::replace "foo foo" foo bar         ;# -> bar foo
::tclutils::tused::replace "foo foo" foo bar -all 1  ;# -> bar bar
::tclutils::tused::delete "# c\nzeile" {^#}          ;# -> zeile
```

Addresses as in sed (`3`, `$`, `/regex/`, range, negation with `!`, step `erste~schritt`) attach to the rule in `process` and `script`. The forms are listed in `docs/tused.md`.

4.5 File Type and Searching the Tree

`tufile` reads only the head of the file and compares it with a signature table. No libmagic, no masks.

Command	Result
detect pfad	dict of the most specific match, or empty
type pfad	short name (pdf, png, zip) or ""
mime pfad	MIME type or ""
describe pfad	a sentence, or data
checkExtension pfad	dict with status: ok, mismatch, noext, unknown, unknownext

```
package require tclutils::tufile
::tclutils::tufile::type /pfad/seite.pdf    ;# -> pdf
::tclutils::tufile::mime /pfad/seite.pdf    ;# -> application/pdf
```

For a page from pdf4tcl the type is `pdf`. According to `docs/tufile.md`, DOCX, XLSX and PPTX share only the ZIP signature and come back as `zip`; the module recognizes ODT, ODS and ODP by the `mimetype` entry at byte 30.

`tufind` walks the tree in Tcl.

Command	What comes back
files wurzel muster ?optionen?	files
directories wurzel muster	directories
all wurzel muster	both
walk wurzel muster script	calls the script for each match

Option	Default	Effect
-recursive	1	into subdirectories
-maxdepth	−1	−1 means no limit
-mindepth	0	shallower entries are dropped
-hidden	0	include dot files
-tails	0	only the file name, not the full path
-type		file, dir, any
-size		N, +N, -N, plus k, m, g
-mtime		age in days: +N older, -N newer

```
package require tclutils::tufind
::tclutils::tufind::files $dir *.pdf -tails 1 -maxdepth 1
```

With `-tails 1` and `-maxdepth 1` these are the names directly in `$dir`, no paths from subdirectories.

4.6 ZIP

`tuzip` reads and writes ZIP with stored and with deflate entries. That is enough for normal archives and for ODF, which is a ZIP. It opens binary files in binary mode.

Command	Effect
names archiv	the member names
entries archiv	a dict per member
readMember archiv name	the content
extract archiv name zieldatei	writes a file
create archiv dateien ?-base dir? ?-compress 1?	new archive
createMembers archiv member	from memory; per dict name, content, method

```
package require tclutils::tuzip
::tclutils::tuzip::create $archiv [list $datei] -base $dir -compress 1
::tclutils::tuzip::names $archiv                 ;# -> notiz.txt
::tclutils::tuzip::readMember $archiv notiz.txt  ;# -> hallo zip
```

`-base` strips the directory from the name. The order in `createMembers` is kept; this way an ODF puts `mimetype` as the first, stored entry.

Since 0.2 `tuzip` writes names as UTF-8; a name with an umlaut gets the UTF-8 flag, and `unzip`, Python and Windows read it correctly. Up to 0.1 names were Latin-1 without the flag, other programs read `Jörg` as `J÷rg`. Reading uses `-encoding auto`: with the flag or valid UTF-8, UTF-8, else CP437, as the ZIP format specifies. For archives from tuzip 0.1 with umlauts, pass `-encoding iso8859-1`. The provider over the same file is described in the chapter Providers.

Paths without a file system, visible characters and amounts: Characters, Numbers, Dates and Paths.

Some modules have a program under `bin/`. The list is in `docs/guide/cli.md`. This book does not describe them one by one.

4.7 In short

- `text`, `file`, `files`: string in, string or list out.
- `tuuniq` removes only neighbors. Run `tusort` first to remove all duplicates.
- `tucut` is not CSV.
- `tugrep -word` matches the word, not the start of a longer word.
- `tuzip` reads the content; the provider decides whether writing is allowed.

Next chapter: More Line Filters

5 More Line Filters

This chapter continues Text and Files. The modules here number, wrap, align, reverse, count and generate lines. Each has a Unix counterpart from coreutils or util-linux; the differences are listed with each module. No module starts a program.

All options go through the same parser. An unknown option throws `{TCLUTILS COMMON OPTION -name}` and lists the known options in the message. An integer below 1 where a positive one is required throws `{TCLUTILS COMMON INTEGER -name}`, a value that is not a boolean `{TCLUTILS COMMON BOOLEAN -name}`. These three codes are not repeated for every module below.

`file` reads the file with the system encoding and converts `\r\n` to `\n`. The result ends with `\n` exactly when the input does:

Trailing \n	Modules
as in the input	tunl, tutac, turev, tuexpand, tutr, tufold, tufmt, tucolumn, tushuf
own rule	tupr (pages, see there)

Up to tclutils 0.68.0, `tufold`, `tufmt`, `tucolumn` and `tushuf` dropped the trailing `\n`.

5.1 Numbering lines: tunl

tunl numbers lines like Unix `nl`.

```
::tclutils::tunl::text text ?option wert ...?
::tclutils::tunl::file pfad ?option wert ...?
```

Option	Default	Effect
-style	nonempty	all every line, nonempty only lines with content, none none
-start	1	first number
-increment	1	step
-width	6	digits of the number, right-aligned
-separator	\t	between number and line

A line without a number starts with `-width` spaces and the separator. Unix `nl` writes only spaces for an empty line, without a tab. A trailing `\n` is kept and does not produce a line of its own. An empty text yields `""`.

`-start` and `-increment` are integers, also negative and of any size. `-width` is an integer of at least 1.

Error	-errorcode
-style other than all, nonempty, none	{TCLUTILS TUNL STYLE}
-start not an integer	{TCLUTILS TUNL START}

Error	-errorcode
-increment not an integer	{TCLUTILS TUNL INCREMENT}
-width not an integer of at least 1	{TCLUTILS TUNL WIDTH}

```
package require tclutils::tunl
::tclutils::tunl::text "a\n\nb" -width 2 -separator " "
                                          ;# -> " 1 a\n   \n 2 b"
::tclutils::tunl::text "a\n\nb" -style all -width 2 -separator " "
                                          ;# -> " 1 a\n 2 \n 3 b"
::tclutils::tunl::text "a\nb" -start 10 -increment 5 \
    -width 2 -separator ": "              ;# -> 10: a\n15: b
```

`tucat -number 1` from Text and Files, by contrast, always numbers with six digits and a tab.

5.2 Wrapping: tufold and tufmt

tufold wraps each line on its own, like Unix `fold`. tufmt joins paragraphs and wraps them anew, like Unix `fmt`.

```
::tclutils::tufold::foldLine zeile ?-width n? ?-words 0|1?
::tclutils::tufold::text     text  ?-width n? ?-words 0|1?
::tclutils::tufold::file     pfad  ?-width n? ?-words 0|1?
::tclutils::tufmt::reflow     text ?-width n?
::tclutils::tufmt::reflowFile pfad ?-width n?
```

Option	Default	Effect
tufold -width	80	at most this many characters per line
tufold -words	0	1 breaks at the last space before the limit
tufmt -width	75	target width of the paragraphs

`foldLine` returns a list of the partial lines, `text` and `file` a string. The width counts characters, not screen columns: a tab is one character. Without `-words`, `tufold` cuts hard after `-width` characters; a space at the cut stays at the end of the line. With `-words 1`, `tufold` breaks at the last space up to column `-width` plus one. All spaces at the break are dropped, both at the end of the line and at the start of the next line. Unix `fold -s` keeps them: from `"ab cd"` with width 3 it makes `"ab "`, `" "` and `"cd"`, whereas `tufold` makes `ab` and `cd`. A word longer than `-width` is cut hard even with `-words 1`. Empty lines stay empty.

`tufmt` separates paragraphs at empty lines; several empty lines become one. Within a paragraph all line breaks and repeated spaces are dropped, then the module fills the lines word by word up to the width. The indentation of the first paragraph line applies to all lines of the paragraph and counts toward the width; a tab in the indentation counts as one character. A word longer than the width stands alone and uncut on its line. The module removes `\r` beforehand.

```
package require tclutils::tufold
package require tclutils::tufmt
::tclutils::tufold::text "abcdefghij" -width 4
                                          ;# -> abcd\nefgh\nij
::tclutils::tufold::foldLine "eins zwei drei vier" -width 9 -words 1
                                          ;# -> {eins zwei} {drei vier}
::tclutils::tufold::foldLine "ab   cd" -width 3 -words 1
                                          ;# -> ab cd
::tclutils::tufmt::reflow "a  b\nc d e" -width 3
                                          ;# -> a b\nc d\ne
::tclutils::tufmt::reflow "  a b c d\n\n\nx" -width 5
                                          ;# -> "  a b\n  c d\n\nx"
```

5.3 Tabs: tuexpand

tuexpand converts tabs to spaces and back, like Unix `expand` and `unexpand`.

```
::tclutils::tuexpand::expand   text ?-tabs n?
::tclutils::tuexpand::unexpand text ?-tabs n? ?-all 0|1?
::tclutils::tuexpand::file     pfad ?-mode m? ?-tabs n? ?-all 0|1?
```

Option	Default	Effect
-tabs	8	distance between tab stops
-all	0	unexpand: 0 only leading spaces, 1 all runs
-mode	expand	file only: which direction

`expand` fills each tab up to the next tab stop. A trailing `\n` is kept.

Without `-all`, `unexpand` converts only the whitespace at the start of the line, spaces as well as tabs; the rest of the line stays as it is, including tabs inside it. `-all 1` converts every run of whitespace. In a run of at least two characters, and in every run at the start of the line, each tab stop reached becomes a tab; what is left after the last tab stop remains spaces. A single space inside the line remains a space, even directly before a tab stop. Spaces directly before a tab are absorbed into it. Unix `unexpand` behaves the same way (`-a` for `-all 1`).

`file` throws `{TCLUTILS TUEXPAND MODE}` for an unknown `-mode` and `{TCLUTILS TUEXPAND OPTION}` for a missing value. `expand` and `unexpand` do not know `-mode`. `-all` applies only to `unexpand`; `expand` and `file -mode expand` throw `{TCLUTILS COMMON OPTION -all}` for it.

```
package require tclutils::tuexpand
::tclutils::tuexpand::expand "ab\tc" -tabs 4  ;# -> ab  c
::tclutils::tuexpand::unexpand "        x  y" ;# -> \tx  y
::tclutils::tuexpand::unexpand "ab  cd      e" -tabs 4 -all 1
                                          ;# -> ab\tcd\t\te
::tclutils::tuexpand::unexpand "a\tb"     ;# -> a\tb
::tclutils::tuexpand::unexpand "abcdefg          x" -all 1
                                          ;# -> abcdefg\t\t x
```

5.4 Columns: tucolumn

tucolumn aligns columns, like `column` from util-linux. There are two modes: `table` for lines with fields (`column -t`), `fill` for a word list that is distributed over several columns.

```
::tclutils::tucolumn::table text  ?option wert ...?
::tclutils::tucolumn::text  text  ?option wert ...?
::tclutils::tucolumn::fill  liste ?-width n? ?-gap n?
::tclutils::tucolumn::file  pfad  ?-mode table|fill? ?option wert ...?
```

Option	Default	Effect
-separator	""	field separator of the input; empty means runs of whitespace
-output	two spaces	separator of the output
-right	0	1 right-aligns all cells
-width	80	fill: target width of the line
-gap	2	fill: spaces between columns, at least 1
-mode	table	file only

`text` is the same as `table`. The column width is the longest cell of the column, in characters. `table` does not pad the last column of each line, so no whitespace appears at the end of a line. Lines may have different numbers of fields. A separator of several characters counts as a whole. Empty lines remain as empty lines; `column -t` drops them without `-e`.

`fill` fills column by column: first the first column from top to bottom, then the next. If not even one column fits into `-width`, there is one column. `file -mode fill` splits the file into words at whitespace. An empty list yields `""`.

`file` throws `{TCLUTILS TUCOLUMN MODE}` for an unknown `-mode` and `{TCLUTILS TUCOLUMN OPTION}` for a missing value.

```
package require tclutils::tucolumn
::tclutils::tucolumn::table "Name Alter\nBob 7"
                                          ;# -> Name  Alter\nBob   7
::tclutils::tucolumn::table "Name;Alter\nBob;7" -separator ";" \
    -output " | "                         ;# -> Name | Alter\nBob  | 7
::tclutils::tucolumn::fill {a b c d e} -width 7
                                          ;# -> "a  c  e\nb  d"
```

5.5 Tables: tutable

tutable renders a table of header and rows as text, either as a Markdown table or with an ASCII frame. There is no Unix counterpart; `tucolumn` is there for aligning finished lines of text.

```
::tclutils::tutable::render koepfe zeilen ?option wert ...?
```

Option	Default	Effect
-style	markdown	markdown (pipe table) or box (+, -, |)
-align	empty	per column l, r or c; missing entries count as l

The return value is a string without a trailing `\n`. `koepfe` determines the number of columns: missing cells of a row stay empty. The width counts characters. The separator line in Markdown carries the alignment (`:--`, `--:`,`:-:`); `l` also gets the colon on the left. In Markdown each column is at least three characters wide, as wide as`:--`; cells and separator line are aligned. In the frame a column is as wide as its longest cell.

Error	-errorcode
-style other than markdown and box	{TCLUTILS TUTABLE OPT}
alignment other than l, r and c	{TCLUTILS TUTABLE OPT}
row with more cells than koepfe	{TCLUTILS TUTABLE COLUMNS}
koepfe empty	{TCLUTILS TUTABLE HEADERS}

```
package require tclutils::tutable
puts [::tclutils::tutable::render {Name Alter} {{Alice 30} {Bob 7}}]
puts [::tclutils::tutable::render {Artikel Menge} \
    {{Äpfel 12} {Birnen 3}} -align {l r} -style box]
```

```
| Name  | Alter |
| :---- | :---- |
| Alice | 30    |
| Bob   | 7     |
+---------+-------+
| Artikel | Menge |
+---------+-------+
| Äpfel   |    12 |
| Birnen  |     3 |
+---------+-------+
```

5.6 Reversing and shuffling: tutac, turev, tushuf

tutac reverses the order of the lines (Unix `tac`), turev the characters in each line (Unix `rev`), tushuf shuffles the lines (Unix `shuf`).

Command	Effect
tutac::text text, file pfad	lines in reverse; trailing \n is kept
tutac::lines liste	the list in reverse, like lreverse
turev::line zeile	the characters in reverse, like string reverse
turev::text text, file pfad	each line on its own; trailing \n is kept

Command	Effect
tushuf::lines liste ?opt?	shuffled list
tushuf::text text ?opt?, file pfad ?opt?	shuffled lines; a trailing \n is kept

`tutac` treats the last line without `\n` like any other: `"a\nb"` becomes `"b\na"`. Unix `tac` returns `ba` and a `\n` here. `turev` reverses by code points; it splits composed characters made of base and accent.

Option tushuf	Default	Effect
-seed	from the clock	integer; same value, same order
-count	−1	at most this many lines; negative means all, 0 none

`tushuf` uses its own random number generator, not `rand()`. With the same `-seed` the order is the same on every platform and under Tcl 8.6 and 9.0. The value may be of any size, also beyond 32 bits under Tcl 8.6; the generator uses its lower 31 bits. A `-seed` that is not an integer throws `{TCLUTILS TUSHUF SEED}`, such a `-count {TCLUTILS TUSHUF COUNT}`.

```
package require tclutils::tutac
package require tclutils::turev
package require tclutils::tushuf
::tclutils::tutac::text "a\nb\nc\n"       ;# -> c\nb\na\n
::tclutils::turev::text "abc\nxyz"        ;# -> cba\nzyx
::tclutils::tushuf::lines {a b c d e} -seed 7
                                          ;# -> b e d a c
::tclutils::tushuf::lines {a b c d e} -seed 7 -count 2
                                          ;# -> b e
```

5.7 Number sequences: tuseq

tuseq generates number sequences like Unix `seq`.

```
::tclutils::tuseq::range ?erste? ?schritt? letzte ?option wert ...?
::tclutils::tuseq::text  ?erste? ?schritt? letzte ?option wert ...?
```

One value means `letzte` (from 1, step 1), two mean `erste letzte`, three `erste schritt letzte`. Options may stand before, between or after the values: an argument that starts with `-` and is not a number introduces an option. Negative numbers count as values. `range` returns a list, `text` a string.

Option	Default	Effect
-format	empty	format pattern per number, like seq -f
-equalwidth	0	padded with zeros to equal width, like seq -w
-separator	\n	text only: separator, like seq -s

If all three values are integers, the module computes with integers, of any size, also beyond 32 bits under Tcl 8.6. Otherwise it computes in floating point and prints the numbers with `%g`; `1.0`

thus becomes 1. A step in the wrong direction yields an empty list. A `-format` for integers (%d) fails on floating-point values with `{TCL VALUE INTEGER}`. `-equalwidth` puts the minus sign before the zeros. `-separator` applies only to `text`; `range` throws `{TCLUTILS COMMON OPTION -separator}` for it.

Error	-errorcode
zero or more than three values	{TCLUTILS TUSEQ ARGS}
value is not a number	{TCLUTILS TUSEQ VALUE}
step 0	{TCLUTILS TUSEQ INCREMENT}
option without value	{TCLUTILS TUSEQ OPTION}

```
package require tclutils::tuseq
::tclutils::tuseq::range 5                ;# -> 1 2 3 4 5
::tclutils::tuseq::range 5 -1 1           ;# -> 5 4 3 2 1
::tclutils::tuseq::range 1 0.5 2          ;# -> 1 1.5 2
::tclutils::tuseq::range 8 11 -equalwidth 1
                                          ;# -> 08 09 10 11
::tclutils::tuseq::range -2 1 -equalwidth 1
                                          ;# -> -2 -1 00 01
::tclutils::tuseq::range 1 3 -format %03d ;# -> 001 002 003
::tclutils::tuseq::text -separator , 3    ;# -> 1,2,3
::tclutils::tuseq::text 3 -separator ,    ;# -> 1,2,3
::tclutils::tuseq::text 1 2 7 -separator ,
                                          ;# -> 1,3,5,7
::tclutils::tuseq::range 5000000000 5000000001
                                          ;# -> 5000000000 5000000001
```

5.8 Replacing characters: tutr

tutr replaces, deletes and squeezes characters, like Unix `tr`.

```
::tclutils::tutr::translate text menge1 menge2
::tclutils::tutr::delete    text menge1
::tclutils::tutr::squeeze   text menge1
::tclutils::tutr::file      pfad menge1 menge2 ?option wert ...?
```

A set consists of single characters and ranges `a-z`. A `-` at the start or end stands for itself. A descending range such as `c-a` throws `{TCLUTILS TUTR RANGE}`. `translate` replaces the n-th character of `menge1` with the n-th of `menge2`; if `menge2` is shorter, its last character applies to the rest. If a character appears in `menge1` more than once, its last mapping applies. All replacements happen in one pass, so `ab` to `ba` swaps. An empty `menge2` throws `{TCLUTILS TUTR SET2}`, unless `menge1` is empty too. `delete` removes the characters of the set. `squeeze` reduces consecutive identical characters of the set to one.

Option file	Default	Effect
-delete	0	1 deletes menge1 instead of replacing

Option file	Default	Effect
-squeeze	0	1 then squeezes the characters from menge2

`file` requires `menge2` even with `-delete 1`; it is then needed only for `-squeeze` and may otherwise be `""`. With an empty `menge2`, `-squeeze 1` and without `-delete`, `file` squeezes only the characters from `menge1`, like `tr -s menge1`. Without `-delete` and `-squeeze`, an empty `menge2` throws `{TCLUTILS TUTR SET2}`.

Differences from Unix `tr`: no classes such as `[:alpha:]` and no escapes of its own; Tcl itself writes `\n` or `\t` into the string.

```
package require tclutils::tutr
::tclutils::tutr::translate "hallo welt" a-z A-Z  ;# -> HALLO WELT
::tclutils::tutr::translate "abcde" a-e xy        ;# -> xyyyy
::tclutils::tutr::delete "a1b2c3" 0-9             ;# -> abc
::tclutils::tutr::squeeze "aaabccc  d" "ac "      ;# -> abc d
::tclutils::tutr::translate "abcabc" aa xy         ;# -> ybcybc
```

5.9 Counting: tuwc

tuwc counts lines, words, characters and bytes, like Unix `wc`.

```
::tclutils::tuwc::text daten
::tclutils::tuwc::file pfad
```

The return value is a dict with `lines`, `words`, `chars`, `bytes`; `file` adds `file` with the path.

Key	Meaning
lines	number of \n, plus one if the text does not end in \n
words	runs without whitespace
chars	characters (code points)
bytes	text: length in UTF-8; file: size of the file in bytes

`lines` differs from `wc -l`: a last line without `\n` is counted. An empty text has 0 lines, `"\n"` one. `file` reads with the system encoding and turns `\r\n` into `\n`; `lines`, `words` and `chars` count this text. For `file`, `bytes` is the file size, as with `wc -c`.

```
package require tclutils::tuwc
set d [::tclutils::tuwc::text "eins zwei\ndrei"]
dict get $d lines                         ;# -> 2
dict get $d words                         ;# -> 3
set d [::tclutils::tuwc::text "Grüße"]
dict get $d chars                         ;# -> 5
dict get $d bytes                         ;# -> 7
```

5.10 Pages: tupr

tupr splits text into pages with a header, like Unix `pr` without multi-column output.

```
::tclutils::tupr::text text ?option wert ...?
::tclutils::tupr::file pfad ?option wert ...?
```

Option	Default	Effect
-length	66	lines per page, including header and footer
-header	""; for file the file name	title in the header
-width	72	width of the header line
-number	0	1 numbers the text lines (%5d, tab)
-date	now, %Y-%m-%d %H:%M	any string for the header

Each page has five header lines: two empty, the info line, two empty. The info line has date and title on the left, separated by three spaces, and `Page n` right-aligned; if it is too long, at least one space stands before `Page`. Then follow `-length` minus 10 text lines, padded with empty lines to the full page length, and five empty footer lines. The pages follow each other without a form feed; the result ends without `\n`. `-width` applies only to the header; the module does not shorten long text lines. `-number 1` counts across all pages. An empty text yields `""`.

A `-length` of 10 or less throws `{TCLUTILS TUPR LENGTH}`.

```
package require tclutils::tupr
set seiten [::tclutils::tupr::text "a\nb\nc" -length 12 \
    -header Liste -date Montag -width 24]
set z [split $seiten \n]
llength $z                      ;# -> 24
lindex $z 2                     ;# -> Montag   Liste    Page 1
lrange $z 5 6                   ;# -> a b
lindex $z 14                    ;# -> Montag   Liste    Page 2
```

Next chapter: Joining, Splitting, Pipelines

6 Joining, Splitting, Pipelines

This chapter continues Text and Files. It covers the modules that combine several line streams, distribute text over files, order pipelines and search text approximately. Each module has a Unix counterpart or follows one loosely. None starts a program; everything runs in Tcl.

Group	Modules	Unix counterpart
Combining line streams	tupaste, tujoin, tucomm	paste, join, comm
Splitting	tucsplit, tusplit	csplit, split
Pipeline helpers	tutee, tuxargs, tutsort	tee, xargs, tsort
Fields and rules	tuawk	awk

Group	Modules	Unix counterpart
Approximate search	tufuzzy, tuagrep	agrep

Common to the text variants: a text that ends in `\n` does not produce an empty last line. Options come in pairs as `-option wert`. Booleans accept anything that `string is boolean` accepts.

6.1 Columns side by side: tupaste

tupaste combines line *n* of each input into one output line, like Unix `paste`.

```
::tclutils::tupaste::lines lineLists ?-delimiter d?
::tclutils::tupaste::texts textList  ?-delimiter d?
::tclutils::tupaste::files paths     ?-delimiter d?
```

`lines` takes a list of line lists, `texts` a list of texts, `files` a list of paths. All three return a string, lines separated by `\n`. `lines` ends without `\n`; `texts` and `files` end with `\n` when one of the inputs does (up to tclutils 0.68.0 never).

Option	Default	Effect
-delimiter	\t	stands between the columns; may also be several characters or empty

If an input is shorter, an empty field takes its place. The number of output lines follows the longest input.

Differences from `paste`: `-delimiter` applies as a whole to every gap; a list of delimiters used in rotation (`paste -d ',;'`) does not exist. `paste -s` (one input as one line) is missing as well. An unknown option and an option without a value report `TCLUTILS COMMON OPTION -name`.

```
package require tclutils::tupaste
::tclutils::tupaste::texts [list "a\nb\nc\n" "1\n2\n"] -delimiter ";"
                                          ;# -> a;1\nb;2\nc;\n
::tclutils::tupaste::lines {{a b} {1 2}} -delimiter " | "
                                          ;# -> a | 1\nb | 2
::tclutils::tupaste::files {namen.txt orte.txt} -delimiter ": "
    ;# -> Ada: London\nLinus: Helsinki
```

In the last call `namen.txt` contains the lines `Ada` and `Linus`, `orte.txt` the lines `London` and `Helsinki`.

6.2 Joining on a key: tujoin

tujoin joins two separate text sources on a key field, modeled on Unix `join`.

```
::tclutils::tujoin::lines leftLines rightLines ?option wert ...?
::tclutils::tujoin::texts leftText  rightText  ?option wert ...?
::tclutils::tujoin::files leftPath  rightPath  ?option wert ...?
```

Option	Default	Effect
-delimiter	\t	field separator of the input; several characters form one literal separator
-joiner	like -delimiter	field separator of the output
-leftfield	1	key field on the left, from 1
-rightfield	1	key field on the right, from 1
-header	0	first line of both sides is a header line
-outer	none	none, left, right, full
-anti	none	none, left, right, both

Each output line consists of the key, the remaining fields on the left and the remaining fields on the right. The return value is a string with `\n` between the lines. `texts` and `files` append a `\n` when one of the inputs ends with one (up to tclutils 0.68.0 never); `lines` does not.

- `-outer none` is the inner join: only lines with a partner.
- `-outer left`, `right`, `full` include lines without a partner from the named side. The missing side is padded with empty fields, as many as the widest line of that side has fields.
- `-anti left`, `right`, `both` return only lines without a partner: key and fields of their own side, without padding.
- With `-header 1` the joined header line comes first. With `-anti` it is omitted.

Differences from `join`: the inputs need not be sorted; the right side is read as an index. The output follows the order on the left; lines without a partner from the right follow afterwards in their order. Duplicate keys yield every combination, as with `join`. The default separator is the tab, not whitespace. `-outer` corresponds to `join -a`, but pads; `-anti` corresponds to `join -v`.

Error	Message	-errorcode
unknown option	unknown option "-x"	TCLUTILS COMMON OPTION -x
option without value	missing value for option "-x"	TCLUTILS COMMON OPTION -x
field number less than 1	-leftfield must be a positive integer	TCLUTILS COMMON INTEGER -leftfield
-header not a boolean	-header requires a boolean value	TCLUTILS COMMON BOOLEAN -header
unknown value	unknown -outer: x	TCLUTILS COMMON VALUE -outer
-outer and -anti together	cannot combine -outer and -anti	TCLUTILS TUJOIN COMBINE
empty -delimiter	delimiter must not be empty	TCLUTILS COMMON DELIMITER

The message for an unknown value lists the allowed values on a second line, for example `Known: none left right full`; for `-anti` it reads `unknown -anti: x` with the `-errorcode TCLUTILS COMMON VALUE -anti`.

```
package require tclutils::tujoin
set L "1;Ada\n2;Linus\n3;Grace\n"
set R "1;Berlin\n2;Helsinki\n4;Paris\n"
::tclutils::tujoin::texts $L $R -delimiter ";"
    ;# -> 1;Ada;Berlin\n2;Linus;Helsinki\n
::tclutils::tujoin::texts $L $R -delimiter ";" -outer full
    ;# -> 1;Ada;Berlin\n2;Linus;Helsinki\n3;Grace;\n4;;Paris\n
::tclutils::tujoin::texts $L $R -delimiter ";" -anti both
                                          ;# -> 3;Grace\n4;Paris\n
::tclutils::tujoin::texts $L $R -delimiter ";" -joiner ,
    ;# -> 1,Ada,Berlin\n2,Linus,Helsinki\n
::tclutils::tujoin::texts "id;name\n1;Ada" "ort;id\nBerlin;1" \
    -delimiter ";" -header 1 -rightfield 2
    ;# -> id;name;ort\n1;Ada;Berlin
```

`tujoin` is not built for CSV with quotes; `tucsv` does the parsing, chapter Data Formats.

6.3 Comparing two sets: tucomm

tucomm compares two sorted line lists like Unix `comm`.

```
::tclutils::tucomm::compareLines aLines bLines ?-sort b? ?-nocase b?
::tclutils::tucomm::text  aText bText ?-sort b? ?-nocase b?
::tclutils::tucomm::files aFile bFile ?-sort b? ?-nocase b?
::tclutils::tucomm::formatColumns comparison ?-separator s?
```

Option	Default	Effect
-sort	0	sort both inputs beforehand, in the order of the comparison
-nocase	0	case-insensitive comparison
-separator	\t	formatColumns only: indentation per column

The return value of the three comparison commands is a dict with the keys `onlyA` (left only), `onlyB` (right only) and `both`. Under `both` stands the line from the left input. Duplicate lines count individually, as with `comm`.

`formatColumns` turns this into text in the style of `comm`: `onlyA` without indentation, `onlyB` with one separator before it, `both` with two. Unlike `comm`, the groups follow one another instead of forming one common sorted sequence.

The comparison is a merge pass with `string compare`, with `-nocase 1` on the lowercased line. This is the order by character codes, as with `comm` and `sort` in the C locale. `-sort 1` sorts both inputs in exactly this order (`lsort` without option, with `-nocase 1` by the lowercased line). Equal lines thus always end up under `both`, also with digits (`a9`, `a10`) and mixed case. Unsorted inputs without `-sort 1` give a wrong result without an error. Sorting beforehand requires `lsort` without option; `lsort -dictionary` orders `a9` before `a10` and does not match the merge pass.

Errors: unknown option `TCLUTILS COMMON OPTION -name`, a boolean that is not one

TCLUTILS COMMON BOOLEAN `-sort` or `-nocase` respectively.

```
package require tclutils::tucomm
set r [::tclutils::tucomm::text "apfel\nbirne\nkiwi\n" \
    "birne\nkiwi\nmango\n"]
dict get $r onlyA                          ;# -> apfel
dict get $r both                           ;# -> birne kiwi
::tclutils::tucomm::formatColumns $r -separator |
    ;# -> apfel\n|mango\n||birne\n||kiwi
::tclutils::tucomm::compareLines [lsort {a10 a9}] {a10}
    ;# -> onlyA a9 onlyB {} both a10
::tclutils::tucomm::compareLines {a9 a10} {a10} -sort 1
    ;# -> onlyA a9 onlyB {} both a10
::tclutils::tucomm::compareLines {a B} {B} -sort 1
    ;# -> onlyA a onlyB {} both B
```

The last two calls show the cases from the paragraph above: `a10` and `B` occur in both inputs and appear under `both`. A line-by-line difference with order is the job of `tudiff`, chapter Encoding, Hashing and Comparing.

6.4 Splitting at patterns: tucsplit

tucsplit writes a text into several files and starts a new file at each line that matches a pattern. The counterpart is Unix `csplit`.

```
::tclutils::tucsplit::splitText text pattern ?option wert ...?
::tclutils::tucsplit::file      path pattern ?option wert ...?
```

Option	Default	Effect
-regexp	1	pattern as regular expression; 0 means string match
-keepmatch	1	the matching line starts the new part; 0 drops it
-empty	0	also write empty parts
-prefix	xx	start of the file name
-suffix	empty	end of the file name, for example .md
-digits	2	digits of the sequence number, from 0
-outdir	.	target directory; created if missing
-encoding	utf-8	encoding of the parts; file reads in it too

The return value is the list of written paths, at least one: a text without a match and an empty text each yield one file. Without `-empty 1` there is no empty part before the first match. Every line keeps its line end; the parts one after another give the input again (since 0.2; up to 0.1 the parts ended without `\n`). `file` reads the bytes in `-encoding` and keeps CRLF; the pattern sees the line without its line end. Existing files of the same name are overwritten.

Differences from `csplit`: a single pattern applies to all lines; there is no list of patterns, no line numbers as boundaries and no repeat count `{*}`. Names and numbering (`xx00`, `xx01`) are as

with `csplit`.

Error	-errorcode
-outdir is a file	TCLUTILS TUCSPLIT OUTDIR
unknown encoding	TCLUTILS TUCSPLIT ENCODING name
unknown option	TCLUTILS COMMON OPTION -name
-digits less than 1	TCLUTILS COMMON INTEGER -digits
not a boolean	TCLUTILS COMMON BOOLEAN -name

```
package require tclutils::tucsplit
set dir [file join [pwd] kapitel]
set md "Vorwort\n# Eins\neins\n# Zwei\nzwei\n"
set teile [::tclutils::tucsplit::splitText $md {^# } -outdir $dir]
lmap p $teile {file tail $p}              ;# -> xx00 xx01 xx02
set teile [::tclutils::tucsplit::splitText $md {^# } -outdir $dir \
    -prefix kap -suffix .md -keepmatch 0]
lmap p $teile {file tail $p}
    ;# -> kap00.md kap01.md kap02.md
```

In the first run `xx00` contains the line `Vorwort\n`, `xx01` the text `# Eins\neins\n`, `xx02` the text `# Zwei\nzwei\n`. In the second run the headings are missing: `kap01.md` contains only `eins\n`.

6.5 Splitting by size: tusplit

tusplit splits text by line count or data by byte count, like Unix `split`.

```
::tclutils::tusplit::splitLines text ?option wert ...?
::tclutils::tusplit::splitBytes data ?option wert ...?
::tclutils::tusplit::file  path ?-lines n | -bytes n? ?option wert ...?
::tclutils::tusplit::lines path n ?option wert ...?
::tclutils::tusplit::bytes path n ?option wert ...?
```

Option	Default	Effect
-lines	1000	lines per part (splitLines, file)
-bytes	1024	bytes per part (splitBytes); for file the option switches to byte mode
-prefix	x	start of the file name
-suffix	empty	end of the file name
-digits	2	digits of the sequence number, from 0
-outdir	.	target directory; created if missing
-encoding	utf-8	splitLines only: encoding of the parts

In both modes `file` splits the bytes of the file: CRLF and the encoding stay as they are (since 0.3). Without `-lines` and `-bytes`, line mode with 1000 lines applies. `lines` and `bytes` are short forms of `file … -lines n` and `file … -bytes n`. The return value is the list of paths; an empty input yields an empty file `x00`.

`splitBytes` expects a byte sequence, that is, characters up to `\xFF`. Convert text with other characters beforehand with `encoding convertto utf-8`. A character above `\xFF` is an error `TCLUTILS TUSPLIT BYTES`, the same under Tcl 8.6 and 9.0; the check runs before any file is created.

Differences from `split`: the number is decimal (`x00`, `x01`, like `split -d`), not `xaa`, `xab`. As with `split`, every line keeps its line end; the parts one after another give the input again. Up to 0.2 every part ended without `\n`, and put together the last line of a part ran into the first line of the next. Splitting into a fixed number of parts (`split -n`) does not exist.

Error	-errorcode
-lines and -bytes together for file	TCLUTILS TUSPLIT MODE
-outdir is a file	TCLUTILS TUSPLIT OUTDIR
character above \xFF for splitBytes	TCLUTILS TUSPLIT BYTES
unknown encoding for splitLines	TCLUTILS TUSPLIT ENCODING name
count or -digits less than 1	TCLUTILS COMMON INTEGER -name
unknown option, option without value	TCLUTILS COMMON OPTION -name

```
package require tclutils::tusplit
set dir [file join [pwd] teile]
set teile [::tclutils::tusplit::splitLines "1\n2\n3\n4\n5\n" \
    -lines 2 -outdir $dir]
lmap p $teile {file tail $p}              ;# -> x00 x01 x02
set teile [::tclutils::tusplit::splitBytes abcdefg -bytes 3 \
    -outdir $dir -prefix teil -suffix .bin]
lmap p $teile {file tail $p}
    ;# -> teil00.bin teil01.bin teil02.bin
```

`x00` contains `1\n2\n`, `x02` only `5\n`. The three binary parts contain `abc`, `def` and `g`.

6.6 Writing to several targets: tutee

tutee writes the same data into several files and, on request, to `stdout`, like Unix `tee`.

```
::tclutils::tutee::write data paths ?option wert ...?
::tclutils::tutee::writeFile inputFile paths ?-append b? ?-stdout b?
::tclutils::tutee::copyChannel inChan outChans ?option wert ...?
```

Option	Default	Commands	Effect
-append	0	write, writeFile	append instead of overwrite
-stdout	0	all	additionally to stdout

Option	Default	Commands	Effect
-nonewline	0	write	no \n at the end
-chunksize	8192	copyChannel	characters per read

- `write` writes `data` with `puts`, that is, with `\n` at the end, except with `-nonewline 1`; unlike the `tee` command, which adds nothing. The return value is `data`.
- `writeFile` copies the bytes of `inputFile` unchanged, CR and encoding included. The return value is the number of bytes. `-nonewline` does not exist here. Up to 0.2 `writeFile` read the file as text and wrote LF in the system encoding.
- `copyChannel` reads `inChan` to the end and writes each block to all channels in `outChans`. The return value is the number of characters copied; for channels opened in binary mode these are bytes. The caller opens and closes the channels.

`write` opens all targets before it writes, and flushes them only once all are open. If opening a target fails, the error comes from `open` (`POSIX ENOENT …`). The channels already opened are then closed again, existing files keep their content, and files that this call created are removed again. `writeFile` behaves the same way.

Option errors: `TCLUTILS COMMON BOOLEAN -name`, `TCLUTILS COMMON INTEGER -chunksize`, `TCLUTILS COMMON OPTION -name`.

```
package require tclutils::tutee
set dir [file join [pwd] tee]
file mkdir $dir
set a [file join $dir a.txt]
set b [file join $dir b.txt]
::tclutils::tutee::write hallo [list $a $b]
                                          ;# -> hallo
::tclutils::tutee::write noch [list $a] -append 1 -nonewline 1
                                          ;# -> noch
set in  [open $a rb]
set out [open [file join $dir kopie.txt] wb]
::tclutils::tutee::copyChannel $in [list $out]
                                          ;# -> 10
close $in; close $out
```

Afterwards `a.txt` contains the text `hallo\nnoch`, `b.txt` the text `hallo\n`, `kopie.txt` the same as `a.txt`.

6.7 Processing lists in batches: tuxargs

tuxargs splits a list into batches and calls a Tcl command for each batch. The counterpart is Unix `xargs`; instead of a program, a command prefix runs.

```
::tclutils::tuxargs::batches items ?-n n? ?-skipempty b?
::tclutils::tuxargs::apply   items commandPrefix ?option wert ...?
::tclutils::tuxargs::command items commandPrefix ?option wert ...?
```

Option	Default	Commands	Effect
-n	100	all	at most this many elements per batch
-skipempty	1	all	remove empty elements beforehand
-collect	1	apply, command	collect results; 0 returns empty

`batches` returns the list of batches. `apply` calls `{*}$commandPrefix {*}$stapel` per batch and, with `-collect 1`, returns the list of results, one element per batch. `command` is the same command under another name. An error in the called command aborts the run and reaches the caller unchanged.

Differences from `xargs`: the input is a Tcl list, not text split at whitespace; quotes and spaces in elements are therefore not an issue. There is no limit by character count and no parallel execution (`xargs -P`). The default `-n 100` is fixed, not dependent on the length of the command line.

Errors: `-n` less than 1 is `TCLUTILS COMMON INTEGER -n`; in addition `TCLUTILS COMMON BOOLEAN -name` and `TCLUTILS COMMON OPTION -name`.

```
package require tclutils::tuxargs
::tclutils::tuxargs::batches {a b c d e} -n 2
                                          ;# -> {a b} {c d} e
::tclutils::tuxargs::batches {a {} b} -n 2
                                          ;# -> {a b}
::tclutils::tuxargs::apply {1 2 3 4 5} ::tcl::mathop::+ -n 2
                                          ;# -> 3 7 5
::tclutils::tuxargs::apply {a b c} {list x} -n 2
                                          ;# -> {x a b} {x c}
::tclutils::tuxargs::apply $pfade {file delete} -n 50 -collect 0
```

The last call deletes the files from `$pfade` in batches of 50.

6.8 Ordering dependencies: tutsort

tutsort sorts topologically, like Unix `tsort`: each pair `u v` means "u comes before `v`".

```
::tclutils::tutsort::pairs tokens
::tclutils::tutsort::text  text
::tclutils::tutsort::file  path
```

`pairs` takes a flat list and returns a list. `text` and `file` split at whitespace, like `tsort`, and return the nodes separated by `\n`. A pair `u u` registers a node without a constraint. The order is stable: of the nodes that become free, the one read first comes first. The output of GNU `tsort` is ordered differently for the same input, but is equally valid.

Error	Message	-errorcode
odd number of words	input contains an odd number of tokens	TCLUTILS TUTSORT ODD
cycle	input contains a cycle involving: …	TCLUTILS TUTSORT CYCLE

On a cycle `tutsort` aborts; GNU `tsort` prints a warning and still returns a sequence. The message names only the nodes that lie on a cycle, in the order of their first occurrence. Nodes that lie only behind a cycle are not included.

```
package require tclutils::tutsort
::tclutils::tutsort::pairs {hemd jacke hose guertel hose schuhe}
    ;# -> hemd hose jacke guertel schuhe
::tclutils::tutsort::text "a b\nb c\nx x"
                                          ;# -> a\nx\nb\nc
::tclutils::tutsort::pairs {a b b a b c}
    ;# Fehler: input contains a cycle involving: a b
```

6.9 Fields and rules: tuawk

tuawk brings the awk model to Tcl: each line is a record, is split into fields, and a list of rules made of pattern and action runs over it. It is not an interpreter of the awk language. Patterns are Tcl expressions, actions are Tcl scripts.

```
::tclutils::tuawk::run  text rules ?-fs s? ?-ofs s? ?-ors s?
::tclutils::tuawk::file path rules ?-fs s? ?-ofs s? ?-ors s?
```

Option	Default	Effect
-fs	empty	field separator; empty means runs of whitespace
-ofs	one space	stands between the arguments of emit
-ors	\n	stands after each output of emit

The return value is the collected output of all `emit` calls as a string. `tuawk` writes nothing to `stdout`. `file` reads the file as text and calls `run`.

6.9.1 Rules

`rules` is a flat list of pairs `muster aktion`. For each record the rules run in their order; each matching rule executes its action.

Pattern	Meaning
BEGIN	action runs once before the first record
END	action runs once after the last record
empty ({})	always matches
/regex/	regular expression against the whole line
otherwise	Tcl expression for expr, for example {$2 > 3} or {$1 eq "x"}

An empty action means `emit $0`, that is, output the line.

6.9.2 Fields and variables

Name	Content
$0	the whole line
$1 ...	the fields, from 1
$NF	number of fields of this line; 0 before the first record
$NR	number of the line, from 1; 0 before the first record
emit arg ...	output: arguments joined with -ofs, followed by -ors

Fields are variables and can be changed in an action (`set 2 …`); `$0` does not change in the process.

Field separator `-fs`: empty splits at runs of whitespace like awk and ignores whitespace at the edges. A single character splits with `split`; two separators in a row yield an empty field. Several characters count as a regular expression.

All rules of a call run in their own frame, like the body of a procedure, in a namespace that is deleted after the run. `set name …` always creates a variable of this frame, never a global variable of the same name; this holds equally under Tcl 8.6 and 9.0. Variables from actions persist across all records up to `END`, like global variables in awk. The actions have the full Tcl command set; rules from an outside source must therefore be treated like code.

In an action, `break` skips the remaining rules for the current record, like `next` in awk; `continue` goes to the next rule.

6.9.3 Differences from awk

- A field beyond `NF` is an error, not an empty string: `can't read "3": no such variable`, `-errorcode TCL LOOKUP VARNAME 3`. If shorter lines are possible, check `$NF` first.
- An unset variable is also an error, not 0. Initial values belong in `BEGIN`.
- `-fs " "` splits at every single space; only the empty separator has the awk meaning "whitespace".
- No `printf`, no `getline`, no custom record separator. For formatting, use `format` in the argument of `emit`.

Error	-errorcode
unknown option, option without value	TCLUTILS TUAWK OPTION
odd number of elements in rules	TCLUTILS TUAWK RULES
field beyond NF	TCL LOOKUP VARNAME n

Errors in patterns and actions reach the caller unchanged.

6.9.4 Examples

```
package require tclutils::tuawk
set daten "Ada 3\nLinus 5\nGrace 4\n"
::tclutils::tuawk::run $daten {{$2 > 3} {emit $1}}
                                          ;# -> Linus\nGrace\n
::tclutils::tuawk::run $daten {/^L/ {}}  ;# -> Linus 5\n
::tclutils::tuawk::run $daten {{} {emit $NR $NF $1}} -ofs ";"
```

```
    ;# -> 1;2;Ada\n2;2;Linus\n3;2;Grace\n
set regeln {
    BEGIN {set s 0}
    {}    {set s [expr {$s + $2}]}
    END   {emit "Summe: $s"}
}
::tclutils::tuawk::run $daten $regeln    ;# -> Summe: 12\n
```

Grouping with a dict, changing separators and output:

```
set log "INFO start\nERROR disk\nINFO ende\n"
::tclutils::tuawk::run $log {
    BEGIN {set n {}}
    {}    {dict incr n $1}
    END   {dict for {k v} $n {emit $k $v}}
}                                          ;# -> INFO 2\nERROR 1\n
::tclutils::tuawk::run $log {/ERROR/ {emit $NR $2}}
                                           ;# -> 2 disk\n
::tclutils::tuawk::run "Ada;3\nLinus;5" {{} {emit $2 $1}} \
    -fs ";" -ofs ,                         ;# -> 3,Ada\n5,Linus\n
::tclutils::tuawk::run "k=v; x=y" {{} {emit $2}} -fs {;\s*}
                                           ;# -> x=y\n
::tclutils::tuawk::run "a b" {{} {emit [format %-4s| $1]}}
                                           ;# -> a   |\n
```

For a complete awk program, `exec awk` remains.

6.10 Approximate comparison: tufuzzy

tufuzzy provides the basics of approximate search. The measure is the Levenshtein distance: the smallest number of insertions, deletions and substitutions of single characters.

```
::tclutils::tufuzzy::distance       a b              ?-nocase b?
::tclutils::tufuzzy::searchDistance pattern text     ?-nocase b?
::tclutils::tufuzzy::similarity     a b              ?-nocase b?
::tclutils::tufuzzy::subsequence    pattern text     ?-nocase b?
::tclutils::tufuzzy::bestMatch      pattern candidates ?-nocase b?
```

Command	Return value
distance	distance between the two strings, an integer
searchDistance	smallest distance from pattern to any substring of text
similarity	1.0 -abstand /längere Länge, between 0.0 and 1.0; two empty strings yield 1.0
subsequence	1 if the characters of pattern occur in text in this order, otherwise 0

Command	Return value
bestMatch	the candidate with the smallest distance; on a tie the first; an empty list yields ""

Option	Default	Effect
-nocase	0	both sides with string tolower beforehand

`subsequence` is the search used by fzf-style selection lists: `tcu` matches `tclutils`, `utc` does not. The computation works on characters, not bytes; umlauts count as one character.

Error	Message	-errorcode
unknown option	unknown option -x	TCLUTILS TUFUZZY OPTION
option without value	missing value for option -x	TCLUTILS TUFUZZY OPTION
extra arguments	unexpected arguments: ...	TCLUTILS TUFUZZY ARGS
-nocase not a boolean	-nocase requires a boolean value	TCLUTILS COMMON BOOLEAN -nocase

```
package require tclutils::tufuzzy
::tclutils::tufuzzy::distance kitten sitting     ;# -> 3
::tclutils::tufuzzy::distance Haus haus -nocase 1
                                                 ;# -> 0
::tclutils::tufuzzy::searchDistance gama "gamma line"
                                                 ;# -> 1
::tclutils::tufuzzy::similarity kitten sitting
                                              ;# -> 0.5714285714285714
::tclutils::tufuzzy::subsequence tcu tclutils    ;# -> 1
::tclutils::tufuzzy::bestMatch tujion {tujoin tupaste tucomm}
                                                 ;# -> tujoin
```

6.11 Approximate search: tuagrep

tuagrep is a grep with error tolerance, built on `tufuzzy::searchDistance`. A line matches if the distance of the pattern to a substring of the line is at most `-maxdist`. The counterpart is Unix `agrep`.

```
::tclutils::tuagrep::search text     pattern ?option wert ...?
::tclutils::tuagrep::match  line     pattern ?option wert ...?
::tclutils::tuagrep::file   filename pattern ?option wert ...?
::tclutils::tuagrep::files  fileList pattern ?option wert ...?
```

Option	Default	Effect
-maxdist	1	largest allowed distance, an integer from 0; 0 means exact substring
-nocase	0	case-insensitive

Option	Default	Effect
-invert	0	the lines without a match
-linenumbers	0	each line as {nummer text}, number from 1
-count	0	only the count
-filenames	0	file only: matches as {datei treffer}
-fileswithmatches	0	file, files only: only the paths with a match

Command	Return value
search	list of matches, with -count 1 the count
match	1 or 0 for one line; honors -maxdist, -nocase, -invert
file	like search; with -fileswithmatches 1 the path or ""
files	matches of all files, always with file name; with -count 1 a dict pfad anzahl; with -fileswithmatches 1 the list of paths

The options are the same as those of `tugrep` from Text and Files. `tuagrep` has no `-max`, no `-word` and no `-wholeline`, and the pattern is always literal: no regular expressions. `agrep`, by contrast, allows patterns with error tolerance and weights for insertion, deletion and substitution; `-maxdist` corresponds to `agrep -k` or `-1` to `-9`.

`search` splits the text at `\n` into the same lines that `file` reads: a `\n` at the end closes the last line and does not start an empty one; an empty text has no line. An empty pattern matches every line. A text has no file name; `search` and `match` accept `-filenames` and `-fileswithmatches` but ignore them. `files` always adds the file name.

Error	Message	-errorcode
unknown option	unknown option -x	TCLUTILS TUAGREP OPTION
option without value	missing value for option -x	TCLUTILS TUAGREP OPTION
extra arguments	unexpected arguments: …	TCLUTILS TUAGREP ARGS
-maxdist not an integer from 0	-maxdist must be a non-negative integer	TCLUTILS TUAGREP MAXDIST
switch not a boolean	-invert requires a boolean value	TCLUTILS COMMON BOOLEAN -name

A file that cannot be opened is reported by `open` with `POSIX ENOENT …`.

```
package require tclutils::tuagrep
set text "gamma line\nbeta line\ndelta"
::tclutils::tuagrep::search $text gama   ;# -> {gamma line}
::tclutils::tuagrep::search $text gama -maxdist 0
                                         ;# ->
::tclutils::tuagrep::search $text gama -linenumbers 1
                                         ;# -> {1 {gamma line}}
```

```
::tclutils::tuagrep::search $text gama -invert 1
                                          ;# -> {beta line} delta
::tclutils::tuagrep::match Hauptstrasse strase
                                          ;# -> 1
::tclutils::tuagrep::search "eins\nzwei\n" xyz -invert 1
                                          ;# -> eins zwei
::tclutils::tuagrep::search $text gama -maxdist x
    ;# Fehler: -maxdist must be a non-negative integer
```

With files; `a.txt` contains `gamma line` and `beta`, `b.txt` contains `nichts` and `gamma`, `c.txt` contains `leer`:

```
::tclutils::tuagrep::files {a.txt b.txt c.txt} gama
    ;# -> {a.txt {gamma line}} {b.txt gamma}
::tclutils::tuagrep::files {a.txt b.txt c.txt} gama -count 1
                                          ;# -> a.txt 1 b.txt 1 c.txt 0
::tclutils::tuagrep::files {a.txt b.txt c.txt} gama \
    -fileswithmatches 1                   ;# -> a.txt b.txt
```

Next chapter: Characters, Numbers, Dates and Paths

7 Characters, Numbers, Dates and Paths

Four modules for what the Tcl core provides only awkwardly or not at all: visible characters (tustr), amounts (tunum), calendar dates (tudate) and paths without a file system (tupath). All four are in the umbrella, none needs Tk.

7.1 tustr

`tustr` extends `string`. What the core can do (`string reverse`, `string map`, `string repeat`) stays with the core.

Command	Effect
isEmpty s	1 for an empty string
truncate s n ?ende?	shortens to n characters; the default for ende is ..., its characters count
padLeft, padRight, center	pad to a width; the default pad character is the space
startsWith, endsWith	1 if the edge matches
removePrefix, removeSuffix	cut off the edge, otherwise unchanged
splitTrim s ?sep?	splits, removes spaces, drops empty pieces; the default for sep is the space
toCamel, toSnake	my-cool_var becomes myCoolVar, myCoolVar becomes my_cool_var; letters of every script count

Command	Effect
slugify s	ASCII lowercase: umlauts spelled out, otherwise everything except a–z and digits becomes -
capitalize s	first letter uppercase
count s teil	non-overlapping matches
graphemes s	list of the visible characters
isPrintable zeichen	1 if exactly one printable character is given

`truncate`, `padLeft`, `padRight` and `center` calculate with `string length`, not with `graphemes`. A negative width or length results in `{TCLUTILS TUSTR ARG}`.

```
package require tclutils::tustr
::tclutils::tustr::truncate HalloWelt 6          ;# -> Hal...
::tclutils::tustr::toCamel my-cool_var           ;# -> myCoolVar
::tclutils::tustr::toSnake myCoolVar             ;# -> my_cool_var
::tclutils::tustr::splitTrim { a , , b } ,       ;# -> a b
::tclutils::tustr::count ababa ba                ;# -> 2
::tclutils::tustr::slugify "Hallo, Welt!"        ;# -> hallo-welt
::tclutils::tustr::slugify "Größe"               ;# -> groesse
::tclutils::tustr::toCamel "Jörg Müller"         ;# -> jörgMüller
```

`slugify` returns ASCII for file names and URLs. It spells Latin letters in ASCII first: ä becomes `ae`, ö `oe`, ü `ue`, ß `ss`, accents are dropped (é becomes `e`, ł becomes `l`). What has no ASCII form, such as `??`, is dropped. `toCamel` and `toSnake` keep letters of every script (since 0.3; up to 0.2 `Größe` became `gr-e` and `toCamel "Jörg Müller"` became `jRgMLler`).

7.1.1 Visible Characters

`graphemes` splits a string into visible characters, a simplified UAX #29. Base characters with combining accents, emoji ZWJ sequences, flag pairs and CR LF stay together. Hangul syllables and Indic conjuncts are not covered. `[join [graphemes $s] ""]` gives `$s` again.

`isPrintable` checks exactly one character against Unicode, not only ASCII: 0 for C0/C1 control characters and DEL, 1 for U+00A0. An empty string or more than one character throws `{TCLUTILS TUSTR ARG}` with `isPrintable expects exactly one character`. Surrogate pairs that Tcl 8.6 delivers from a channel count as one character in both commands.

```
llength [::tclutils::tustr::graphemes "e\u0301x"]   ;# -> 2
string length "e\u0301x"                           ;# -> 3
::tclutils::tustr::isPrintable \u00e4               ;# -> 1
::tclutils::tustr::isPrintable \x07                 ;# -> 0
```

7.2 tupath

`tupath` wraps `file`. `clean` and `relative` do not query the file system; `normalize` and `readlink` do.

Command	Effect
normalize pfad	absolute, resolves ., .. and symbolic links (file normalize)
clean pfad	., .. and // only lexically; an empty path becomes .
isAbsolute pfad	1 or 0
components pfad	file split
join teile...	file join
relative basis ziel	way from the base to the target, both absolute or both relative
commonPath liste	longest common directory, otherwise ""
readlink pfad	target of a symbolic link

```
package require tclutils::tupath
::tclutils::tupath::clean a/./b/../c             ;# -> a/c
::tclutils::tupath::clean /a/../../b             ;# -> /b
::tclutils::tupath::clean ""                     ;# -> .
::tclutils::tupath::commonPath {/a/b/c /a/b/d}   ;# -> /a/b
::tclutils::tupath::relative /a/b /a/b/c/d       ;# -> c/d
::tclutils::tupath::relative /a/b /a/b           ;# -> .
```

Since tclutils 0.64.0 an absolute path keeps its root: `..` stops there. On Windows, according to the code, this also applies to `C:/`, `/` (drive-relative) and `//server/share`. The module version stayed 0.1.

`relative` with one relative and one absolute path throws `{TCLUTILS TUPATH RELATIVE}`. `commonPath` without a path throws `{TCLUTILS TUPATH ARGS}`. `readlink` on a regular file throws `{TCLUTILS TUPATH NOTLINK}`. If the paths share no first component, `commonPath` returns `""`.

7.3 tudate

`tudate` calculates in the local time zone, on top of `clock`.

Command	Effect
parse text ?-format fmt?	seconds since the epoch; without -format a list of formats, then a free-form attempt
iso sekunden ?-time 0?	yyyy-mm-dd, with -time 1 additionally THH:MM:SS
add sekunden zahl einheit	calendar addition; unit second, minute, hour, day, week, month, year, also in the plural
diff a b ?-unit days?	a minus b, as an integer; units seconds, minutes, hours, days, weeks
relative sekunden ?-base sek?	English day phrase; without -base the base is now

Command	Effect
today	today's date as ISO text
zones	sorted names for clock format -timezone

```
package require tclutils::tudate
set t [::tclutils::tudate::parse 15.03.2024]
::tclutils::tudate::iso $t                                  ;# ->
    2024-03-15
::tclutils::tudate::iso [::tclutils::tudate::add $t 1 month]
                                                            ;# ->
    2024-04-15
::tclutils::tudate::relative $t -base $t                    ;# -> today
::tclutils::tudate::relative [::tclutils::tudate::add $t 3 day] -base
    $t
                                                            ;# -> in 3 days
expr {"Europe/Berlin" in [::tclutils::tudate::zones]}   ;# -> 1
```

Without `-format`, `parse` tries in turn the ISO date and ISO date-time (also without seconds and as `YYYYMMDD`), `d.m.Y` (also with a time), `d/m/Y`, `Y/m/d`, `m/d/Y`, `H:M:S`, `H:M` and English month names (`29 Oct 2025`, `Oct 29 2025`). `02/03/2026` is 2 March; `m/d/Y` only applies when `d/m/Y` gives no date, so `10/29/2026` is 29 October. A string of digits and separators only never goes to the free-form attempt.

`parse` rejects a day or month that does not exist, with and without `-format`, on Tcl 8.6 as on 9.0: `2026-02-29`, `31.04.2026`, `29/02/2026` and `Feb 30 2026` give `{TCLUTILS TUDATE PARSE}`. `clock scan` alone turns these into 1 March, 1 May and 2 May 2028 on 8.6; `parse` formats every match back and drops it when the numbers differ. Up to tclutils 0.68.0 these days slipped through on 8.6.

```
::tclutils::tudate::iso [::tclutils::tudate::parse 10/29/2026]
                                                            ;# ->
    2026-10-29
catch {::tclutils::tudate::parse 2026-02-29} m o
dict get $o -errorcode                                      ;# -> TCLUTILS
    TUDATE PARSE
```

`relative` answers in English: `today`, `tomorrow`, `yesterday`, `in N days`, `N days ago`. An unreadable date results in `{TCLUTILS TUDATE PARSE}`, an unknown unit in `{TCLUTILS TUDATE UNIT}`.

`diff -unit days` divides the seconds by 86400 and truncates. It does not count calendar days. From March 15, 2024 to April 15, 2024 it returns 30, not 31: in the local zone (CEST, `TZ` not set) one hour is missing because of daylight saving time. `add` itself goes through `clock add` and lands on April 15.

`zones` reads the first source that has entries: Tcl's tzdata, otherwise `/usr/share/zoneinfo`. It checks names from the system directory with `clock`, because Tcl 8.6 without its own tzdata rejects names like `Etc/GMT+1`. `UTC` is always included. The list does not mix the two sources.

7.4 tunum

`tunum` reads amounts as they appear in tables and writes them in German format. `tunumfmt` is a different module: it reads sizes like `1.5K`, `2Mi` and `2KiB`.

Command	Effect
parse text ?-default {}? ?-locale auto?	Tcl double, or -default if nothing is recognized
sum werte ?-default 0?	sum, unreadable entries are dropped; always locale auto
isNumber text	1 if parse with auto finds a value
format zahl ?-locale de? ?-decimals 2?	period as thousands separator, decimal comma

Locale `auto` (default of `parse`): a comma followed by digits at the end is the decimal comma, periods are thousands separators. Otherwise the period is decimal and the commas are thousands separators. `€`, `$`, `£`, `¥` and spaces are dropped.

Locale `de-strict`: the period is always a thousands separator and is dropped, the comma is decimal, currency symbols stay. `956.06` thus becomes `95606.0`. For the period as decimal separator, use `auto`.

```
package require tclutils::tunum
::tclutils::tunum::parse "1.234,56"                ;# -> 1234.56
::tclutils::tunum::parse "1,234.56"                ;# -> 1234.56
::tclutils::tunum::parse "12 \u20AC"               ;# -> 12.0
::tclutils::tunum::parse "abc"                     ;# -> (leer)
::tclutils::tunum::sum {1,50 2,00 abc 4,20}        ;# -> 7.7
::tclutils::tunum::parse 956.06 -locale de-strict  ;# -> 95606.0
::tclutils::tunum::format 1234.5                   ;# -> 1.234,50
::tclutils::tunum::format 1234.5 -decimals 0       ;# -> 1.235
::tclutils::tunum::format 2.675                    ;# -> 2,68
::tclutils::tunum::format -2.5 -decimals 0         ;# -> -3
format %.2f 2.675                                  ;# -> 2.67
```

Since module 0.4, `format` rounds half away from zero on the decimal digits, the same on every platform. Up to tclutils 0.63.0 this went through `format %.*f`, and there the C runtime decided. The different behavior of the Windows runtime before 0.4 is described in the CHANGELOG. The caller appends a currency: `"[::tclutils::tunum::format $n] EUR"`.

The procedure is named `format` and lives in the namespace `tclutils::tunum`. The call stays `::tclutils::tunum::format`; that way it does not hide Tcl's `format`. `-decimals` below 0 results in `{TCLUTILS TUNUM DECIMALS}`, a locale other than `de` results in `{TCLUTILS TUNUM LOCALE}`. `parse -locale` knows `auto` and `de-strict`.

7.5 In short

- Visible length: `tustr::graphemes`, not `string length`.
- `slugify` returns ASCII and spells umlauts out: `Größe` becomes `groesse`.
- `tupath::clean` and `relative` read no file; `..` stops at the root.

- `tudate::diff -unit days` divides seconds; `relative` answers in English.
- Amounts: `tunum::format`. `de-strict` treats every period as a thousands separator.
- Nationwide public holidays: chapter Notes and Calendars.

Next chapter: Data Formats

8 Data Formats

CSV, JSON and INI are pure Tcl and need no third-party package. Amounts are read by `tunum` in the chapter Characters, Numbers, Dates and Paths, the text of a PDF in the chapter Text from PDF Files. Base64, digests and dicts are described in the chapter Encoding, Hashing and Comparing. Markdown, iCalendar, vCard and notes are described in the chapter Notes and Calendars.

8.1 CSV: tucsv

tucsv splits lines, joins them again and reads a table into dicts. It is not an import framework.

Command	Effect
parseLine zeile	one line to a list; -delimiter (default comma), -quote (default "), -trim (default 0)
joinLine felder	a list to a line; the same -delimiter and -quote, plus -alwaysQuote (default 0)
parse text ?optionen?	lines to a list of field lists; a field may contain a line break
text zeilen ?optionen?	field lists back to text; additionally -newline, -bom, -profile
dicts zeilen	the first line becomes the header, each further line a dict
file pfad ?optionen?	a file, like parse; additionally -encoding (default utf-8), -profile
writeFile pfad zeilen ?optionen?	writes a file; like text, plus -encoding

A field with a comma, a quote or a line break is put in quotes. A quote inside the field is doubled. `-alwaysQuote 1` puts every field in quotes.

```
package require tclutils::tucsv
::tclutils::tucsv::parseLine {a,"b,c","d""e"}
                                          ;# -> a {b,c} {d"e}
::tclutils::tucsv::joinLine [list a {b,c} {d"e}]
                                          ;# -> a,"b,c","d""e"
set zeilen [::tclutils::tucsv::parse "name;city\nAlice;Berlin"
    -delimiter {;}]
                                          ;# -> {name city} {Alice
    Berlin}
```

```
::tclutils::tucsv::dicts $zeilen          ;# -> {name Alice city
    Berlin}
::tclutils::tucsv::parse "a,\"b\nc\",d"   ;# -> a {b\nc} d
```

`-strict 1` is the default. An unclosed quote throws `{TCLUTILS TUCSV QUOTE}` with `unterminated quoted field`. `-strict 0` takes the rest as a field and does not throw. `parse` removes a leading UTF-8 BOM by itself, including the one from `-profile excel-de`.

Since `tucsv` 0.2 there are profiles for Excel. `-profile excel-de` sets a semicolon, CRLF and a UTF-8 BOM, so that a German Excel recognizes umlauts and columns. `-profile rfc4180` sets a comma and CRLF. An option after the profile wins over the profile. An unknown profile throws `{TCLUTILS TUCSV PROFILE <name>}`.

```
package require tclutils::tucsv 0.2
set zeilen {{Name Ort} {Müller Köln} {{A, B} Essen}}
::tclutils::tucsv::writeFile liste.csv $zeilen -profile excel-de
                                          ;# BOM, "Name;Ort" CRLF ...
::tclutils::tucsv::file liste.csv -delimiter {;}
                                          ;# -> {Name Ort} {Müller
    Köln} {{A, B} Essen}
::tclutils::tucsv::text {{a b}} -profile excel-de -delimiter ,
                                          ;# BOM + a,b
```

`file` and `writeFile` read and write UTF-8 unless `-encoding` says otherwise. `writeFile` writes the line ends from `-newline` unchanged; without `-newline` the platform default applies.

Fields without quotes, cut only at the delimiter, belong to `tucut` in the chapter Text and Files.

8.2 JSON: tujson

tujson reads and writes JSON. It is not JSON5: no comments, no trailing commas, no JSONPath, no streaming.

`parse` and `fromJson` return Tcl values. The distinction between object, array and scalar is lost in the process.

JSON	Tcl from parse
object	dict, keys in the order of the text
array	list
string	string
number	the digit text from the source
true / false	1 / 0
null	empty string

If an object holds a key twice, the last value wins, as in JavaScript; RFC 8259 only says that names should be unique.

```
package require tclutils::tujson
set d [::tclutils::tujson::parse
    {{"name":"Gregor","items":[1,2],"ok":true,"x":null}}]
dict get $d name                          ;# -> Gregor
dict get $d items                         ;# -> 1 2
dict get $d ok                            ;# -> 1
dict get $d x                             ;# -> (leer)
```

Malformed text throws `{TCLUTILS TUJSON PARSE offset}`. For `{` alone the message is `invalid JSON at offset 1: expected object key string`.

`parseTyped` keeps the kind. Each node is `{art wert}`:

Kind	Value
object	dict, key to a node
array	list of nodes
string	the string
number	the digits from the source
boolean	true or false
null	empty

```
::tclutils::tujson::parseTyped {{"tags":["x"],"n":1}}
    ;# -> object {tags {array {{string x}}} n {number 1}}
```

`toJson` writes such a tree back. The constructors `str`, `num`, `bool`, `null`, `obj` and `arr` build it by hand. `-indent` N with N > 0 breaks lines, N spaces per level. The default 0 is a single line. An unknown kind throws `{TCLUTILS TUJSON TYPE}`.

```
set doc [::tclutils::tujson::obj [list \
    name [::tclutils::tujson::str Alice] \
    n    [::tclutils::tujson::num 1] \
    z    [::tclutils::tujson::null]]]
::tclutils::tujson::toJson $doc
    ;# -> {"name":"Alice","n":1,"z":null}
```

`quote` and `escape` put a string in JSON quotes. `validate` checks without building a value. `minify` and `pretty` format text, `fileMinify` and `filePretty` a file. On the command line: `tclsh bin/tujson.tcl validate|minify|pretty|parse datei`.

`true` as `1` and `null` as empty are intended behavior of `parse`. If the kind matters, use `parseTyped`.

8.3 INI: tuini

tuini reads INI into a dict: section to a dict of key to value. Keys before the first `[section]` are in the section `""`. The order is kept. Comments with `#` or `;` are dropped.

Command	Effect
parse text	the dict
toIni dict	INI text, schlüssel=wert without spaces around =
sections dict	the section names
keys dict ?abschnitt?	the keys
get dict abschnitt schlüssel ?vorgabe?	the value, or the default
has dict abschnitt schlüssel	1 or 0
setValue dict abschnitt schlüssel wert	a copy with the new value
addSection dict abschnitt	a copy in which the section exists
removeKey, removeSection	a copy without the key or section

`setValue` does not change the dict passed in.

```
package require tclutils::tuini
set ini [::tclutils::tuini::parse "# k\n\n\[lager\]\nort = Nord\n"]
::tclutils::tuini::sections $ini             ;# -> lager
::tclutils::tuini::get $ini lager ort        ;# -> Nord
::tclutils::tuini::get $ini lager fehlt -    ;# -> -
set ini [::tclutils::tuini::setValue $ini lager gang 3]
::tclutils::tuini::toIni $ini                ;# ->
    \[lager\]\nort=Nord\ngang=3
```

If a key appears twice in a section, the last value wins; Windows (`GetPrivateProfileString`) takes the first. Blanks at the edges of key and value are dropped, so a value cannot end in a blank.

An unclosed section throws `{TCLUTILS TUINI SYNTAX}`, for example `unterminated section header: [offen`. What INI cannot hold, `setValue`, `addSection` and `toIni` refuse since 0.2 with `{TCLUTILS TUINI VALUE}`: a line break in a value; in a key `=`, a line break, `;`, `#` or `[` at the start, blanks at the edges; in a section name `]`, a line break, blanks at the edges. Up to 0.1 `toIni` wrote such text, and `parse` failed on it.

```
catch {::tclutils::tuini::setValue {} lager notiz "a\nb"} m
set m                                    ;# -> value of lager.notiz:
    no line breaks in INI
```

`tusettings`, chapter tusettings, checks the same limits when saving and also refuses blanks at the edges of a value.

8.4 XML: tuxml

tuxml writes small elements. It does not validate a document and knows no XPath. `escape` converts `&`, `<` and `>`; with `-quotes 1` also quotes. `unescape` resolves the predefined entities and `&#…;`. `textElement` escapes the text, `element` inserts it raw.

```
package require tclutils::tuxml
::tclutils::tuxml::declaration               ;# -> <?xml version="1.0"
    encoding="UTF-8"?>
::tclutils::tuxml::textElement lager {ort Nord} {a < b}
                                             ;# -> <lager ort="Nord">a
    &lt; b</lager>
```

A name that does not start with a letter or `_` throws `{TCLUTILS TUXML NAME <name>}`, an attribute name `{TCLUTILS TUXML ATTR <name>}`.

8.5 SQLite: tusqlite

tusqlite opens no database and does not load `sqlite3`. The caller passes an open handle. `insert` builds the statement in its own scope so that a value does not bind to a variable of the caller. `rows` returns dicts in the order of the rows. `value` takes the first column of the first row, otherwise the default. `value db sql -default d -params dict` keeps the default and the placeholders apart; the positional form `value db sql default ?dict?` still works. Without options `value db $sql {id 1}` took the dict as the default and returned it when no row matched.

Since `tusqlite` 0.2, `rows` and `value` take a dict with values for the placeholders `:name`. The query also runs in its own scope: it does not reach the caller's variables, and a placeholder without a value in the dict becomes SQL NULL. `update` and `delete` take the condition as a dict (`column value`, joined with AND) and return the number of changed rows. They refuse an empty condition dict, so that a mistake does not hit every row.

A value equal to `[tusqlite::null]` becomes SQL NULL. `rows` returns this cell as an empty string. `quoteId` puts a name in quotes and doubles a quote inside it.

```
package require sqlite3
package require tclutils::tusqlite
sqlite3 db :memory:
db eval {create table lager (ort text, gang integer)}
::tclutils::tusqlite::insert db lager {ort Nord gang 3}
                                             ;# -> 1
::tclutils::tusqlite::rows db {select ort, gang from lager}
                                             ;# -> {ort Nord gang 3}
::tclutils::tusqlite::value db {select gang from lager where ort =
    'x'} -
                                             ;# -> -
::tclutils::tusqlite::insert db lager {ort Sued gang 5}
                                             ;# -> 2
::tclutils::tusqlite::rows db {select ort, gang from lager where gang
    > :min} {min 4}
                                             ;# -> {ort Sued gang 5}
::tclutils::tusqlite::value db {select gang from lager where ort = :o}
    - {o Nord}
                                             ;# -> 3
::tclutils::tusqlite::update db lager {gang 4} {ort Nord}
```

```
                                                ;# -> 1
::tclutils::tusqlite::delete db lager {ort Sued}
                                                ;# -> 1
```

An empty dict for `insert` or for the new values of `update` throws `{TCLUTILS TUSQLITE EMPTY}`, an empty condition dict `{TCLUTILS TUSQLITE WHERE}`, a placeholder name with characters other than letters, digits and `_` `{TCLUTILS TUSQLITE PARAM <name>}`.

8.6 LDIF: tuldif

tuldif reads LDIF according to RFC 2849 into a list of entries and writes them back. An entry is an ordered list of pairs `{attribut wert}`. `dn` is an attribute like any other; a multi-valued attribute appears several times in the list.

Command	Effect
parse text ?-binary namen?	list of entries; a blank line separates two entries
toLdif einträge ?-binary namen?	LDIF text, a blank line after each entry
dn eintrag	value of dn, otherwise the empty string
get eintrag attribut	all values of the attribute; case does not matter
attributes eintrag	attribute names in the order of first appearance
toDict eintrag	dict of attribute to list of values
addAttr eintrag attribut wert	a copy with the pair appended
setAttr eintrag index attribut wert	a copy with the pair at index replaced
removeAttr eintrag index	a copy without the pair at index

`parse` skips comment lines with `#` and the `version:` line. A line that starts with a space continues the previous one. A value `attr:: …` is decoded through `tubase64` and read as UTF-8, because LDIF values are UTF-8 according to RFC 2849. Bytes that do not form valid UTF-8 stay bytes, one character per byte. With `attr:< url`, the URL itself is the value; no file is read.

`toLdif` writes a value as Base64 if it starts with a space, `:` or `<`, ends with a space or contains a character outside ASCII 32–126. Before Base64, the value is converted to UTF-8; `Hällo` becomes `SMOkbGxv`, and a character above 255 such as `€` works the same way. `toLdif` does not wrap long lines. Comments from the input are gone after `parse`.

Binary attributes (since 0.3) have bytes as their value, one character per byte. `parse` leaves the decoded bytes as they are, even when they happen to be valid UTF-8, and `toLdif` writes them unchanged as Base64. Up to 0.2 `toLdif` converted them to UTF-8 first, and the Base64 line held other bytes. Binary are `jpegPhoto`, `photo`, `audio`, `thumbnailPhoto`, the certificate and revocation list attributes (`userCertificate`, `cACertificate`, …), `objectGUID`, `objectSid`, every attribute with the option `;binary`, and the names from `-binary` to `parse` and `toLdif`. A character above `\xFF` in a binary value throws

{TCLUTILS TULDIF BINARY}.

```
package require tclutils::tuldif
set ldif "dn: cn=Alice,dc=ex\ncn: Alice\nmail: a@ex.com\n"
append ldif "mail: alice@ex.com\n\ndn: cn=Bob,dc=ex\ncn: Bob\n"
set e [::tclutils::tuldif::parse $ldif]
llength $e                                  ;# -> 2
set alice [lindex $e 0]
::tclutils::tuldif::dn $alice               ;# -> cn=Alice,dc=ex
::tclutils::tuldif::get $alice MAIL         ;# -> a@ex.com alice@ex.com
::tclutils::tuldif::attributes $alice       ;# -> dn cn mail
::tclutils::tuldif::toDict $alice
    ;# -> dn cn=Alice,dc=ex cn Alice mail {a@ex.com alice@ex.com}
set e2 [::tclutils::tuldif::parse "dn: x\ncn:: SGFsbG8=\n"]
::tclutils::tuldif::get [lindex $e2 0] cn ;# -> Hallo
::tclutils::tuldif::toLdif [list {{dn x} {cn { Lager}}}]
                                            ;# -> dn: x\ncn:: IExhZ2Vy\n
::tclutils::tuldif::toLdif [list {{dn x} {cn {Lager }}}]
                                            ;# -> dn: x\ncn:: TGFnZXIg\n
::tclutils::tuldif::toLdif [list {{dn x} {cn Hällo}}]
                                            ;# -> dn: x\ncn:: SMOkbGxv\n
set e3 [::tclutils::tuldif::parse "dn: x\ncn:: SMOkbGxv\n"]
::tclutils::tuldif::get [lindex $e3 0] cn ;# -> Hällo
```

A binary value that `parse` returns as bytes goes through UTF-8 again in `toLdif` and therefore does not come out byte for byte the same. `tuldif` is meant for text values.

A line without a colon throws `{TCLUTILS TULDIF SYNTAX}` with `LDIF line without ':' -> zeile`. `tuldif` reads only content records. A change record throws, at the word `changetype`, the code `{TCLUTILS TULDIF CHANGETYPE}` with `LDIF change records are not supported (changetype: modify)`.

```
set aend "dn: cn=A\nchangetype: modify\nreplace: mail\n"
catch {::tclutils::tuldif::parse $aend} m o
dict get $o -errorcode            ;# -> TCLUTILS TULDIF CHANGETYPE
```

8.7 Bookmarks: tubookmark

tubookmark reads and writes the Netscape bookmark format, the HTML file through which browsers import and export bookmarks. A bookmark is a dict with `title`, `url`, `folder`, `folders`, `tags` and `adddate`. `folders` is the list of folder names from the top, empty for the top level; `folder` is the same joined with `/`, for display. `tags` is a list.

Command	Effect
parse html	flat list of bookmark dicts
serialize lesezeichen ?-title text?	the HTML file as text

Option	Default	Effect
-title	Bookmarks	content of <TITLE> and <H1>

`parse` reads the HTML tag by tag, not line by line. Each `<A …>…</A>` is a bookmark, even if several links stand on one line. Folders result from `<H3>` and the nested `<DL>`. Entities in the title, `HREF`, `TAGS` and `ADD_DATE` are resolved by `tuxml::unescape`. `TAGS` is split at the comma, `ADD_DATE` stays text. Values may stand in double or single quotes or without (`HREF='…'`, since 0.3). If a `<DD>` follows a link, its text goes into the key `description`. If the bookmark lies in the folder with `PERSONAL_TOOLBAR_FOLDER="true"` (the toolbar) or below it, `toolbar` holds the path of that folder as a list. Both keys exist only when the file has them; `serialize` writes them back. Other attributes such as `ICON`, the dates of folders and a `<DD>` after a folder are dropped.

`serialize` keeps the order of the list: a folder stands where its first bookmark is, folders and links stay mixed. It takes the path from `folders`, else it splits `folder` at `/`; a folder name with `/` thus stays one folder. Up to 0.2 `serialize` sorted the folders, put them before the links and made two folders out of `A/B`. Title and folder name are escaped (&, <, >), the attribute values `HREF`, `TAGS` and `ADD_DATE` additionally `"` as `"`. Each line of the header starts in the first column.

```
package require tclutils::tubookmark
set html "<DL><p>\n<DT><A HREF=\"https://tcl.tk/\" TAGS=\"tcl,lang\">"
append html "Tcl &amp; Tk</A>\n<DT><H3>Dev</H3>\n<DL><p>\n"
append html "<DT><A HREF=\"https://core.tcl-lang.org/\">Fossil</A>\n"
append html "</DL><p>\n</DL><p>\n"
set bms [::tclutils::tubookmark::parse $html]
llength $bms                                ;# -> 2
dict get [lindex $bms 0] title              ;# -> Tcl & Tk
dict get [lindex $bms 0] tags               ;# -> tcl lang
dict get [lindex $bms 1] folder             ;# -> Dev
set neu [::tclutils::tubookmark::serialize $bms -title Lager]
llength [::tclutils::tubookmark::parse $neu]
                                            ;# -> 2
set zwei "<DT><A HREF=\"a\">A</A><DT><A HREF=\"b\">B</A>"
llength [::tclutils::tubookmark::parse $zwei]
                                            ;# -> 2
set bm [dict create title T url u folder "" tags {a"b} adddate 1]
set h [::tclutils::tubookmark::serialize [list $bm]]
string match {*TAGS="a&quot;b"*} $h         ;# -> 1
set t [dict get [lindex [::tclutils::tubookmark::parse $h] 0] tags]
lindex $t 0                                 ;# -> a"b
```

There are no module-specific error codes. An unknown option to `serialize` throws `{TCLUTILS COMMON OPTION -name}`.

8.8 What Has No Section of Its Own Here

TDBC and MaxMind are described in the chapter TDBC and MaxMind. ODF and the PDF structure

are described in the chapter ODF and PDF Structure.

The view of the same data in the user interface is in *tkutils* (`tkujson`, `tkuical`). `tupdf` reads the structure, `tupdftext` the text.

8.9 In short

- CSV with quotes: `tucsv`. Cutting at a delimiter: `tucut`.
- `tujson::parse` turns `true` into `1` and `null` into empty. `parseTyped` keeps the kind.
- `tuini::setValue` returns a new dict.
- INI comments are `#` and `;`. `toIni` writes no spaces around the equals sign.
- `tuxml` escapes in `textElement`. `element` inserts the content raw.
- `tusqlite` binds SQL NULL through `null`. `rows` returns the empty string.
- `tusqlite::rows` and `value` bind `:name` from the dict, not from variables.
- CSV for a German Excel: `tucsv::writeFile … -profile excel-de`.
- `tuldif`: entry = list of pairs. Base64 values are UTF-8; change records throw `CHANGETYPE`.
- `tubookmark`: each `<A>` is a bookmark; `folders` holds the folder path as a list.

Next chapter: TDBC and MaxMind

9 TDBC and MaxMind

tutdbc 0.1 works with a TDBC connection that the caller has already opened. The module loads no driver and is not in the umbrella. tummdb 0.1 reads a MaxMind DB of format version 2.0. It is in the umbrella and builds on `tubin`.

9.1 TDBC: tutdbc

```
package require tclutils::tutdbc
::tclutils::tutdbc::insert  verbindung tabelle dict
::tclutils::tutdbc::rows    verbindung sql ?bindungen?
::tclutils::tutdbc::value   verbindung sql ?bindungen? ?vorgabe?
::tclutils::tutdbc::execute verbindung sql ?bindungen?
::tclutils::tutdbc::transaction verbindung skript
::tclutils::tutdbc::quoteId name
::tclutils::tutdbc::null
```

Placeholders in the SQL are written `:name`. The bindings are a dict. `rows` returns a list of dicts. An SQL NULL cell is the empty string; the column stays in the dict. `value` takes the first column of the first row, otherwise the default. `insert` returns the number of affected rows. An empty dict throws `{TCLUTILS TUTDBC EMPTY}`.

`null` is the sentinel for SQL NULL. The same effect occurs when the key is missing from the dict. A stored `0` stays `0`.

```
package require tdbc::sqlite3
package require tclutils::tutdbc
tdbc::sqlite3::connection create db :memory:
db allrows {create table lager (ort text, gang integer)}
```

```
::tclutils::tutdbc::insert db lager {ort Nord gang 3}
                                            ;# -> 1
::tclutils::tutdbc::rows db {select ort, gang from lager}
                                            ;# -> {ort Nord gang 3}
::tclutils::tutdbc::value db {select gang from lager where ort = :o}
    {o x} -
                                            ;# -> -
::tclutils::tutdbc::quoteId {a"b}           ;# -> "a""b"
```

`insert` with `gang` set to `[null]` writes SQL NULL. After `insert db lager [list ort Sued gang [null]]` the row for `Sued` has the key `gang` with an empty value. `gang 0` comes back as `0`.

An error in the script of `transaction` rolls back the changes and rethrows the original error. After one insert and an aborted second insert, `count(*)` stays at 1, the message is `abbruch`, and the `-errorcode` is that of the aborted script.

9.2 MaxMind: tummdb

```
::tclutils::tummdb::open     pfad
::tclutils::tummdb::metadata handle
::tclutils::tummdb::lookup   handle ip
::tclutils::tummdb::close    handle
```

`open` and `close` are not in `namespace export`. The call stays qualified; otherwise it would hit Tcl's `open` and `close`. `open` reads the whole file into memory and returns a handle such as `tummdb1`. `metadata` is the dict from the end of the file. `lookup` returns the record, or the empty string if the address has no record.

A file without the marker `MaxMind.com` throws `{TCLUTILS TUMMDB FORMAT}`. `metadata` and `lookup` with an unknown handle throw `{TCLUTILS TUMMDB HANDLE}`. `close` on such a handle returns the empty string and does not throw.

Further codes from the module: `RECORDSIZE` if the node width is not supported, and `TYPE` if a data field has an unknown kind.

9.3 In short

- `tutdbc` loads no driver. The caller opens the connection.
- SQL NULL stays a column with an empty value. `0` stays `0`.
- `transaction` rolls back on error and passes the error on.
- Always write `tummdb::open` and `close` with the namespace.

Next chapter: Encoding, Hashing and Comparing

10 Encoding, Hashing and Comparing

Base64 and Base32, CRC, the digests and xxHash32, changing the encoding, UUIDs, query strings, nested dicts, a line comparison and applying a diff. Everything is in the umbrella. CRC uses `zlib` from Tcl itself; the digests are pure Tcl and need no tcllib.

10.1 Base64: tubase64

`encode` and `decode` go through `binary encode/decode base64`. `encodeFile` reads the file in binary mode, `decodeFile` reads the text.

Option	Default	Effect
-maxlen	0	line break after this many characters; 0 keeps the line whole
-encoding	binary	encode: binary takes each character as one byte, utf-8 converts text first; decode: utf-8 turns the bytes back into text

```
package require tclutils::tubase64
::tclutils::tubase64::encode hallo          ;# -> aGFsbG8=
::tclutils::tubase64::decode aGFsbG8=       ;# -> hallo
::tclutils::tubase64::encode hallohallo -maxlen 4
                                          ;# -> aGFs\nbG9o\nYWxs\nbw==
```

`-maxlen -1` throws `{TCLUTILS TUBASE64 MAXLEN}`. A text without valid Base64, such as `!!!`, comes back as an empty string, without an error code.

Without the option `encode` takes bytes. A character above 255 throws `{TCLUTILS TUBASE64 BYTES}` on Tcl 8.6 as on 9.0; up to tclutils 0.68.0 Tcl 8.6 silently kept only the low byte. Text goes in with `-encoding utf-8`:

```
::tclutils::tubase64::encode "Jörg €" -encoding utf-8   ;# ->
    SsO2cmcg4oKs
::tclutils::tubase64::decode SsO2cmcg4oKs -encoding utf-8 ;# -> Jörg €
::tclutils::tubase64::encode Jörg                        ;# -> SvZyZw==
```

The last example encodes `ö` as the single byte `f6`. `tuhash` and `tuxxhash`, in contrast, convert text to UTF-8 on their own; they are mostly used on text, `tubase64`, `tubase32` and `tucrc` mostly on file and image data.

10.2 Base32: tubase32

tubase32 encodes according to RFC 4648, optionally with the base32hex alphabet. Tcl itself provides only Base64; the implementation is pure Tcl. `encodeFile` reads the file in binary mode, `decodeFile` reads the text.

Command	Effect
encode daten ?-pad b? ?-hex b?	Base32 text in uppercase
decode text ?-hex b?	the bytes
encodeFile pfad ?-pad b? ?-hex b?	the bytes of the file, encoded
decodeFile pfad ?-hex b?	the text of the file, decoded

Option	Default	Effect
-pad	1	pads with = to a full group of eight characters; 0 omits it
-hex	0	1 uses the alphabet 0–9, A–V (base32hex, RFC 4648 §7)

`decode` skips `=`, space, tab and newline and accepts lowercase as well as uppercase letters. `decode` checks the end as in RFC 4648 §3.5: the number of characters must be one that `encode` produces (a multiple of 8 plus 0, 2, 4, 5 or 7), and the bits after the last full byte must be 0. `decode MY` gives `f`, `decode MZ` is an error.

```
package require tclutils::tubase32
::tclutils::tubase32::encode foobar        ;# -> MZXW6YTBOI======
::tclutils::tubase32::encode foobar -pad 0 ;# -> MZXW6YTBOI
::tclutils::tubase32::encode foobar -hex 1 ;# -> CPNMUOJ1E8======
::tclutils::tubase32::decode mzxw6ytboi    ;# -> foobar
::tclutils::tubase32::encode [encoding convertto utf-8 Größe]
                                           ;# -> I5ZMHNWDT5SQ====
::tclutils::tubase32::decode MY            ;# -> f
catch {::tclutils::tubase32::decode MZ} m o
dict get $o -errorcode                     ;# -> TCLUTILS TUBASE32 BITS
```

The input of `encode` is bytes. A character above 255 throws `{TCL VALUE BYTES}` under Tcl 9.0; under Tcl 8.6 only the lower byte is kept, silently. Text must therefore go through `encoding convertto utf-8` first.

Unlike `tubase64`, `tubase32` does not return an empty string for invalid text but throws an error. A character outside the alphabet throws `{TCLUTILS TUBASE32 CHAR}`, for example `invalid base32 character: "1"`. An impossible length such as `MZX` throws `{TCLUTILS TUBASE32 LENGTH}`, nonzero bits at the end as with `MZ` `{TCLUTILS TUBASE32 BITS}`. An invalid boolean word for `-pad` or `-hex` throws `{TCLUTILS COMMON BOOLEAN -pad}` or `-hex` respectively.

10.3 CRC: tucrc

`crc32` and `adler32` pass the bytes on to `zlib`. A second argument is the start value for a continued checksum. After it, `-encoding binary|utf-8` may follow, as with `tubase64`: without the option a character above 255 is `{TCLUTILS TUCRC BYTES}`, with `utf-8` the checksum equals that of `crc32File` on a UTF-8 file with this text. `hex32` turns it into eight hex characters in uppercase. `crc32File` and `adler32File` read the file in binary mode.

```
package require tclutils::tucrc
::tclutils::tucrc::crc32 hallo              ;# -> 3111268817
::tclutils::tucrc::hex32 [::tclutils::tucrc::crc32 hallo]
                                            ;# -> B97231D1
::tclutils::tucrc::adler32 hallo            ;# -> 102498833
```

A wrong argument reports `wrong # args` with `{TCLUTILS TUCRC OPTION}`, an unknown encoding `{TCLUTILS TUCRC ENCODING}`.

10.4 Digests: tuhash

`sha256`, `sha1` and `md5` are separate implementations of FIPS 180-4 and RFC 1321. The file commands hash the bytes of the file. For text, `-encoding` applies:

Value	Effect
utf-8	default; the text is first converted to UTF-8
binary	each character is one byte 0–255

```
package require tclutils::tuhash
::tclutils::tuhash::sha256 ""                ;# ->
    e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
::tclutils::tuhash::sha256 abc               ;# ->
    ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
::tclutils::tuhash::md5 ""                   ;# ->
    d41d8cd98f00b204e9800998ecf8427e
::tclutils::tuhash::sha1 ""                  ;# ->
    da39a3ee5e6b4b0d3255bfef95601890afd80709
```

Another encoding throws `{TCLUTILS TUHASH ENCODING}`. The same empty digests are the test vectors of the standards.

10.5 xxHash32: tuxxhash

tuxxhash computes XXH32, a fast hash without cryptographic claims, in pure Tcl. It serves to detect changes and to pre-sort possible duplicates, not to protect against forgery. There are no 64-bit or 128-bit variants.

Command	Effect
xxh32 daten ?seed? ?-encoding e?	eight hex characters in lowercase
xxh32file pfad ?seed?	the same for the bytes of the whole file

`seed` is an integer, default `0`. Only its lower 32 bits count. As with `tuhash`, `-encoding` applies to text; it comes after `seed`:

Value	Effect
utf-8	default; the text is first converted to UTF-8
binary	each character is one byte 0–255

```
package require tclutils::tuxxhash
::tclutils::tuxxhash::xxh32 ""               ;# -> 02cc5d05
::tclutils::tuxxhash::xxh32 hello            ;# -> fb0077f9
::tclutils::tuxxhash::xxh32 hello 1          ;# -> fcfffba9
::tclutils::tuxxhash::xxh32 Größe            ;# -> 3a44ca22
set b [encoding convertto utf-8 Größe]
::tclutils::tuxxhash::xxh32 $b 0 -encoding binary
                                            ;# -> 3a44ca22
```

xxh32 Größe hashes the seven UTF-8 bytes and matches xxh32file of a UTF-8 file with this text. A character above 255 such as € works the same way. -encoding binary takes the characters as bytes, for example for data from binary format or from a channel with -translation binary.

A seed that is not an integer throws {TCLUTILS TUXXHASH SEED}. Another value for -encoding throws {TCLUTILS TUXXHASH ENCODING}, a character above 255 with -encoding binary {TCLUTILS TUXXHASH BYTES}, an argument other than -encoding {TCLUTILS TUXXHASH OPTION}. A file that cannot be opened throws {TCLUTILS TUXXHASH IO}. With 32 bits, random collisions occur in large collections; equal values are only candidates for a comparison of the bytes or with sha256.

10.6 Changing the Encoding: tuiconv

tuiconv converts between the encodings Tcl knows, and reads and writes files with an explicit encoding. The channels are binary; the conversion is done with encoding convertfrom and encoding convertto.

Command	Effect
encodings	the names from encoding names, sorted
convert bytes von nach ?-replace b?	bytes in von to bytes in nach
readFile pfad kodierung ?-replace b?	the text of the file
writeFile pfad text kodierung ?-replace b?	writes the file, returns pfad
convertFile ein aus von nach ?-replace b?	reads ein, writes aus, returns aus

A name must appear exactly in encoding names, such as utf-8, iso8859-1 or cp1252. Spellings such as utf8 or latin1 throw {TCLUTILS TUICONV ENCODING name} with unknown encoding "utf8". The list is longer under Tcl 9.0 than under 8.6.

```
package require tclutils::tuiconv
set utf8 [encoding convertto utf-8 Größe]
set cp [::tclutils::tuiconv::convert $utf8 utf-8 cp1252]
binary encode hex $cp                       ;# -> 4772f6df65
set zurueck [::tclutils::tuiconv::convert $cp cp1252 utf-8]
encoding convertfrom utf-8 $zurueck         ;# -> Größe
```

Since 0.2 the conversion is equally strict under Tcl 8.6 and 9.0. A character the target encoding does not know, for example ł to cp1252, throws {TCLUTILS TUICONV UNMAPPABLE cp1252}; bytes that are invalid in the source encoding throw {TCLUTILS TUICONV INVALID name}. -replace 1 on all commands substitutes instead, with? when writing. writeFile converts before it opens the file: on an error the file keeps its content. Up to 0.1 Tcl 8.6 silently inserted?, Tcl 9.0 threw a Tcl error without its own code.

```
catch {::tclutils::tuiconv::convert [encoding convertto utf-8 ł] utf-8
    cp1252} m
set m             ;# -> character U+0142 at index 0 cannot be written
    in cp1252
```

```
binary encode hex [::tclutils::tuiconv::convert \
    [encoding convertto utf-8 ł] utf-8 cp1252 -replace 1]
                                          ;# -> 3f
```

10.7 UUID: tuuuid

v4 is random, v7 carries the Unix time in milliseconds in the first 48 bits and random data after that (RFC 9562). `generate` takes `-version 4` or `7`, default `4`. `nil` is the nil UUID. `validate` checks the form `8-4-4-4-12`, upper and lower case. `version` reads the digit at position 14.

```
package require tclutils::tuuuid
::tclutils::tuuuid::nil                    ;# ->
    00000000-0000-0000-0000-000000000000
::tclutils::tuuuid::validate [::tclutils::tuuuid::nil]
                                          ;# -> 1
::tclutils::tuuuid::version [::tclutils::tuuuid::nil]
                                          ;# -> 0
::tclutils::tuuuid::version [::tclutils::tuuuid::v4]
                                          ;# -> 4
::tclutils::tuuuid::version [::tclutils::tuuuid::v7]
                                          ;# -> 7
```

`version nein` throws `{TCLUTILS TUUUID INVALID}`. `generate -version 9` throws `{TCLUTILS TUUUID VERSION}`. The random bytes come from `/dev/urandom`, otherwise from `rand()`.

10.8 Query Strings: tuurl

Letters, digits and `-._~` stay unreserved. Everything else is percent-encoded with `%` over the UTF-8 bytes.

Command	-plus	Effect
encode, decode	0	space becomes %20; + stays +
buildQuery, parseQuery	1	space becomes +, and + becomes a space again when reading

```
package require tclutils::tuurl
::tclutils::tuurl::encode "a b"             ;# -> a%20b
::tclutils::tuurl::encode "a b" -plus 1     ;# -> a+b
::tclutils::tuurl::encode Größe             ;# -> Gr%C3%B6%C3%9Fe
::tclutils::tuurl::decode %C3%BC            ;# -> ü
::tclutils::tuurl::buildQuery {ort {Nord Lager} gang 3}
                                          ;# -> ort=Nord+Lager&gang=3
::tclutils::tuurl::parseQuery {ort=Nord+Lager&gang=3}
                                          ;# -> ort {Nord Lager} gang 3
```

A key without `=` gets the empty value. If a key occurs more than once in the query (a=1&a=2,

for example with checkboxes), `parseQuery` keeps only the last value; if every value counts, split the query at `&` yourself. An invalid boolean word for `-plus` throws `{TCLUTILS COMMON BOOLEAN -plus}`.

10.9 Nested Dicts: tudict

The core already does `dict get $d a b`. tudict adds the path with a default, the leaf paths, a flat form, deep merging and inverting.

Command	Effect
getOr dict vorgabe schlüssel …	the value, or the default if a key is missing
paths dict	leaf paths, each path a list of keys
flatten dict ?sep?	a flat dict, keys joined with .
mergeDeep dict …	from left to right; nested dicts merge, scalars overwrite
invert dict	swaps key and value; for a duplicate value the last one wins

The default comes before the path.

```
package require tclutils::tudict
::tclutils::tudict::getOr {ui {theme dark}} - ui theme
                                            ;# -> dark
::tclutils::tudict::getOr {ui {theme dark}} - ui fehlt
                                            ;# -> -
::tclutils::tudict::paths {a {b {c 1}} d 2}
                                            ;# -> {a b c} d
::tclutils::tudict::flatten {a {b {c 1}} d 2}
                                            ;# -> a.b.c 1 d 2
::tclutils::tudict::invert {a 1 b 2}        ;# -> 1 a 2 b
::tclutils::tudict::invert {a 1 b 1}        ;# -> 1 b
```

Without a key, `getOr` throws `{TCLUTILS TUDICT ARG}`. A value that looks like a dict becomes one: `flatten` of `{name {a b}}` gives `name.a b`, because `a b` has an even length. `mergeDeep` is shown with an example in the appendix Practice.

10.10 Line Comparison: tudiff

`text` returns a list of `equal`, `delete` and `insert`, each with the line. `unifiedText` builds a unified diff from it, `contextText` the context form. `files`, `unified` and `context` read two files. `directory` compares two trees.

Option	Default	Effect
-maxcells	2000000	upper limit for the table, old lines times new lines

Option	Default	Effect
-fromlabel, -tolabel	old, new	header lines of unifiedText and contextText

```
package require tclutils::tudiff
::tclutils::tudiff::text "a\nb" "a\nc"      ;# -> {equal a} {delete b}
    {insert c}
::tclutils::tudiff::unifiedText "a\nb" "a\nc"
                                            ;# -> --- old\n+++ new\n@@
    -1,2 +1,2 @@\n a\n-b\n+c
```

`-maxcells 0` throws `{TCLUTILS TUDIFF OPTION MAXCELLS}`. Two texts with two lines each and `-maxcells 1` throw `{TCLUTILS TUDIFF TOO_LARGE}`, because 2 × 2 exceeds the limit.

By default line ends do not count: CRLF, CR and LF are the same, and so is a missing final newline (`-eol ignore`). `-eol strict` (since 0.2) compares each line with its end. The operations then carry the end, and `unifiedText` marks a missing newline as GNU diff does. `files`, `unified` and `context` also take `-encoding`.

```
::tclutils::tudiff::text "a\r\nb" "a\nb\n"              ;# -> {equal
    a} {equal b}
::tclutils::tudiff::unifiedText "a\nb\n" "a\nb" -eol strict
;# -> --- old\n+++ new\n@@ -1,2 +1,2 @@\n a\n-b\n+b\n\\ No newline at
    end of file
```

10.11 Applying a Diff: tupatch

tupatch is the counterpart of `tudiff`: it applies a unified diff to a text. That can be the output of `unifiedText`, but also `diff -u` or `git diff` with several hunks.

Command	Effect
text quelle patch ?optionen?	the changed text
fromFile pfad patch ?optionen?	like text, with the source from the file
parse patch	list of the hunks as dicts

Option	Default	Effect
-reverse	0	1 applies the diff in reverse and so undoes it
-strict	1	checks context and deletion lines against the source; 0 follows the diff blindly

A hunk from `parse` has `oldStart`, `oldLen`, `newStart`, `newLen`, `body`, `oldNoNl` and newNoNl. `body` is a list of pairs of kind (`, +, -`) and line. Header lines such as `---`, `+++`, `diff` and `index` are dropped. `oldNoNl` and `newNoNl` are 1 when `\ No newline at end of file` follows the last old or new line. An empty line ends a hunk.

Each hunk applies at the line from its header. `tupatch` does not search for an offset. The lines are joined with `\n`. If the source ends with `\n`, so does the result; the marker `\ No newline`

`at end of file` works as with `diff -u` (since 0.3; up to 0.2 the last `\n` was always missing from the result). If the diff has no hunk, the source comes back unchanged.

```
package require tclutils::tudiff
package require tclutils::tupatch
set alt "alpha\nbeta\ngamma"
set neu "alpha\nBETA\ngamma"
set p [::tclutils::tudiff::unifiedText $alt $neu]
::tclutils::tupatch::text $alt $p           ;# -> alpha\nBETA\ngamma
::tclutils::tupatch::text $neu $p -reverse 1
                                          ;# -> alpha\nbeta\ngamma
llength [::tclutils::tupatch::parse $p]     ;# -> 1
```

If a line does not match, `text` throws `{TCLUTILS TUPATCH CONTEXT}`, for example with the source `alpha\nbeta\nGAMMA`: `patch does not apply: expected "gamma" but found "GAMMA" at source line 3`. A hunk beyond the end of the source, or one that overlaps an earlier one, throws `{TCLUTILS TUPATCH RANGE}`. An unknown option throws `{TCLUTILS COMMON OPTION -name}`. `-reverse` and `-strict` accept any boolean value (`1`, `yes`, `true`, `on` and their counterparts); `-strict foo` throws `{TCLUTILS COMMON BOOLEAN -strict}`.

10.12 In short

- Base64 without line breaks: `-maxlen 0`. Invalid text becomes empty.
- Digests of text are over UTF-8. Files hash the bytes.
- `tuurl`: `-plus` is off for a single value, on for the query.
- `getOr`: first the dict, then the default, then the keys.
- `tudiff` keeps the whole comparison table in memory.
- Base32: `tubase32`, the input is bytes; `-hex 1` for base32hex. Invalid text throws an error.
- xxh32 hashes text as UTF-8 like `tuhash`; `-encoding binary` for bytes.
- `tuiconv` knows only the names from `encoding names`. Characters it cannot represent and invalid bytes are an error; `-replace 1` substitutes.
- `tupatch` applies without offset search. `-reverse 1` undoes the diff.

Next chapter: Binary Data and File Information

11 Binary Data and File Information

The modules in this chapter display bytes, search and change them in a file, compare two files, and read size and metadata. `tucode` adds the character tables for ASCII and Latin-1. All modules are pure Tcl.

Group	Modules
Viewing	tuhexdump, tuod, tustrings
Changing	tuhexedit
Comparing	tucmp

Group	Modules
Size and metadata	tusize, tustat
Character tables	tucode

Commands that take data as an argument expect bytes: a Tcl string in which every character has a value from 0 to 255. `read` on a channel with `-translation binary` returns such a string, and the `file` variants of the modules read their file the same way. A text with characters above 255, such as `€`, is not a byte string. `tuhexdump`, `tuod`, `tustrings`, `tucmp` and `tuhexedit::writeBytes` check this up front and throw `{TCLUTILS <MODUL> BYTES}` under both Tcl 8.6 and 9, for example `{TCLUTILS TUOD BYTES}`. Convert text to bytes first with `encoding convertto utf-8`.

```
package require tclutils::tuod
::tclutils::tuod::bytes [encoding convertto utf-8 €]
                                          ;# -> 226 130 172
catch {::tclutils::tuod::bytes €} msg opts ;# -> 1
dict get $opts -errorcode         ;# -> TCLUTILS TUOD BYTES
```

Checksums, hashes, Base64 and line comparison are covered in the chapter Encoding, Hashing and Comparing, file type detection with `tufile` in the chapter Text and Files.

11.1 Hexdump: tuhexdump

tuhexdump prints bytes as lines with an address, hex values and an ASCII column, in the form of `hexdump -C`.

```
::tclutils::tuhexdump::data bytes ?option wert ...?
::tclutils::tuhexdump::file pfad  ?option wert ...?
```

Option	Default	Effect
-width	16	bytes per line, integer of at least 1
-offset	0	first byte; the address counts from here
-length	-1	number of bytes; negative means up to the end

The return value is a string with lines separated by `\n`, without a trailing `\n`. Each line has the address with eight hex digits, the bytes in lowercase, and between `|` the characters 32 to 126; every other byte appears there as `.`. The last line is padded with spaces to the full width. Empty data yields an empty string.

Differences from `hexdump -C` and `xxd`: no gap after the eighth byte, no folding of identical lines into `*`, no final line with the end address.

```
package require tclutils::tuhexdump
set daten "Hallo, Welt!\n\x00\x01\xff"
puts [::tclutils::tuhexdump::data $daten -width 8]
```

```
00000000  48 61 6c 6c 6f 2c 20 57  |Hallo, W|
00000008  65 6c 74 21 0a 00 01 ff  |elt!....|
```

```
::tclutils::tuhexdump::data "Hallo, Welt!" -offset 7 -length 4 -width 4
                                ;# -> 00000007  57 65 6c 74  |Welt|
```

At the default width a line is 77 characters long.

Error	-errorcode
-width not an integer of at least 1	{TCLUTILS COMMON INTEGER -width}
-offset negative or not an integer	{TCLUTILS TUHEXDUMP OFFSET}
-length not an integer	{TCLUTILS TUHEXDUMP LENGTH}
characters above 255 in the data	{TCLUTILS TUHEXDUMP BYTES}
unknown option, option without value	{TCLUTILS COMMON OPTION -name}

11.2 Octal, Decimal, Characters: tuod

tuod prints bytes one by one as octal, hex or decimal numbers or as characters. `bytes` returns the byte values as an unsigned list; `binary scan` with `c` alone would give the value `-1` for `\xff`.

```
::tclutils::tuod::bytes bytes ?option wert ...?
::tclutils::tuod::data  bytes ?option wert ...?
::tclutils::tuod::file  pfad  ?option wert ...?
```

Option	Default	Effect
-format	octal	octal (%03o), hex (%02x), decimal (%03d), char
-width	16	bytes per line, at least 1
-offset	0	first byte
-length	-1	number of bytes; negative means up to the end
-address	hex	hex (8 digits), octal (7), decimal (8), none

`-format char` shows the characters 32 to 126 and `.` otherwise. Two spaces separate the address from the values, one space separates the values. `bytes` evaluates only `-offset` and `-length` and accepts the other options without effect.

Differences from `od`: Unix `od` without options shows two-byte words in octal and the address in octal. `tuod` shows each byte on its own and the address in hex; `-address octal` gives the address as `od` does. Instead of `\n` and `\0` as in `od -c`, `.` appears. Identical lines are not folded into `*`, and there is no final line with the end address.

```
package require tclutils::tuod
::tclutils::tuod::bytes "A\x00\xff"       ;# -> 65 0 255
::tclutils::tuod::data ABC                ;# -> 00000000  101 102 103
::tclutils::tuod::data "Hallo\n" -format hex -address none
                                          ;# -> 48 61 6c 6c 6f 0a
::tclutils::tuod::data abcdef -format char -offset 2 -length 3
```

```
                                          ;# -> 00000002  c d e
set daten "Hallo\n\x00\xff"
puts [::tclutils::tuod::data $daten -format char \
    -address octal -width 4]
```

```
0000000  H a l l
0000004  o . . .
```

Error	-errorcode
unknown -format	{TCLUTILS TUOD FORMAT wert}
unknown -address	{TCLUTILS TUOD ADDRESS wert}
-width not an integer of at least 1	{TCLUTILS TUOD WIDTH}
-offset negative or not an integer	{TCLUTILS TUOD OFFSET}
-length not an integer	{TCLUTILS TUOD LENGTH}
characters above 255 in the data	{TCLUTILS TUOD BYTES}
unknown option, option without value	{TCLUTILS COMMON OPTION -name}

`data` and `file` check `-format` and `-address` before the first byte, so a wrong value is caught even with empty data.

11.3 Readable Strings: tustrings

tustrings searches binary data for runs of printable ASCII characters, like Unix `strings`.

```
::tclutils::tustrings::extract bytes ?-minlength n?
::tclutils::tustrings::file    pfad  ?-minlength n?
```

Option	Default	Effect
-minlength	4	shorter runs are discarded; integer of at least 1

Bytes 32 to 126 are printable. Every other byte ends a run, including tab and end of line. The return value is a Tcl list of the runs in order of occurrence, without positions.

Differences from `strings`: GNU `strings` counts the tab as part of the string, `tustrings` splits there. There is no position output like `strings -t`. Umlauts are not printable ASCII characters, in Latin-1 either, and also split.

```
package require tclutils::tustrings
set bin "\x00\x01%PDF-1.7\x00ab\x00Hallo Welt\n\xffKey\tWert"
::tclutils::tustrings::extract $bin  ;# -> %PDF-1.7 {Hallo Welt} Wert
::tclutils::tustrings::extract $bin -minlength 2
                           ;# -> %PDF-1.7 ab {Hallo Welt} Key Wert
::tclutils::tustrings::extract "Größe 12"  ;# -> {e 12}
```

There is no `-encoding` option; the printable bytes are always 32 to 126.

Error	-errorcode
-minlength not an integer of at least 1	{TCLUTILS COMMON INTEGER -minlength}
characters above 255 in the data	{TCLUTILS TUSTRINGS BYTES}
unknown option, option without value	{TCLUTILS COMMON OPTION -name}

11.4 Reading, Searching, Patching: tuhexedit

tuhexedit reads bytes at a position, searches for byte and text patterns, and overwrites bytes in the file. It is a library, not an editor with a user interface. Unix has no single tool for this; overwriting at a fixed position corresponds to `dd ... conv=notrunc`.

```
::tclutils::tuhexedit::readBytes  pfad offset länge
::tclutils::tuhexedit::writeBytes pfad offset bytes ?sicherung?
::tclutils::tuhexedit::patch      pfad offset hex ?-backup 1?
::tclutils::tuhexedit::backup     pfad ?zielname?
::tclutils::tuhexedit::findHex    pfad hex
::tclutils::tuhexedit::findString pfad text ?kodierung?
::tclutils::tuhexedit::dump       pfad ?option wert ...?
```

Command	Return value
readBytes	the bytes; fewer at the end of the file, empty past the end
writeBytes, patch	the path, as passed
backup	the name of the copy; default pfad.bak
findHex, findString	list of offsets from 0, overlapping matches included
dump	hexdump like tuhexdump::file, with its options

An offset is an integer from 0 or hex with `0x`, such as `0x100`; `0x` without a valid hex digit is an error. Hex patterns for `findHex` and `patch` may contain spaces, `:`, `-`, and `0x` before each byte; `{50 4B 03 04}`, `504b0304` and `0x50,0x4B,0x03,0x04` are the same pattern. `findString` converts the text to bytes with the encoding before searching, default `utf-8`. An ä in a Latin-1 file is therefore found only by `findString pfad ä iso8859-1`. A character the encoding cannot represent is an error.

`writeBytes` and `patch` first create a copy `pfad.bak` and overwrite an existing copy. `writeBytes` with `0` as the fourth argument and `patch` with `-backup 0` skip the copy. The file is overwritten in place; nothing is inserted or removed, and the file length stays the same. If the bytes from the offset reach past the end of the file, the command throws an error before it creates the copy; file and copy then stay unchanged. `findHex`, `findString` and `patch` read the whole file, `readBytes` only the requested part.

```
package require tclutils::tuhexedit
set fh [open probe.bin wb]
puts -nonewline $fh ABCDEFGH
close $fh
```

```
::tclutils::tuhexedit::readBytes probe.bin 2 3    ;# -> CDE
::tclutils::tuhexedit::findString probe.bin CD    ;# -> 2
::tclutils::tuhexedit::findHex probe.bin {47 48}  ;# -> 6
::tclutils::tuhexedit::patch probe.bin 0x2 {58 59}
                                                  ;# -> probe.bin
::tclutils::tuhexedit::readBytes probe.bin 0 8    ;# -> ABXYEFGH
::tclutils::tuhexedit::readBytes probe.bin.bak 0 8
                                                  ;# -> ABCDEFGH
::tclutils::tuhexedit::dump probe.bin -width 8
              ;# -> 00000000  41 42 58 59 45 46 47 48  |ABXYEFGH|
catch {::tclutils::tuhexedit::patch probe.bin 7 {58 59}} msg opts
                                                  ;# -> 1
dict get $opts -errorcode     ;# -> TCLUTILS TUHEXEDIT RANGE
```

Error	-errorcode
offset negative, not a number, or 0x without a hex digit	{TCLUTILS TUHEXEDIT OFFSET}
length negative or not a number	{TCLUTILS TUHEXEDIT LENGTH}
writing past the end of the file	{TCLUTILS TUHEXEDIT RANGE}
writeBytes with characters above 255	{TCLUTILS TUHEXEDIT BYTES}
character not in the encoding (findString)	{TCLUTILS TUHEXEDIT ENCODING}
empty search pattern	{TCLUTILS TUHEXEDIT NEEDLE}
hex with an odd number of digits	{TCLUTILS TUBIN HEX ODD}
hex with other characters	{TCLUTILS TUBIN HEX BADCHAR}
unknown option to patch	{TCLUTILS COMMON OPTION -name}
unknown encoding	{TCL LOOKUP ENCODING name}
file missing	{POSIX ENOENT ...}

`writeBytes` does not create a new file; a missing file throws `{POSIX ENOENT ...}`. All checks run before writing, so a rejected call changes nothing.

11.5 Comparing Byte by Byte: tucmp

tucmp compares two byte strings or two files and reports the first difference, like Unix `cmp`.

```
::tclutils::tucmp::data            bytes1 bytes2
::tclutils::tucmp::files           pfad1 pfad2
::tclutils::tucmp::equal           pfad1 pfad2
::tclutils::tucmp::firstDifference pfad1 pfad2
```

`data` and `files` return a dict:

Key	Content
equal	1 for identical data, 0 otherwise
offset	first differing position from 0; -1 for identical data
byte1, byte2	byte value 0–255 at offset; empty where the data has ended
size1, size2	number of bytes of both data

`equal` returns only `equal`, `firstDifference` only `offset`. If one side is the beginning of the other, `offset` is set to the length of the shorter side and its byte is empty.

Differences from `cmp`: the offset counts from 0, `cmp` counts byte and line from 1. There is no line number, and the result is a dict instead of an exit status. `files` reads both files completely into memory.

```
package require tclutils::tucmp
dict get [::tclutils::tucmp::data abc abc] equal    ;# -> 1
dict get [::tclutils::tucmp::data abc abc] offset   ;# -> -1
::tclutils::tucmp::data abc abx
::tclutils::tucmp::data abc abcd
```

```
equal 0 offset 2 byte1 99 byte2 120 size1 3 size2 3
equal 0 offset 3 byte1 {} byte2 100 size1 3 size2 4
```

`data` compares bytes. A character above 255 is an error, even if both strings are equal; convert text first with `encoding convertto`.

```
set cmp [::tclutils::tucmp::data [encoding convertto utf-8 a€] \
    [encoding convertto utf-8 aä]]
dict get $cmp byte1                         ;# -> 226
dict get $cmp size1                         ;# -> 4
```

Error	-errorcode
characters above 255 in data	{TCLUTILS TUCMP BYTES}
file missing	{POSIX ENOENT ...}

11.6 Sizes: tusize

tusize adds up file sizes below a directory and writes byte counts in readable form, a small `du`.

```
::tclutils::tusize::bytes   pfad
::tclutils::tusize::entries verzeichnis
::tclutils::tusize::human   n ?-si 0|1?
```

Command	Return value
bytes	size of the file, or the sum of all files below it
entries	list {bytes pfad} per direct entry, sorted by path

Command	Return value
human	number with unit

Option human	Default	Effect
-si	0	0: base 1024, B KiB MiB GiB TiB PiB EiB ZiB YiB; 1: base 1000, B kB MB GB TB PB EB ZB YB

`bytes` includes dot files. It counts the file size, not the space used on disk, and the directories themselves do not count. Below the base, `human` prints the integer with `B`, otherwise the value with one decimal place. It does not go beyond `YiB` (YB); larger values stay in that unit.

The unit always names its base: `KiB` is 1024, `kB` is 1000. 1500 and 1536 bytes are both `1.5 KiB` because only one decimal is shown. `tunumfmt::toHuman` instead writes the bare `K` for 1000 like `numfmt` (`1.5K`); for bytes use `-to iec` there (`1.5Ki`).

`bytes` does not descend into a symbolic link and does not follow it. It counts the link itself with `file lstat`; under Unix its size is the length of the target path. This applies to links to files, links to directories and links without a target. A circular link therefore does not lead to a loop.

Differences from `du`: without options `du` counts used blocks and the directories themselves. `tusize::bytes` is closer to `du -sb`, without the directory entries.

```
package require tclutils::tusize
file mkdir projekt/sub
foreach {pfad n} {projekt/a.txt 100 projekt/sub/b.bin 3000} {
    set fh [open $pfad wb]
    puts -nonewline $fh [string repeat x $n]
    close $fh
}
::tclutils::tusize::bytes projekt        ;# -> 3100
::tclutils::tusize::entries projekt
                          ;# -> {100 projekt/a.txt} {3000 projekt/sub}
::tclutils::tusize::human 500            ;# -> 500 B
::tclutils::tusize::human 1536           ;# -> 1.5 KiB
::tclutils::tusize::human 1500 -si 1     ;# -> 1.5 kB
::tclutils::tusize::human 5368709120     ;# -> 5.0 GiB
::tclutils::tusize::human [expr {2**70}] ;# -> 1.0 ZiB
```

On a real tree the sum depends on the content; the result of `entries` then has the same form, a list of pairs of number and path.

Error	-errorcode
path missing (bytes)	{TCLUTILS TUSIZE NOTFOUND}
not a directory or missing (entries)	{TCLUTILS TUSIZE NOTDIR}
n not an integer (human)	{TCLUTILS TUSIZE VALUE}
-si not a boolean	{TCLUTILS COMMON BOOLEAN -si}
unknown option	{TCLUTILS COMMON OPTION -name}

11.7 Metadata: tustat

tustat returns the values of `file stat` as a dict and formats a selection of them as text, similar to Unix `stat`.

```
::tclutils::tustat::file   pfad
::tclutils::tustat::files  pfade
::tclutils::tustat::render dict ?-timeformat format?
```

`file` returns a dict with the keys from `file stat` (`atime`, `blksize`, `blocks`, `ctime`, `dev`, `gid`, `ino`, `mode`, `mtime`, `nlink`, `size`, `uid`) plus `path` (as passed), `normalized` (result of `file normalize`), `type` (result of `file type`: `file`, `directory`, `link` and so on) and `modeOctal` (`mode` in the familiar octal form, such as `100644`). For a link, `target` is added, the result of `file readlink`. `files` returns a list of such dicts in the order of the paths.

Option render	Default	Effect
-timeformat	%Y-%m-%d %H:%M:%S	format for clock format, local time

`render` prints the fields `path`, `type`, `size`, `mtime`, `mode`, `ino`, `dev`, `uid` and `gid` in this order, one per line, with the name padded to ten characters followed by the value. If a field is missing from the dict, its line is missing. `render` prints no other fields.

`file` does not follow a link, like Unix `stat` without `-L`: size, permissions and times come from `file lstat` and belong to the link itself. A link without a target is therefore not an error. `mode` is a decimal number with file type and permissions; `render` prints it as such.

```
package require tclutils::tustat
set d [::tclutils::tustat::file projekt/a.txt]
dict get $d size                          ;# -> 100
dict get $d type                          ;# -> file
dict get $d path                          ;# -> projekt/a.txt
llength [::tclutils::tustat::files {projekt/a.txt projekt}]
                                          ;# -> 2
file link -symbolic projekt/l.txt a.txt
set l [::tclutils::tustat::file projekt/l.txt]
dict get $l type                          ;# -> link
dict get $l target                        ;# -> a.txt
puts [::tclutils::tustat::render $d]
```

The output of `render` has this form; time, permissions and numbers depend on the file and the machine:

```
path       projekt/a.txt
type       file
size       100
mtime      JJJJ-MM-TT hh:mm:ss
mode       Dezimalzahl
ino        Dezimalzahl
dev        Dezimalzahl
uid        Dezimalzahl
gid        Dezimalzahl
```

`file` and `files` have no error codes of their own; a missing path throws `{POSIX ENOENT ...}` from `file type`. An unknown option to `render` throws `{TCLUTILS COMMON OPTION -name}`.

11.8 Character Tables: tucode

tucode prints tables of the characters 0 to 255, ASCII and Latin-1, plus selected character groups, and looks up single characters. The Unix counterparts are the manual pages `ascii(7)` and `iso_8859-1(7)`.

```
::tclutils::tucode::lookup     wert
::tclutils::tucode::lookupChar zeichen
::tclutils::tucode::table      von bis ?option wert ...?
::tclutils::tucode::render     von bis ?option wert ...?
::tclutils::tucode::ascii      ?option wert ...?
::tclutils::tucode::latin1     ?option wert ...?
::tclutils::tucode::ansi       ?option wert ...?
::tclutils::tucode::signs      ?gruppe ...? ?option wert ...?
::tclutils::tucode::groups
```

`lookup` takes a decimal number, hex with `0x`, a code point with `U+` (`U+00E4`, also lowercase u+e4) or a Latin-1 character. With several characters the first one counts. An empty value, a character above 255 and a code point above `U+00FF` are errors. `lookupChar` is the same command. The return value is a dict:

Key	Content
code	value 0–255
hex, oct	two uppercase hex digits, three octal digits
glyph	display: ^@ to ^_ and ^? for control characters, SP for the space, . for 128–159 and the soft hyphen
name	short name: NUL, DEL, nbsp, ssharp, auml; for 32–126 the character itself
char	the character

`table` and `render` are the same command for the range `von` to `bis`, both from 0 to 255. `ascii` is `0 127`, `latin1` is `128 255`, `ansi` is `latin1`. The entries run column by column from top to bottom.

Option	Default	Effect
-columns	4	columns side by side
-showName	1	short name per entry
-compact	0	1: matrix with 16 characters per line, display only

In the full table each name is padded to the length of the longest name in the range, so the columns line up. With four columns it is 73 characters wide for ASCII and 85 for Latin-1.

`signs` prints groups with a title, a rule and one entry per line; without a group, all of them. `groups` returns the names sorted. The groups `controls`, `whitespace`, `quotes`, `dashes`, `german`, `currency`, `math` and `latin1hi` contain Latin-1 characters; `arrows`, `boxdraw` and `boxdraw2` contain Unicode characters above 255 and show them as `U+XXXX`. `signs` accepts the options before or after the groups; the effective one there is `-showName`.

```
package require tclutils::tucode
::tclutils::tucode::lookup 0xDF
```

```
code 223 hex DF oct 337 glyph ß name ssharp char ß
```

```
dict get [::tclutils::tucode::lookup 65] name       ;# -> A
dict get [::tclutils::tucode::lookup ä] code        ;# -> 228
dict get [::tclutils::tucode::lookup U+00E4] name   ;# -> auml
dict get [::tclutils::tucode::lookup 0] glyph       ;# -> ^@
dict get [::tclutils::tucode::lookup 160] name      ;# -> nbsp
puts [::tclutils::tucode::table 0 7 -columns 2]
puts [::tclutils::tucode::table 64 95 -compact 1]
puts [::tclutils::tucode::signs currency]
puts [::tclutils::tucode::signs dashes -showName 0]
```

```
Dec Hex  Chr  Name
----------------------------------------
  0 00  ^@   NUL |   4 04  ^D   EOT
  1 01  ^A   SOH |   5 05  ^E   ENQ
  2 02  ^B   STX |   6 06  ^F   ACK
  3 03  ^C   ETX |   7 07  ^G   BEL

    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
40  @  A  B  C  D  E  F  G  H  I  J  K  L  M  N  O
50  P  Q  R  S  T  U  V  W  X  Y  Z  [  \  ]  ^  _

Currency symbols
----------------------------------------
 36 24  $    $
162 A2  ¢    cent
163 A3  £    pound
164 A4  ¤    curren
165 A5  ¥    yen
```

```
Dashes and hyphen
----------------------------------------
 45 2D  -
173 AD  .
```

In the compact form each cell is three characters wide; the digits of the header line stand above their columns.

Error	-errorcode
value outside 0–255, von > bis, not an integer	{TCLUTILS TUCODE RANGE}
empty value, character above 255 (lookup €), U+ above U+00FF	{TCLUTILS TUCODE LOOKUP}
unknown group	{TCLUTILS TUCODE GROUP}
-columns not an integer of at least 1	{TCLUTILS COMMON INTEGER -columns}
-showName, -compact not a boolean	{TCLUTILS COMMON BOOLEAN -name}
unknown option, option without value	{TCLUTILS COMMON OPTION -name}

11.9 In short

- Data commands expect bytes 0–255. Convert text first with `encoding convertto utf-8`; otherwise `{TCLUTILS <MODUL> BYTES}` follows.
- `tuhexdump` and `tuod` do not fold identical lines.
- `tustrings` splits at the tab.
- `patch` and `writeBytes` create a `.bak` copy unless given `-backup 0` or `0`, insert nothing, and do not write past the end of the file.
- `tucmp` counts the offset from 0, not from 1 like `cmp`.
- `tusize` counts file sizes, not blocks; links with their own size.
- `tustat::file` does not follow links; `target` names the target.

Next chapter: Notes and Calendars

12 Notes and Calendars

Markdown, iCalendar with recurrence rules, vCard, a note tree in memory and in SQLite, a monthly calendar as text, and the nationwide public holidays in Germany. Everything is in the umbrella and is pure Tcl; `tunotesdb` needs `sqlite3` with FTS5 when called. `tuvcard` also uses `tubase64`.

12.1 Markdown: tumd

tumd converts a subset of CommonMark into an HTML fragment, without an `<html>` frame. Headings are ATX (# to `######`). In addition there are paragraphs, fenced code, block quotes, flat lists and horizontal rules. Inline: code in backticks, `**fett**`, `*kursiv*`, links, images and autolinks. Nested lists, Setext headings, tables and reference links stay as they are and are not recognized as such. An indented list becomes a flat list.

`frontmatter` separates a YAML header between two `---` lines. If the closing rule is missing, the header stays empty and the text whole. `toHtml` omits the header. `headings` returns pairs of level and text. `toc` builds a list with anchors, `toHtml` sets the same ones as `id`. The anchor is made as on GitHub: lowercase, `*` and backticks are dropped, as is everything except letters, digits, `_`, `-` and spaces; spaces become `-`. Letters of every script stay. A second equal anchor gets `-1`, a third `-2` (since 0.2; up to 0.1 only `a–z` and digits stayed, and `Jörg` and `Jrg` had the same anchor `jrg`).

```
package require tclutils::tumd
::tclutils::tumd::toHtml "# Größe\n\nEin **Regal** und `x < y`."
                                          ;# -> <h1
    id="größe">Größe</h1>\n<p>Ein <strong>Regal</strong> und <code>x
    &lt; y</code>.</p>
::tclutils::tumd::headings "# Größe"       ;# -> {1 Größe}
::tclutils::tumd::toc "# Größe"            ;# -> - [Größe](#größe)
::tclutils::tumd::frontmatter "---\ntitle: Lager\n---\n# Lager\n"
                                          ;# -> {title: Lager} {#
    Lager\n}
::tclutils::tumd::toHtml "- a\n  - b"      ;# ->
    <ul>\n<li>a</li>\n<li>b</li>\n</ul>
```

The anchor of `Größe` is `größe`. `tustr::slugify` turns it into `groesse`.

12.2 iCalendar: tuical

`parse` reads the text into a list of components. A component is a dict with `type`, `props` and `components`. A property is a dict with `name`, `value` and `params`. The value stays raw, as in the file. `escapeText` and `unescapeText` handle `\,;,,` and the newline according to RFC 5545. `eventInfo`, `todoInfo` and `journalInfo` unescape the text fields. A continuation line starts with a space or tab. `BEGIN` and `END` may be lower case. A quoted parameter value may hold `;` and `:` (`ALTREP="http://x"`); the quotes are dropped on reading and written again where the value needs them. `setProperty` and `addProperty` refuse a value with a line break (`{TCLUTILS TUICAL VALUE}`), because the raw break would end the line; pass text through `escapeText` first.

`events`, `todos` and `journals` find `VEVENT`, `VTODO` and `VJOURNAL` in nested components as well. `newComponent` builds a component from a flat `Name Wert` list. `toIcs` writes it back, each line with CR/LF, since 0.2 the last one too. Long lines are folded after 75 bytes, never inside a UTF-8 character; up to tclutils 0.68.0 it counted characters, so a line of umlauts became 142 bytes long.

```
package require tclutils::tuical
set ics "BEGIN:VCALENDAR\nBEGIN:VEVENT\nUID:1\nSUMMARY:Lager\\,
    Nord\nDTSTART:20240315T090000\nEND:VEVENT\nEND:VCALENDAR\n"
set ev [lindex [::tclutils::tuical::events [::tclutils::tuical::parse
    $ics]] 0]
dict get [::tclutils::tuical::eventInfo $ev] summary
                                          ;# -> Lager, Nord
dict get [::tclutils::tuical::eventInfo $ev] dtstart
                                          ;# -> 20240315T090000
::tclutils::tuical::escapeText "a;b\nc"    ;# -> a\;b\nc
```

A line without a colon and an `END` without `BEGIN` throw `{TCLUTILS TUICAL SYNTAX}`. A property list of odd length throws `{TCLUTILS TUICAL ARG}`.

12.3 Recurrences: turrule

turrule expands an iCalendar recurrence rule `RRULE` (RFC 5545) into individual occurrences. Calculations are done in UTC.

Command	Effect
parse rrule	dict of the parts; keys in uppercase, values as written
occurrences ?optionen?	list of occurrences
eventsInRange ics von bis	list of dicts uid summary start for all VEVENT in the period; loads tuical

Option	Default	Effect
-dtstart		start, required; JJJJ-MM-TT, JJJJMMTT, also with time THHMMSS or THH:MM:SS; a trailing Z is dropped
-rule		the rule without RRULE:
-from	-dtstart	earlier occurrences do not appear
-to		later occurrences do not appear
-count	0	at most this many occurrences; 0 for no limit

Rule part	Scope
FREQ	DAILY, WEEKLY, MONTHLY, YEARLY
INTERVAL	interval between periods, default 1
COUNT	number of occurrences, counted from -dtstart
UNTIL	latest point in time; as a date without time, the whole day
BYDAY	weekdays MO to SU; MONTHLY, YEARLY: also with an ordinal such as 1MO, 4TH or -1FR

Rule part	Scope
BYMONTHDAY	days of the month, negative from the end of the month; not with WEEKLY
BYMONTH	months 1–12; YEARLY takes these months, otherwise it restricts
WKST	only MO

YEARLY works like this: with BYMONTH, each named month applies, and an ordinal in BYDAY counts within the month (BYMONTH=11;BYDAY=4TH is the fourth Thursday in November). Without BYMONTH, BYMONTHDAY applies in every month, and an ordinal in BYDAY counts within the year (BYDAY=20MO is the 20th Monday). Without these parts the anniversary of -dtstart remains. If BYDAY and BYMONTHDAY appear together, only days that satisfy both remain (BYDAY=FR;BYMONTHDAY=13). A monthly occurrence on the 31st skips months without a 31st day, a yearly occurrence on February 29 skips years without a leap day.

Other parts such as BYSETPOS, BYWEEKNO, BYYEARDAY, BYHOUR or a WKST other than MO throw {TCLUTILS TURRULE UNSUPPORTED} instead of being dropped.

COUNT counts from -dtstart; -from and -to only trim the output. A -dtstart without time yields dates, one with time yields JJJJ-MM-TTTHH:MM:SS with the same time. UNTIL without time includes the whole day: an occurrence at 09:00 on the day of UNTIL is included.

```
package require tclutils::turrule
::tclutils::turrule::parse "FREQ=WEEKLY;BYDAY=TU,TH;COUNT=4"
    ;# -> FREQ WEEKLY BYDAY TU,TH COUNT 4
::tclutils::turrule::occurrences -dtstart 2025-10-01 \
    -rule "FREQ=WEEKLY;BYDAY=TU,TH;COUNT=4"
    ;# -> 2025-10-02 2025-10-07 2025-10-09 2025-10-14
::tclutils::turrule::occurrences -dtstart 2025-01-31 \
    -rule "FREQ=MONTHLY;BYMONTHDAY=-1" -count 3
    ;# -> 2025-01-31 2025-02-28 2025-03-31
::tclutils::turrule::occurrences -dtstart 20251002T090000 \
    -rule "FREQ=DAILY;INTERVAL=2" -count 2
    ;# -> 2025-10-02T09:00:00 2025-10-04T09:00:00
set ics "BEGIN:VCALENDAR\nBEGIN:VEVENT\nUID:4\nSUMMARY:Standup\n"
append ics "DTSTART:20251002T090000\nRRULE:FREQ=WEEKLY\n"
append ics "END:VEVENT\nEND:VCALENDAR\n"
set ev [::tclutils::turrule::eventsInRange $ics 2025-10-01 2025-10-10]
lmap e $ev {dict get $e start}
    ;# -> 2025-10-02T09:00:00 2025-10-09T09:00:00
::tclutils::turrule::occurrences -dtstart 2025-01-01 \
    -rule "FREQ=YEARLY;BYMONTH=11;BYDAY=4TH;COUNT=2"
    ;# -> 2025-11-27 2026-11-26
::tclutils::turrule::occurrences -dtstart 2025-01-01 \
    -rule "FREQ=MONTHLY;BYDAY=FR;BYMONTHDAY=13;COUNT=2"
    ;# -> 2025-06-13 2026-02-13
::tclutils::turrule::occurrences -dtstart 2025-01-06T09:00:00 \
    -rule "FREQ=WEEKLY;UNTIL=20250120"
    ;# -> 2025-01-06T09:00:00 2025-01-13T09:00:00 2025-01-20T09:00:00
```

Without `-to`, `-count`, `COUNT` and `UNTIL`, `occurrences` throws `{TCLUTILS TURRULE UNBOUNDED}`. A rule without `FREQ` or with a frequency other than the four above throws `{TCLUTILS TURRULE FREQ}`, an unreadable date `{TCLUTILS TURRULE DATE}`. An unknown weekday abbreviation such as `XX`, the ordinal 0, or an ordinal with `DAILY` and `WEEKLY` throw `{TCLUTILS TURRULE BYDAY}`; a month outside 1–12 in `BYMONTH` or a day outside ±1–31 in `BYMONTHDAY` throws `{TCLUTILS TURRULE VALUE}`.

12.4 vCard: tuvcard

`parse` returns a list of cards. A card is a list of property dicts, the same structure as in `tuical`. `property` returns the first value of a name, `get` all values, `fullName` the property `FN`. `toVcf` writes CR/LF, after the last line too, and folds after 75 bytes like `tuical`. The case of the name does not matter. A group prefix is not part of the name: `fullName` finds `item1.FN:Ada`, which is written back as `item1.FN`. `TYPE="work;cell"` is one value `work;cell`. A line break in a value for `addProperty` or `setProperty` is `{TCLUTILS TUVCARD VALUE}`.

```
package require tclutils::tuvcard
set vcf "BEGIN:VCARD\nFN:Ada\nN:Lovelace;Ada;;;\nTEL:1\nEND:VCARD\n"
set card [lindex [::tclutils::tuvcard::parse $vcf] 0]
::tclutils::tuvcard::fullName $card          ;# -> Ada
::tclutils::tuvcard::get $card TEL           ;# -> 1
::tclutils::tuvcard::names $card             ;# -> FN N TEL
```

A line without a colon throws `{TCLUTILS TUVCARD SYNTAX}`.

12.5 Note Tree: tunotes

A store is a dict mapping id to note. A note has `id`, `parent_id`, `title`, `content`, `created`, `modified` and `tags`. `create`, `update` and `delete` receive the name of the variable and change it. `get`, `search`, `count`, `roots` and `children` receive the value. The id is `clock microseconds`.

`search` compares title and content with `string match`, ignoring case. An asterisk in the query is a pattern character. `toJson` and `fromJson` go through `tujson`. `fromJson` fills missing fields with empty values. `load` of a missing file yields an empty store. `save` always writes the file as UTF-8 (0.2). `load` reads a file that is not valid UTF-8 in the system encoding; that is how 0.1 wrote it. `delete` with the third argument `1` (default) removes the branch; with `0` the children are attached to the root.

```
package require tclutils::tunotes
set store [dict create]
set id [::tclutils::tunotes::create store Lager "Regal 3" {ort}]
dict get [::tclutils::tunotes::get $store $id] title
                                             ;# -> Lager
dict get [::tclutils::tunotes::get $store $id] tags
                                             ;# -> ort
::tclutils::tunotes::count $store              ;# -> 1
set gelesen [::tclutils::tunotes::fromJson {{"7":{"title":"Lager"}}}]
dict get [dict get $gelesen 7] title         ;# -> Lager
dict get [dict get $gelesen 7] parent_id     ;# ->
```

An unknown id throws `{TCLUTILS TUNOTES NOTFOUND}`, as does a `create` with a missing parent.

12.6 Notes in SQLite: tunotesdb

tunotesdb stores the same notes as `tunotes` in SQLite, one row per note, with an FTS5 index for full-text search. Like `tusqlite`, the module does not load `sqlite3`: the caller opens the database and passes the handle. The note dict has the same structure as in `tunotes`; `parent_id` is empty for a root.

`init db` creates the table, the FTS5 index and three triggers if they are missing, and turns on `journal_mode = WAL` and `foreign_keys`. A repeated call does no harm.

Command	Effect
create db titel inhalt ?tags? ?elter?	the new id from clock microseconds
update db id titel inhalt ?tags? ?elter?	the id; elter defaults to KEEP and then stays
get db id, exists db id	the note dict, or 1 or 0
delete db id ?kaskade?	1 (default) deletes the branch, 0 turns the children into roots
move db id elter	the id; a note below itself or below a child is rejected
ids db, count db	all ids, or their number
roots db, children db elter	roots, or direct children; children db "" are the roots
descendants, ancestors, parent, path, depth, siblings, subtree	the tree via WITH RECURSIVE; path starts at the root
search db frage ?optionen?	hit dicts id title snippet rank, best first
byTag db tag, tags db	ids with the tag, or all tags, sorted
toStore db, fromStore db store ?-clear 1?	the bridge to a tunotes store

Option of search	Default	Effect
-limit	50	at most this many hits; 0 for all
-snippet	10	words of context in the snippet; the hit is enclosed in [ ]

The query goes to FTS5 unchanged: several words mean AND, `"wort folge"` is a phrase, `wort*` a prefix, and `OR` and `NOT` apply as written. The tokenizer `unicode61` with `remove_diacritics 2` searches without regard to case and accents. Tags are stored as a Tcl list in one column; `byTag` compares in Tcl.

```
package require sqlite3
package require tclutils::tunotesdb
sqlite3 db :memory:
::tclutils::tunotesdb::init db
set a [::tclutils::tunotesdb::create db Lager "Regal 3 Nord" ort]
set b [::tclutils::tunotesdb::create db Regal "Fach 2, Schrauben" \
    {} $a]
::tclutils::tunotesdb::count db             ;# -> 2
expr {[::tclutils::tunotesdb::parent db $b] == $a}
                                           ;# -> 1
::tclutils::tunotesdb::depth db $b          ;# -> 1
set h [lindex [::tclutils::tunotesdb::search db schraube*] 0]
dict get $h snippet                         ;# -> Fach 2, [Schrauben]
::tclutils::tunotesdb::tags db              ;# -> ort
```

`fromStore` reads the store in one transaction. A note with an existing id is changed in place; its children in the database remain. The order in the store does not matter, and a child may come before its parent: the foreign key check waits until the end of the transaction. Every `parent_id` must be in the store or in the database; otherwise `fromStore` throws `NOTFOUND` and rolls everything back.

```
set st [::tclutils::tunotesdb::toStore db]
dict set st $a title Lagerhalle
::tclutils::tunotesdb::fromStore db [dict create $a [dict get $st $a]]
                                                ;# -> 1
::tclutils::tunotesdb::count db                   ;# -> 2
expr {[::tclutils::tunotesdb::path db $b] eq [list $a $b]}
                                                ;# -> 1
```

The `-errorcode` is `{TCLUTILS TUNOTESDB grund}`:

Reason	Case
NODB	the handle is not a usable sqlite3 command
NOFTS5	SQLite without FTS5
NOTFOUND	no note with the id, or no such parent (also a parent_id passed to fromStore)
CYCLE	move below itself or a descendant
BADQUERY	FTS5 cannot parse the query, for example "offen
BADOPTION	unknown option, or -limit not a number ≥ 0
BADNOTE	a note passed to fromStore lacks a field

12.7 Monthly Calendar: tucal

tucal prints calendars as text, like the `cal` command. The PNG counterpart is `tumonthpng` in the chapter Images without Tk: PNG, SVG, Colors.

Command	Effect
month ?monat? ?jahr? ?optionen?	one month; without arguments the current month, without a year the current year (UTC)
now ?…?	the same as month
render jahr monat ?optionen?	one month, year first
three ?monat? ?jahr? ?optionen?	previous month, month and next month side by side
year jahr ?optionen?	the whole year in four rows of three months each

A single four-digit number from 1900 passed to `month` counts as a year and yields `year`. In `year` the months run row by row as in `cal`: the first row shows January, February and March.

Option	Default	Effect
-mondayfirst	0	1 starts the week on Monday
-weeknumbers	0	1 puts the week number on the left; with Monday first per ISO (%V), otherwise %U
-iso	0	1 sets -mondayfirst 1 and -weeknumbers 1
-locale		locale of clock format for month and weekday names, such as de_DE; empty means English (Su Mo Tu …)

Every option needs a value; `-iso` alone is not enough. The return value is text with `\n`, without a trailing newline. A day cell is three characters wide in `month`, `render` and `three`, and two in `year`; a month is thus 27 or 20 characters wide, 4 more with week numbers. The month title is centered above the day columns. `three` and `year` place the months side by side with two spaces between them; the year in `year` is centered over the full width. No line ends with a space.

```
package require tclutils::tucal
puts [::tclutils::tucal::month 5 2026 -locale de_DE -iso 1]
set kal [::tclutils::tucal::month 2 2026]
llength [split $kal \n]                      ;# -> 6
set titel [lindex [split $kal \n] 0]
string first F $titel                        ;# -> 7
string trim $titel                           ;# -> February 2026
set j [split [::tclutils::tucal::year 2026] \n]
regexp -all -inline {[A-Z][a-z]+} [lindex $j 2]
                                            ;# -> January February March
```

The output of `puts`:

```
              Mai 2026
     Mo  Di  Mi  Do  Fr  Sa  So
18                    1   2   3
19    4   5   6   7   8   9  10
20   11  12  13  14  15  16  17
21   18  19  20  21  22  23  24
22   25  26  27  28  29  30  31
```

A month outside 1–12 throws `{TCLUTILS TUCAL MONTH}`, a year less than 1 `{TCLUTILS TUCAL YEAR}`. More than two numbers throw `{TCLUTILS TUCAL ARGS}`, an option without a value `{TCLUTILS TUCAL OPTION}`, an unknown option `{TCLUTILS COMMON OPTION -name}`.

12.8 Public Holidays: tuholiday

`easter` calculates Easter Sunday in the Gregorian calendar (Meeus/Jones/Butcher) and returns an ISO date. `holidays` returns a dict mapping date to name, sorted. `-region` knows only `de`: the days that apply in all German states, plus Easter Sunday and Whit Sunday. Epiphany, Corpus

Christi, All Saints' Day and Reformation Day are missing. `isHoliday` returns the name or the empty string.

Offset from Easter	Name
−2	Karfreitag (Good Friday)
0	Ostersonntag (Easter Sunday)
+1	Ostermontag (Easter Monday)
+39	Christi Himmelfahrt (Ascension Day)
+49	Pfingstsonntag (Whit Sunday)
+50	Pfingstmontag (Whit Monday)

Fixed dates are New Year's Day, Labor Day (May 1), the Day of German Unity and the two Christmas holidays. For 2024 the dict has eleven entries. Easter Sunday and Whit Sunday are statutory holidays in a few states only; `-sundays 0` (since 0.2, for `holidays` and `isHoliday`) leaves them out and returns the nine days that apply everywhere.

```
package require tclutils::tuholiday
::tclutils::tuholiday::easter 2024           ;# -> 2024-03-31
::tclutils::tuholiday::easter 2025           ;# -> 2025-04-20
::tclutils::tuholiday::isHoliday 2024-03-31  ;# -> Ostersonntag
::tclutils::tuholiday::isHoliday 2024-03-15  ;# ->
::tclutils::tuholiday::isHoliday 2024-03-31 -sundays 0  ;# ->
dict size [::tclutils::tuholiday::holidays 2024 -sundays 0]  ;# -> 9
```

`easter xyz` throws `{TCLUTILS TUHOLIDAY YEAR}`. `-region at` throws `{TCLUTILS TUHOLIDAY REGION}`, `-sundays maybe` `{TCLUTILS TUHOLIDAY SUNDAYS}`.

12.9 Calendar Views: tical

The modules of this chapter deliver data and text: recurrences, holidays, iCalendar, a month like `cal`. Views for month, week and day with renderers for the terminal, the Tk canvas and PDF come with the separate library tical 1.1. It has its own holidays and an iCalendar export (with RRULE); its core does not need tclutils. Its optional import (`tical::io::import`) reads iCalendar with `tuical` and expands recurrences with `turrule`. It is described in the book *Utils-Stack*.

12.10 In short

- `tumd` is a subset. The anchor of `Größe` is `größe`.
- iCalendar and vCard values stay raw. `eventInfo` unescapes text.
- `toIcs` and `toVcf` write CR/LF, after the last line too.
- `tunotes`: change through the variable name, read through the value.
- Public holidays: only `de`, eleven names, no state-specific days.
- `turrule` needs a limit: `-to`, `-count`, `COUNT` or `UNTIL`. Unsupported parts throw `UNSUPPORTED`.
- `tunotesdb`: open the handle yourself, then `init`. `fromStore` changes existing notes in

place.
- `tucal` is text like `cal`; as PNG `tumonthpng`.

Next chapter: Text from PDF Files

13 Text from PDF Files

tupdftext 0.1 is new in tclutils 0.67.0. It reads the text of a PDF page by page, in pure Tcl. Tcl's `zlib` unpacks the streams. Neither poppler nor a C extension is needed. It is intended for full-text search: words, spaces and line breaks. Columns and tables come in the order of the content stream.

The sister module `tupdf` reads the structure of a PDF (version, objects, trailer, metadata, ZUGFeRD). `tupdftext` reads the text.

13.1 Commands

```
package require tclutils::tupdftext
::tclutils::tupdftext::pages   datei ?-notext error|empty?
::tclutils::tupdftext::text    datei ?-separator s? ?-notext
    error|empty?
::tclutils::tupdftext::summary datei
```

`pages` returns a list with one string per page, in page order. Lines are separated by `\n`, runs of spaces are collapsed, ligatures (fi, fl) are spelled out, and soft hyphens are removed.

`text` joins the pages with `-separator`. The default is the form feed `\f`, as in `pdftotext`. A single page has no separator.

`summary` returns a dict:

Key	Meaning
pages	number of pages
encrypted	1 if an /Encrypt entry is present
version	version from the header line, such as 1.4
textPages	pages with at least one character
unmapped	glyphs that came out as U+FFFD (two-byte font without /ToUnicode)
fonts	list of dicts name, subtype, toUnicode

With `-notext empty`, a PDF without text yields a list of empty strings, one per page. Without the option this is the error `NOTEXT`.

The command is called `summary`, not `info`. A procedure `info` in the module's namespace would have hidden Tcl's `info` from all procedures of the module.

```
set seiten [::tclutils::tupdftext::pages $datei]
llength $seiten                                          ;# -> 1
lindex $seiten 0                                         ;# -> Lager
    Nord, Regal 3\nGroesse: 12 Stueck
dict get [::tclutils::tupdftext::summary $datei] version ;# -> 1.4
dict get [::tclutils::tupdftext::summary $datei] unmapped
                                                         ;# -> 0
::tclutils::tupdftext::text $zweiSeiten                  ;# -> Seite
    eins\fSeite zwei
```

The comments apply to two files created with pdf4tcl: one page in Helvetica with the two lines above, and two pages with "Seite eins" and "Seite zwei". The same, but empty, page without the option throws `{TCLUTILS TUPDFTEXT NOTEXT}`; with `-notext` `empty` the list is one empty element. A missing file yields `FILE`, a file without a `%PDF-` header `FORMAT`. Tcl 8.6 and Tcl 9 return the same text.

13.2 Errors

Every error carries `-errorcode {TCLUTILS TUPDFTEXT grund}`:

Reason	When
FILE	the file is not readable
FORMAT	no %PDF- header, no objects or no pages
ENCRYPTED	the file is encrypted; even an empty user password is not read
NOTEXT	no page has text: a scan without OCR, or images only
OPTION	unknown option or missing value

13.3 What Comes Through

The module looks for objects as `N G obj` at the start of a line. It does not need the cross-reference table; a damaged one does no harm. A later definition of the same number wins, as in an incremental update. Objects in object streams (`/ObjStm`) fill numbers that do not appear directly.

Filters: `FlateDecode` (also with PNG predictor), `ASCIIHexDecode`, `ASCII85Decode`, `LZWDecode`, and chains of these. Pages come from the page tree starting at `/Root`, with inherited `/Resources`. Without a usable tree: all `/Type /Page` objects in file order.

Text operators: `Tj`, `TJ`, `'`, `"` with `Tf`, `Tc`, `Tw`, `Tz`, `TL`, `Ts`, `Td`, `TD`, `Tm`, `T*`, `q`, `Q`, `cm`. Form XObjects (`Do`) with their own resources and their own matrix, up to depth 8; a form that draws itself, once. Embedded images are skipped.

Fonts: Type0 (two-byte codes, `Identity-H`/`-V` or an embedded CMap), Type1, TrueType, Type3. Unicode from `/ToUnicode` (`bfchar`, `bfrange`), otherwise from `/Encoding` (`WinAnsiEncoding`, `MacRomanEncoding`, `StandardEncoding`, `PDFDocEncoding` and `/Differences` with glyph names). Spaces and line breaks come from the positions: a jump of half a line downward is a new line, a gap of more than 0.15 em within the line is a space.

13.4 What It Does Not Do

- decrypt (ENCRYPTED)
- read text from images
- map a two-byte font without `/ToUnicode` (U+FFFD, counted in `unmapped`)
- keep columns apart
- join words that are hyphenated at the end of a line (`pdftotext` does this)

Compared with `pdftotext` (poppler 24.02) on 13 PDFs, recorded in `docs/tupdftext.md`: 99.5 to 100 % of the words for eleven files, 91 % for two pages of formulas. A book with 108 pages takes about 3.6 s under Tcl 8.6.14. Tcl 9.0.4 returns the same text there; characters outside the BMP are one character under Tcl 9 and a surrogate pair under Tcl 8.6. This comparison was not rerun here. No encrypted file was created in this run; the module description names ENCRYPTED for it.

13.5 Command Line

```
tclsh bin/tupdf.tcl text doc.pdf
tclsh bin/tupdf.tcl text doc.pdf -page 3
```

`book-search.tcl` and mdserver use the module for the text in PDFs.

13.6 In short

- `pages` per page, `text` with `\f` in between, `summary` for the key figures.
- A scan and an encrypted file are errors, not a silent empty string.
- `-notext empty` turns the error into an empty page string.
- Columns, hyphenation at the end of a line and text in images stay out.

Next chapter: ODF and PDF Structure

14 ODF and PDF Structure

tupdf 0.2 reads the structure of a PDF: header line, object numbers, page markers, trailer, info and an indication of ZUGFeRD. `tupdftext` reads the text of the pages, see the chapter Text from PDF Files.

tuodf 0.2 writes and reads a plain OpenDocument with paragraphs. tudocx and tuxlsx 0.1 do the same for Word and Excel files (Office Open XML). All are in the umbrella.

14.1 PDF: tupdf

```
package require tclutils::tupdf
::tclutils::tupdf::version  datei
::tclutils::tupdf::objects  datei
::tclutils::tupdf::object   datei nummer
::tclutils::tupdf::trailer  datei
::tclutils::tupdf::metadata datei
::tclutils::tupdf::summary  datei
::tclutils::tupdf::zugferd  datei
```

`version` takes the number after `%PDF-` in the first 1024 bytes. `objects` returns the numbers from `N G obj`, sorted and unique. The module unpacks compressed object streams with `zlib` before counting. `object` returns the raw text from `id gen obj` to `endobj`. `pages` in `summary` counts `/Type /Page` and leaves out `/Pages`.

The following file has the header `%PDF-1.4`, four objects, one page and an info object with `Title (Lager)` and `Author (Nord)`.

```
::tclutils::tupdf::version $datei                    ;# -> 1.4
::tclutils::tupdf::objects $datei                    ;# -> 1 2 3 4
dict get [::tclutils::tupdf::summary $datei] pages   ;# -> 1
dict get [::tclutils::tupdf::metadata $datei] Title  ;# -> Lager
```

`summary` also carries `version`, `size`, `objects`, `encrypted`, `linearized`, `acroform` and `zugferdDetected`. Existing info fields are in the same dict. Here `encrypted`, `linearized` and `acroform` are 0, and `zugferdDetected` is 0 as well.

`zugferd` is a search in the file text, not a conformance check. Without an attachment and without the markers, the lists in the dict stay empty:

```
dict get [::tclutils::tupdf::zugferd $datei] detected ;# -> 0
dict get [::tclutils::tupdf::zugferd $datei] profile  ;# -> {}
```

The keys are `detected`, `profile`, `attachmentNames`, `mimeTypes` and `hints`. A match sets `profile` to `factur-x`, `zugferd` or `unknown`.

Code	When
{TCLUTILS TUPDF FORMAT}	no %PDF-header, or endobj missing after the object
{TCLUTILS TUPDF ARG}	the object number is not an integer
{TCLUTILS TUPDF NOTFOUND}	the number does not appear in the text after inflating

`object $datei x` throws `ARG`. `object $datei 9` on the file above throws `NOTFOUND`.

14.2 ODF: tuodf

```
package require tclutils::tuodf
::tclutils::tuodf::createText datei absätze ?-title T? ?-creator C?
::tclutils::tuodf::parts      datei
::tclutils::tuodf::part       datei name
::tclutils::tuodf::paragraphs datei
::tclutils::tuodf::text       datei
```

`createText` returns the file path. The container is a ZIP. `mimetype` is the first entry and stays uncompressed. `-title` and `-creator` go into `meta.xml`.

```
set z [::tclutils::tuodf::createText lager.odt \
    [list {Lager Nord} {a < b} {}] -title Lager -creator Nord]
::tclutils::tuodf::parts $z
    ;# -> mimetype content.xml styles.xml meta.xml
    META-INF/manifest.xml
::tclutils::tuodf::part $z mimetype
    ;# -> application/vnd.oasis.opendocument.text
::tclutils::tuodf::paragraphs $z          ;# -> {Lager Nord} {a < b} {}
::tclutils::tuodf::text $z                ;# -> Lager Nord\na < b\n
```

`a < b` comes back as text. The XML contains `<`. An empty entry in the list is an empty paragraph. `text` joins the paragraphs with `\n`, so the result ends with a newline. `meta.xml` contains `<dc:title>Lager</dc:title>` and `<dc:creator>Nord</dc:creator>`.

`paragraphs` reads paragraphs (`text:p`) and, since 0.2, headings (`text:h`), in document order. It finds paragraphs inside tables and lists, in the cells of an `.ods` and on the slides of an `.odp` as well; the structure itself (rows, columns, levels) is not kept. A missing ZIP file is reported by `tuzip`, not with an error code of `tuodf`'s own.

14.3 Word: tudocx

```
package require tclutils::tudocx
::tclutils::tudocx::createText file paragraphs ?-title T? ?-creator C?
::tclutils::tudocx::parts      file
::tclutils::tudocx::part       file name
::tclutils::tudocx::paragraphs file
::tclutils::tudocx::text       file
```

The commands are named as in `tuodf`. `createText` writes the five parts a `.docx` needs at least; Word, LibreOffice and python-docx open it. A tab in a paragraph becomes `w:tab`, a line break `w:br`.

```
set d [::tclutils::tudocx::createText lager.docx \
    [list {Lager Nord} "Regal\t3" {}] -title Lager]
::tclutils::tudocx::parts $d
    ;# -> {[Content_Types].xml} _rels/.rels word/document.xml
    docProps/core.xml docProps/app.xml
::tclutils::tudocx::paragraphs $d        ;# -> {Lager Nord}
    {Regal<TAB>3} {}
```

`paragraphs` reads the main part named in `_rels/.rels`, usually `word/document.xml`: every paragraph in document order, headings, list items and table cells included. Deleted text of tracked changes (`w:delText`) and field codes (`w:instrText`) are left out. Headers, footers, footnotes and comments are in other parts and are not read. A missing main part is `{TCLUTILS TUDOCX PART …}`.

14.4 Excel: tuxlsx

```
package require tclutils::tuxlsx
::tclutils::tuxlsx::writeRows file rows ?-sheet NAME? ?-numbers
    auto|0? ?-title T? ?-creator C?
::tclutils::tuxlsx::sheets    file
::tclutils::tuxlsx::rows      file ?-sheet NAME|NUMBER?
::tclutils::tuxlsx::text      file
::tclutils::tuxlsx::part      file name
::tclutils::tuxlsx::parts     file
```

`writeRows` writes one sheet. With `-numbers auto` a value that looks like a decimal number is stored as a number, everything else as text; `007` stays text.

```
set x [::tclutils::tuxlsx::writeRows lager.xlsx \
    {{Artikel Menge} {Schraube 120} {Mutter 007}} -sheet Lager]
::tclutils::tuxlsx::sheets $x            ;# -> Lager
::tclutils::tuxlsx::rows $x              ;# -> {Artikel Menge}
    {Schraube 120} {Mutter 007}
::tclutils::tuxlsx::rows $x -sheet Nord  ;# error {TCLUTILS TUXLSX
    SHEET Nord}
```

`rows` reads any workbook, also those of Excel and LibreOffice. The values come as the file stores them: text from the shared string table or inline in the cell, numbers as written, a date as its serial number, booleans as `1` and `0`, a formula with its last computed value, an error as text (`#DIV/0!`). Gaps are filled with `""`. `tuxlsx` does not read formats. `text` returns all sheets, cells separated by a tab, rows by a newline.

14.5 In short

- `tupdftext` reads the page text. `tupdf` returns version, objects, info.
- `/Pages` does not count as a page. `zugferd` does not check the file for conformance.
- `tuodf` writes paragraphs. `mimetype` is the first, uncompressed entry.
- `tudocx` and `tuxlsx` are named and work like `tuodf`; `tuxlsx::rows` returns values as the file stores them (a date as its serial number).
- An empty paragraph at the end adds a newline to `text`.

Next chapter: Diagrams

15 Diagrams

Boxes, arrows, bars and circles are produced in pure Tcl, without a browser and without Node.

The input is text in a subset of the Mermaid notation, the output is SVG or PNG. For node-and-edge graphs the model is a dict of tudiagram: `layout` computes the boxes in pixels, `toSvg` and `toPng` draw from them. Each of the other kinds has a module of its own with its own model and its own drawing.

tuflow 0.3 is the entry point for all kinds. When loaded, `tuflow` pulls in only `tudiagram` 0.5. The module for a kind is loaded on the first call with its keyword. The list is also in the chapter Umbrella and Modules.

Kind	Keyword	Module	parse returns graph	PNG options
Flowchart	flowchart, graph	tuflow	yes	-scale, -fontfile
States	stateDiagram, stateDiagram-v2	tustate	yes	-scale, -fontfile
Classes	classDiagram, classDiagram-v2	tuclass	yes	-scale, -fontfile
Entities	erDiagram	tuer	yes	-scale, -fontfile
Requirements	requirementDiagram	turequirement	yes	-scale, -fontfile
Mind map	mindmap	tumindmap	yes	-scale, -fontfile
C4	C4Context, C4Container, C4Component, C4Dynamic, C4Deployment	tuc4	yes	-scale, -fontfile
Blocks	block-beta	tublock	yes	-scale, -fontfile
Git history	gitGraph	tugit	yes	-scale, -fontfile
Architecture	architecture, architecture-beta	tuarchitecture	yes	-scale, -fontfile
Sequence	sequenceDiagram	tusequence	no, UNSUPPORTED	surface from the content
Gantt	gantt	tugantt	no	width 800, height from 400 by tasks
Pie	pie	tupie	no	480 × 320, -legend
XY	xychart-beta	tuxychart	no	520 × 340
Quadrant	quadrantChart	tuquadrant	no	420 × 420
Journey	journey	tujourney	no	600 × 360
Timeline	timeline	tutimeline	no	640 × 380

Kind	Keyword	Module	parse returns graph	PNG options
Sankey	sankey, sankey-beta	tusankey	no	700 × 400
Radar	radar, radar-beta	turadar	no	600 × 600
Treemap	treemap, treemap-beta	tutreemap	no	640 × 400
Bit fields	packet, packet-beta	tupacket	no	width 640, height from the rows
Kanban	kanban	tukanban	no	surface from columns and cards

The sizes in the last column are the defaults of `-width` and `-height`; every kind without a graph also takes `-scale` and `-fontfile`. Mermaid knows further kinds (`zenuml`, `xychart` without `-beta`). Such a text passes as a flowchart, and its first word becomes a node.

15.1 From Text to File: tuflow

```
::tclutils::tuflow::parse    text                ;# -> tudiagram-Modell
::tclutils::tuflow::toSvg    text ?opt wert ...? ;# -> SVG-Text
::tclutils::tuflow::toPng    text ?opt wert ...? ;# -> PNG-Bytes
::tclutils::tuflow::writeSvg text datei ?opt wert ...?  ;# -> datei
::tclutils::tuflow::writePng text datei ?opt wert ...?  ;# -> datei
```

`toSvg` and `toPng` read the first word of the first line that is neither empty nor a `%%` comment, regardless of upper and lower case. If it names a kind without a graph, the facade calls `parse` and `toSvg` or `toPng` of that kind's module. Otherwise it calls its own `parse` and draws the model with `tudiagram`. `writeSvg` writes UTF-8 with LF, `writePng` writes binary; both return the file name. In all diagram modules writing goes through a temporary file (`common::replaceFile`): if it fails, the old file stays and no channel is left open.

```
package require tclutils::tuflow
set t {flowchart LR
    A --> B
}
string range [::tclutils::tuflow::toSvg $t] 0 3   ;# -> <svg
string range [::tclutils::tuflow::toPng $t] 1 3   ;# -> PNG
::tclutils::tuflow::writePng $t fluss.png         ;# -> fluss.png
```

Option	Default	Effect
-scale	1	integer from 1; PNG: surface and drawing times the factor
-fontfile	empty	TTF or OTF file for the PNG font
-width	per kind	width in pixels, only kinds without a graph
-height	per kind	height in pixels, only kinds without a graph

Option	Default	Effect
-legend	1	only pie: 0 omits the legend

For a graph, `toPng` takes only `-scale` and `-fontfile`, and `toSvg` takes no option at all. For the other kinds `-scale` also applies to SVG: the surface stays at `-width` × `-height`, while text, lines and spacing grow by the factor. An option the kind does not know is dropped by the facade without an error; `-scale 0`, however, throws the kind's ARG error, for a graph `{TCLUTILS TUDIAGRAM ARG}`. `-width 0` with a fixed surface throws `{TCLUTILS COMMON INTEGER width}` from the surface.

```
set p "pie\n\"a\" : 1\n"
set a [::tclutils::tuflow::toPng $p -farbe rot]
string equal $a [::tclutils::tuflow::toPng $p]   ;# -> 1
catch {::tclutils::tuflow::toPng $p -scale 0} m o
dict get $o -errorcode                    ;# -> TCLUTILS TUPIE ARG
```

Calling a kind's module directly pays off in three cases: when the model is needed (`parse` of every kind returns a dict), when an unknown option should show up as an error (the modules check strictly with `{TCLUTILS COMMON OPTION ...}`), and when a `tudiagram` model is changed before drawing, for example with `setMeta -theme`. Every module without a graph has the same five commands: `parse text`, and `toSvg`, `toPng`, `writeSvg`, `writePng` with the model instead of the text.

For a kind without a graph, `tuflow::parse` returns the error `{TCLUTILS TUFLOW UNSUPPORTED}`, also for `sankey` without `-beta`.

```
catch {::tclutils::tuflow::parse "pie\n\"a\" : 1\n"} m o
dict get $o -errorcode               ;# -> TCLUTILS TUFLOW UNSUPPORTED
catch {::tclutils::tuflow::parse "sankey\na,b,1\n"} m o
dict get $o -errorcode               ;# -> TCLUTILS TUFLOW UNSUPPORTED
```

15.2 Model: tudiagram

```
::tclutils::tudiagram::create   ?-title t? ?-direction d? ?-theme t?
                                ?-fontfile f?
::tclutils::tudiagram::addNode  d id ?-label t? ?-shape s?
                                ?-style dict?
::tclutils::tudiagram::addEdge  d von nach ?-label t? ?-style s?
                      ?-arrow a? ?-startMark m? ?-endMark m?
::tclutils::tudiagram::addGroup d id ?-label t? ?-members ids?
::tclutils::tudiagram::setMeta  d ?-schlüssel wert ...?
::tclutils::tudiagram::validate d           ;# -> Liste der Probleme
::tclutils::tudiagram::layout   d           ;# -> Dict mit Koordinaten
::tclutils::tudiagram::theme    name        ;# -> Farb-Dict
::tclutils::tudiagram::toSvg    d
::tclutils::tudiagram::toPng    d ?-scale n?
::tclutils::tudiagram::writeSvg d datei
::tclutils::tudiagram::writePng d datei ?-scale n?
```

`create` returns the dict with the keys `version`, `meta`, `nodes`, `edges` and `groups`. `meta` holds `title`, `direction`, `theme`, `fontfile`, `nodeGap` 30, `rankGap` 70 and `padding` 20. The direction is `LR`, `TB`, `RL` or `BT`, default `LR`. The themes are `default`, `pipeline` and `mono`. Instead of the name, a dict with at least `fill` also works; missing colors come from `default`. Any other value throws `{TCLUTILS TUDIAGRAM THEME}`, in `theme`, `create` and `setMeta`. The title is centered above the graph: `layout` moves all nodes and edges down by the title band (`titleH` in `meta`, 26 pixels) and widens the surface when the title is wider than the graph. `addNode`, `addEdge` and `addGroup` return the extended dict; they do not change the one passed in. `setMeta` sets any keys in `meta` and checks only `-direction` and `-theme`.

Shape	Names
Node -shape	box (default), rounded, dot, circle, stadium, diamond, hexagon, cylinder
Node -style	dict with fill, stroke, text as #rrggbb; missing keys come from the theme
Edge -style	solid (default), dashed, dotted, thick
Edge -arrow	end (default), none, both, start
Edge -startMark, -endMark	none (default), exactlyOne, zeroOrOne, oneOrMany, zeroOrMany

A node without `-label` shows its id. A `dot` shows no label. The marks at the edge ends are crow's feet as in an ER diagram. `addEdge` checks neither the ends nor the values of `-style` and `-arrow`.

`validate` does not throw. An empty list means: there are nodes, and every edge names known ends. `layout` sets `x`, `y`, `width` and `height` on every node, `points`, `arrowStart` and `back` on every edge, and in `meta` the overall size `width` and `height` as well as the height of the title band `titleH`. The font metrics are the fixed 6×8 grid, even without a surface. An edge from a node to itself does not enter the ranking and is not drawn. A cycle is broken at one edge, which then runs backward (`back` 1).

```
package require tclutils::tudiagram
set d [::tclutils::tudiagram::create -title Lager -direction LR]
set d [::tclutils::tudiagram::addNode $d A -label Eingang]
set d [::tclutils::tudiagram::addNode $d B -label Pruefung \
    -shape diamond]
set d [::tclutils::tudiagram::addEdge $d A B -label ja]
::tclutils::tudiagram::validate $d           ;# -> (leer)
set L [::tclutils::tudiagram::layout $d]
dict keys [lindex [dict get $L nodes] 0]
                        ;# -> id label shape style x y width height
dict get [dict get $L meta] width            ;# -> 394
dict get [dict get $L meta] titleH           ;# -> 26
```

After `layout`, A is at x 20, y 46, 112×44, and B at x 202, y 46, 172×56; without a title both are at y 20. `toSvg` starts with `<svg xmlns="http://www.w3.org/2000/svg"`.

Groups are labeled frames around the nodes in `-members`. A name in `-members` that is not a node is dropped when drawing. If a group lies entirely inside another, the outer one gets more spacing. With groups, `layout` places the members of each group in a band of their own so that the frames do not overlap.

A second id `A` throws `{TCLUTILS TUDIAGRAM DUPID}`. The direction `XX` throws `{TCLUTILS TUDIAGRAM DIR}`, a shape `triangle` `{TCLUTILS TUDIAGRAM ARG}`. A model without nodes yields the entry `EMPTY no nodes` from `validate`, and `layout` then throws `{TCLUTILS TUDIAGRAM EMPTY}`. An edge to `Z` and `Y` adds one `NONODE` entry to `validate` for each of the two ends.

```
catch {::tclutils::tudiagram::create -theme neon} m o
dict get $o -errorcode               ;# -> TCLUTILS TUDIAGRAM THEME
```

15.3 Flowchart: tuflow parse

`parse` reads `graph` and `flowchart` and returns a `tudiagram` model. The header line is optional, and upper and lower case do not matter (`FLOWCHART td`, `Graph LR`); without a direction `LR` applies, `TD` becomes `TB`. An id consists of letters, digits and `_`, umlauts included: `Büro[Amt]` is the node `Büro` (up to tclutils 0.68.0 ASCII only, and the line was dropped). The shape is given by the brackets; quotation marks around the label are removed.

Notation	Shape
A alone	box, label A
A[Text] and A[[Text]]	box
A(Text)	rounded
A([Text])	stadium
A((Text))	circle
A{Text}	diamond
A{{Text}}	hexagon
A[(Text)]	cylinder

Arrow	-style	-arrow
-->, --o, --x	solid	end
---	solid	none
-.->	dotted	end
-.-	dotted	none
==>	thick	end
===	thick	none

The label is written as `A -->|Text| B`, `A -- Text --> B`, `A -. Text.-> B` or `A == Text ==> B`. Chains `A --> B --> C` are accepted. `&` joins several nodes on one side: `A &`

`B --> C & D` yields one edge for each pair, here four. `%%` is a comment, statements may be separated by `;`; `a;` inside a label (`A[Jörg;Büro]`, `-->|a;b|`) or in quotes does not separate. If an id is given twice with a shape, the last one applies. `subgraph name [Beschriftung]` up to `end` becomes a group with the nodes inside it, also nested; a `direction` inside it is dropped. An edge to the group id is dropped.

Not read:

- `style`, `classDef`, `class`, `click` and `linkStyle`: skipped;
- `A:::klasse`: the class is dropped, the node stays;
- `A[/Text/]` and the other trapezoid and parallelogram shapes: `box` with the label `/Text/`;
- `--o` and `--x` draw an ordinary arrowhead, not a circle and not a cross.

```
package require tclutils::tuflow
set src {flowchart TD
    A[Eingang] --> B{Pruefung}
    B -->|ja| C[(Lager)]
}
set m [::tclutils::tuflow::parse $src]
dict get [dict get $m meta] direction        ;# -> TB
set e [lindex [dict get $m edges] 1]
list [dict get $e label] [dict get $e style] ;# -> ja solid
set w [::tclutils::tuflow::parse "Graph RL\nA & B --> C\n"]
dict get [dict get $w meta] direction        ;# -> RL
lmap k [dict get $w edges] {dict get $k from}  ;# -> A B
```

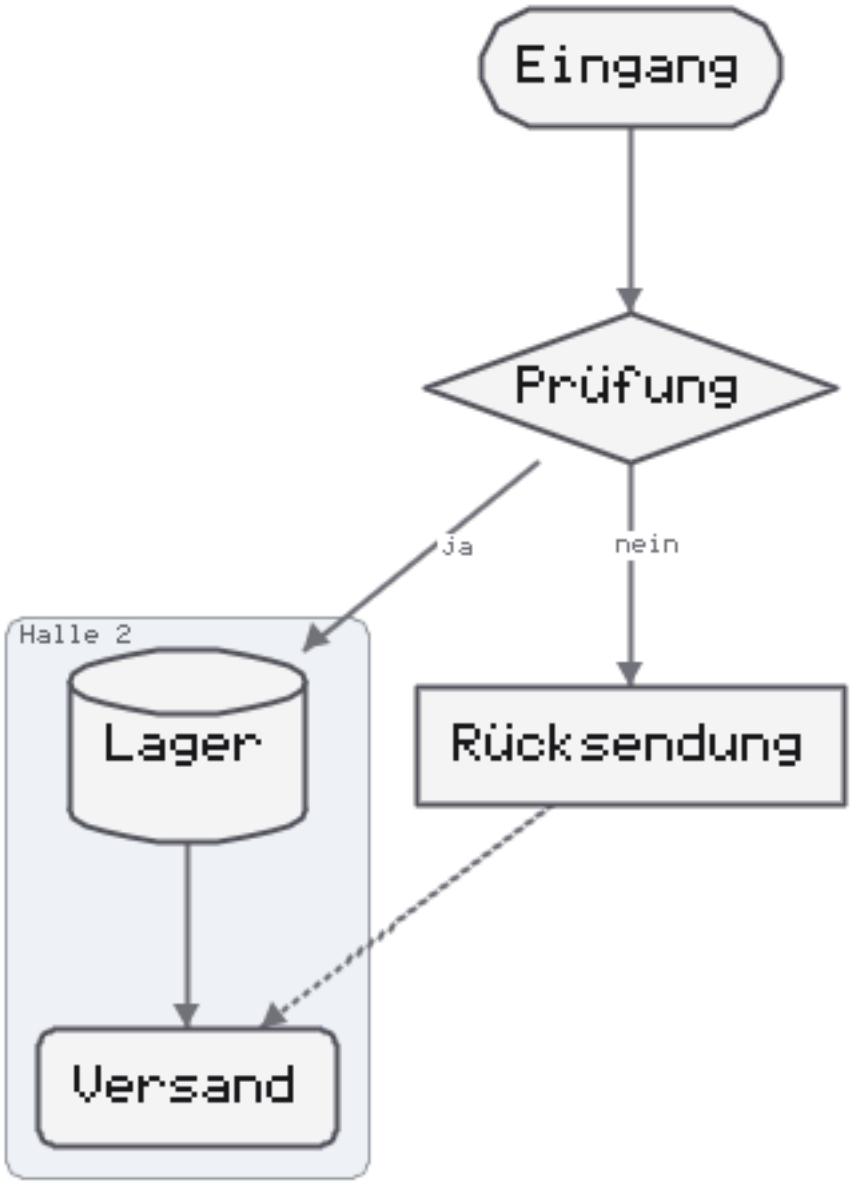
The nodes are `A` as `box` "Eingang", `B` as `diamond` "Pruefung" and `C` as `cylinder` "Lager". An edge is a dict with `from`, `to`, `label`, `style`, `arrow`, `startMark` and `endMark`.

```
set g {flowchart LR
    subgraph kasten [Kasten]
        A[a] --> B[b]
    end
}
dict get [::tclutils::tuflow::parse $g] groups
                 ;# -> {id kasten label Kasten members {A B}}
```

`A-3` throws `{TCLUTILS TUFLOW BADID}`. A text without nodes throws `{TCLUTILS TUFLOW EMPTY}`.

The following script produces the image below.

```
set fluss {flowchart TB
    A([Eingang]) --> B{Prüfung}
    B -->|ja| C[(Lager)]
    B -->|nein| D[Rücksendung]
    subgraph halle [Halle 2]
        C --> E(Versand)
    end
    D -.-> E
}
::tclutils::tuflow::writePng $fluss 056-fluss.png  ;# -> 056-fluss.png
```



Flowchart with group, 370 × 494 pixels

15.4 State Diagram: tustate

tustate reads `stateDiagram` and `stateDiagram-v2`. A state becomes a `rounded` node, a transition an edge with an arrowhead. The direction is `TB`.

Notation	Result
A --> B and A --> B : Text	edge, text as label
[*] --> A	start point __start as dot
A --> [*]	end point __end as dot
A : Text	label of state A; the id may have umlauts (Büro : Jörg)
A alone	state without an edge
state X <<fork>>, <<join>>	state X as rounded
direction LR	LR; RL becomes LR, BT and TD become TB

Not read:

- `state X { … }`: the frame is dropped, the transitions inside stay;
- `note …: Text` and `note … end note`: skipped;
- `state "Lang" as L`: skipped, `L` is only created with the first edge;
- an edge from `A` to `A`: in the model, not drawn.

```
set st {stateDiagram-v2
    [*] --> Bereit
    Bereit --> Laeuft : start
    Laeuft --> [*]
    Bereit : wartet
}
set m [::tclutils::tuflow::parse $st]
lmap n [dict get $m nodes] {dict get $n label}
                                ;# -> __start wartet Laeuft __end
lmap n [dict get $m nodes] {dict get $n shape}
                                ;# -> dot rounded rounded dot
```

A text without a state throws `{TCLUTILS TUSTATE EMPTY}`.

15.5 Class Diagram: tuclass

tuclass reads `classDiagram` and `classDiagram-v2`. Each class becomes a `box` node with three parts: name, attributes, methods. A line `----------` separates the parts. A member with `(` is a method, all others are attributes. The direction is `TB`.

Notation	Result
class X { … } over several lines	class with one member per line
class X { +a; +b() }	block on one line, ; separates the members; not inside quotes (0.3)

Notation	Result
class X, class X { }	class without members
X : +int alter, X : +laut()	member of X
A <|-- B	edge B to A, label extends
A *-- B	edge A to B, composition
A o-- B	aggregation
A ..> B	dependency, dashed
A ..|> B	extends, dashed
A --> B	without label
A --> B : Text	text as label
A "1" --> "*" B	edge A to B; the cardinalities are dropped

If the arrow starts with `<`, the edge points to the left class. Every edge has a plain arrowhead; the module does not draw the UML heads (triangle, diamond). A member before the closing brace (`+b()  }`) counts.

Not read:

- `class X~T~`, `class X:::k`, `<<interface>> X`: skipped.

```
set cl {classDiagram
    class Tier {
        +String name
        +laut()
    }
    Tier <|-- Hund
}
set m [::tclutils::tuflow::parse $cl]
split [dict get [lindex [dict get $m nodes] 0] label] \n
           ;# -> Tier ---------- {+String name} ---------- +laut()
lrange [lindex [dict get $m edges] 0] 0 5
                            ;# -> from Hund to Tier label extends
set k [::tclutils::tuflow::parse \
    "classDiagram\nclass Ort { }\nKunde \"1\" --> \"*\" Ort : hat\n"]
lmap n [dict get $k nodes] {dict get $n label} ;# -> Ort Kunde
dict get [lindex [dict get $k edges] 0] label   ;# -> hat
```

A text without a class throws `{TCLUTILS TUCLASS EMPTY}`.

15.6 ER Diagram: tuer

tuer 0.4 reads `erDiagram`. An entity becomes a `box` node with the name on the first line and one attribute on each further line. A relationship becomes an edge without an arrowhead, with crow's feet at both ends. The direction is `TB`.

Notation	Result
NAME { … }	entity; attribute line typ name
typ name PK	attribute typ name (PK); further words are dropped
A ||--o{ B : Text	solid edge, label Text
A }|..|| B : Text	dashed edge
NAME alone	entity without attributes

Each side of the cardinality becomes a mark: `{` or `}` means many, `o` means zero. `||` becomes `exactlyOne`, `|o` and `o|` `zeroOrOne`, `}|` and `|{` `oneOrMany`, `}o` and `o{` `zeroOrMany`. Quotation marks around the label are removed: `: "gibt auf"` becomes the label `gibt auf`.

An entity name consists of letters of any script, digits, `_` and `-`, so `Büro {` works too. Three words are a relationship only when the middle one contains `--` or `...`. Lines with `direction`, `title`, `accTitle` and `accDescr` are skipped; any other line that is neither a block nor an entity nor a relationship throws `{TCLUTILS TUER SYNTAX}`. Up to 0.3 `Büro {` was not recognised, and the attribute line `string name PK` became an edge from `string` to `PK`.

```
set er "erDiagram
    KUNDE ||--o\{ AUFTRAG : gibt
    KUNDE \{
        string name PK
    \}
"
set m [::tclutils::tuflow::parse $er]
set e [lindex [dict get $m edges] 0]
dict get $e startMark                          ;# -> exactlyOne
dict get $e endMark                            ;# -> zeroOrMany
split [dict get [lindex [dict get $m nodes] 0] label] \n
                           ;# -> KUNDE {string name (PK)}
```

A text without an entity throws `{TCLUTILS TUER EMPTY}`.

15.7 Requirements: turequirement

turequirement reads `requirementDiagram`. Each requirement and each element becomes a `box` node. The first line names the kind in `<<…>>`, the second the name, followed by the fields `id`, `text`, `type`, `risk`, `verifymethod` and `docref` in this order, as far as present. The direction is `TB`.

Notation	Result
requirement n { … }	node <<requirement>>
functionalRequirement, performanceRequirement, interfaceRequirement, physicalRequirement, designConstraint	likewise, with this kind

Notation	Result
element n { … }	node <<element>>
feld: wert in the block	field line; fields other than the six are dropped
a - satisfies -> b	edge a to b, label satisfies
b <- derives - a	edge a to b

A relationship to an unknown name creates a node with that name. Long field values are not wrapped; the box becomes wide. All kinds draw the same shape.

```
set rq {requirementDiagram
    requirement schnell {
        id: 1
        risk: high
    }
    element server {
        type: simulation
    }
    server - satisfies -> schnell
}
set m [::tclutils::tuflow::parse $rq]
split [dict get [lindex [dict get $m nodes] 0] label] \n
           ;# -> <<requirement>> schnell {id: 1} {risk: high}
dict get [lindex [dict get $m edges] 0] label  ;# -> satisfies
```

A text without a requirement and element throws `{TCLUTILS TUREQUIREMENT EMPTY}`.

15.8 Mind Map: tumindmap

tumindmap reads `mindmap`. Indentation determines the level: a line indented deeper than the previous one is its child; a shallower line attaches to the nearest ancestor with smaller indentation. A tab counts as four spaces. The ids are generated (`n1`, `n2`, …), the edges have no arrowhead. The result is a top-down tree (TB), not a radial layout.

Notation	Shape
Text	rounded
id((Text)), ((Text))	circle
id{{Text}}	hexagon
id[Text]	box
id(Text)	rounded
)Text(and))Text((	rounded

A bracket counts only if it encloses the whole text; `API (REST)` stays text. Lines with `::icon(…)` or `class` are dropped. A second line at the level of the root becomes a second root.

```
set mm {mindmap
    root((Lager))
        Eingang
        [Ausgang]
}
set m [::tclutils::tuflow::parse $mm]
lmap n [dict get $m nodes] {dict get $n shape}
                                ;# -> circle rounded box
lmap e [dict get $m edges] {dict get $e to}  ;# -> n2 n3
```

A text without a line after the header throws `{TCLUTILS TUMINDMAP EMPTY}`.

15.9 C4: tuc4

tuc4 reads `C4Context`, `C4Container`, `C4Component`, `C4Dynamic` and `C4Deployment`. The direction is `TB`. An element `Typ(id, "Name", …)` becomes a node with the id, and the first quoted text as its label; technology and description are dropped. The id consists of letters, digits and `_`, umlauts included: `Person(größe, "Weg")` has the id `größe` (up to tclutils 0.68.0 `gr`). The fill color comes from the type.

Type	Shape	Color
Person, Person_Ext	rounded	violet
System, Container, Component	box	blue, green, orange
type with Db, such as SystemDb, ContainerDb	cylinder	by type
type with _Ext	as above	gray
Rel(a, b, "Text"), BiRel, Rel_U, Rel_D, Rel_L, Rel_R	edge a to b with arrowhead	
title Text	title, centered above the graph	

Not read:

- boundaries such as `System_Boundary(…) { … }` and `Deployment_Node`: the frame is dropped, the elements inside stay;
- `BiRel` gets only one arrowhead; the direction in `Rel_U` to `Rel_R` is dropped;
- `UpdateElementStyle`, `UpdateRelStyle` and all lines starting with `Update`: skipped.

A relationship to an unknown id creates a `box` node without color.

```
set c4 {C4Context
    title Lager
    Person(kunde, "Kunde")
    System(lag, "Lager")
    Rel(kunde, lag, "holt")
}
set m [::tclutils::tuflow::parse $c4]
dict get [dict get $m meta] title            ;# -> Lager
dict get [lindex [dict get $m nodes] 0] style
                                  ;# -> fill #e8def8 stroke #6a1b9a
```

A C4 text without elements throws `{TCLUTILS TUC4 EMPTY}`.

15.10 Blocks: tublock

tublock reads `block-beta`. Each word of a line is a block; brackets and quotation marks keep spaces together. The shapes match the flowchart: `a["Text"] box`, `a("Text") rounded`, `a(("Text")) circle`, `a{"Text"} diamond`, `a{{"Text"}} hexagon`, `a(["Text"]) stadium`, `a[("Text")] cylinder`, `a[["Text"]] box`. The blocks are laid out as a top-down graph (TB), not in a grid. Ids may have umlauts: `Büro[Jörg]` is the block `Büro` with the label `Jörg` (up to tclutils 0.68.0 one single token).

Notation	Result
a --> b, a ==> b, a -.-> b	edge with arrowhead
a --- b	edge without arrowhead
a -->|Text| b, a -- Text --> b	edge with label
a["Text"]:2	the span :2 is dropped
columns N, space, space:N	skipped
block:id … end	the frame is dropped, the blocks inside stay
style, classDef, class, click	skipped

Arrow blocks such as `a<["Text"]>(right)` become a node with exactly this character sequence as its id.

```
set bl {block-beta
    columns 3
    a["Eingang"] b(("Prüfung")) c["Lager"]
    a --> b
    b -->|ok| c
}
set m [::tclutils::tuflow::parse $bl]
lmap n [dict get $m nodes] {dict get $n shape}
                                  ;# -> box circle box
dict get [lindex [dict get $m edges] 1] label  ;# -> ok
```

A text without a block throws `{TCLUTILS TUBLOCK EMPTY}`.

15.11 Git History: tugit

tugit reads `gitGraph`. Each commit becomes a node, each parent relationship an edge from the parent to the child commit. The first branch is called `main`. The direction is `LR`, with `gitGraph TB:` `TB`. Each branch gets a fill color; the module does not draw lanes as Mermaid does.

Notation	Result
commit	node c1, c2, … as rounded
commit id: "X"	node X
commit tag: "T"	label with [T]
branch name	new branch from the current state, becomes current
checkout name	switch branch
merge name	box "merge name" with two parents
cherry-pick id: "X"	rounded "pick X"

`commit type: …` is skipped. A commit id may occur only once, as in Mermaid: a second `commit id: "X"` or a `merge … id: "X"` with a taken id throws `{TCLUTILS TUGIT DUPID}` (up to tclutils 0.68.0 the second commit was dropped and an edge ran from `X` to itself). Generated ids (`c1`, `c2`, …) skip ids the text already uses.

```
set gg {gitGraph
    commit
    branch dev
    commit
    checkout main
    merge dev
}
set m [::tclutils::tuflow::parse $gg]
lmap n [dict get $m nodes] {dict get $n label}
                                ;# -> c1 c2 {merge dev}
llength [dict get $m edges]                ;# -> 3
```

A text without a commit throws `{TCLUTILS TUGIT EMPTY}`, a duplicate commit id `{TCLUTILS TUGIT DUPID}`.

15.12 Architecture: tuarchitecture

tuarchitecture reads `architecture-beta` and `architecture`. Services become nodes, groups become frames. The direction is `LR`.

Notation	Result
service id(icon)[Text]	node, shape by icon; the id may have umlauts

Notation	Result
service id(icon)[Text] in g	node in group g
group id(icon)[Text]	frame around its members
group id(icon)[Text] in g	nested group
junction id	node dot
a:R -- L:b	edge without arrowhead
a:R --> L:b, <--, <-->	arrowhead at the target, at the start, at both
a{group}:R -- L:b	{group} is dropped, the edge stays

The icon determines the shape: `database`, `db`, `disk`, `storage` yield `cylinder`, `cloud` and `internet` `rounded`, all others `box`. The sides `L`, `R`, `T`, `B` are read but not used; `tudiagram` determines where edges attach. An end that is not declared as a service becomes a `box`. The label is the text in `[…]`, without it the id.

```
set ar {architecture-beta
    group api(cloud)[API]
    service db(database)[Datenbank] in api
    service web(server)[Web] in api
    web:R --> L:db
}
set m [::tclutils::tuflow::parse $ar]
lmap n [dict get $m nodes] {dict get $n shape}  ;# -> cylinder box
dict get $m groups      ;# -> {id api label API members {db web}}
```

A text without a service throws `{TCLUTILS TUARCHITECTURE EMPTY}`.

15.13 Sequence Diagram: tusequence

tusequence reads `sequenceDiagram` and draws on its own, without `tudiagram`. The surface follows from the content (`-width 0 -height 0`); a number above 0 fixes it.

Notation	Result
participant A, participant A as Text	participant, in order of mention
actor A as Text	participant, drawn as a box
A->>B: Text	solid, filled arrowhead
A-->>B: Text	dotted, filled arrowhead
A->B, A-->B	open arrowhead
A-)B, A--)B	open arrowhead
A-xB, A--xB	cross
A<<->>B, A<<-->>B	filled arrowheads at both ends, solid or dotted

Notation	Result
A->>+B: Text, A->>-B: Text	activates B, deactivates A
activate A, deactivate A	activation bar
Note left of A: Text, right of, over A,B	note
loop, opt, alt, par, critical, break, rect … end	frame across the full width
else, and, option	dividing line in the frame
autonumber	messages with 1., 2., …
title Text	title, centered above the participants

A participant is also created by the first message or note that names it. A message needs the colon; `A->>B` without `:` is skipped.

Not read:

- `create`, `destroy`, `box`, `link`: skipped;
- stick figures for `actor`.

A title moves the drawing down by a band of 24 pixels; if it is wider than the participants, the surface becomes wider.

`parse` returns a dict with `type sequence`, `participants` (ids in order), `labels` (id to label), `isActor`, `steps`, `autonumber` and `title`. Each step is a dict with `kind message`, `note`, `activate`, `deactivate`, `blockStart`, `blockElse` or `blockEnd`.

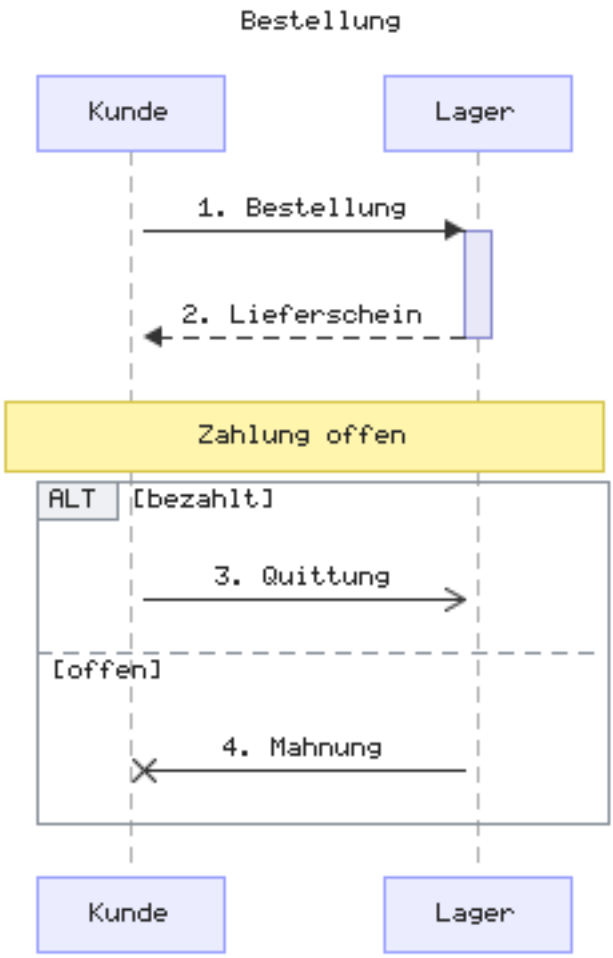
```
package require tclutils::tusequence
set sq "sequenceDiagram\nK->>+L: bestellt\nL-->>-K: liefert\n"
set m [::tclutils::tusequence::parse $sq]
dict get $m participants                    ;# -> K L
lindex [dict get $m steps] 0
        ;# -> kind message from K to L op ->> act + text bestellt
set bi "sequenceDiagram\nK<<->>L: Abgleich\n"
set b [::tclutils::tusequence::parse $bi]
dict get $b participants                    ;# -> K L
```

-errorcode	Cause
{TCLUTILS TUSEQUENCE EMPTY}	no participant
{TCLUTILS TUSEQUENCE ARG}	-scale not an integer from 1
{TCLUTILS TUSEQUENCE FONT}	-fontfile names no file

The following script produces the image below.

```
set seq {sequenceDiagram
    title Bestellung
    autonumber
    actor K as Kunde
    participant L as Lager
```

```
    K->>+L: Bestellung
    L-->>-K: Lieferschein
    Note over K,L: Zahlung offen
    alt bezahlt
        K-)L: Quittung
    else offen
        L-xK: Mahnung
    end
}
::tclutils::tuflow::writePng $seq 056-sequenz.png
                                                ;# -> 056-sequenz.png
```



Sequence diagram with title, 242 × 374 pixels

15.14 Gantt: tugantt

tugantt reads `gantt`. Each task is a row with a bar; the section is on the left, the name to the right of the bar. The default width is 800. Without `-height` the surface is 400 pixels high and grows when the tasks need more room: each row takes 26 pixels, and at 400 pixels 13 rows fit with a title. With `-height` the height stays fixed.

Notation	Result
title Text	title, centered at the top
dateFormat F	input format, default YYYY-MM-DD
axisFormat F	axis format, default as dateFormat
section Name	section, alternating background
Name : done, a1, 2026-03-02, 5d	task with tag, id, start, duration
Name : 2026-03-20, 2026-03-30	start and end as dates
Name : after a1, 3d	start at the latest end of the given ids
Name : 3d	start at the end of the previous task
tags active, done, crit	light blue, gray, red
tag milestone	yellow diamond in the middle of the span

`dateFormat` and `axisFormat` translate `YYYY`, `YY`, `MM`, `DD`, `HH`, `mm` and `ss` into `clock` formats; `%` formats such as `%d.%m.` stay as they are. `X` means seconds since 1970. Durations are a number and a unit: `ms`, `s`, `m` or `min`, `h`, `d`, `w`; any other unit throws `{TCLUTILS TUGANTT DURATION}` when drawing (since 0.3; up to 0.2 it counted as 0). An id may contain letters of any script (`büro`). The first task needs a start: without a date and without `after` referring to an earlier task, drawing throws `{TCLUTILS TUGANTT DATE}`, just as Mermaid does not draw a chart without a start date. The axis has up to six ticks, rounded to whole units of the axis format: seconds if it shows seconds, minutes for minutes, hours for hours, otherwise days. A tick with the same label as a previous one is dropped, so each date appears only once.

Not read: `excludes`, `todayMarker`, `tickInterval`, `weekday`, `vert` (skipped), `until`, excluding weekends, compact display.

`parse` does not check the dates yet. It returns `type gantt`, `title`, `dateFormat`, `axisFormat`, `haveSection` and `tasks`; each task is a dict with `title`, `section`, `tags`, `id`, `startTok` and `endTok`. The times are computed only when drawing.

```
package require tclutils::tugantt
set gt "gantt\nsection Bau\nRohbau :crit, b1, 2026-03-02, 2w\n"
set m [::tclutils::tugantt::parse $gt]
set a [lindex [dict get $m tasks] 0]
dict keys $a       ;# -> title section tags id startTok endTok
dict get $a startTok                          ;# -> 2026-03-02
dict get $a endTok                            ;# -> 2w
```

-errorcode	Cause
{TCLUTILS TUGANTT EMPTY}	no line with : after the header
{TCLUTILS TUGANTT DURATION}	duration with an unknown unit, such as 5x; only when drawing

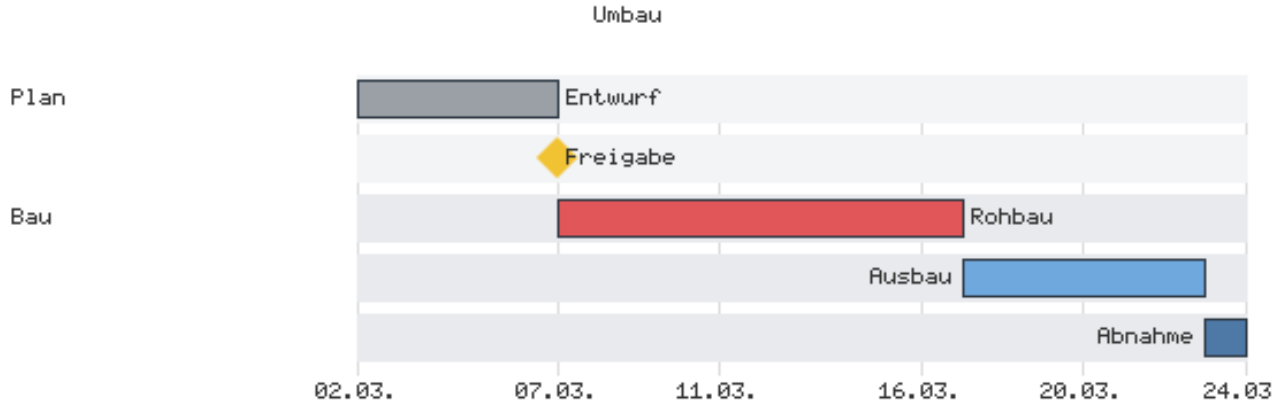
-errorcode	Cause
{TCLUTILS TUGANTT DATE}	date does not match dateFormat or does not exist, or the first task has no start; only when drawing
{TCLUTILS TUGANTT ARG}	-scale not an integer from 1
{TCLUTILS TUGANTT FONT}	-fontfile names no file

A date must exist: `tugantt` writes the parsed date back in the same format and compares it with the input, ignoring leading zeros. So `2026-13-45` and `2026-02-30` throw `DATE` alike under Tcl 8.6 and 9.0, while `2026-3-2` is accepted.

```
set g1 [::tclutils::tugantt::parse "gantt\nA : 3d\n"]
catch {::tclutils::tugantt::toSvg $g1} m o
dict get $o -errorcode                  ;# -> TCLUTILS TUGANTT DATE
set g2 [::tclutils::tugantt::parse "gantt\nA : 2026-13-45, 1d\n"]
catch {::tclutils::tugantt::toSvg $g2} m o
dict get $o -errorcode                  ;# -> TCLUTILS TUGANTT DATE
```

The following script produces the image below.

```
set gantt {gantt
    title Umbau
    dateFormat YYYY-MM-DD
    axisFormat %d.%m.
    section Plan
    Entwurf  :done, e1, 2026-03-02, 5d
    Freigabe :milestone, m1, after e1, 0d
    section Bau
    Rohbau   :crit, b1, after e1, 10d
    Ausbau   :active, after b1, 6d
    Abnahme  : 1d
}
::tclutils::tuflow::writePng $gantt 056-gantt.png \
    -width 560 -height 200                  ;# -> 056-gantt.png
```



Gantt chart, 560 × 200 pixels

15.15 Pie Chart: tupie

tupie reads `pie`. A line `"Beschriftung": Zahl` becomes a slice; the quotation marks may be omitted, the label extends to the last colon. Each slice shows its share in percent, and on the right is the legend with label and share.

Notation	Result
pie, pie title Text	header, title optional
pie showData	legend also shows the value
title Text	title on a line of its own
"Nord" : 42, Sued : 2.5	slice

A value is a decimal number, also with an exponent: `42`, `2.5`, `1e3`. Leading zeros count as decimal, `010` is 10. A negative value throws `{TCLUTILS TUPIE VALUE}` with the line number. A line whose value is not a number is not a slice. A slice with 0 stays in the model. The colors repeat after ten slices.

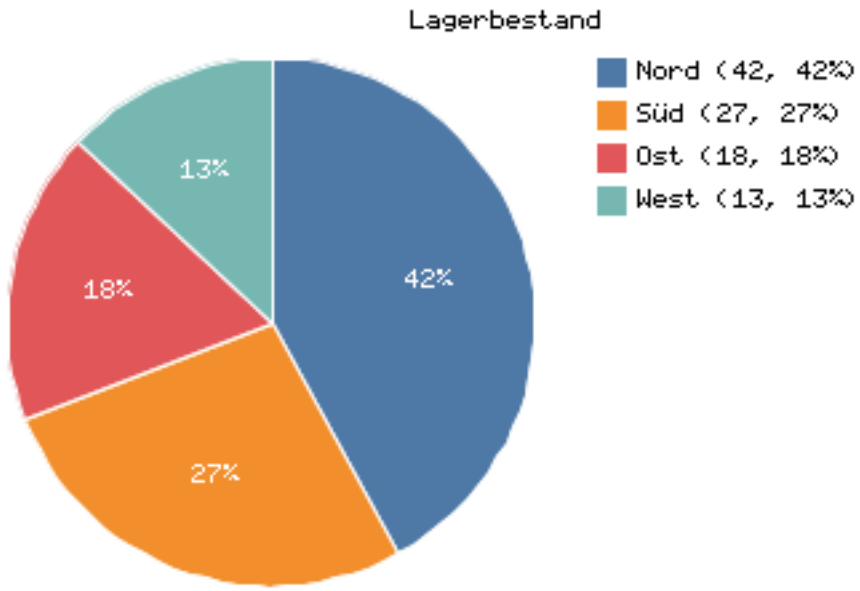
Option	Default	Effect
-width	480	width in pixels
-height	320	height in pixels
-legend	1	0 omits the legend
-scale	1	factor, integer from 1
-fontfile	empty	TTF or OTF, PNG only

```
package require tclutils::tupie
set pt "pie title Lager\n\"Nord\" : 1\n\"Sued\" : 3\n"
::tclutils::tupie::parse $pt
             ;# -> title Lager showData 0 slices {{Nord 1} {Sued 3}}
dict get [::tclutils::tupie::parse "pie\nA : 1e3\n"] slices
                                                ;# -> {A 1e3}
catch {::tclutils::tupie::parse "pie\nA : 3\nB : -1\n"} m
set m              ;# -> line 3: negative pie slice value "-1"
```

-errorcode	Cause
{TCLUTILS TUPIE EMPTY}	no slice
{TCLUTILS TUPIE VALUE}	negative value, or sum not positive
{TCLUTILS TUPIE ARG}	-scale not an integer from 1
{TCLUTILS TUPIE FONT}	-fontfile names no file

The following script produces the image below.

```
set pie {pie showData title Lagerbestand
    "Nord" : 42
    "Süd" : 27
    "Ost" : 18
    "West" : 13
}
::tclutils::tuflow::writePng $pie 056-pie.png \
    -width 420 -height 240                    ;# -> 056-pie.png
```



Pie chart, 420 × 240 pixels

15.16 XY Chart: tuxychart

tuxychart reads `xychart-beta` with bar and line series. The default surface is 520 × 340.

Notation	Result
xychart-beta	header; an appended horizontal is kept in the model, the drawing is vertical
title "Text"	title
x-axis "Titel" [a, b, c]	axis with categories, title optional
x-axis "Titel" 1 --> 5	numeric axis from 1 to 5
y-axis "Titel" 0 --> 100	title and range, both optional
bar [1, 2, 3]	bar series, several allowed
line [1, 2, 3]	line series, several allowed

Without a range, `y-axis` chooses the scale itself. The title of the y axis is horizontal above the axis. Several bar series stand side by side per category. The facade does not recognize `xychart` without `-beta`.

```
package require tclutils::tuxychart
set xy {xychart-beta
    x-axis [Jan, Feb, Mar]
    y-axis "EUR" 0 --> 100
    bar [40, 65, 50]
}
set m [::tclutils::tuxychart::parse $xy]
dict get $m categories                        ;# -> Jan Feb Mar
list [dict get $m ymin] [dict get $m ymax]   ;# -> 0 100
dict get $m series             ;# -> {type bar values {40 65 50}}
```

-errorcode	Cause
{TCLUTILS TUXYCHART EMPTY}	no series
{TCLUTILS TUXYCHART VALUE}	value in a series not a number
{TCLUTILS TUXYCHART COUNT}	with categories: a series does not have one value per category (since 0.2)
{TCLUTILS TUXYCHART ARG}	-scale not an integer from 1
{TCLUTILS TUXYCHART FONT}	-fontfile names no file

15.17 Quadrant: tuquadrant

tuquadrant reads `quadrantChart`. Both axes run from 0 to 1. The default surface is 420 × 420.

Notation	Result
title Text	title
x-axis links --> rechts	labels of the ends; without --> only the left one
y-axis unten --> oben	likewise
quadrant-1 Text to quadrant-4 Text	1 top right, 2 top left, 3 bottom left, 4 bottom right
Name: [0.3, 0.6]	point; values outside 0..1 are clamped to the edge

Settings after the point such as `radius: 9` or `color:` are dropped; the points take colors from a fixed sequence. The label of the y axis is horizontal on the left. The header is not required: any text with at least one recognized line is valid.

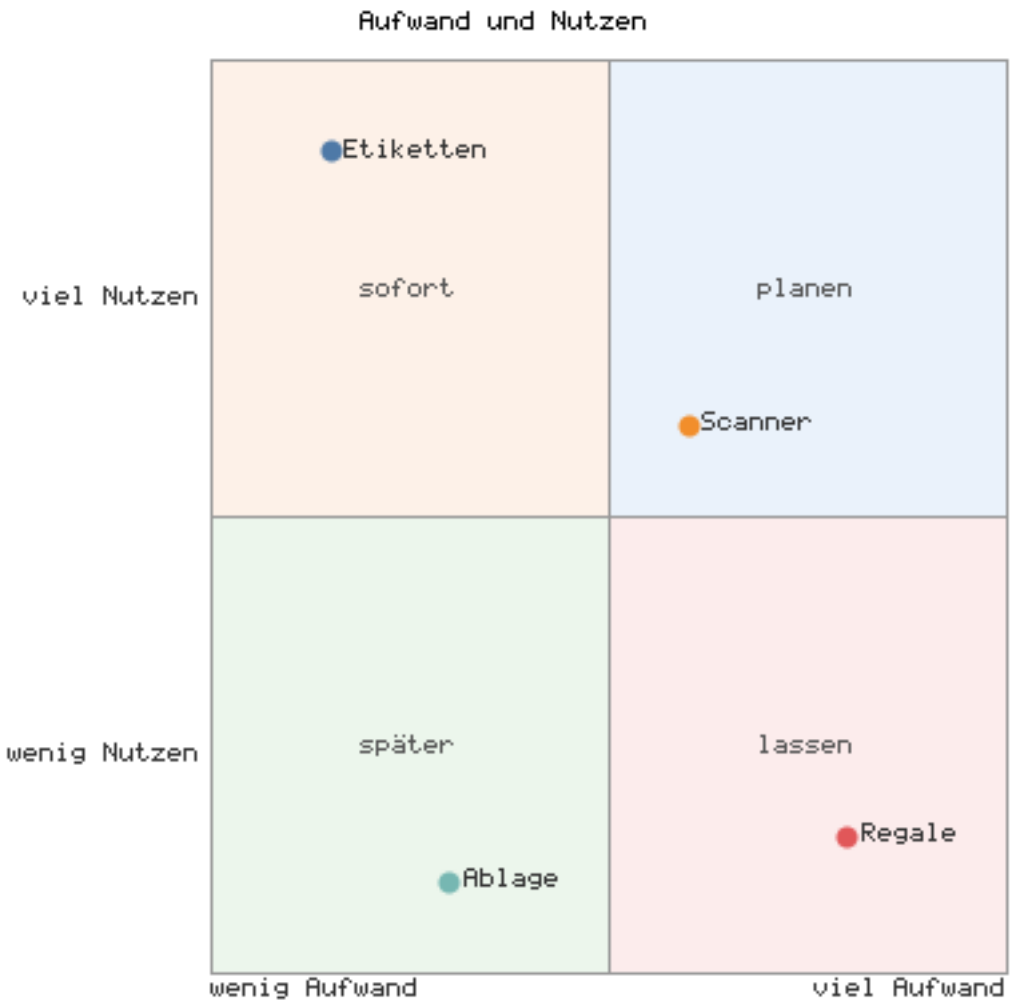
```
package require tclutils::tuquadrant
set q {quadrantChart
    x-axis wenig --> viel
    A: [0.3, 0.6]
    B: [1.4, -0.2]
```

```
}
set m [::tclutils::tuquadrant::parse $q]
list [dict get $m xleft] [dict get $m xright] ;# -> wenig viel
dict get $m points                ;# -> {A 0.3 0.6} {B 1.0 0.0}
```

-errorcode	Cause
{TCLUTILS TUQUADRANT EMPTY}	no recognized line
{TCLUTILS TUQUADRANT ARG}	-scale not an integer from 1
{TCLUTILS TUQUADRANT FONT}	-fontfile names no file

The following script produces the image below.

```
set quad {quadrantChart
    title Aufwand und Nutzen
    x-axis wenig Aufwand --> viel Aufwand
    y-axis wenig Nutzen --> viel Nutzen
    quadrant-1 planen
    quadrant-2 sofort
    quadrant-3 später
    quadrant-4 lassen
    Etiketten: [0.15, 0.9]
    Scanner: [0.6, 0.6]
    Regale: [0.8, 0.15]
    Ablage: [0.3, 0.1]
}
::tclutils::tuflow::writePng $quad 056-quadrant.png \
    -width 400 -height 400                  ;# -> 056-quadrant.png
```



Quadrant, 400 × 400 pixels

15.18 Journey: tujourney

tujourney reads `journey`. The tasks run from left to right in the order of the text; a line connects their scores. The sections are bands at the top, below each task the actors appear as colored dots, plus a legend. The default surface is 600 × 360.

Notation	Result
title Text	title
section Name	section for the following tasks
Aufgabe: 5: Ich, Du	task, score, actors
Aufgabe: 2	task without actors

The score is a number from 0 made of digits with an optional fractional part; `2.5` is valid. Mermaid scores from 1 to 5; values above 5 are valid, and the axis then extends to the next integer above the largest value. Any other score (`gut`, `-1`) throws `{TCLUTILS TUJOURNEY VALUE}` with the line number. Lines with `accTitle` and `accDescr` are dropped. Long names are horizontal and may overlap.

```
package require tclutils::tujourney
set jo {journey
    title Tag
    section Morgen
        Kaffee: 5: Ich, Du
        Bus: 2
}
set m [::tclutils::tujourney::parse $jo]
dict get $m tasks
              ;# -> {Kaffee 5 {Ich Du} Morgen} {Bus 2 {} Morgen}
dict get $m actors                            ;# -> Ich Du
set j [::tclutils::tujourney::parse "journey\nTee: 2.5\n"]
lindex [dict get $j tasks] 0 1                ;# -> 2.5
```

-errorcode	Cause
{TCLUTILS TUJOURNEY EMPTY}	no task
{TCLUTILS TUJOURNEY VALUE}	score not a number from 0
{TCLUTILS TUJOURNEY ARG}	-scale not an integer from 1
{TCLUTILS TUJOURNEY FONT}	-fontfile names no file

15.19 Timeline: tutimeline

tutimeline reads `timeline`. The time axis runs horizontally, the time points are below it, and their events are boxes above it. Sections are colored bands at the top. The default surface is 640 × 380.

Notation	Result
title Text	title
section Name	section for the following time points
2022 : Muster : Alpha	time point with two events
: Beta	another event for the previous time point

The text is split at a colon followed by whitespace or the end of the line. `12:30: Mittag` is the time point `12:30` with the event `Mittag`; `2020:Start` without a space stays a time point without an event. Long texts are not wrapped.

```
package require tclutils::tutimeline
set tl "timeline\n2021 : Idee\n2022 : Muster : Alpha\n     : Beta\n"
dict get [::tclutils::tutimeline::parse $tl] periods
               ;# -> {2021 Idee {}} {2022 {Muster Alpha Beta} {}}
set uhr "timeline\n12:30 : Mittag\n"
dict get [::tclutils::tutimeline::parse $uhr] periods
                                        ;# -> {12:30 Mittag {}}
```

-errorcode	Cause
{TCLUTILS TUTIMELINE EMPTY}	no time point
{TCLUTILS TUTIMELINE ARG}	-scale not an integer from 1
{TCLUTILS TUTIMELINE FONT}	-fontfile names no file

15.20 Sankey: tusankey

tusankey reads `sankey-beta` and `sankey`. The header is followed by CSV lines `Quelle,Ziel,Wert`. A name with a comma is put in quotation marks; a quotation mark inside it is doubled (""). The default surface is 700 × 400.

The nodes are placed in columns by the longest path from a source. The height of a node follows the larger value of inflow and outflow, the width of a band follows its value. A band has the color of its source.

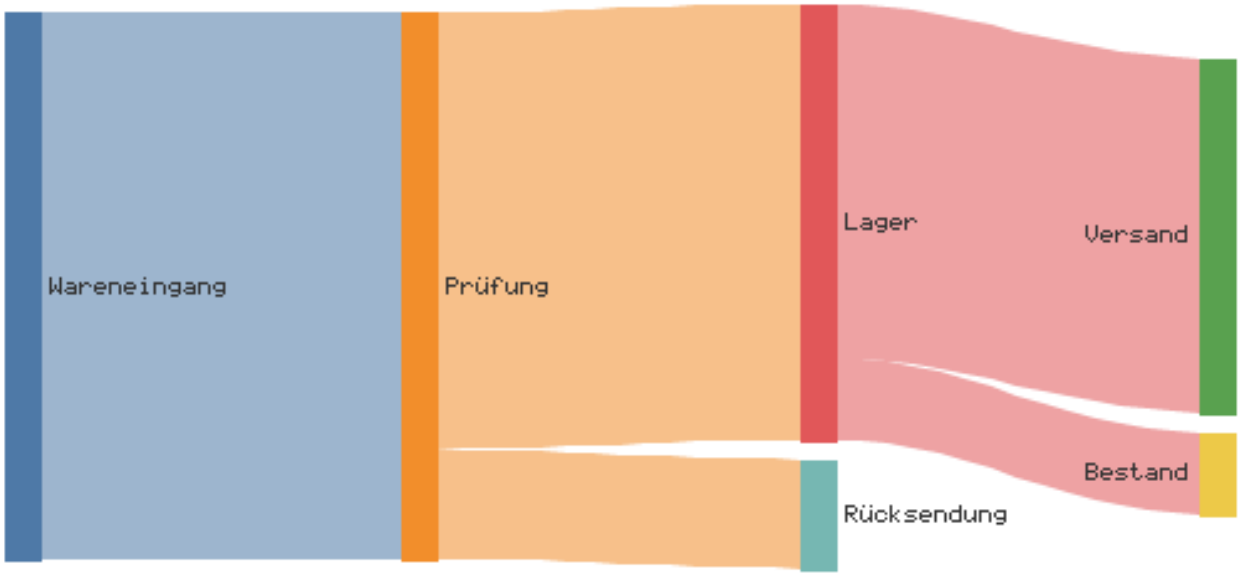
A line with fewer than three fields, an empty name or a value that is not a number or below 0 throws `{TCLUTILS TUSANKEY VALUE}` (since 0.2; up to 0.1 it was dropped silently). A line with the value 0 is dropped: it draws nothing, and a node that only occurs in such lines does not appear. Fields after the third are skipped. A cycle in the flow does not abort; the columns are then only approximate.

```
package require tclutils::tusankey
set sk "sankey-beta\nQuelle,Werk,10\n\"A,B\",Werk,2\nWerk,X,0\n"
set m [::tclutils::tusankey::parse $sk]
dict get $m nodes                              ;# -> Quelle Werk A,B
dict get $m links          ;# -> {Quelle Werk 10} {A,B Werk 2}
```

-errorcode	Cause
{TCLUTILS TUSANKEY EMPTY}	no valid line
{TCLUTILS TUSANKEY VALUE}	line not source,target,value with a number from 0
{TCLUTILS TUSANKEY ARG}	-scale not an integer from 1
{TCLUTILS TUSANKEY FONT}	-fontfile names no file

The following script produces the image below.

```
set sankey {sankey-beta
    Wareneingang,Prüfung,100
    Prüfung,Lager,80
    Prüfung,Rücksendung,20
    Lager,Versand,65
    Lager,Bestand,15
}
::tclutils::tuflow::writePng $sankey 056-sankey.png \
    -width 520 -height 260                     ;# -> 056-sankey.png
```



Sankey diagram, 520 × 260 pixels

15.21 Radar: turadar

turadar reads `radar-beta` and `radar`. Each axis starts at the center, the first one pointing up. Each series becomes a closed polygon with a semi-transparent fill. The default surface is 600 × 600.

Notation	Result
radar-beta title Text, title Text	title
axis a, b, c	axes; also over several lines
axis m["Mathe"]	axis with label
curve id["Name"]{1, 2, 3}	series, values in axis order
curve id{ b: 3, a: 1 }	series, values by axis id
max N, min N	scale; default min 0, max largest value
ticks N	number of rings, default 5
graticule circle, polygon	shape of the rings, default circle
showLegend true, false	legend

Every series needs exactly one value per axis. If the values are in axis order and there are more or fewer than axes, `parse` throws `COUNT`; if the key form leaves out an axis or a key names no axis, it throws `AXIS`. Up to tclutils 0.68.0 a missing value counted as 0, and extra or misspelled ones were dropped. Without `["Name"]` the series uses its id as its name. The legend appears when at least one series has a name in `["…"]`; `showLegend` sets it explicitly. Unknown lines are dropped. The first line must be the header, otherwise `EMPTY`. The lines are straight, not smoothed; `theme` and `config` are dropped.

```
package require tclutils::turadar
set rd {radar-beta
    axis t["Tcl"], c["C"], p
    curve a["Anna"]{4, 3, 2}
    max 5
}
set m [::tclutils::turadar::parse $rd]
dict get $m axes                              ;# -> Tcl C p
dict get $m curves                            ;# -> {Anna {4.0 3.0 2.0}}
dict get $m showLegend                        ;# -> 1
set r2 "radar-beta\naxis a, b, c\ncurve x{1, 2, 3}\n"
dict get [::tclutils::turadar::parse $r2] showLegend  ;# -> 0
```

-errorcode	Cause
{TCLUTILS TURADAR EMPTY}	header missing or no series
{TCLUTILS TURADAR AXES}	fewer than three axes
{TCLUTILS TURADAR AXIS}	key form without a value for an axis, or a key without an axis
{TCLUTILS TURADAR COUNT}	number of values in axis order does not match the axes
{TCLUTILS TURADAR VALUE}	value, min or max not a number
{TCLUTILS TURADAR ARG}	-scale not an integer from 1
{TCLUTILS TURADAR FONT}	-fontfile names no file

15.22 Treemap: tutreemap

tutreemap reads `treemap-beta` and `treemap`. The indentation determines the level; a tab counts as four spaces. Each leaf gets an area according to its value, a branch the sum of its leaves. The rectangles are as square as possible, using the squarify method. A branch below the root gets a header bar with its name, a leaf its name and value if there is room. The default surface is 640 × 400.

Notation	Result
"Name"	branch
"Name": 30	leaf with value
Name: 30, Name	likewise without quotation marks
"Name": 5:::klasse, classDef, style	style is dropped, also a class with umlauts (0.3)

A node with a value and children is a branch; the value is dropped. Texts are cut off, not wrapped. A leaf with a value of 0 or less is removed from the model, as is a branch that no longer has a leaf afterward. Sums and areas come only from the remaining leaves: with `-5, 10`

and `5`, `10` and `5` share the area in the ratio 2 to 1.

`parse` returns a tree of dicts with `name`, `value` and `children`. The root has the name `""`.

```
package require tclutils::tutreemap
set tm {treemap-beta
"Lager"
    "Nord": 30
    "Sued": 15
"Rest": 4
}
set m [::tclutils::tutreemap::parse $tm]
dict get $m value                              ;# -> 49.0
lmap k [dict get $m children] {dict get $k name}  ;# -> Lager Rest
set n [::tclutils::tutreemap::parse "treemap\nN: -5\nA: 10\nB: 5\n"]
dict get $n value                              ;# -> 15.0
```

-errorcode	Cause
{TCLUTILS TUTREEMAP EMPTY}	first line not a header, or no node
{TCLUTILS TUTREEMAP VALUE}	line not readable or value not a number
{TCLUTILS TUTREEMAP ARG}	-scale not an integer from 1
{TCLUTILS TUTREEMAP FONT}	-fontfile names no file

15.23 Bit Fields: tupacket

tupacket reads `packet-beta` and `packet`. Each line after the header is a field of bits. A row has 32 bits; a field that crosses the row boundary is split and keeps its color. The bit numbers appear above each part. Bits between the fields that no field covers appear in light gray. The width is 640, the height follows from the number of rows (`-height 0`).

Notation	Result
packet-beta title Text, title Text	title
0-15: "Quellport"	field bit 0 to 15
64: "F"	field of one bit
16-31: Zielport, 'Text'	label without or with single quotation marks

The fields are sorted by their first bit. The module does not know the relative notation `+16: "Text"` from newer Mermaid versions; it throws `VALUE`. Long labels are cut off.

```
package require tclutils::tupacket
set pk "packet-beta\n0-15: \"Quellport\"\n16-31: Zielport\n32: F\n"
set m [::tclutils::tupacket::parse $pk]
dict keys $m                        ;# -> title bitsPerRow fields
lrange [dict get $m fields] 1 2     ;# -> {16 31 Zielport} {32 32 F}
```

-errorcode	Cause
{TCLUTILS TUPACKET EMPTY}	first line not a header, or no field
{TCLUTILS TUPACKET VALUE}	line is not a field
{TCLUTILS TUPACKET RANGE}	end before start, or fields overlap
{TCLUTILS TUPACKET ARG}	-scale not an integer from 1
{TCLUTILS TUPACKET FONT}	-fontfile names no file

15.24 Kanban: tukanban

tukanban reads `kanban`. The first line after the header sets the indentation of the columns. Lines with this or smaller indentation are column titles, deeper lines are cards of the current column. Each column has a colored header, with the cards below it. The surface follows from the number of columns and the tallest stack of cards (`-width 0 -height 0`).

Notation	Result
Offen	column
Spec schreiben	card
API@{ ticket: T-1, priority: High }	card with an extra line

Of the settings in `@{ … }`, `priority` is drawn as a colored marker (`High` red, `Medium` orange, `Low` green, `Critical` and `Blocked` dark red, otherwise gray); all settings appear as a gray line below the text. The Mermaid form `id[Text]` is not split up: `todo[Erledigt]` appears as the title as is. Long texts are cut off. With a fixed `-height`, cards that no longer fit in the column are dropped.

```
package require tclutils::tukanban
set kb {kanban
    Offen
        Spec schreiben
        API@{ ticket: T-1, priority: High }
    Fertig
}
set cols [dict get [::tclutils::tukanban::parse $kb] columns]
lmap c $cols {lindex $c 0}                    ;# -> Offen Fertig
lindex $cols 0 1 1         ;# -> API {ticket: T-1  priority: High}
```

-errorcode	Cause
{TCLUTILS TUKANBAN EMPTY}	first line not a header, or no column
{TCLUTILS TUKANBAN ARG}	-scale not an integer from 1
{TCLUTILS TUKANBAN FONT}	-fontfile names no file

15.25 PNG and Fonts

Without `-fontfile`, PNG renders the labels with a 6×8 bitmap font; it contains the German umlauts. -fontfile names a TTF or OTF file; then `tupngdraw fillcontours` draws real outlines. This requires the package *Glyphs* (A. Buratti), which is not part of tclutils: put the directory with its `pkgIndex.tcl` on `auto_path`. If Glyphs is missing, the bitmap is used without an error. A font file that does not exist throws `{TCLUTILS TUDIAGRAM FONT}`, and for the kinds without a graph the module's `FONT` error. SVG ignores `-fontfile`; there the viewer chooses the font (`monospace`). The layout dimensions stay on the 6×8 grid in every case.

```
package require tclutils::tuflow
set t "flowchart LR\n  A --> B"
catch {::tclutils::tuflow::toPng $t -fontfile /gibt/es/nicht.ttf} m o
dict get $o -errorcode           ;# -> TCLUTILS TUDIAGRAM FONT
```

Under Tcl 9.0 and 8.6 the images in this chapter come out byte for byte identical. The models and errors match as well, including the date check in `tugantt`.

15.26 In short

- One entry point: `tuflow::toSvg`, `toPng`, `writeSvg`, `writePng` choose the kind by the first word.
- `tuflow::parse` returns only graphs, and `UNSUPPORTED` for every other kind.
- Graph PNG knows only `-scale` and `-fontfile`, graph SVG no option. The facade drops unknown options.
- Building returns a new dict. `validate` does not throw, `addEdge` does not check the ends.
- Titles in the `tudiagram` model and in `sequenceDiagram` are centered above the drawing.
- Unrecognized lines are mostly dropped without an error; `sankey` silently drops bad values, `pie` and `journey` throw `VALUE`.
- PNG font: without Glyphs the 6×8 bitmap stays, without an error.

Next chapter: Images without Tk: PNG, SVG, Colors

16 Images without Tk: PNG, SVG, Colors

The modules in this chapter write, read and inspect images in pure Tcl. They need neither Tk nor an extension, only the `zlib` command from the core and TclOO. This lets them run on a server without a display. Boxes and arrows from a model are drawn by `tudiagram`, see Diagrams; it builds on the surfaces `tupngdraw` and `tusvg` from this chapter.

Module	Version	Purpose
tucolor	0.2	color names, hex, HSV, nearest name
tupng	0.5	encode and decode PNG
tupngdraw	0.13	surface, writes PNG
tutablepng	0.2	table as PNG
tucodepng	0.1	character table as PNG

Module	Version	Purpose
tumonthpng	0.5	month, quarter, year as PNG
tupngpad	0.2	bring PNGs to one size
tusvg	0.3	SVG surface, SVG document, icons
tuimage	0.1	detect format and size, data: URI
tuico	0.2	Windows .ico with PNG content
tudhash	0.2	perceptual hash for similar images

An image occurs in four data forms. Each module states which one a command expects.

Form	Structure	Where
row list	list of rows, each row a list of pixels, all rows of equal length	tupng::encode*, write*
RGBA bytes	byte sequence of length width × height × 4, row by row, per pixel R G B A with 8 bits	tupng::decode, encodeRGBARaw, tupngdraw paste, tupngpad
number list	flat, row by row: width × height gray values or × 3 color values	tudhash
surface	TclOO object, the command name is the handle	tupngdraw::new, tusvg::new

The modules do not write colors uniformly. The table shows what each one accepts.

Notation	tucolor	tupng	tupngdraw	tusvg (object)
name	148 CSS/X11 names	no	15 names	passed through unchecked
#rgb	yes	no	no	passed through unchecked
#rrggbb	yes	yes	yes	passed through unchecked
RRGGBB, RRGGBBAA, #rrggbbaa	no	yes	yes	passed through unchecked
{r g b}	yes	yes	yes	rgb(r,g,b)
{r g b a}	no	yes	yes	rgba(r,g,b,a/255)

The 15 names of `tupngdraw` are `black`, `white`, `red`, `green`, `lime`, `blue`, `yellow`, `cyan`, `magenta`, `gray`, `grey`, `silver`, `orange`, `transparent` and `none`; the last two are `{0 0 0 0}`. For all other names, `tucolor::rgb` returns the triple.

16.1 Colors: tucolor

tucolor resolves a color into an RGB triple and converts to hex and HSV. The name table is built into the module; it matches the values of Tk but does not need Tk.

```
::tclutils::tucolor::rgb     farbe      ;# -> {r g b}, je 0..255
::tclutils::tucolor::hex     farbe      ;# -> #rrggbb
::tclutils::tucolor::names              ;# -> 148 Namen, sortiert
::tclutils::tucolor::exists  name       ;# -> 0 | 1
::tclutils::tucolor::nearest farbe      ;# -> nächster Name
::tclutils::tucolor::toHsv   farbe      ;# -> {h s v}
::tclutils::tucolor::fromHsv {h s v}    ;# -> {r g b}
```

`farbe` is a name, `#rgb`, `#rrggbb` or a triple `{r g b}` of decimal integers 0 to 255. Names are case-insensitive; leading and trailing spaces are dropped. `#rgb` doubles each digit (`#f80` is `#ff8800`). `nearest` looks for the smallest Euclidean distance in RGB space; on a tie, the name that comes first in the table wins (`aqua` before `cyan`, `gray` before `grey`). `toHsv` returns `h` in degrees 0 to 360, `s` and `v` in percent, all rounded. A round trip can differ by one unit per channel. `fromHsv` takes `h` from 0 to 360 and `s`, `v` from 0 to 100 as decimal numbers, also with fractional digits; 360 is the same as 0.

```
package require tclutils::tucolor
::tclutils::tucolor::rgb SteelBlue          ;# -> 70 130 180
::tclutils::tucolor::rgb #f80               ;# -> 255 136 0
::tclutils::tucolor::hex {255 136 0}        ;# -> #ff8800
::tclutils::tucolor::hex navy               ;# -> #000080
::tclutils::tucolor::exists " Navy "        ;# -> 1
::tclutils::tucolor::nearest #fe0000        ;# -> red
::tclutils::tucolor::nearest {0 255 255}    ;# -> aqua
::tclutils::tucolor::toHsv steelblue        ;# -> 207 61 71
::tclutils::tucolor::fromHsv {207 61 71}    ;# -> 71 131 181
::tclutils::tucolor::fromHsv {300 100 100}  ;# -> 255 0 255
::tclutils::tucolor::fromHsv {120.5 50 50}  ;# -> 64 128 64
::tclutils::tucolor::rgb {010 20 30}        ;# -> 10 20 30
llength [::tclutils::tucolor::names]        ;# -> 148
```

Every unrecognized color throws `{TCLUTILS TUCOLOR NAME}`, including `FF0000` without `#`, `{256 0 0}`, a quadruple `{255 0 0 255}`, `transparent` and a triple with a hex number, a sign or a fractional part such as `{0x10 0 0}`. A leading zero in the triple is not an octal number, the same under Tcl 8.6 and 9.0: `{08 0 0}` gives `8 0 0`. `fromHsv` throws `{TCLUTILS TUCOLOR HSV}` if the list does not have three elements, a value is not a number or lies outside its range, for example `{0 150 100}` or `{-60 100 100}`.

16.2 Encoding and Decoding PNG: tupng

tupng writes PNG files with 8 bits per channel in four color types and reads PNG files back into RGBA bytes. It is based solely on `zlib` from the core.

```
::tclutils::tupng::encodeRGB     bild ?option wert ...?
::tclutils::tupng::encodeRGBA    bild ?option wert ...?
::tclutils::tupng::encodeGray    bild ?option wert ...?
::tclutils::tupng::encodeIndexed palette bild ?option wert ...?
::tclutils::tupng::encodeRGBARaw bytes breite hoehe ?option wert ...?
::tclutils::tupng::writeRGB      datei bild ?option wert ...?
::tclutils::tupng::writeRGBA     datei bild ?option wert ...?
::tclutils::tupng::writeGray     datei bild ?option wert ...?
::tclutils::tupng::writeIndexed  datei palette bild ?option wert ...?
::tclutils::tupng::writeRGBARaw  datei bytes breite hoehe ?opt w ...?
::tclutils::tupng::decode        bytes
::tclutils::tupng::readPNG       datei
```

Command	Color type	Pixel in the row list
encodeRGB	2, truecolor	RRGGBB, #rrggbb, {r g b}; an alpha value is dropped
encodeRGBA	6, truecolor with alpha	as above, plus RRGGBBAA, {r g b a}; without alpha, 255 applies
encodeGray	0, grayscale	integer 0..255
encodeIndexed	3, palette	index into the palette

Option	Default	Effect
-compression	6	zlib level 0..9
-filter	best	best, none, sub, up, average, paeth

`best` tries all five PNG filters per row and takes the one with the smallest sum of absolute values. This costs time; `up` is considerably faster (figures at the end of this section).

`encode*` return the PNG bytes, `write*` the file name. `encodeRGBARaw` takes RGBA bytes instead of a row list and always writes color type 6. The palette of `encodeIndexed` has 1 to 256 entries in the notation of `encodeRGBA`. If an entry has alpha below 255, the module writes a `tRNS` chunk, otherwise not.

`decode` returns a dict:

Key	Content
width, height	size in pixels
colortype	color type of the file: 0, 2, 3, 4 or 6
bitdepth	always 8
rgba	RGBA bytes, length width × height × 4

Every color type is converted to RGBA. Grayscale becomes `g g g`. A `tRNS` color key makes, for grayscale and truecolor (type 2), every pixel with exactly this value transparent (alpha 0); all others get 255. For a palette, the alpha entry from `tRNS` applies. `decode` of `encodeRGBARaw` returns the same bytes. Before evaluating, `decode` checks the CRC of every chunk.

```
package require tclutils::tupng
set img {{FF0000 00FF00} {0000FF FFFFFF}}
set png [::tclutils::tupng::encodeRGB $img]
string range $png 1 3                          ;# -> PNG
set d [::tclutils::tupng::decode $png]
list [dict get $d width] [dict get $d colortype]  ;# -> 2 2
binary encode hex [dict get $d rgba]
                               ;# -> ff0000ff00ff00ff0000ffffffffffff
set pal {FF0000 0000FF00}
set png [::tclutils::tupng::encodeIndexed $pal {{0 1} {1 0}}]
binary encode hex [dict get [::tclutils::tupng::decode $png] rgba]
                               ;# -> ff0000ff0000ff000000ff00ff0000ff
set png [::tclutils::tupng::encodeGray {{0 128 255}}]
binary encode hex [dict get [::tclutils::tupng::decode $png] rgba]
                               ;# -> 000000ff808080ffffffffff
set raw [binary format cu* {255 0 0 255 0 0 255 128}]
set png [::tclutils::tupng::encodeRGBARaw $raw 2 1]
expr {[dict get [::tclutils::tupng::decode $png] rgba] eq $raw}
                               ;# -> 1
```

Errors when encoding:

-errorcode	**Cause**
{TCLUTILS TUPNG DIM}	no row, empty row, rows of unequal length; for encodeRGBARaw width or height not positive or wrong byte count
{TCLUTILS TUPNG COLOR}	pixel not readable, for example red or a gray value 300
{TCLUTILS TUPNG PALETTE}	palette empty or longer than 256
{TCLUTILS TUPNG INDEX}	index outside the palette
{TCLUTILS TUPNG OPT}	-compression not 0..9, -filter unknown
{TCLUTILS COMMON OPTION -level}	unknown option, here -level

Errors when decoding carry `{TCLUTILS TUPNG DECODE grund}`: `SIGNATURE` (no PNG signature), `IHDR` (header missing), `BITDEPTH` (not 8 bits), `INTERLACE`, `COLORTYPE`, `IDAT` (no image data), `FILTER` (unknown row filter). A chunk whose CRC does not match or that ends before its CRC throws `{TCLUTILS TUPNG CRC}`. If the CRC matches but the compressed data is invalid, `zlib` itself reports `{TCL ZLIB DATA}`.

Limits: the encoder writes only 8 bits, no interlacing, no ancillary chunks such as `gAMA`, `pHYs` or `tEXt`. The decoder rejects 16-bit images and palettes or grayscale with 1, 2 or 4 bits (`BITDEPTH`), as well as interlacing (`INTERLACE`). It skips ancillary chunks other than `tRNS` after the CRC check.

Run time (measured) on an Intel Xeon at 2.8 GHz, Tcl 9.0.4 and 8.6.14 (values for 8.6 in parentheses):

Operation	Image	Time
readPNG	1024 × 768 RGB, noise	1.48 s (1.45 s)
encodeRGBARaw -filter up	1024 × 768 RGBA	0.75 s (0.75 s)
encodeRGBARaw with best	1024 × 768 RGBA	4.5 s (4.9 s)
decode	256 × 256 RGBA, drawing	0.04 s (0.04 s)

This is enough for icons, tables and calendar sheets; for full-size photos, a tool written in C is the better choice.

16.3 Drawing: tupngdraw

tupngdraw is a surface as a TclOO object. It holds the pixels as an RGBA list and at the end outputs a PNG via `tupng::encodeRGBARaw`. Colors are blended by "source over": a semi-transparent fill is laid over whatever lies underneath.

```
set p [::tclutils::tupngdraw::new ?option wert ...?]
```

Option	Default	Effect
-width	100	width in pixels, integer from 1
-height	100	height in pixels, integer from 1
-background	white	color of the empty surface

After `new`, fill color and stroke color are black, stroke width 1, and antialiasing is on.

Method	Effect
width, height	size
pixel x y	{r g b a} at the position; coordinates are rounded
setfill farbe, setstroke farbe	fill and stroke color
setlinewidth n	stroke width, integer from 1
setantialias bool	antialiasing for strokes and fills
clear ?farbe?	whole surface anew, without a color using the background
setpixel x y ?-color f?	one pixel, blended
line x1 y1 x2 y2 ?...?	line segment
rect x1 y1 x2 y2 ?...?	rectangle, both corners included
circle cx cy r ?...?	circle
ellipse cx cy rx ry ?...?	ellipse
polygon {x0 y0 x1 y1 ...} ?...?	polygon, at least three points

Method	Effect
arc cx cy r a0 a1 ?...?	arc, pie slice or chord
fillcontours {kontur ...} ?...?	several contours as one area
text x y text ?...?	text in the built-in 6×8 font
textwidth text ?-scale n? ?-spacing s?	width in pixels
paste x y rgba b h ?-scale n?	insert RGBA bytes
data ?-compression n? ?-filter f?	PNG bytes
write datei ?-compression n? ?-filter f?	PNG file, returns the name
destroy	release the object

Options of the shapes:

Option	Default	Applies to	Effect
-fill	0	rect, circle, ellipse, polygon, arc	interior with the fill color
-outline	1	rect, circle, ellipse, polygon	outline with the stroke color
-color	stroke color	all strokes, setpixel, text, fillcontours	color for this call only
-fillcolor	fill color	shapes with -fill	fill color for this call only
-aa	surface value	strokes, fills, fillcontours	antialiasing for this call only
-width	stroke width	line, arc	width for this call only
-caps	round	line	round, butt, square
-join	round	rect, polygon, arc	round, bevel, mitre
-rx, -ry	0	rect	corner radius; -ry 0 takes over -rx
-style	arc	arc	arc, pie, chord
-rule	nonzero	fillcontours	nonzero or evenodd
-scale	1	text, textwidth, paste	each pixel as n × n
-spacing	0	text, textwidth	additional pixels per character

`arc` measures angles in degrees; 0 points to the right, increasing angles run clockwise on the image because y grows downward. `text` places the top left corner of the first character. The font knows ASCII 32 to 126 and `ä ö ü Ä Ö Ü ß`; other characters stay empty. `textwidth` is length × (6 + spacing) × scale; the spacing also counts after the last character. Without an argument, `data` and `write` filter with `-filter up`, not with `best`.

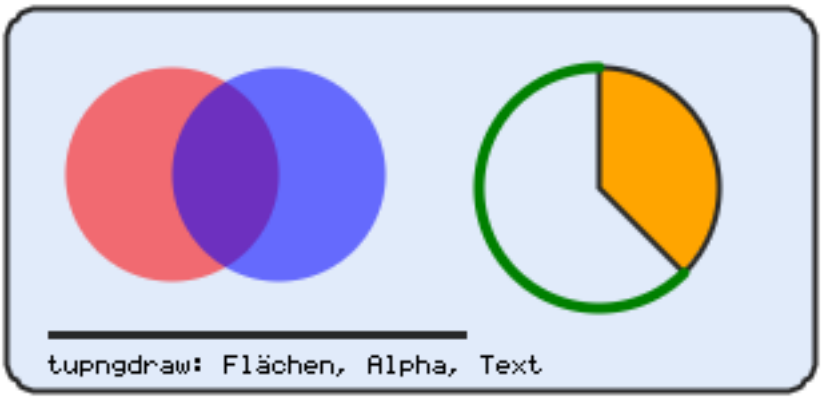
```
package require tclutils::tupngdraw
set p [::tclutils::tupngdraw::new -width 40 -height 20]
$p pixel 0 0                                ;# -> 255 255 255 255
$p setfill {255 0 0 128}
$p rect 0 0 9 9 -fill 1 -outline 0
$p pixel 5 5                                ;# -> 255 127 127 255
$p rect 20 2 23 5 -fill 1 -fillcolor black -outline 0
list [$p pixel 23 5] [$p pixel 24 6]
                              ;# -> {0 0 0 255} {255 255 255 255}
$p textwidth Hallo                          ;# -> 30
$p textwidth Hallo -scale 2 -spacing 1      ;# -> 70
string range [$p data] 1 3                  ;# -> PNG
$p destroy
```

`fillcontours` fills an area made of several contours. With `evenodd`, an inner contour becomes a hole; with `nonzero`, the winding direction decides, and two squares running the same way give a solid area.

```
set p [::tclutils::tupngdraw::new -width 20 -height 20]
set aussen {2 2 18 2 18 18 2 18}
set innen  {6 6 14 6 14 14 6 14}
$p fillcontours [list $aussen $innen] -color black -rule evenodd
list [$p pixel 10 10] [$p pixel 4 10]
                              ;# -> {255 255 255 255} {0 0 0 255}
$p destroy
```

The following script produces the image below it: a rectangle with rounded corners, two semi-transparent circles, a pie slice, an arc, a line and text.

```
package require tclutils::tupngdraw
set p [::tclutils::tupngdraw::new -width 320 -height 160]
$p setstroke {40 40 40}
$p setlinewidth 2
$p setfill {225 235 250}
$p rect 8 8 311 151 -fill 1 -rx 12
$p setfill {255 0 0 140}
$p circle 70 70 40 -fill 1 -outline 0
$p setfill {0 0 255 140}
$p circle 110 70 40 -fill 1 -outline 0
$p arc 230 75 45 -90 45 -style pie -fill 1 -fillcolor orange
$p arc 230 75 45 45 270 -style arc -width 4 -color {0 128 0}
$p line 24 130 180 130 -width 3 -caps butt
$p text 24 138 "tupngdraw: Flächen, Alpha, Text" -color black
$p write 058-zeichnung.png -compression 9
$p destroy
```



Drawing with tupngdraw, 320 × 160 pixels

Under Tcl 9.0 and under Tcl 8.6, the same file results byte for byte. Tcl 8.6 reads a script file in the system encoding; if that is not UTF-8, ä turns into a wrong character. The remedy is `source -encoding utf-8` or `\u00e4` in the text.

`paste` blends by the same rule as all other commands ("source over"): the resulting alpha is a_s + a_d × (1 − a_s); the colors are weighted accordingly (s source, d destination, alpha as a fraction 0 to 1). On an opaque surface the result is an opaque blend; on a transparent one the pixel stays as it is. Pixels with alpha 0 leave the surface unchanged.

```
set p [::tclutils::tupngdraw::new -width 2 -height 1 \
    -background transparent]
$p paste 0 0 [binary format cu* {255 0 0 128}] 1 1
$p setpixel 1 0 -color {255 0 0 128}
list [$p pixel 0 0] [$p pixel 1 0]
                              ;# -> {255 0 0 128} {255 0 0 128}
$p clear white
$p paste 0 0 [binary format cu* {255 0 0 128}] 1 1
$p pixel 0 0                                    ;# -> 255 127 127 255
$p destroy
```

-errorcode	Cause
{TCLUTILS TUPNGDRAW COLOR}	color not readable, including a name outside the 15, for example navy
{TCLUTILS TUPNGDRAW RANGE}	pixel outside the surface
{TCLUTILS TUPNGDRAW CAPS}, JOIN, STYLE, RULE	value of -caps, -join, -style, -rule unknown
{TCLUTILS TUPNGDRAW POLY}	polygon with fewer than three points
{TCLUTILS COMMON INTEGER name}	width, height, radius, stroke width or scale not from 1; name is for example width, radius, -linewidth
{TCLUTILS COMMON BOOLEAN name}	-fill, -aa or setantialias not a boolean
{TCLUTILS COMMON OPTION name}	unknown option

Every pixel is set in Tcl. An antialiased filled circle with radius 100 on 256 × 256 pixels takes, as measured, 0.14 s under Tcl 9.0.4 and 0.10 s under 8.6.14 (Xeon, 2.8 GHz); `data` with `-filter` up 0.04 s, with `-filter best` 0.33 s.

16.4 Table as an Image: tutablepng

tutablepng renders a list of rows as a table into a PNG: header row, alignment per column, alternating row color, grid. Drawing is done with `tupngdraw` in the 6×8 font.

```
::tclutils::tutablepng::render zeilen ?option wert ...?
::tclutils::tutablepng::write  datei zeilen ?option wert ...?
```

`zeilen` is a list of rows, each a list of cells. The widest row determines the number of columns; shorter rows get empty cells. `render` returns PNG bytes (RGBA, color type 6), `write` the file name.

Option	Default	Effect
-header	1	first row as header
-align	l	l, c, r for all columns or a list per column; missing entries take over the last one
-scale	1	font scale
-padding	6	margin inside the cell, pixels, integer from 0
-spacing	1	pixels between characters, integer from 0
-background	white	background
-gridcolor	{180 180 185}	grid
-textcolor	{25 25 25}	text
-headerbg	{230 230 238}	header row, background
-headertext	{0 0 0}	header row, text
-zebra	empty	color of every second data row; empty means off
-border	1	grid lines

The column width is the longest cell × (6 + `-spacing`) × `-scale` + 2 × `-padding`, the row height 8 × `-scale` + 2 × `-padding`. Colors follow `tupngdraw`. Each cell appears exactly as it stands in the list; `4.80`, `1e3`, `0x1F` or `010` also stay unchanged.

```
package require tclutils::tutablepng
package require tclutils::tuimage
set rows {{Artikel Menge Preis} {Schrauben 120 4,80} {Muttern 80}}
set png [::tclutils::tutablepng::render $rows -align {l r r}]
::tclutils::tuimage::dimensions $png         ;# -> 169 60
set png [::tclutils::tutablepng::render {{4.80 010}} -header 0]
::tclutils::tuimage::dimensions $png         ;# -> 73 20
```

The image below is produced by this script:

```
package require tclutils::tutablepng
set rows {
    {Artikel     Lager  Menge  Preis}
    {Schrauben   Nord   120    4,80}
    {Muttern     Nord   80     2,10}
    {Scheiben    Süd    450    1,95}
    {Dübel       Süd    60     3,25}
}
::tclutils::tutablepng::write 058-tabelle.png $rows \
    -scale 2 -align {l l r r} -zebra {245 245 248}
```

Artikel	Lager	Menge	Preis
Schrauben	Nord	120	4,80
Muttern	Nord	80	2,10
Scheiben	Süd	450	1,95
Dübel	Süd	60	3,25

Table with tutablepng, 384 × 140 pixels

`{}` as the row list and rows without any cells throw `{TCLUTILS TUTABLEPNG EMPTY}`. An unknown `-align` value throws `{TCLUTILS TUTABLEPNG ALIGN}`. `-header` and `-border` without a boolean throw `{TCLUTILS COMMON BOOLEAN ...}`, `-scale 0` throws `{TCLUTILS COMMON INTEGER -scale}`, an unreadable color `{TCLUTILS TUPNGDRAW COLOR}`. If `-padding` or `-spacing` is not an integer from 0, the result is `{TCLUTILS TUTABLEPNG PADDING}` or `{TCLUTILS TUTABLEPNG SPACING}`.

16.5 Character Table as an Image: tucodepng

tucodepng draws a code table: in each cell the hex value at the top, the character in the middle and, on request, the name at the bottom. The data come from `tucode`, see Binary Data and File Information.

```
::tclutils::tucodepng::render von bis ?option wert ...?
::tclutils::tucodepng::write  datei von bis ?option wert ...?
::tclutils::tucodepng::ascii  ?option wert ...?   ;# 0..127
::tclutils::tucodepng::latin1 ?option wert ...?   ;# 128..255
::tclutils::tucodepng::all    ?option wert ...?   ;# 0..255
```

Option	Default	Effect
-columns	16	cells per row
-scale	2	scale of the character in the middle
-theme	default	default, dark, light

Option	Default	Effect
-showcode	1	hex value at the top
-shownames	0	name at the bottom, for example NUL, auml
-title	empty	heading above the table
-textcmd	empty	custom command for the character in the middle

Control characters 0 to 31 and 127 show their abbreviation, 32 shows SP, each at scale 1. Hex value and name always use the 6×8 font. From 128 to 255 the font knows only ä ö ü Ä Ö Ü ß; the other cells stay empty in the middle unless `-textcmd` draws. The command is called for every printable character as `{*}$cmd bild x y b h zeichen farbe`; `bild` is the `tupngdraw` surface.

A cell is 28 × `-scale` wide, with names at least 48 pixels, and 12 × `-scale` high, plus 10 pixels for the hex value and 10 for the name. The heading is 12 × `-scale` high.

```
package require tclutils::tucodepng
package require tclutils::tuimage
set png [::tclutils::tucodepng::ascii]
::tclutils::tuimage::dimensions $png         ;# -> 896 272
set png [::tclutils::tucodepng::render 64 95 -columns 8 -scale 1]
::tclutils::tuimage::dimensions $png         ;# -> 224 88
```

`von` or `bis` not an integer, `von` greater than `bis` or a range outside 0..255 throw `{TCLUTILS TUCODEPNG RANGE}`. An unknown theme throws `{TCLUTILS TUCODEPNG THEME}`.

16.6 Month Sheet: tumonthpng

tumonthpng draws a month as a calendar sheet: title, weekday header, column with ISO calendar weeks, six weeks. Today, public holidays, notes, weekends and days of other months are highlighted. Quarter and year place several months side by side.

```
::tclutils::tumonthpng::render        jahr monat ?opt wert ...?
::tclutils::tumonthpng::write         datei jahr monat ?opt wert ...?
::tclutils::tumonthpng::renderQuarter jahr monat ?opt wert ...?
::tclutils::tumonthpng::writeQuarter  datei jahr monat ?opt wert ...?
::tclutils::tumonthpng::renderYear    jahr ?opt wert ...?
::tclutils::tumonthpng::writeYear     datei jahr ?opt wert ...?
```

`render*` return PNG bytes, `write*` the file name. Dates are ISO dates `JJJJ-MM-TT`.

Option	Default	Effect
-theme	default	default, dark, light
-scale	2	scale
-today	empty	the "today" day; empty means system date
-holidays	empty	dict date → name

Option	Default	Effect
-notes	empty	dict date → text; the day gets a dot
-firstweekday	1	first day of the week, 1 Monday to 7 Sunday; shifts grid and header
-showweeks	1	column "KW" (calendar week)
-weekdays	{Mo Di Mi Do Fr Sa So}	names for the header, always Monday to Sunday
-monthnames	{Januar ... Dezember}	month names in the title
-title	empty	render: title instead of "month year"; quarter and year: heading above the whole sheet
-select	empty	list of dates and ranges von..bis
-selectstyle	outline	outline, fill, both
-selectcolor	empty, then #1565C0	color of the selection
-selectwidth	empty, then max(2, scale)	frame width of the selection
-selectalpha	0.3	share of the selection color with fill
-textcmd	empty	custom command for every label
-cols	3	renderYear only: months per row

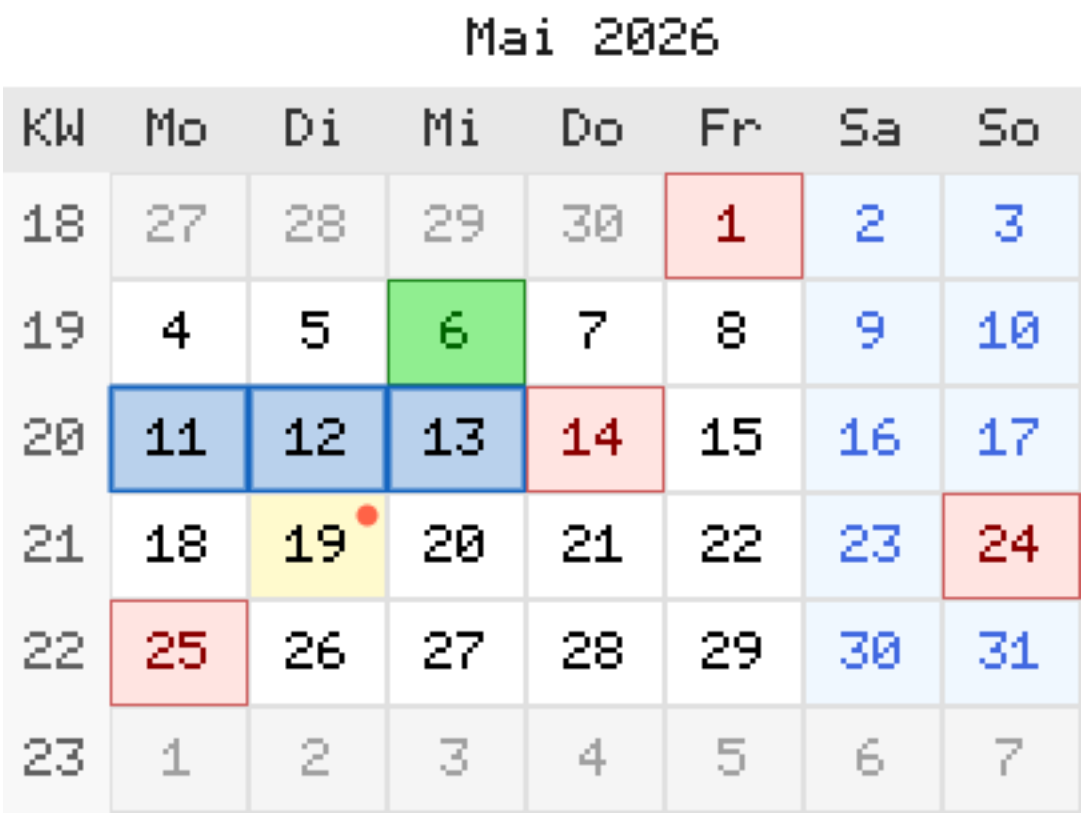
A day gets exactly one coloring, in this order: today, public holiday, note, other month, weekend, normal. The module draws the dot of a note in any case. The names in `-holidays` and the texts in `-notes` do not appear in the image; they only mark the day. The calculation runs in UTC via `clock`; the result does not depend on the time zone.

A month block is 7 × 26 × `-scale` wide, with the week column 20 × `-scale` more, and 154 × `-scale` high. Quarter and year put 10 × `-scale` between the blocks; a `-title` adds a strip of 18 × `-scale` across all blocks there. The year is an integer from 1 to 9999.

```
package require tclutils::tumonthpng
package require tclutils::tuimage
proc dims {png} { ::tclutils::tuimage::dimensions $png }
dims [::tclutils::tumonthpng::render 2026 6 -today 2026-06-06]
                                                ;# -> 404 308
dims [::tclutils::tumonthpng::render 2026 6 -scale 1 -showweeks 0]
                                                ;# -> 182 154
dims [::tclutils::tumonthpng::renderQuarter 2026 11 -scale 1]
                                                ;# -> 626 154
dims [::tclutils::tumonthpng::renderYear 2026 -scale 1 -cols 4]
                                                ;# -> 838 482
dims [::tclutils::tumonthpng::renderQuarter 2026 11 -scale 1 \
    -title "Q4 2026"]                           ;# -> 626 172
```

The quarter starting November 2026 shows November, December and January 2027. The month sheet below combines `tumonthpng` with the public holidays from `tuholiday` (Notes and Calendars):

```
package require tclutils::tumonthpng
package require tclutils::tuholiday
::tclutils::tumonthpng::write 058-monat.png 2026 5 \
    -today 2026-05-06 \
    -holidays [::tclutils::tuholiday::holidays 2026] \
    -notes {2026-05-19 Inventur} \
    -select {2026-05-11..2026-05-13} -selectstyle both
```



May 2026 with tumonthpng, 404 × 308 pixels

`-textcmd` is called as `{*}$cmd bild x y b h text farbe` for the title, weekdays, "KW", week numbers and day numbers, and draws the text itself, for example with real font outlines via `tupngdraw fillcontours`.

`-firstweekday` shifts grid and header together: with `-firstweekday 7`, the first column starts on Sunday, and the header shows `So Mo Di Mi Do Fr Sa`. `-weekdays` always lists the names from Monday to Sunday; the module rotates the list itself.

-errorcode	Cause
{TCLUTILS TUMONTHPNG MONTH}	month not 1..12
{TCLUTILS TUMONTHPNG YEAR}	year not an integer 1..9999, for example abc
{TCLUTILS TUMONTHPNG FIRSTWEEKDAY}	-firstweekday not 1..7
{TCLUTILS TUMONTHPNG THEME}	theme unknown

-errorcode	Cause
{TCLUTILS TUMONTHPNG SELECT}	entry in -select not JJJJ-MM-TT or range
{TCLUTILS TUMONTHPNG SELECTSTYLE}	-selectstyle unknown
{TCLUTILS COMMON BOOLEAN -showweeks}	-showweeks not a boolean
{TCLUTILS COMMON INTEGER -scale}, -cols	scale or columns not from 1

A `-today` such as `morgen` is not an error; it just matches no day.

16.7 Uniform Size: tupngpad

tupngpad brings several PNG files to a common size. Each image is cropped to its content, centered with a margin and laid on a background color. Typical input is cut-out images with a transparent background.

```
::tclutils::tupngpad::bbox  rgba b h               ;# -> {x0 y0 x1 y1}
::tclutils::tupngpad::trim  rgba b h               ;# -> {rgba b h}
::tclutils::tupngpad::padTo rgba b h zb zh ?option wert ...?
::tclutils::tupngpad::batch dateien zielordner ?option wert ...?
```

bbox finds all pixels with alpha above 0 and returns the corners inclusive; for a fully transparent image it returns the empty list. `trim` crops to this box; if nothing is visible, the input comes back unchanged. padTo places the content on a surface `zb` × `zh` and returns PNG bytes.

Option of padTo	Default	Effect
-background	white	color of the surface, notation as in tupngdraw
-align	center	center, nw, n, ne, w, e, sw, s, se
-filter	empty	PNG filter; empty means up

Option of batch	Default	Effect
-margin	4	margin in pixels, integer from 0
-background	white	as above
-trim	1	crop to the content first
-align	center	as above
-square	0	square output
-size	empty	{B H} for the inner area, integers from 1; empty means largest input image

batch reads each file with `tupng::readPNG`, writes it under the same name into the target directory (created if needed) and returns a dict: `size {b h}` and `files` with alternating input and output paths. All outputs have color type 6. A `-size` that is too small cuts off the content. Semi-transparent pixels are blended with the background, see `paste` under `tupngdraw`: on an opaque background they become opaque; with `-background transparent` they stay

semi-transparent.

```
package require tclutils::tupngpad
set px {}
for {set y 0} {$y < 3} {incr y} {
    for {set x 0} {$x < 4} {incr x} {
        if {$y == 1 && $x in {1 2}} {
            lappend px 255 0 0 255
        } else {
            lappend px 0 0 0 0
        }
    }
}
set rgba [binary format cu* $px]
::tclutils::tupngpad::bbox $rgba 4 3          ;# -> 1 1 2 1
lassign [::tclutils::tupngpad::trim $rgba 4 3] r w h
list $w $h                                    ;# -> 2 1
set png [::tclutils::tupngpad::padTo $r $w $h 6 4 -align nw]
set d [::tclutils::tupng::decode $png]
binary encode hex [string range [dict get $d rgba] 0 11]
                              ;# -> ff0000ffff0000ffffffffff
::tclutils::tupng::writeRGBARaw in1.png $rgba 4 3
set img2 {{00000000 00FF00} {00FF00 00FF00}}
::tclutils::tupng::writeRGBA in2.png $img2
set res [::tclutils::tupngpad::batch {in1.png in2.png} aus -margin 2]
dict get $res size                            ;# -> 6 6
```

`batch` checks all options before it reads a file.

-errorcode	Cause
{TCLUTILS TUPNGPAD MARGIN}	-margin below 0 or not an integer
{TCLUTILS TUPNGPAD TRIM}	-trim not a boolean
{TCLUTILS TUPNGPAD SQUARE}	-square not a boolean
{TCLUTILS TUPNGPAD SIZE}	-size not empty and not {B H} with integers from 1
{TCLUTILS COMMON VALUE -align}	-align unknown; the message names the allowed values

Missing files and broken PNGs are reported by `open` (`{POSIX ENOENT ...}`) and `tupng::decode` with their own codes.

16.8 SVG: tusvg

tusvg generates SVG text. It has two interfaces: a surface `new`, whose methods resemble those of `tupngdraw`, and an older procedural interface with a document as a dict, gradients, groups and 149 icon names. The images in this book are PNG; SVG appears here only as text.

16.8.1 Surface

```
set c [::tclutils::tusvg::new ?-width n? ?-height n? ?-background f?]
```

Defaults as in `tupngdraw`: 100 × 100, background `white`. With `-background none` or empty, the background rectangle is omitted. The methods `width`, `height`, `setfill`, `setstroke`, `setlinewidth`, `rect`, `line`, `polygon`, `circle`, `ellipse`, `text`, `textwidth`, `data`, `write` and `destroy` take the same arguments as in `tupngdraw` (`-fill`, `-outline`, `-color`, `-fillcolor`, `-rx`, `-ry`, `-width`, `-caps`, `-scale`, `-spacing`). In addition:

Method	Effect
text ... -anchor a	start (default), middle, end
polyline punkte ?-color f? ?-width n?	open polyline
path d ?-fill? ?-outline? ?-color? ?-fillcolor?	SVG path
addDef text	entry in <defs>
icon name x y groesse ?...?	insert an icon, see below

`data` returns the SVG text; `write` writes it in UTF-8 and returns the file name. `textwidth` calculates as in `tupngdraw`; `text` also sets `textLength` so that the text takes exactly this width in the viewer. A box sized by `textwidth` thus stays the same size in SVG and PNG. Differences from `tupngdraw`: colors are not checked (`nichtsda` and `{999 0 0}` pass through); `-aa` and `-join` are unknown options; `arc`, `pixel`, `setpixel`, `clear`, `setantialias`, `paste` and `fillcontours` are missing; `rect` has the width `x2 − x1`, while `tupngdraw` counts both corner pixels, that is, one pixel more.

```
package require tclutils::tusvg
set c [::tclutils::tusvg::new -width 120 -height 40]
$c setfill {200 220 255}
$c rect 4 4 115 35 -fill 1 -rx 6
$c text 10 16 "A<B" -color navy
$c textwidth "A<B"                          ;# -> 18
puts [$c data]
$c destroy
```

Output:

```
<svg xmlns="http://www.w3.org/2000/svg" width="120" height="40"
 viewBox="0 0 120 40">
  <rect x="0" y="0" width="120" height="40" fill="white"/>
  <rect x="4" y="4" width="111" height="31" rx="6"
 fill="rgb(200,220,255)" stroke="black" stroke-width="1"/>
  <text x="10" y="23" font-family="monospace" font-size="8"
 textLength="18" lengthAdjust="spacingAndGlyphs"
 text-anchor="start" fill="navy">A&lt;B</text>
</svg>
```

The long lines are wrapped here; in the original, each element is on one line.

-errorcode	Cause
{TCLUTILS TUSVG COLOR}	color as a list with 2 or more than 4 elements
{TCLUTILS TUSVG CAPS}	-caps unknown
{TCLUTILS TUSVG ANCHOR}	-anchor unknown
{TCLUTILS TUSVG POLY}	polygon with fewer than three points
{TCLUTILS COMMON ...}	options, booleans, integers as in tupngdraw

The method `icon` takes the options of the procedure `icon` (`-color`, `-strokeWidth`). It inserts the icon as a nested `<svg>` element with `x`, `y`, width and height `groesse` and `viewBox="0 0 24 24"`, without an XML declaration.

```
set c [::tclutils::tusvg::new -width 40 -height 40]
$c icon save 4 4 16 -color navy
set d [$c data]
regexp -all {<svg} $d                          ;# -> 2
regexp {<svg x="4" y="4" width="16"} $d        ;# -> 1
$c destroy
```

16.8.2 Document as a Dict

```
set svg [::tclutils::tusvg::create breite hoehe ?-viewBox v? ?-id i?]
::tclutils::tusvg::rect        svg x y b h ?option wert ...?
::tclutils::tusvg::circle      svg cx cy r ?...?
::tclutils::tusvg::ellipse     svg cx cy rx ry ?...?
::tclutils::tusvg::line        svg x1 y1 x2 y2 ?...?
::tclutils::tusvg::polyline    svg punkte ?...?
::tclutils::tusvg::polygon     svg punkte ?...?
::tclutils::tusvg::path        svg d ?...?
::tclutils::tusvg::textElement svg x y text ?...?
::tclutils::tusvg::linearGradient svg id x1 y1 x2 y2 stops
::tclutils::tusvg::radialGradient svg id cx cy r stops
set g [::tclutils::tusvg::group svg ?-transform t? ?-opacity o?]
::tclutils::tusvg::addToGroup g element
::tclutils::tusvg::addGroup   svg $g
::tclutils::tusvg::toString   $svg               ;# -> SVG-Text
::tclutils::tusvg::write      $svg datei         ;# -> datei
```

Here `rect` differs from the surface: corner, width, height. The element procedures take the name of the dict variable and append to it. Their options are named like SVG attributes in camel case: `-fill`, `-stroke`, `-strokeWidth`, `-opacity`, `-id`, for lines `-strokeLinecap` and `-strokeLinejoin`, for text `-fontSize`, `-fontFamily`, `-fontWeight`, `-textAnchor`. The default for `-fill` is `#000000`; for `line`, `polyline` and `path` it is `none` with stroke `#000000`. Unknown options are silently ignored; values are not checked. `punkte` is given as text `"x,y x,y ..."` and goes into the output unchanged. The gradients go into `<defs>` and return `url(#id)`, suitable for `-fill`. A stop is `{offset farbe ?deckkraft?}`, offset and coordinates in percent. Elements go into a group when the element procedure is given the group

variable. `toString` starts with the XML declaration.

```
set svg [::tclutils::tusvg::create 40 20 -viewBox "0 0 40 20"]
::tclutils::tusvg::rect svg 2 2 36 16 -fill "#eef" -stroke navy -rx 3
::tclutils::tusvg::textElement svg 20 14 "Lager & Co" \
    -fontSize 10 -textAnchor middle
set g [::tclutils::tusvg::group svg -transform "translate(5,5)"]
::tclutils::tusvg::rect g 0 0 4 4
::tclutils::tusvg::addGroup svg $g
puts [::tclutils::tusvg::toString $svg]
```

Output:

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" width="40" height="20"
 viewBox="0 0 40 20">
  <rect x="2" y="2" width="36" height="16" rx="3" fill="#eef"
 stroke="navy" stroke-width="1"/>
  <text x="20" y="14" fill="#000000" font-size="10"
 font-family="sans-serif" text-anchor="middle">Lager &amp;amp; Co</text>
  <g transform="translate(5,5)">
    <rect x="0" y="0" width="4" height="4" fill="#000000"/>
  </g>
</svg>
```

16.8.3 Icons

```
::tclutils::tusvg::icons                        ;# -> Liste der Namen
::tclutils::tusvg::icon     name groesse ?-color f? ?-strokeWidth w?
::tclutils::tusvg::saveIcon name groesse datei ?-color f?
```

`icons` lists all 149 names that `icon` knows, including duplicate names such as `trash` and `delete` or `rotate-left` and `rotate_left`. `icon` returns a document dict with `viewBox "0 0 24 24"`; `toString` turns it into text. The default for `-color` is `#333333`; without an argument, the stroke width grows with the size (1.5 up to 16, 1.75 up to 24, 2.0 up to 32, otherwise 2.5). An unknown name gives a substitute icon with a question mark, without an error. The icons consist only of lines and areas, without `<text>`.

```
llength [::tclutils::tusvg::icons]            ;# -> 149
set ic [::tclutils::tusvg::icon save 16]
dict get $ic width                            ;# -> 16
dict get $ic viewBox                          ;# -> 0 0 24 24
```

16.9 Detecting the Image Format: tuimage

tuimage reads format, MIME type and size from the first bytes of an image. It does not decode any pixels. It also handles `data:` URIs.

```
::tclutils::tuimage::type       bytes    ;# -> png|jpeg|gif|bmp|webp|""
::tclutils::tuimage::mime       bytes    ;# -> image/png ... oder ""
::tclutils::tuimage::dimensions bytes    ;# -> {breite hoehe} oder {}
::tclutils::tuimage::inspect    bytes    ;# -> Dict
::tclutils::tuimage::dataUri    mime bytes
::tclutils::tuimage::fromDataUri uri     ;# -> {mime m bytes b}
```

Format	Signature	Size from
PNG	\x89PNG\r\n\x1a\n	IHDR
JPEG	\xff\xd8\xff	first SOF marker
GIF	GIF87a, GIF89a	logical screen descriptor
BMP	BM	info header; negative height becomes positive
WebP	RIFF ... WEBP	VP8 , VP8L or VP8X

`inspect` returns `type`, `mime`, `width` and `height`; for an unknown format all four are empty. The check is minimal: every string that starts with `BM` counts as BMP. If the file is too short for the header, `dimensions` stays empty. The bytes must have been read in binary mode.

`dataUri` puts `data:mime; base64,` in front of the Base64 form without checking the MIME type. `fromDataUri` decodes Base64 only if `base64` is in the header, not inside a quoted parameter such as `charset="utf-8;base64"` (0.2); otherwise the content comes back unchanged, without percent decoding. Invalid Base64 such as `!!!` gives empty bytes, not an error. Base64 is handled by `tubase64`, see Encoding, Hashing and Comparing.

```
package require tclutils::tuimage
package require tclutils::tupng
set png [::tclutils::tupng::encodeGray {{0 255}}]
::tclutils::tuimage::inspect $png
                        ;# -> type png mime image/png width 2 height 1
::tclutils::tuimage::type "BM irgendwas"     ;# -> bmp
::tclutils::tuimage::dimensions Hallo        ;# ->
set uri [::tclutils::tuimage::dataUri image/png $png]
string range $uri 0 21          ;# -> 
set r [::tclutils::tuimage::fromDataUri $uri]
expr {[dict get $r bytes] eq $png}           ;# -> 1
::tclutils::tuimage::fromDataUri "data:,Hallo%20Welt"
                               ;# -> mime {} bytes Hallo%20Welt
```

A string without `data:` at the start throws `{TCLUTILS TUIMAGE DATAURI}`. The other commands do not throw.

16.10 Windows Icons: tuico

tuico writes and reads `.ico` files. The entries are PNG images, which Windows accepts in icons since Vista and which carry an alpha channel. The module does not convert any pixels; the PNGs come, for example, from `tupngdraw`. All commands are also available as the ensemble `tclutils::tuico write|info|extract`.

```
::tclutils::tuico::write   datei eintraege         ;# -> Anzahl Bytes
::tclutils::tuico::info    datei                   ;# -> Dicts
::tclutils::tuico::extract datei groesse ?ziel?    ;# -> PNG-Bytes
```

`eintraege` is a list of pairs `{groesse png}`. The size is the edge length 1 to 256; the PNG must be exactly that size and square. `write` checks all entries before it writes; an error leaves no half-written file behind. The entries appear in the given order, each with 1 plane and 32 bits; 256 is stored in the directory as

1. `info` returns per entry `width`, `height`, `bpp`, `format` (`png`

or `bmp`), `offset` and `length`, with 256 instead of 0. `extract` finds the first entry with this width and, on request, also writes it to `ziel`. `info` recognizes BMP entries, but the module cannot write them.

```
package require tclutils::tuico
package require tclutils::tupngdraw
set entries {}
foreach s {32 16} {
    set p [::tclutils::tupngdraw::new -width $s -height $s \
        -background transparent]
    $p setfill {30 144 255}
    $p circle [expr {$s/2}] [expr {$s/2}] [expr {$s/2 - 1}] \
        -fill 1 -outline 0
    lappend entries [list $s [$p data]]
    $p destroy
}
::tclutils::tuico::write app.ico $entries    ;# -> 752
lmap e [::tclutils::tuico::info app.ico] {dict get $e width}
                                             ;# -> 32 16
dict get [lindex [::tclutils::tuico::info app.ico] 1] format
                                             ;# -> png
string length [::tclutils::tuico::extract app.ico 16 x16.png]
                                             ;# -> 247
```

-errorcode	Cause
{TCLUTILS TUICO NOENTRIES}	empty list
{TCLUTILS TUICO TOOMANY}	more than 65535 entries
{TCLUTILS TUICO BADENTRY}	entry is not a pair
{TCLUTILS TUICO BADSIZE}	size not 1..256
{TCLUTILS TUICO BADPNG}	content is not a PNG
{TCLUTILS TUICO SIZEMISMATCH}	PNG size does not match the stated size
{TCLUTILS TUICO BADFILE}	file too short or not an .ico
{TCLUTILS TUICO TRUNCATED}	directory entry truncated

-errorcode	Cause
{TCLUTILS TUICO NOSUCHSIZE}	extract does not find the size
{TCLUTILS TUICO READFAILED}, WRITEFAILED	file cannot be opened

`info` reads all fields of the directory as unsigned, as the format specifies. If the header announces more entries than the file contains, `info` throws `TRUNCATED`, also for more than 32767.

16.11 Similar Images: tudhash

tudhash computes a 64-bit difference hash (dHash). Similar images get similar hashes; the distance is the number of differing bits. This reveals images that have been scaled down, saved again or slightly changed and whose bytes therefore differ.

```
::tclutils::tudhash::fromGray b h grauwerte    ;# -> 16 Hexzeichen
::tclutils::tudhash::fromRGB  b h rgbwerte     ;# -> 16 Hexzeichen
::tclutils::tudhash::distance hexA hexB        ;# -> 0..64
::tclutils::tudhash::similar  hexA hexB ?max?  ;# -> 0 | 1
```

`grauwerte` are b × h numbers 0..255, `rgbwerte` b × h × 3, each row by row. `fromRGB` computes gray as `(77 R + 150 G + 29 B) >> 8`. The image is averaged down to 9 × 8 cells (mean over each cell, integer); then each cell compares itself with its right neighbor, bit 1 means "right is brighter". `distance` accepts upper- and lowercase letters. `similar` is `distance <= max`, default 10. The downscaling is a simple box filter; hashes from other tools with the same bit order but a different filter can differ by a few bits.

```
package require tclutils::tudhash
set g {}
for {set y 0} {$y < 8} {incr y} {
    for {set x 0} {$x < 9} {incr x} { lappend g [expr {$x * 20}] }
}
::tclutils::tudhash::fromGray 9 8 $g         ;# -> ffffffffffffffff
set g2 [lmap v $g {expr {255 - $v}}]
::tclutils::tudhash::fromGray 9 8 $g2        ;# -> 0000000000000000
::tclutils::tudhash::distance FFFF000000000000 ffff000000000001
                                             ;# -> 1
::tclutils::tudhash::similar ffffffffffffffff fffffffffffffff0
                                             ;# -> 1
::tclutils::tudhash::similar ffffffffffffffff fffffffffffffff0 3
                                             ;# -> 0
```

For PNG files, `tupng` supplies the pixels without Tk. The following example draws the same motif in two sizes and once shifted, and compares the hashes.

```
package require tclutils::tudhash
package require tclutils::tupngdraw
proc pngHash {png} {
    set d [::tclutils::tupng::decode $png]
```

```
    binary scan [dict get $d rgba] cu* px
    set rgb {}
    foreach {r g b a} $px { lappend rgb $r $g $b }
    ::tclutils::tudhash::fromRGB \
        [dict get $d width] [dict get $d height] $rgb
}
proc bild {w h dx} {
    set p [::tclutils::tupngdraw::new -width $w -height $h]
    $p setfill {0 0 160}
    $p circle [expr {$w/3 + $dx}] [expr {$h/2}] [expr {$h/3}] \
        -fill 1 -outline 0
    $p rect [expr {$w*2/3}] [expr {$h/4}] [expr {$w-4}] \
        [expr {$h*3/4}] -fill 1 -fillcolor orange -outline 0
    set png [$p data]
    $p destroy
    return $png
}
set a [pngHash [bild 90 60 0]]              ;# -> 0018191919191900
set b [pngHash [bild 45 30 0]]              ;# -> 0018191909191900
set c [pngHash [bild 90 60 25]]             ;# -> 0006050505050700
::tclutils::tudhash::distance $a $b         ;# -> 1
::tclutils::tudhash::distance $a $c         ;# -> 20
```

The half size lies one bit away, the shift twenty. For an image with 256 × 256 pixels, `fromRGB` takes, as measured, 0.018 s under Tcl 9.0.4 and 0.017 s under 8.6.14 (Xeon, 2.8 GHz).

-errorcode	Cause
{TCLUTILS TUDHASH DIM}	width or height not an integer from 1
{TCLUTILS TUDHASH DATA}	list has the wrong length
{TCLUTILS TUDHASH HASH}	hash not 16 hex characters

Both commands check width and height first, so `a` as the width throws `DIM`. Neither command checks the values themselves for 0..255.

Next chapter: Providers

17 Providers

17.1 One API, several backends

tuprovider describes files and folders uniformly: `list`, `stat`, `get`, `put`, `mkdir`, `move`, `delete`, … — depending on the capabilities (`caps`).

Backend	Package	typical use
local	tclutils::tuprovider	file system
ZIP	tuprovider::zip	archive, read only

Backend	Package	typical use
DAV	tuprovider::dav	WebDAV
FTP	tuprovider::ftp	needs tcllib ftp on open
SFTP	tuprovider::sftp	its own require, OpenSSH sftp

17.2 Caps per scheme

`caps` returns a list of the operations the backend actually supports. The table applies to 0.69.0 (the same since 0.64.0):

Scheme	caps
local	copy delete get list mkdir move put stat
zip	get list stat
dav	delete get list put stat
ftp	delete get list mkdir move put stat
sftp	delete get list mkdir move put stat

The table in `docs/tuprovider.md` is authoritative; the test `tests/tuprovider-caps.test` reads exactly these lines. ZIP is therefore read-only. WebDAV has neither `mkdir` nor `copy`/`move`: the DAV provider only forwards to `tudav`, and `tudav` has no public MKCOL/MOVE/COPY. Copying across provider boundaries is done in the app with `get` + `put`.

```
package require tclutils::tuprovider
set p [::tclutils::tuprovider::open local]
lsort [$p caps]      ;# -> copy delete get list mkdir move put stat
```

An unknown scheme aborts without `-errorcode`. The message names the schemes that are loaded: with `tuprovider` only, it reads `unknown scheme 'unbekannt' (have: local)`. ZIP and DAV appear in it as soon as their package is loaded.

17.2.1 Entry

`list` and `stat` return dicts. They contain at least `name`, `path`, `type` (`file` or `dir`), `size` and `mtime`. `path` goes back unchanged into `list`, `stat` and `get`. `get` returns the bytes, without encoding. Anyone who wants text converts it themselves.

```
$p put [file join $dir a.txt] eins
dict get [$p stat [file join $dir a.txt]] size    ;# -> 4
dict get [$p stat [file join $dir a.txt]] type    ;# -> file
$p get [file join $dir a.txt]                      ;# -> eins
```

`put` of the local provider writes through a temporary file; if it fails, the old file stays (0.3). `mtime` can be empty. With the ZIP provider the member path is `/notiz.txt`, the time stays empty, `size` is the byte count.

17.2.2 Writes the caps forbid

ZIP has `get`, `list` and `stat`. `put` on it answers without `-errorcode`: `provider ::tclutils::tuprovider::Zip does not support 'put'`. Copying from one provider to another is `get` followed by `put` on the target, not `copy`. `copy` stays within the same provider.

`head` is not a cap. The base class reads via `get` and truncates. SFTP transfers the whole file in doing so. The local provider reads only the requested bytes.

17.3 Apps on top

tkutils widgets (`tkufilelist`, `tkufiletree`) and ctrlutils controllers (`cufileops`) speak only the provider API. Recipes and file view: book *Utils-Stack*. HOWTOs in the library tree: `docs/guide/howto-*.md`.

17.4 Summary

- Provider = uniform storage API.
- Caps control what the UI may offer. ZIP is read-only, and `put` on it has no `-errorcode`. DAV has no `mkdir`, `copy`, `move`.

Next chapter: Building Blocks for Applications

18 Building Blocks for Applications

Seventeen small modules that an application needs around its actual task: log and events, checking and calculating, environment and deployment, output in the terminal and on the page. All are pure Tcl without Tk and are part of the umbrella `tclutils`. Platform-dependent are `tuappinfo`, `tusysinfo`, `tudeploy`, `tuexe`, `tuopen` and `tuterm`; `tuzipfs` needs Tcl 9 for most commands.

Group	Modules
Log and events	tulog, tuevent, turegistry
Checking and calculating	tuvalidate, tulist, tumath, tunumany
Environment and deployment	tuappinfo, tusysinfo, tuedid, tudeploy, tuexe, tuopen, tuzipfs
Output	tuterm, tupagespec, tulayout

The modules report unknown options and options without a value in different ways: through `::tclutils::common::parseOptions` with `{TCLUTILS COMMON OPTION <opt>}` or with a code of their own. Each section names the code.

18.1 tulog

tulog is a log with four levels. A logger is a command; the levels `info` and `error` are subcommands and therefore do not hide any Tcl commands.

```
::tclutils::tulog::new ?option wert ...?        ;# -> Logger-Befehl
$log debug|info|warn|error nachricht
$log log stufe nachricht
$log setLevel stufe
$log level
$log destroy
::tclutils::tulog::assert ausdruck ?nachricht?
```

Option	Default	Effect
-channel	stderr	channel for the output
-level	info	lowest level that is written
-name	""	name after the level
-timestamp	1	timestamp %Y-%m-%dT%H:%M:%S, local time

The levels rise in the order `debug`, `info`, `warn`, `error`. A message appears if its level is at least that of the logger. Each line has the form `ZEIT [STUFE] name: nachricht`; without a timestamp, `ZEIT` is dropped, without a name the line reads `[STUFE]: nachricht`. The channel is flushed after each line.

new returns the command name `::tclutils::tulog::logger<n>`. The subcommands for writing, `setLevel` and `destroy` return `""`, `level` returns the current level. After `destroy` the command no longer exists.

```
package require tclutils::tulog
set log [::tclutils::tulog::new -name app -channel stdout \
    -timestamp 0]
$log info "Start"
$log debug "unterdrückt"
$log warn "Platte fast voll"
$log setLevel debug
$log level                                  ;# -> debug
$log destroy
```

Output on `stdout`:

```
[INFO] app: Start
[WARN] app: Platte fast voll
```

`assert` evaluates the expression with `expr` in the caller's context. If it is false, it fails with `{TCLUTILS TULOG ASSERT}`; the message is the given message or `assertion failed: <ausdruck>`.

```
set n 3
::tclutils::tulog::assert {$n > 2}          ;# -> (leer)
catch {::tclutils::tulog::assert {$n > 5}} m
set m                          ;# -> assertion failed: $n > 5
```

Case	Code
unknown level for -level, log, setLevel	{TCLUTILS TULOG LEVEL}
-timestamp not boolean	{TCLUTILS COMMON BOOLEAN -timestamp}
unknown option for new	{TCLUTILS COMMON OPTION <opt>}
unknown subcommand, wrong number of arguments	without -errorcode
assertion false	{TCLUTILS TULOG ASSERT}

new checks all options before it creates the logger; after an error no logger is left behind. `new` does not check `-channel`. A channel that does not exist only shows up at the first output: `can not find channel named "…"` with `{TCL LOOKUP CHANNEL …}`.

18.2 tuevent

tuevent is an event bus following the publish/subscribe pattern. Each bus is a token; subscribers are command prefixes.

Command	Effect	Return value
create	new bus	token tuevent<n>
subscribe bus ereignis handler	register a handler; the same handler only once	""
unsubscribe bus ereignis handler	unregister a handler; if it is missing, nothing happens	""
emit bus ereignis ?arg ...?	call the handlers	number of handlers called
handlers bus ereignis	registered handlers	list in order of registration
events bus	events with handlers	sorted list
clear bus ?ereignis?	delete the handlers of one event or of all	""
destroy bus	delete the bus with all registrations	""

`emit` calls each handler at global level, with the arguments appended, in the order of registration. An error in a handler aborts `emit` and goes to the caller; the remaining handlers then do not run. A token that `create` did not return or that `destroy` has deleted gives `{TCLUTILS TUEVENT BUS}` with `unknown event bus: "<token>"`.

```
package require tclutils::tuevent
set bus [::tclutils::tuevent::create]
set log {}
proc merke {id} { lappend ::log $id }
::tclutils::tuevent::subscribe $bus saved merke
::tclutils::tuevent::subscribe $bus saved merke
```

```
::tclutils::tuevent::emit $bus saved 42      ;# -> 1
set log                                      ;# -> 42
::tclutils::tuevent::emit $bus closed        ;# -> 0
::tclutils::tuevent::events $bus             ;# -> saved
::tclutils::tuevent::destroy $bus
```

18.3 turegistry

turegistry stores values under keys, for example database connections or services that several parts of an application need. Each registry is a token.

Command	Effect	Return value
create	new registry	token turegistry<n>
put reg schlüssel wert	store a value, replace the old one	the value
get reg schlüssel ?vorgabe?	read a value	value or default
has reg schlüssel	key present?	1 or 0
keys reg ?muster?	keys, filtered with string match	sorted list
remove reg schlüssel	delete a key	1 if it was present, otherwise 0
destroy reg	delete the registry	""

`get` without a default fails on a missing key with `{TCLUTILS TUREGISTRY KEY}`, message `registry key not set: "<schlüssel>"`. An empty default `{}` counts as a default. An unknown token gives `{TCLUTILS TUREGISTRY REG}`.

```
package require tclutils::turegistry
set reg [::tclutils::turegistry::create]
::tclutils::turegistry::put $reg db conn1    ;# -> conn1
::tclutils::turegistry::put $reg db.timeout 30
::tclutils::turegistry::get $reg db          ;# -> conn1
::tclutils::turegistry::get $reg cache leer  ;# -> leer
::tclutils::turegistry::keys $reg db.*       ;# -> db.timeout
::tclutils::turegistry::remove $reg db       ;# -> 1
::tclutils::turegistry::has $reg db          ;# -> 0
::tclutils::turegistry::destroy $reg
```

More than one default value for `get` is a call error without `-errorcode`, but only if the key is missing. If it is set, `get` returns the value and ignores the surplus arguments.

18.4 tuvalidate

tuvalidate contains checks that return `1` or `0`. They do not fail on the value being checked; errors occur only for wrong arguments. The checks for e-mail and URL recognize the usual form; they are not complete RFC checks.

Command	1 if
email s	lokal@domain.tld; local part from A-Z a-z 0-9 . _ % + -, domain from A-Z a-z 0-9 . -, ending at least two ASCII letters
url s	starts with http://or https://, no whitespace, the first character after it is not /$. ? #
ipv4 s	four groups of 1–3 digits, each at most 255, no leading zero
port s	decimal digits only, value 1 to 65535
alpha s	only A-Z a-z, not empty
alnum s	only A-Z a-z 0-9, not empty
numeric s	string is double -strict
integer s	string is integer -strict
length s min max	min <= [string length s] <= max
pattern s re	regexp re s matches
inList s liste	s is an element of the list (exact comparison)

```
package require tclutils::tuvalidate
::tclutils::tuvalidate::email a@b.com             ;# -> 1
::tclutils::tuvalidate::email "jörg@b.de"    ;# -> 0
::tclutils::tuvalidate::url ftp://tcl.tk/         ;# -> 0
::tclutils::tuvalidate::ipv4 10.0.0.256           ;# -> 0
::tclutils::tuvalidate::ipv4 010.0.0.1            ;# -> 0
::tclutils::tuvalidate::ipv4 0.0.0.0              ;# -> 1
::tclutils::tuvalidate::port 0x50                 ;# -> 0
::tclutils::tuvalidate::port 080                  ;# -> 1
::tclutils::tuvalidate::numeric 1,5               ;# -> 0
::tclutils::tuvalidate::length abc 3 20           ;# -> 1
::tclutils::tuvalidate::pattern 2024-03-15 {^\d{4}-\d\d-\d\d$}
                                                  ;# -> 1
```

`alpha` and `alnum` know only ASCII; for Unicode letters use `string is alpha`. `ipv4` checks strictly like `inet_pton`: a group with a leading zero (`010`) gives 0, the group `0` alone is valid. `port` accepts only the ASCII digits `0` to `9`, without sign, without whitespace and without `0x`; `0x50` and `" 80"` give 0. Leading zeros are allowed and count as decimal: `080` is port 80, the same under Tcl 8.6 and 9.

Case	Code
length: min or max not an integer	{TCLUTILS TUVALIDATE ARG}
pattern: invalid regular expression	{TCLUTILS TUVALIDATE REGEX}

18.5 tulist

tulist supplements the list commands of the core. `lreverse`, `lsort -unique`, `lsearch` and `lmap` remain with the core. `map`, `filter`, `reduce`, `all` and `any` take a command prefix, not a script body; it runs at global level with the element appended. Local variables of the caller are not visible in it; `apply` with parameters solves this.

Command	Effect
unique liste	remove duplicates, the order of first occurrence is kept
flatten liste ?tiefe?	resolve sublists, default for tiefe is 1; tiefe <= 0 returns the list unchanged
chunk liste größe	pieces of at most größe elements
zip liste ...	tuples, ends with the shortest list; without a list {}
sum liste	sum, empty list 0
avg liste	mean as a floating-point number
min liste, max liste	smallest, largest element by < and >
reduce liste start cmd	left fold: acc = cmd acc element
map liste cmd	results of cmd element
filter liste cmd	elements for which cmd element is true
all liste cmd, any liste cmd	1/0; empty list: all 1, any 0
take liste n, drop liste n	first n elements or the rest; n <= 0 gives {} or the whole list

```
package require tclutils::tulist
::tclutils::tulist::unique {a b a c b}            ;# -> a b c
::tclutils::tulist::flatten {{1 2} {3 {4 5}}}     ;# -> 1 2 3 {4 5}
::tclutils::tulist::flatten {{1 2} {3 {4 5}}} 2   ;# -> 1 2 3 4 5
::tclutils::tulist::chunk {1 2 3 4 5} 2           ;# -> {1 2} {3 4} 5
::tclutils::tulist::zip {a b c} {1 2}             ;# -> {a 1} {b 2}
::tclutils::tulist::avg {1 2 3 4}                 ;# -> 2.5
::tclutils::tulist::max {b a c}                   ;# -> c
::tclutils::tulist::reduce {1 2 3 4} 0 ::tcl::mathop::+
                                                  ;# -> 10
::tclutils::tulist::map {1 2 3} {apply {{x} {expr {$x*$x}}}}
                                                  ;# -> 1 4 9
::tclutils::tulist::filter {1 2 3 4} {apply {{x} {expr {$x%2==0}}}}
                                                  ;# -> 2 4
::tclutils::tulist::drop {a b c d} 2              ;# -> c d
```

`min` and `max` compare numbers numerically, otherwise as text. `sum` and `avg` calculate with `expr`; a non-numeric element fails with `{ARITH DOMAIN {non-numeric string}}`. So does a non-numeric `n` for `take` and `drop`.

Case	Code
chunk: size not a positive integer	{TCLUTILS TULIST SIZE}
avg, min, max of an empty list	{TCLUTILS TULIST EMPTY}

18.6 tumath

tumath supplements `expr` with calculations that are missing there. `round`, `ceil`, `floor` and `abs` remain with `expr`.

Command	Return value	Error
clamp x lo hi	x, limited to lo to hi	lo > hi: ARG
inRange x lo hi	1 if lo <= x <= hi	–
percent teil ganzes	teil /ganzes * 100 as a floating-point number	ganzes == 0: DIVZERO
sign x	-1, 0 or 1	–
gcd a b	greatest common divisor, not negative; gcd 0 0 is 0	not integers: ARG
lcm a b	least common multiple; 0 if an argument is 0	through gcd
factorial n	n!, with Tcl's big integers	n negative or not an integer: ARG
roundTo x stellen	round(x * 10**stellen) /10**stellen, floating-point number	stellen negative or not an integer: ARG

The codes are `{TCLUTILS TUMATH ARG}` and `{TCLUTILS TUMATH DIVZERO}`.

```
package require tclutils::tumath
::tclutils::tumath::clamp 12 0 10            ;# -> 10
::tclutils::tumath::percent 30 200           ;# -> 15.0
::tclutils::tumath::gcd -24 36               ;# -> 12
::tclutils::tumath::lcm 4 6                  ;# -> 12
::tclutils::tumath::factorial 25   ;# -> 15511210043330985984000000
::tclutils::tumath::roundTo 3.14159 2        ;# -> 3.14
::tclutils::tumath::roundTo 2.5 0            ;# -> 3.0
::tclutils::tumath::roundTo 1.005 2          ;# -> 1.0
```

`roundTo` calculates with floating-point numbers. `1.005 * 100` gives `100.49999…`, hence `1.0` instead of `1.01`. For amounts with a fixed number of decimal places, `tunum::format` rounds on the decimal digits, see Characters, Numbers, Dates and Paths.

18.7 tunumany

tunumany is a single entry point for numbers in two notations. Amounts with thousands separators and currency go to `tunum::parse`, sizes with an SI or IEC unit (`1.5K`, `2Mi`) to

`tunumfmt::fromHuman`. The two parsers themselves stay separate; `tunum` is described in the chapter Characters, Numbers, Dates and Paths.

```
::tclutils::tunumany::parse text ?-default wert? ?-prefer weg?
```

Option	Default	Effect
-default	""	return value if nothing is readable
-prefer	auto	auto, si or locale: which parser is asked first

With `auto`, `parse` chooses the SI path if the trimmed text is a number with a point or comma as decimal separator, optional spaces and exactly one of the letters `K M G T P E Z Y` (also lowercase) with an optional `i`. Everything else goes to `tunum::parse` with locale `auto`. If the chosen parser returns nothing, `parse` asks the other one. `parse` loads both modules only when called; if one is missing, that path is not available.

```
package require tclutils::tunumany
::tclutils::tunumany::parse "1.234,56"       ;# -> 1234.56
::tclutils::tunumany::parse "1,234.56"       ;# -> 1234.56
::tclutils::tunumany::parse "12 €"      ;# -> 12.0
::tclutils::tunumany::parse 42               ;# -> 42.0
::tclutils::tunumany::parse 1.5K             ;# -> 1500
::tclutils::tunumany::parse 2Mi              ;# -> 2097152
::tclutils::tunumany::parse 2KiB             ;# -> 2048
::tclutils::tunumany::parse "3 G"            ;# -> 3000000000
::tclutils::tunumany::parse 1.5m             ;# -> 1500000
::tclutils::tunumany::parse 1,5K             ;# -> 1500
::tclutils::tunumany::parse 1,500K -default x ;# -> x
::tclutils::tunumany::parse x -default 0     ;# -> 0
```

Lowercase letters stand for the same units: `m` is mega, not milli. A decimal comma before the unit counts: `1,5K` is 1500 (up to tclutils 0.68.0 the empty string). With exactly three digits after the comma (`1,500K`) it could also group thousands; `parse` does not read that form and returns the default. Because the default `""` also stands for unreadable input, a `-default` that cannot be a number tells the two cases apart. Integers without a unit go through `tunum` and come back as floating-point numbers. An `i` directly after the unit letter makes the unit an IEC unit, also with a trailing `B`: `2KiB` gives 2048, `2KB` gives 2000.

Case	Code
unknown option, option without value (parse 1.5 -default)	{TCLUTILS TUNUMANY OPTION}
-prefer not auto, si, locale	{TCLUTILS TUNUMANY PREFER}

18.8 tuappinfo

tuappinfo collects information about Tcl, the program, the platform, the environment, search paths and loaded packages in a text report, for example for an info dialog or an error report to the developer. The caller handles the display.

```
::tclutils::tuappinfo::trackTm datei
::tclutils::tuappinfo::buildReport ?-title text? ?-anonymize 0|1?
::tclutils::tuappinfo::writeLog datei ?-title text? ?-anonymize 0|1?
```

Option	Default	Effect
-title	Application Information	heading ==== <titel> ====
-anonymize	0	hide user, host and path information

`trackTm` records a module file for the last section and returns the list of all recorded files. `buildReport` returns the report as one string, `writeLog` writes it as UTF-8 through a temporary file (`common::replaceFile`) and returns the file name; if writing fails, an old log stays. Up to 0.2 it wrote in the system encoding.

The report has these sections in this order:

Section	Content
Header	title, Timestamp:, PID:
--- Tcl / Tk ---	info patchlevel, tk_patchLevel (only with Tk; up to 0.2 tk_version), info nameofexecutable, operating system and version, machine, Host:
--- Environment ---	USER, USERNAME, HOME, TEMP, TMP, PATH, if set
--- auto_path ---	one entry per line
--- Loaded Packages ---	each package with its package provide version
--- tm::path ---	tcl::tm::path list
--- Loaded TM Modules ---	the files recorded with trackTm, otherwise (none tracked)

Platform-dependent according to the code: `Host:` appears only if the environment variable `COMPUTERNAME` is set; Windows sets it, under Linux and macOS the line is usually missing. `USERNAME`, `TEMP` and `TMP` are typical Windows variables, `USER` and `HOME` typical ones under Linux and macOS.

`-anonymize 1` replaces:

Item	Replacement
USER, USERNAME	<user-xxxxxxxx>
Host	<host-xxxxxxxx>
HOME	<HOME>
TEMP, TMP	<TEMP>
PATH	<anonymized>
paths below the home directory	<HOME>/…
all other paths	<path>/<letzter Teil>

Item	Replacement
recorded module files	<tm-xxxxxxxx>.tm

xxxxxxxx is a stable, non-cryptographic hash of eight hex digits; equal names give equal values. The path rule applies to `Executable` and to each entry in `auto_path` and `tm::path`. The home directory is `$HOME`, without `HOME` `$USERPROFILE`; a path counts as below it if it is equal to it or starts with `<heim>/`. `/usr/share/tcltk` becomes `<path>/tcltk`, `$HOME/lib/tm` becomes `<HOME>/lib/tm`. Separators appear as `/`; under Windows `tuappinfo` compares the home directory without regard to case. `Platform`, `Machine`, the versions and the package list stay unchanged.

```
package require tclutils::tuappinfo
::tclutils::tuappinfo::trackTm /opt/app/lib/tm/foo-1.0.tm
set r [::tclutils::tuappinfo::buildReport -title "MeineApp 1.0"]
lindex [split $r \n] 0                ;# -> ==== MeineApp 1.0 ====
string match "*(none tracked)*" $r    ;# -> 0
```

The other lines depend on the computer: under Linux, for example, `Tcl version: 9.0.4`, `Platform: Linux <kernel>`, `HOME: /home/<name>`.

Case	Code
unknown option, option without value	{TCLUTILS TUAPPINFO OPTION}
-anonymize not boolean	{TCLUTILS TUAPPINFO VALUE}

`writeLog` checks the options through `buildReport` before it creates the file.

18.9 tusysinfo

tusysinfo describes the computer: operating system, processor, memory, model, disks and monitors, on Linux, macOS and Windows. `tuappinfo` describes the Tcl runtime, `tusysinfo` the machine beneath it.

```
::tclutils::tusysinfo::collect ?option wert ...?        ;# -> Dict
::tclutils::tusysinfo::report  info ?-title text? ?-anonymize 0|1?
::tclutils::tusysinfo::toJson  info ?-indent n?
```

The module reads files and the registry first and calls a program only where nothing else has the data. If a program is missing, its fields stay empty; `collect` does not fail because of it.

System	Sources
Linux	/etc/os-release, /proc/cpuinfo, /proc/meminfo, /sys/class/dmi/id, /sys/block, /sys/class/drm/*/edid; no program, no root
macOS	sw_vers, uname, sysctl, system_profiler -json (a few seconds)
Windows	registry (package registry) for system, CPU, model and the monitors' EDID; one PowerShell call (CIM) for memory, disks and the active monitors

Option	Default	Effect
-sections	all	any of os cpu memory machine disks displays
-serials	0	1: serial numbers of computer, disks, monitors
-commands	1	0: call no program
-timeout	10000	limit in ms per program call
-platform	this computer	linux, macos, windows
-root	empty	Linux: read /etc, /proc, /sys below this folder (tests)

`collect` returns a dict with `platform`, `host` and per section a dict (`os`, `cpu`, `memory`, `machine`) or a list of dicts (`disks`, `displays`). Sizes are in bytes. A monitor carries the fields from its EDID (section `tuedid`), the connector and `present` (1 connected, 0 not connected). On Windows a monitor without a status counts as not connected. If PowerShell gives no answer or one that cannot be read, the dict has a list `warnings` and `report` shows a section "Warnings"; memory, disks and monitor state are then missing instead of empty.

```
package require tclutils::tusysinfo
set i [::tclutils::tusysinfo::collect -sections {os memory}]
lsort [dict keys $i]                  ;# -> host memory os platform
dict get $i os id                     ;# -> debian (depends on the
    computer)
puts [::tclutils::tusysinfo::report $i -anonymize 1]
```

`report` shows disks in GB as on the label and memory in GiB, `toJson` writes numbers as numbers. The command line `bin/tusysinfo.tcl` prints the same, with `--json`, `--serials`, `--anonymize` and `--sections`.

Serial numbers, host names and MAC addresses identify a computer and, at an office desk, its user. Serial numbers come only with `-serials 1`, MAC addresses not at all; `report -anonymize 1` hides the host name and serial numbers.

Further details:

- Linux: only root may read the computer's serial number (`product_serial`). Virtual machines often have no DMI data and no monitor.
- Windows: the registry keeps every monitor that was ever connected, often the same one under several entries (other port, other driver). `tusysinfo` lists each EDID once and sets `present` from `Get-PnpDevice -Class Monitor`: 1 for status OK, otherwise 0, empty without that list. In a remote desktop session the physical monitors count as not connected; `report` then writes "(not connected)". `WmiMonitorID` reports "not supported" on some computers and is therefore not used. Windows 11 still reports "Windows 10 ..." as `ProductName`; from build 22000 on it shows Windows 11. SSD or HDD comes from `Get-PhysicalDisk`; an NVMe disk counts as SSD.
- Firmware placeholders such as "Default string" or "To be filled by O.E.M." are left empty.
- macOS: `system_profiler` takes a few seconds; `-sections {os cpu memory}` leaves it out. For a monitor it gives maker, model, product code, week and year, but no size; `widthCm`, `heightCm` and `diagonal` stay 0. There is one entry per APFS container (`disk3`), its volumes

in `volumes`; the size is that of the container, a little below the drive's.
- Linux: `disks` shows the devices, not loop devices, RAM disks and device mapper volumes (LVM, LUKS); RAID (`md*`) stays. Debian testing/sid has no `VERSION_ID`; `version` then comes from `/etc/debian_version`, e.g. `forky/sid`.
- Checked on a Linux PC with two monitors (Debian sid), a Windows 11 PC and a Mac mini (Apple Silicon, macOS 27). The tests replay recorded output of these computers.

Case	Code
unknown option, option without a value	{TCLUTILS TUSYSINFO OPTION}
unknown section	{TCLUTILS TUSYSINFO SECTION}
unknown platform	{TCLUTILS TUSYSINFO PLATFORM}

18.10 tuedid

tuedid reads a monitor's EDID block. In it the monitor describes itself in 128 bytes: manufacturer, product code, serial number, week and year of manufacture, size in centimetres and the preferred resolution; text fields carry the model name and a serial number as text.

```
::tclutils::tuedid::parse    bytes       ;# -> Dict
::tclutils::tuedid::parseHex text        ;# hex digits; spaces and ":"
    ignored
::tclutils::tuedid::vendor   kürzel      ;# DEL -> Dell
```

On Linux the bytes are in `/sys/class/drm/card*-*/edid`, on Windows in the registry (`Enum\DISPLAY\…\Device Parameters`, value `EDID`); `xrandr --verbose` and `ioreg` on macOS print them as hex. `tusysinfo` reads them itself.

```
package require tclutils::tuedid
set fh [open /sys/class/drm/card0-DP-1/edid rb]
set d [::tclutils::tuedid::parse [read $fh]]
close $fh
dict get $d vendor                    ;# -> Dell
dict get $d diagonal                  ;# -> 27.2 (inches)
dict get $d preferred                 ;# -> width 1920 height 1080
    refresh 60.0
```

Key	Content
valid	1 for a correct checksum; a block with a wrong sum is still read
manufacturer, vendor	PNP id (DEL) and the name for known ids
productCode, serial, serialText, name	product code (hex), serial number as a number and as text, model name
week, year	manufacture, 0 when unknown
widthCm, heightCm, diagonal	size; 0 for projectors without a size

Key	Content
preferred	first detailed resolution: width, height, refresh

Extension blocks (CEA-861, DisplayID) are counted in `extensions`, not read. Fewer than 128 bytes raise `{TCLUTILS TUEDID SHORT}`, a missing EDID header `{TCLUTILS TUEDID HEADER}`, text that is not hex `{TCLUTILS TUEDID HEX}`.

18.11 tudeploy

tudeploy finds the module directories and resources that an application brings along next to itself, adds them to `tcl::tm::path` and loads packages from them. It knows no particular application; directories and packages come as arguments. The module is also an ensemble: `::tclutils::tudeploy roots` and `::tclutils::tudeploy::roots` are the same call.

```
::tclutils::tudeploy::baseDirs
::tclutils::tudeploy::platformTag
::tclutils::tudeploy::roots ?option wert ...?
::tclutils::tudeploy::addModulePaths ?option wert ...?
::tclutils::tudeploy::require pakete ?option wert ...?
::tclutils::tudeploy::resourceDirs name ?option wert ...?
::tclutils::tudeploy::sourceModule paket ?option wert ...?
```

`baseDirs` returns the directories of the main script (`argv0`) and of the running program (`info nameofexecutable`), normalized, without duplicates, only existing ones. `platformTag` returns `<os><bits>`:

tcl_platform	Result
platform = windows	windows32 or windows64
platform = unix, os = Darwin	macos32 or macos64
platform = unix, any other os	linux32 or linux64

The bits come from `tcl_platform(pointerSize)`. BSD systems therefore also get `linux…`.

18.11.1 roots and addModulePaths

Option	Default	Effect
-base	[baseDirs]	base directories
-env	{}	names of environment variables; their values come unchanged at the start
-roots	vendor {libs common} libs {lib tm}	subpaths per base, each as a list of path parts
-parents	1	also search the parent directories of the bases

`roots` returns the existing directories in this order: first the values of the `-env` variables that are set and not empty; then, for each base, each subpath from `-roots`; with `-parents 1`

after that the same for each parent directory. A root directory contains package folders: `<wurzel>/<paket>/<modul>-<version>.tm` gives `package require <paket>::<modul>`.

`addModulePaths` takes the same options, calls `tcl::tm::path add` for each root and returns the roots that are in the module path afterwards, newly added or already present. If `tcl::tm::path` rejects a directory because it lies inside one already added or contains one, `addModulePaths` skips it without an error and leaves it out of the return value.

```
package require tclutils::tudeploy
::tclutils::tudeploy::platformTag     ;# linux64, windows64, macos64 …
::tclutils::tudeploy::roots -base [list $appdir] -parents 0
```

With `$appdir/vendor` and `$appdir/lib/tm` present, `roots` returns the list `$appdir/vendor $appdir/lib/tm`.

18.11.2 require

`require pakete ?option wert ...?` adds the roots and runs `package require` for each package.

Option	Default	Effect
-tmadd	1	call addModulePaths with the remaining options first
-fail	0	fail on missing packages instead of returning 0
-base, -env, -roots, -parents	as for roots	passed to addModulePaths

The return value is `1` if all packages are loaded, otherwise `0`. With `-fail 1`, `require` fails with `{TCLUTILS TUDEPLOY REQUIRE}`; the message names the missing packages: `package(s) not found: foo::nix`. Even with `-tmadd 0`, `require` checks the options of `roots`; `-tmadd` and `-fail` must be boolean.

```
if {![::tclutils::tudeploy::require {meinlib::kern meinlib::ui}]} {
    error "Module fehlen"
}
```

18.11.3 resourceDirs

`resourceDirs name ?option wert ...?` returns the existing directories in which a bundled resource can be located, for example a decoder. Per base, with `-parents 1` also per parent directory, in this order:

1. `vendor/<name>/<tag>`
2. `vendor/<name>`
3. `vendor/<tag>`
4. `vendor`
5. `bin`
6. the base itself

Option	Default	Effect
-base, -env, -parents	as for roots	
-tag	[platformTag]	platform folder

`resourceDirs` does not know `-roots`. An empty name gives `{TCLUTILS TUDEPLOY USAGE}`. The list fits directly into `tuexe::find -dirs`:

```
set dirs [::tclutils::tudeploy::resourceDirs webp -env MYAPP_WEBP]
set exe [::tclutils::tuexe::find dwebp -dirs $dirs]
```

18.11.4 sourceModule

`sourceModule paket ?option wert ...?` loads a package whose `.tm` is not in the place its name requires, for example a package `qpdf` as `vendor/qpdf/lib/qpdf-0.2.tm`. A separate entry `vendor/qpdf/lib` in the module path fails because it lies below `vendor`. `sourceModule` therefore searches the directories for `<letzter Namensteil>-<version>.tm`, sorted by version in descending order, and runs `source` at global level until the package is present.

Option	Default	Effect
-dirs	{}	directories directly; otherwise roots with the remaining options
-fail	0	fail on failure instead of returning 0
-base, -env, -roots, -parents	as for roots	only without -dirs

Return value `1` or `0`; with `-fail 1` `{TCLUTILS TUDEPLOY REQUIRE}` with `could not source module for package: <paket>`. `sourceModule` checks the options first; if the package is already loaded after that, it returns `1` at once. It skips files whose version part is not a valid version and files whose `source` fails.

```
::tclutils::tudeploy::sourceModule qpdf -env QPDF_TM \
    -roots {{vendor qpdf lib}}
```

Case	Code
unknown option, option without value, argument without -	{TCLUTILS TUDEPLOY OPTION}
-tmadd, -fail not boolean	{TCLUTILS TUDEPLOY VALUE}
resourceDirs without a name	{TCLUTILS TUDEPLOY USAGE}
package missing, with -fail 1	{TCLUTILS TUDEPLOY REQUIRE}

An application must first make `tclutils` itself loadable before it can use `tudeploy`. How `tclutils` gets into the module path is described in the chapter Installation.

18.12 tuexe

tuexe searches for external programs in bundled directories and in the PATH, trying several

names.

```
::tclutils::tuexe::find namen ?-dirs liste? ?-pathfirst 0|1? ?-all 0|1?
::tclutils::tuexe::all namen ?-dirs liste? ?-pathfirst 0|1?
::tclutils::tuexe::exists namen ?-dirs liste? ?-pathfirst 0|1?
```

Option	Default	Effect
-dirs	{}	additional directories
-pathfirst	0	1: first PATH, then -dirs
-all	0	1: all matches instead of the first

namen is a name or a list. For each name in turn, `find` first searches all `-dirs`, then the PATH (the other way round with `-pathfirst 1`), and returns the first match as a normalized path, otherwise `""`. A later name is tried only if the earlier one is found nowhere: with `{gs gswin64c}`, a `gs` in the PATH wins over a `gswin64c` in `-dirs`. `all` returns all matches without duplicates in search order, `exists` returns `1` or `0`.

In the PATH, `tuexe` searches with `auto_execok` and takes its first word. In the `-dirs`, according to the code:

Platform	Checked
Linux, macOS	<dir>/<name>; regular file and executable
Windows	<dir>/<name>, then <dir>/<name><ext> for each extension from PATHEXT (lowercase), without PATHEXT .exe .com .bat .cmd; regular file, no check for executability

```
package require tclutils::tuexe
::tclutils::tuexe::find {gs gswin64c gswin32c}   ;# Pfad oder ""
::tclutils::tuexe::find qpdf -dirs [list $vendorBin]
::tclutils::tuexe::exists gibtsnicht             ;# -> 0
```

Under Linux, `find sh` returns, for example, `/usr/bin/sh`; with an executable `sh` in `$vendorBin` and `-dirs [list $vendorBin]` its path, with `-pathfirst 1` in addition `/usr/bin/sh` again.

An unknown option gives `{TCLUTILS TUEXE OPTION}` with `unknown option '<opt>'`, an option without a value (`find sh -dirs`) the same code with `option '<opt>' requires a value`.

18.13 tuopen

tuopen opens a URL or file with the system's default program, a file in the editor or a folder in the file manager, and names the configuration directory of an application.

Command	Effect	Return value
command ziel ?-platform p? ?-os o?	build the command for opening, do not run it	command list

Command	Effect	Return value
launch ziel ?-platform p? ?-os o?	start the command in the background	command list
editCommand ziel ?-editor cmd? ?-platform p? ?-os o?	build the editor command	command list
edit ziel ?-editor cmd? ?-platform p? ?-os o?	start the editor	command list
openDir ziel ?-platform p? ?-os o?	open a folder; if ziel is a file, its folder	command list
configDir app ?-platform p? ?-os o?	configuration directory	path
configFile app datei ?-platform p? ?-os o?	file in it	path

Option	Default	Effect
-platform	tcl_platform(platform)	windows or other
-os	tcl_platform(os)	if it contains darwin (case-insensitive), macOS applies
-editor	""	custom editor command, also several words; takes precedence over everything else

What is produced per platform according to the code:

	Windows	macOS	Linux and other Unix
command, URL	rundll32 url.dll,FileProtocolHandler <ziel>	open <ziel>	xdg-open <ziel>
command, otherwise	cmd.exe /c start "" <ziel>	open <ziel>	xdg-open <ziel>
editCommand	notepad <ziel>	open -e <ziel>	$EDITOR <ziel>, otherwise xdg-open <ziel>
configDir	%APPDATA%\<app>, otherwise %USERPROFILE%\AppData\Roaming\<app>	~/Library/Application Support/<app>	$XDG_CONFIG_HOME/<app>, otherwise ~/.config/<app>

Under Windows, a target counts as a URL if it starts with a scheme of at least two characters and `:` (`https:`, `mailto:`); `C:/…` is a drive. `rundll32` passes the URL on without `cmd.exe`, so an & in the query stays part of the URL. `cmd.exe` would split the command there.

`$EDITOR` is read as a list; `vim -n` gives `vim -n <ziel>`. The home directory is `$HOME`, without `HOME file normalize ~`. `configDir` creates nothing. tusettings names the same directory under Linux and macOS and writes INI files into it; under Windows only the fallback without `APPDATA` differs.

`command`, `editCommand` and `configDir` start nothing; with `-platform` and `-os` they show the result for each platform:

```
package require tclutils::tuopen
::tclutils::tuopen::command https://tcl.tk/ -platform windows
            ;# -> rundll32 url.dll,FileProtocolHandler https://tcl.tk/
::tclutils::tuopen::command C:/doc/a.pdf -platform windows
                           ;# -> cmd.exe /c start {} C:/doc/a.pdf
::tclutils::tuopen::command a.pdf -platform unix -os Darwin
                           ;# -> open a.pdf
::tclutils::tuopen::editCommand notiz.txt -editor {code -w}
                           ;# -> code -w notiz.txt
::tclutils::tuopen::configDir meineapp -platform unix -os Linux
                           ;# ~/.config/meineapp, mit $HOME aufgelöst
```

`launch`, `edit` and `openDir` run the command with `exec … &` and return it. `launch` returns as soon as the process has started; it does not report whether the program can open the file afterwards.

```
::tclutils::tuopen::launch https://tcl.tk/
::tclutils::tuopen::openDir [file join $dir bericht.pdf]
```

Case	Code
ziel empty or whitespace only	{TCLUTILS TUOPEN TARGET}
app empty or whitespace only	{TCLUTILS TUOPEN APP}
exec fails, for example without xdg-open	{TCLUTILS TUOPEN LAUNCH}, cannot open "<ziel>": … or cannot edit …
unknown option	{TCLUTILS COMMON OPTION <opt>}

`namespace export` lists all seven commands; after `namespace import ::tclutils::tuopen::*` they are available without a prefix.

18.14 tuzipfs

tuzipfs wraps `zipfs` from Tcl 9: mounting and reading ZIP archives and building standalone programs (zipkits). The package also loads under Tcl 8.6 and is part of the umbrella. Commands that need `zipfs` fail there with `{TCLUTILS TUZIPFS UNAVAILABLE}`, message `zipfs is not available in this Tcl interpreter; Tcl 9 is required`. `tuzip` reads and writes ZIP files under Tcl 8.6, see Text and Files.

Command	Effect	Tcl 8.6
available	1 if the command zipfs exists	0
requireAvailable	1, otherwise an error	UNAVAILABLE
root	zipfs root, that is //zipfs:/	UNAVAILABLE

Command	Effect	Tcl 8.6
mounts	zipfs mount: pairs of mount point and archive	UNAVAILABLE
mount archiv ?name?	mount, returns //zipfs:/<name>	UNAVAILABLE
unmount punkt	unmount, returns punkt	UNAVAILABLE
listFiles punkt ?-glob m? ?-recursive b?	entries, sorted	UNAVAILABLE
find punkt ?...?	the same as listFiles	UNAVAILABLE
exists pfad	zipfs exists	UNAVAILABLE
readFile pfad	content as bytes	UNAVAILABLE
withMounted archiv var rumpf ?-mount name?	mount, run the body, unmount	UNAVAILABLE
rcopy quelle ziel	recursive byte copy, returns ziel	works
copyStdlib ziel ?-tk b? ?-from q?	copy tcl_library (and tk_library)	works
mkimg datei vfs ?...?	zipfs mkimg with checks	UNAVAILABLE
buildImage -out datei -vfs dir ?...?	copyStdlib, then mkimg	option check and copyStdlib work, mkimg fails

18.14.1 Reading an archive

`mount` without a name generates `tclutils_<pid>_<clicks>`. A missing archive is reported by `zipfs` itself with `{POSIX ENOENT …}`.

Option of listFiles	Default	Effect
-glob	*	pattern against the last name part or the whole path
-recursive	1	1: all entries through zipfs find; 0: only the top level through glob

The list also contains directories. With `-recursive 0`, `-glob` applies only to the top level. `readFile` reads in binary mode; text is produced with `encoding convertfrom`.

`withMounted` mounts, sets the variable in the caller's context to the mount point, runs the body there and always unmounts afterwards, also after an error. The return value is the result of the body.

```
package require tclutils::tuzipfs
if {[::tclutils::tuzipfs::available]} {
    ::tclutils::tuzipfs::withMounted $zip mp {
        set xml [::tclutils::tuzipfs::listFiles $mp -glob *.xml]
        set txt [::tclutils::tuzipfs::readFile $mp/notiz.txt]
    }
}
```

For an archive with `notiz.txt` (content `Hallo`) and `sub/a.xml`, under Tcl 9 this gives `xml = //zipfs:/tclutils_<pid>_<clicks>/sub/a.xml` and `txt = Hallo`; without `-glob` the list would also contain `…/notiz.txt` and the directory `…/sub`.

18.14.2 Building a program

`zipfs mkimg` replaces the ZIP archive attached to the basekit. A basekit from BAWT or Magicsplat carries its standard library in this archive (`//zipfs:/app/tcl_library`). An image with only the application loses it and does not start. `copyStdlib` therefore puts it into the VFS tree beforehand.

`rcopy` copies byte by byte and can also read from a mounted `zipfs`. It copies files and folders whose name starts with a dot; it skips `.` and `..`.

Option of copyStdlib	Default	Effect
-tk	0	also copy tk_library
-from	running	running: info library and $::tk_library of the running interpreter; otherwise a directory with tcl_library/(and tk_library/), for example the mount point of another basekit

Option of mkimg	Default	Effect
-strip	the VFS directory	path prefix that is dropped in the archive
-basekit	running program	template; if not given, the running program must itself be a zipkit
-password	""	password for the archive

Option of buildImage	Default	Effect
-out	–	output file, required
-vfs	–	finished VFS tree, required
-basekit	""	passed to mkimg
-stdlib	none	none, cli (only tcl_library) or gui (with tk_library)
-stdlibfrom	running	passed to copyStdlib -from

```
file mkdir app.vfs
file copy meineapp.tcl [file join app.vfs main.tcl]
::tclutils::tuzipfs::buildImage -out meineapp -vfs app.vfs \
    -basekit basekit-tcl -stdlib cli
```

`buildImage` returns the output file. `main.tcl` at the root of the archive starts when the program is called. Whether the result runs without an installed Tcl depends on the basekit: only a statically built one is standalone.

Case	Code
zipfs missing	{TCLUTILS TUZIPFS UNAVAILABLE}
-tk 1 without tk_library	{TCLUTILS TUZIPFS NOTK}
-from without tcl_library	{TCLUTILS TUZIPFS NOSTDLIB}
mkimg: VFS not a directory	{TCLUTILS TUZIPFS NODIR}
buildImage without -out/-vfs, wrong -stdlib	{TCLUTILS TUZIPFS OPTION}
-recursive, -tk not boolean	{TCLUTILS COMMON BOOLEAN <opt>}
unknown option	{TCLUTILS COMMON OPTION <opt>}

18.15 tuterm

tuterm generates ANSI control sequences (SGR) for font attributes and colors in the terminal and removes them again.

Command	Return value
style ?spec ...?	control sequence; "" if switched off
wrap text ?spec ...?	control sequence, text, reset; only the text if switched off
off attr	sequence for switching off an attribute
names ?was?	all (default): categories; fg/bg: color names; an attribute: on off; reset and anything unknown: {}
strip text	text without sequences of the form ESC [… m
enable ?bool?	read or set the switch; returns the state
auto	set the switch from NO_COLOR; returns the state
enableVT, disableVT	VT mode of the Windows console; 1 on success

Spec	Effect
reset	reset everything
bold dim italic underline blink reverse invisible strike	attribute on

Spec	Effect
<attr>:on, <attr>:off	attribute on or off
fg:<farbe>, bg:<farbe>	foreground and background; fgcolor: and bgcolor: are equivalent

A color is a name (`black` … `white`, `gray`, `bright_black` … `bright_white`), an index `0` to 255 (256 colors) or `#rrggbb` (24 bit). `gray` and `bright_black` are the same color. Each spec is a separate argument; `"fg:red bold"` as one argument is an unknown color.

```
package require tclutils::tuterm
proc zeige {s} { string map [list \x1b {\e}] $s }
zeige [::tclutils::tuterm::style bold fg:red]   ;# -> \e[1m\e[31m
zeige [::tclutils::tuterm::wrap OK fg:green]    ;# -> \e[32mOK\e[0m
zeige [::tclutils::tuterm::style fg:208 bg:#00aa00]
                          ;# -> \e[38;5;208m\e[48;2;0;170;0m
zeige [::tclutils::tuterm::off underline]       ;# -> \e[24m
::tclutils::tuterm::strip [::tclutils::tuterm::wrap OK bold]
                                                ;# -> OK
::tclutils::tuterm::enable 0                    ;# -> 0
::tclutils::tuterm::wrap OK bold                ;# -> OK
::tclutils::tuterm::enable 1                    ;# -> 1
```

After loading, `tuterm` is switched on, even if `NO_COLOR` is set or the output goes to a file. Only `auto` evaluates `NO_COLOR`: if the variable exists, even empty, it switches off, otherwise on. `tuterm` does not check whether `stdout` is a terminal. When switched off, `style` does not check the specs; typing errors then go unnoticed.

Platform according to the code: outside Windows, `enableVT` and `disableVT` return `1` at once. Under Windows they set or clear the bit `0x0004` in the console mode of `stdout` through the package `twapi`; if `twapi` is missing or the call fails, they return `0`.

Case	Code
unknown spec or attr: with a value other than on/off	{TCLUTILS TUTERM STYLE}
unknown color, index outside 0–255	{TCLUTILS TUTERM VALUE}
off for reset or a color category	{TCLUTILS TUTERM CATEGORY}
enable with a non-boolean value	{TCLUTILS COMMON BOOLEAN bool}

`enable` accepts every boolean value of Tcl (`0`/`1`, `yes`/`no`, `true`/`false`, `on`/`off`) and returns `0` or `1`; after an error the switch stays unchanged.

18.16 tupagespec

tupagespec translates page specifications such as `1-3,5,7-` into a list of page numbers and a list back into the short form.

```
::tclutils::tupagespec::parse angabe gesamt ?-base 1?
::tclutils::tupagespec::count angabe gesamt
::tclutils::tupagespec::compact seiten
```

Part	Meaning
empty, all, *	all pages
N	one page
A-B	range; A > B is swapped
A-	A to gesamt
-B	1 to B
end, last	stands for gesamt

Parts are separated by commas; `parse` ignores whitespace around a comma and empty parts, case does not matter. Whitespace inside a part (`1 - 3`) is a syntax error.

Option	Default	Effect
-base	1	1: pages from 1; 0: each number one less

`parse` returns the pages sorted and without duplicates, `count` their number (without `-base`). `compact` sorts, removes duplicates and combines runs into `A-B`; an empty list gives `""`.

```
package require tclutils::tupagespec
::tclutils::tupagespec::parse "1-3,5" 10        ;# -> 1 2 3 5
::tclutils::tupagespec::parse "5-" 8            ;# -> 5 6 7 8
::tclutils::tupagespec::parse "-3" 8            ;# -> 1 2 3
::tclutils::tupagespec::parse "4-last" 6        ;# -> 4 5 6
::tclutils::tupagespec::parse "5-3,1,1" 10      ;# -> 1 3 4 5
::tclutils::tupagespec::parse "2-4" 10 -base 0  ;# -> 1 2 3
::tclutils::tupagespec::count "1-3,5" 10        ;# -> 4
::tclutils::tupagespec::compact {8 7 1 2 2 3}   ;# -> 1-3,7-8
```

Case	Code
part not readable	{TCLUTILS TUPAGESPEC SYNTAX}, invalid page-spec part: '<teil>'
page outside 1..gesamt	{TCLUTILS TUPAGESPEC RANGE}, page <n> out of range (1..<gesamt>)
unknown option	{TCLUTILS TUPAGESPEC OPTION}
-base not 0 or 1; gesamt not a non-negative integer	{TCLUTILS TUPAGESPEC VALUE}

Page 0 and ranges past the end (`8-12` with 10 pages) are `RANGE` errors, not silent truncations.

Numbers are sequences of the ASCII digits `0` to `9` and always decimal, also with a leading zero; the same applies under Tcl 8.6 and

1. This also applies to a `gesamt` made of digits only:

```
::tclutils::tupagespec::parse 08 10             ;# -> 8
::tclutils::tupagespec::parse 07-09 10          ;# -> 7 8 9
::tclutils::tupagespec::parse 9-end 010         ;# -> 9 10
```

18.17 tulayout

tulayout calculates page layouts in millimeters, origin at the top left: paper sizes, conversion to pixels, grid and blocks. A block is a dict; a layout has a sub-dict `blocks` from block ID to block. The Tk editor for it is described in the book *tkutils* (`tkulayoutcanvas`).

Key	Meaning
label	label
x, y, w, h	position and size in mm
show	1 visible, 0 hidden
lockedY	1: vertical position automatic

18.17.1 Page and scale

Command	Return value
pageSize papier ?ausrichtung?	{breite höhe} in mm
mmToPx mm maßstab	mm * maßstab
pxToMm px maßstab	px / maßstab
snap mm ?-grid 5.0?	rounded to the grid
fitScale seiteB seiteH sichtB sichtH ?-marginPx 20?	largest scale at which the page including the margin fits into the view

`pageSize` knows `a4` (210 × 297), `a5` (148 × 210) and `letter` (216 × 279), case-insensitive. A custom size in mm is given as a list `{breite höhe}` or as `BxH` or `BXH`, for example `54x56` or `80.5X200`; both parts must be positive numbers. `1.2.3x5` is a `PAPER` error. Only `landscape` swaps width and height; any other value counts as portrait. `fitScale` calculates `min((sichtB - 2*rand) / seiteB, (sichtH - 2*rand) / seiteH)`.

```
package require tclutils::tulayout
::tclutils::tulayout::pageSize A4 landscape     ;# -> 297 210
::tclutils::tulayout::pageSize 54x56            ;# -> 54 56
::tclutils::tulayout::mmToPx 210 2              ;# -> 420.0
::tclutils::tulayout::snap 12.6                 ;# -> 15.0
::tclutils::tulayout::snap 7 -grid 2            ;# -> 8
::tclutils::tulayout::fitScale 210 297 800 600 -marginPx 0
                                        ;# -> 2.0202020202020203
```

18.17.2 Blocks

Command	Effect
defaultBlockKeys	x y w h show label lockedY
isAutoY block	1 if lockedY is true or y == 0
normalizeBlock block ?-defaults d?	only label x y w h show lockedY, missing ones from d; x y w h as floating-point numbers, show as 0/1
validateBlocks blocks	normalize and check each block; returns the new dict
ensureBlocks blocks definitionen	complete blocks with their definitions, create missing ones, check everything
blockOrder blocks ?definitionen?	IDs: without definitions sorted; with definitions first their order, then the rest
blockRect block ?-height h?	{x1 y1 x2 y2}
mergeBlocks basis vorlage ?-keys k?	take keys k from the template into blocks of the same name in the base
mergeLayout ziel vorlage ?-keys k?	mergeBlocks on blocks of two layouts

The default for `-defaults` is `x 0 y 0 w 40 h 8 show 1`, for `-keys x y w show`. `mergeBlocks` skips blocks of the template that are missing in the base and takes over values unchanged, without normalizing them. `blockRect` takes the height from `-height`, otherwise from the block, otherwise 8.

```
set blocks {
    kopf    {label Kopf x 20 y 30 w 170 h 15 show 1}
    tabelle {label Tabelle x 20 y 0 w 170 h 40 show 1}
}
::tclutils::tulayout::isAutoY [dict get $blocks tabelle]  ;# -> 1
::tclutils::tulayout::blockRect [dict get $blocks kopf]
                                    ;# -> 20.0 30.0 190.0 45.0
set m [::tclutils::tulayout::mergeBlocks $blocks {kopf {x 25 h 99}}]
dict get $m kopf x                                          ;# -> 25
dict get $m kopf h                                          ;# -> 15
::tclutils::tulayout::blockOrder $blocks     ;# -> kopf tabelle
```

`ensureBlocks` completes each existing block with the keys it lacks: first from its definition, then from the default values of `-defaults`. A value in the block takes precedence over the definition. An ID that appears only in the definitions is created from its definition alone. `blocks` may be empty; then everything comes from the definitions. `ensureBlocks` checks the result like `validateBlocks`.

```
set e [::tclutils::tulayout::ensureBlocks {kopf {x 20}} \
    {kopf {w 170 h 15} fuss {x 20 y 280 w 170 h 10}}]
dict get $e kopf          ;# -> x 20.0 y 0.0 w 170.0 h 15.0 show 1
dict get $e fuss                ;# -> x 20.0 y 280.0 w 170.0 h 10.0
::tclutils::tulayout::ensureBlocks {} {kopf {w 170 h 15}}
                                        ;# -> kopf {w 170.0 h 15.0}
```

Case	Code
unknown paper, size not readable or not positive	{TCLUTILS TULAYOUT PAPER}
scale not positive	{TCLUTILS TULAYOUT SCALE}
grid not positive	{TCLUTILS TULAYOUT GRID}
unknown option, option without value	{TCLUTILS TULAYOUT OPTION}
validateBlocks: blocks empty or not a dict; ensureBlocks: blocks, definitions or a block not a dict	{TCLUTILS TULAYOUT BLOCKS}
empty block ID	{TCLUTILS TULAYOUT BLOCKID}
x, y, w, h not a number or negative, show not boolean	{TCLUTILS TULAYOUT VALUE}
layout without blocks	{TCLUTILS TULAYOUT LAYOUT}
view minus twice the margin not positive	{TCLUTILS TULAYOUT VIEWPORT}

The message for `VALUE` names the block and the key, for example `block 'kopf': bad x 'abc'` or `block 'kopf': height must be >= 0`. `snap`, `normalizeBlock`, `blockRect`, `mergeBlocks`, `mergeLayout` and `fitScale` check options in the same way: only the option named for each is allowed.

Next chapter: Network and HTTPS

19 Network and HTTPS

19.1 tuhttps

tuhttps is the shared HTTPS policy for clients that use `::http::register`. Certificates are verified; they can be turned off only via `-insecure 1`. An internal CA is given with `-cafile`. `tuhttps::with prefix script` registers `https` only for the duration of a script and afterwards restores the previous registration, even on errors. Do not use `return` in the script (comment in the code). An application that itself registers `https` afterwards, or asynchronous `::http::geturl -command` calls during `with`, are not seen by `tuhttps`: the registration is temporary, not locked.

Clients: `tufetch`, `tudav`, `tupostgrest`. Raw REST without the tclutils policy: book *Tcl REST*.

19.2 tls 1.x and tls 2.0

tls	CA store
2.0 (Tcl 9)	OpenSSL default paths, no file needed
1.x (Tcl 8.6, also Windows)	does not find the system store by itself

Only under tls < 2.0 does `tuhttps` look for a CA bundle, in this order: `$env(SSL_CERT_FILE)`, then fixed paths (`/etc/ssl/certs/ca-certificates.crt`, `/etc/pki/tls/certs/ca-bundle.crt`, `/etc/ssl/ca-bundle.pem`, `/etc/ssl/cert.pem`, `/usr/local/etc/openssl/cert.pem`). If it finds none, it throws `{TCLUTILS TUHTTPS NOCA}` and names the way out (`-cafile`, `SSL_CERT_FILE`). It never falls back to "unverified". Under Windows with tls 1.x: set `SSL_CERT_FILE` or pass `-cafile`.

Under Windows with tls 2.0, OpenSSL decides. If it finds no store, the handshake fails. `-cafile` remains the way out.

19.3 What socketCmd returns

`socketCmd` returns the command prefix for `::http::register`. It does not open a socket. Under Tcl 8.6.18 with tls 2.0.1:

Call	Prefix
-host example.org	::tls::socket -autoservername 1 -require 1
-insecure 1 -host example.org	::tls::socket -autoservername 1 -request 0 -require 0
-host 127.0.0.1	::tls::socket -autoservername 0 -require 1

An IP gets no SNI (`-autoservername 0`). tls 2.0 and later appends no `-cafile`. The `caFile` search runs anyway and returns the first readable file. On this Linux that is `/etc/ssl/certs/ca-certificates.crt`. It is needed only when `socketCmd` sees a tls older than 2.0. An unreadable `-cafile` throws `{TCLUTILS TUHTTPS CAFILE}` under tls 2 as well.

19.4 tufetch

tufetch 0.4 fetches via native `http` + `tls`, otherwise via `curl` or `wget`. All three ways verify the certificate. `-cafile` and `-insecure` apply to all three.

Command	Effect
get url ?optionen?	body as UTF-8 text
download url pfad ?optionen?	body as bytes into the file; returns native, curl or wget

Option	Default	Effect
-timeout	30000	milliseconds

Option	Default	Effect
-redirects	5	after that {TCLUTILS TUFETCH REDIRECT}
-method	get	get or post
-headers		flat list name value
-data		body, together with -method post
-type		Content-Type of this body
-cafile		CA file for verification
-insecure	0	any certificate is accepted

If under tls 1.x no CA bundle at all is found, the native way steps aside and `curl`/`wget` take over (they bring their own CA stores). If neither of the two exists, `{TCLUTILS TUFETCH NOMETHOD}` comes with the way out in the message (`no HTTP method available: …` plus the text of `NOCA`). Entirely without tls and without curl/wget it reads `no HTTP method available -- install the Tcl tls package or curl/wget`.

`{TCLUTILS TUFETCH CAFILE}`, in contrast, means: the file named with `-cafile` is not readable. This code exists only on the native way, that is, with `tls`. Without `tls`, `tufetch` passes the file on to curl (`--cacert`) or wget (`--ca-certificate=`). An error from curl then arrives as `CONNECT`, one from wget as `CHILDSTATUS` (according to the code). `tuhttps` reports the case as `{TCLUTILS TUHTTPS CAFILE}`. Without `tls` it reports `{TCLUTILS TUHTTPS NOTLS}` there — the tls check runs before the `-cafile` check.

Further codes from `tufetch`: `{TCLUTILS TUFETCH HTTP <Klasse> <Status>}` (for example `HTTP 4XX 404`), `URL` (only `http://` and `https://`), `TIMEOUT`, `CONNECT` and `REDIRECT`. Prefix traps such as `trap {TCLUTILS TUFETCH HTTP}` catch a whole class. Via wget an HTTP status is not translated; it appears as `CHILDSTATUS` from `exec`.

Without a server, these two errors remain:

```
package require tclutils::tufetch
::tclutils::tufetch::get ftp://example.com/a
    ;# -> {TCLUTILS TUFETCH URL}  unsupported or invalid URL
::tclutils::tufetch::get https://example.com/ -cafile
    /gibt/es/nicht.pem
    ;# -> {TCLUTILS TUFETCH CAFILE}  cannot read CA file

package require tclutils::tuhttps
try {
    ::tclutils::tuhttps::socketCmd -cafile /gibt/es/nicht.pem
} trap {TCLUTILS TUHTTPS CAFILE} msg {
    puts "CA-Datei: $msg"
} trap {TCLUTILS TUHTTPS NOTLS} msg {
    puts "kein tls: $msg"
}
```

With tls the message reads `cannot read CA file: /gibt/es/nicht.pem`, without tls `https needs the tls package: can't find package tls`. If under tls 1.x no bundle at

all is found, NOCA comes with the text to set -cafile or SSL_CERT_FILE.

19.5 tusparql

tusparql is a lean SPARQL client. The transport is handled by tufetch, the query strings are built by tuurl, the result is read by tujson (SPARQL 1.1 Query Results JSON). The same prerequisites as for tufetch therefore apply: http with tls, otherwise curl or wget. The module does not read XML results.

Command	Effect
query endpunkt sparql ?optionen?	SELECT; list of dicts, per row variable to value
ask endpunkt sparql ?optionen?	ASK; 1 or 0

Option	Default	Effect
-method	get	post sends the query as the body; any other value counts as get
-timeout	30000	milliseconds, passed on to tufetch
-headers		further header lines, flat list name value; an Accept in it replaces -accept
-accept	application/sparql-results+json	value of the Accept header

According to the code, get appends?query=…&format=json to the endpoint, with + for the space. post sends the query raw with the Content-Type application/sparql-query. Accept goes out exactly once: if -headers already contains an Accept (in any capitalization), that one applies, otherwise the value of -accept. tusparql always reads the response as JSON. An endpoint that already has a query string, such as …/sparql?key=1, gets the parameters appended with &.

A result row contains only value per variable; type, datatype and xml:lang are dropped. A variable without a binding, for example behind OPTIONAL, is missing from the row's dict. The order of the variables comes from head vars. If this list is missing, the variables of all rows apply in the order of their first appearance, not only those of the first row.

The example needs a reachable endpoint; the result depends on its data and is therefore shown only in outline.

```
package require tclutils::tusparql
set ep https://query.wikidata.org/sparql
set q {SELECT ?ort WHERE { ?ort wdt:P31 wd:Q515 } LIMIT 2}
::tclutils::tusparql::query $ep $q -timeout 10000
    ;# -> {ort http://www.wikidata.org/entity/Q…} {ort …}
::tclutils::tusparql::ask $ep {ASK { wd:Q64 wdt:P31 wd:Q515 }}
    ;# -> 1
```

The URL of a get call for ASK {} to https://example.org/sparql is https://example.org/sparql?query=ASK+%7B%7D&format=json. A response with two rows, the second of which does not bind gang, yields {ort Nord gang 3} {ort Süd}.

A response without `boolean` makes `ask` throw `{TCLUTILS TUSPARQL NOTASK}`. Transport errors and HTTP status come unchanged from `tufetch` (`{TCLUTILS TUFETCH …}`), a response text that is not JSON as `{TCLUTILS TUJSON PARSE offset}`. An unknown option throws `{TCLUTILS COMMON OPTION -name}`.

19.6 Summary

- One place for HTTPS: `tuhttps`.
- tls 1.x needs a CA bundle; otherwise `NOCA` or `NOMETHOD`.
- CAFILE = `-cafile` not readable, not "CA missing".
- Keep `-insecure` deliberate and local.
- SPARQL: `tusparql` over `tufetch`, JSON results only, rows as dicts without type.
- Query strings: chapter Encoding, Hashing and Comparing (`tuurl`).

Next chapter: tusettings

20 tusettings

20.1 Purpose

tusettings **0.1** stores settings per app as INI in the platform config directory, plus directories for cache/data/temp and a recent list. Pure Tcl, no Tk. New in 0.64.0, in the umbrella since then. The INI is written by `tuini`; it can be read and changed by hand. The module writes and reads files as UTF-8.

```
package require tclutils::tusettings
set s [::tclutils::tusettings::load meineapp -defaults {
    window {width 800 height 600}
}]
dict set s window width 1024
::tclutils::tusettings::save meineapp $s     ;# -> Pfad der
    settings.ini
::tclutils::tusettings::recentAdd meineapp /pfad/zur/datei
::tclutils::tusettings::value $s window width ;# -> 1024
```

`save` returns the path, under Linux `$HOME/.config/meineapp/settings.ini`.

The commands live in the namespace `::tclutils::tusettings`. Tcl resolves the short form `tusettings::load` only inside `namespace eval ::tclutils`; at global level it aborts with `invalid command name`.

20.2 API (short)

Command	Role
dir kind app ?-create bool?	config / cache / data / temp
file app ?name?	<config>/<name>.ini, default name settings
load app ?-defaults d? ?-name n?	INI to dict, laid over the defaults

Command	Role
save app settings ?-name n?	dict to INI (temp file, then rename); returns the path
value settings section key ?default?	read section/key
recentAdd app path ?-list l? ?-max n?	path to the top; returns the new list
recentList app ?-list l? ?-existing bool?	list, newest first
recentRemove app path ?-list l? /recentClear app ?-list l?	delete an entry or the list

`load` lays the file over `-defaults`; new keys of a later app version thus get their default. If the file is missing, `load` returns the defaults. `dir` creates something only with `-create 1`; `save` and `recentAdd` create the config directory themselves. `-list` (default `recent`) selects the recent file.

20.3 Directories per platform

kind	Linux / BSD	Windows	macOS
config	$XDG_CONFIG_HOME or ~/.config	%APPDATA%	~/Library/Application Support
cache	$XDG_CACHE_HOME or ~/.cache	%LOCALAPPDATA%	~/Library/Caches
data	$XDG_DATA_HOME or ~/.local/share	%APPDATA%	~/Library/Application Support
temp	$TMPDIR or /tmp	%TEMP% or %TMP%	$TMPDIR or /tmp

The app name is appended to each directory. Under Windows, without the environment variables, `<home>/AppData/Roaming` or `<home>/AppData/Local` (temp: `…/Local/Temp`) apply. macOS is recognized by `tcl_platform(os) = Darwin`.

Without `XDG_*` and `TMPDIR`, under Linux these are `~/.config/meineapp`, `~/.cache/meineapp`, `~/.local/share/meineapp` and `/tmp/meineapp`. The columns for Windows and macOS are as written in the code.

20.4 Recent list

One text file per list (`<config>/<list>.txt`, one path per line, UTF-8). `recentAdd` normalizes the path, moves it to the top, removes an older entry for the same file (under Windows ignoring case) and keeps `-max` entries (default 10). `recentList -existing 1` omits files that have disappeared.

20.5 Limits and errors

INI cannot hold everything. `save` rejects: values with a line break or leading/trailing spaces; empty keys, keys with `=`, a line break, leading/trailing spaces, or starting with `;`, `#`, `[`; section

names with `]` or a line break. The check runs before writing, so the file stays unchanged. If the writing itself fails, the old file stays as well, the temp file is removed and no channel is left open (0.2). Comments in the file do not survive `save`.

```
try {
    ::tclutils::tusettings::save meineapp {window {title " x"}}
} trap {TCLUTILS TUSETTINGS VALUE} msg {
    puts "abgelehnt: $msg"
}
```

The message reads `value of window.title: no line breaks or outer blanks in INI`.

Code	Case
{TCLUTILS TUSETTINGS APP}	app name empty or with path characters
{TCLUTILS TUSETTINGS KIND}	dir with an unknown kind
{TCLUTILS TUSETTINGS SYNTAX}	INI file cannot be parsed
{TCLUTILS TUSETTINGS VALUE}	value, key or section not INI-compatible; -max not a positive integer; path with a line break

Unknown options: `{TCLUTILS COMMON OPTION <opt>}`. On `SYNTAX` the caller decides whether to fall back to the defaults; the message names the file and the faulty line, for example `…/settings.ini: line without '=': garbage line`.

`tkutils::tkugeometry` stores window geometry through it; details in the book *tkutils*.

20.6 Summary

- INI settings + recent list, platform-typical paths.
- Full namespace `::tclutils::tusettings::…`.
- GUI geometry: book *tkutils*.

Next chapter: Tasks with tutask

21 Tasks with tutask

21.1 Purpose

tutask **0.2** (since tclutils 0.66.0) runs jobs as tasks on coroutines. A task waits for an operation with `await`; the event loop keeps running in the meantime. On top of that come timeout, cancellation with cleanup, groups, a thread pool and a sandbox.

Pure Tcl from 8.6 on (`package require Tcl 8.6-`). Only the pool needs the package Thread and loads it only on the first `pool`. The umbrella loads `tutask` as well.

`tutask` 0.2 requires `tclutils::common 0.2`. When copying over an older tree, delete `tutask-0.1.tm`; otherwise both versions lie side by side.

```
package require tclutils::tutask
namespace import ::tclutils::tutask::*
```

Exported are `spawn`, `run`, `current`, `await`, `sleep`, `timer`, `onwrite`, `pipe`, `group`, `pool`, `sandbox`, `tasks` and `stats`. The examples in this chapter assume the import.

21.2 The contract: starters

Every operation is a command prefix, a starter. `await` appends a callback `cb` and calls the starter. The starter returns a cancel command (a prefix, may be empty) and later reports exactly once:

```
starter cb                    ;# -> Abbruchbefehl
{*}$cb ok wert
{*}$cb error meldung ?optionen?
```

Only the first report counts. A late report after timeout or cancellation is discarded. Your own operations follow the same lines:

```
proc meinOp {cb} {
    set id [after 100 [list {*}$cb ok fertig]]
    return [list after cancel $id]
}
run apply {{} { await meinOp }}      ;# -> fertig
```

Supplied: `timer`, `pipe`, `onwrite`, tasks and groups (via `wait`), `$pool op` and `$sandbox op`.

21.3 Tasks

```
set t [spawn apply {{n} {
    sleep 50
    return [expr {$n * 2}]
}} 21]
$t state         ;# -> running
$t wait          ;# -> 42
$t state         ;# -> ok
$t destroy
```

Command	Effect
spawn ?-detach bool? cmd ?arg ...?	task object; runs immediately up to the first await
run ?-timeout ms? cmd ?arg ...?	spawn, wait, destroy; returns the value
current	running task or empty
$t state	running, ok, error or cancelled
$t result	value; otherwise the task's error, CANCELLED or NOTDONE
$t wait ?-timeout ms?	inside a task: await; outside: blocks in vwait

Command	Effect
$t cancel	cancellation; 1 if the task was still running
$t ondone cmd	calls cmd with the task object appended as soon as it is done
$t destroy	cancels a running task first

`run` is meant for scripts and tests. A timeout in `wait` cancels the task; `state` is `cancelled` afterwards. `-timeout 0` (default) means: no timeout. A task with `-detach 1` destroys itself after it ends.

21.4 Waiting: await and sleep

`await starter ?-timeout ms?` returns the value of the operation or throws its error with its own `-errorcode`. If the timeout expires, `await` calls the operation's cancel command and throws `{TCLUTILS TUTASK TIMEOUT}`. `sleep ms` pauses the task without stopping the event loop.

```
try {
    run apply {{} { await [timer 500 x] -timeout 50 }}
} trap {TCLUTILS TUTASK TIMEOUT} msg {
    puts $msg                       ;# -> timeout after 50 ms
}
```

`await` works in any coroutine; `cancel` needs a task. Outside any coroutine, `await` throws `{TCLUTILS TUTASK NOTASK}`.

`-timeout` must be an integer of 0 or more. From 0.66.0 on, `await`, `run`, `$t wait` and `$g wait` check this the same way: a value such as `abc` or `-5` throws `{TCLUTILS TUTASK OPTION}` with `-timeout must be a non-negative integer (ms)`. `run` checks before `spawn`; a wrong value leaves no task behind. Up to 0.65.0 only `await` checked. `run`, `$t wait` and `$g wait` outside a task silently accepted the value and waited without a timeout.

```
catch {run -timeout abc {after 10}} msg opts
dict get $opts -errorcode    ;# -> TCLUTILS TUTASK OPTION
tasks                        ;# -> (leer)
```

21.5 Cancelling with cleanup

`$t cancel` resumes the task with a cancellation request. Its `await` throws `{TCLUTILS TUTASK CANCELLED}` there. The `finally` branches have run by the time `cancel` returns. The operation the task was waiting for is cancelled along with it.

```
set t [spawn apply {{} {
    try { sleep 10000 } finally { puts aufgeraeumt }
}}]
$t cancel        ;# gibt "aufgeraeumt" aus, -> 1
$t state         ;# -> cancelled
$t destroy
```

- From within the task itself, `cancel` takes effect only at the next `await`.
- A task that catches `CANCELLED` and waits again is cancelled again.
- `destroy` from within the task itself throws `{TCLUTILS TUTASK INSIDE}`; use `cancel` there.
- If a task waits via `wait` for another task and is cancelled, this cancels the other one as well: both end `cancelled`, both `finally` branches run.

21.6 Operations

Operation	Value
timer ms ?wert?	after ms milliseconds, wert (default empty)
onwrite varName	the new content as soon as the variable is written
pipe cmdlist ?-encoding enc? ?-profile p?	output of the pipeline, stdout and stderr combined

`onwrite` takes a global or fully qualified variable; without a leading `::` it prepends `::`. Typical use: a button sets the variable.

`pipe` starts the pipeline non-blocking (`open |… 2>@1`). A nonzero exit code throws Tcl's `{CHILDSTATUS pid code}`. Cancellation and timeout terminate the processes: `kill` under Unix, `taskkill /T /F` under Windows. `-profile` exists only under Tcl 9; under 8.6 the option throws `{TCLUTILS TUTASK OPTION}` with `pipe: -profile needs Tcl 9`.

```
run apply {{} { await [pipe {echo hallo}] }}     ;# -> "hallo\n"
try {
    run apply {{} { await [pipe {sh -c {exit 3}}] }}
} trap CHILDSTATUS {msg opts} {
    lindex [dict get $opts -errorcode] 2        ;# -> 3
}
```

Under Unix, `kill` hits only the processes that `pid` names for the channel. If the pipeline in turn starts children (such as `sh -c "sleep 30 & …"`), these keep running after cancellation or timeout. The roadmap in the tclutils tree plans a process group kill (`setsid`) for this. According to the CHANGELOG, Windows terminates the pipeline with `taskkill /T /F`.

21.7 Groups

```
set g [group]
$g spawn apply {{} { sleep 30; return a }}
$g spawn apply {{} { return b }}
$g wait                       ;# -> a b
$g destroy
```

- `-mode all` (default): all results in `spawn` order. The first error cancels the remaining tasks and is thrown.
- `-mode first`: the first finished task wins, the others are cancelled. `$g winner` names the winner.

- `$g wait` also takes `-timeout ms`. `$g tasks` lists the tasks; `$g destroy` destroys them as well.
- Empty group: `all` returns an empty list, `first` throws `{TCLUTILS TUTASK EMPTY}`. Any other mode throws `{TCLUTILS COMMON VALUE -mode}`.
- If a task that waits for a group is cancelled, this cancels the group's tasks.

`$g destroy` directly after `$g wait` is safe, even if `wait` ran outside a task with `-mode first` or `-mode all` ended with an error: late reports from cancelled tasks are discarded. Up to 0.65.0, `bgerror` then reported `invalid command name "::oo::Obj…::my"`.

21.8 Thread pool

```
set p [pool -size 2 -init {proc rechne {n} { expr {$n * 2} }}]
run apply {{p} { await [$p op {rechne 21}] -timeout 10000 }} $p
# -> 42
$p stats         ;# -> workers 1 idle 1 queued 0 jobs 0
$p destroy
```

- `-size` (default 2) is the maximum number of workers; they are created only when needed. `-init` runs in every new worker.
- Each job runs in a worker thread with its own interpreter. There are no shared variables.
- Errors keep the worker's `-errorcode` and `-errorinfo`.
- A job in the queue is removed on cancellation. A running job gets `thread::cancel`; the worker lives on (after a timeout on `while 1 {}` the same pool accepts the next job).
- `$p destroy` releases the workers; waiting tasks receive `CANCELLED`.
- Without the package `Thread`, `pool` throws `{TCLUTILS TUTASK NOTHREAD}`. If `-init` fails, the first job throws `{TCLUTILS TUTASK INIT}`.

Measured: two jobs with `after 300` each in a group took about 300 ms together, so they ran in parallel.

`pool -size 0` throws `-size must be a positive integer`. With `-init {error boom}` the waiting task gets `{TCLUTILS TUTASK INIT}` with `pool -init failed: boom`; afterwards `$p stats` shows `workers 0 idle 0 queued 0 jobs 0`. Up to 0.65.0, `bgerror` additionally reported an error in this case, and the job stayed in the queue.

21.9 Sandbox

```
set s [sandbox -commands 1000 -time 500]
$s eval {expr {6*7}}                  ;# -> 42
catch {$s eval {while 1 {info level}}} msg opts
dict get $opts -errorcode
# -> TCLUTILS TUTASK LIMIT COMMANDS
$s eval {set x 1}                     ;# -> 1 (Grenzen wieder frisch)
run apply {{s} { await [$s op {string repeat ab 2}] }} $s   ;# -> abab
$s destroy
```

The sandbox is a safe interpreter (`-safe 1`, default; `-safe 0` for a normal one). Options: `-commands n` and `-time ms` (0 each = no limit), `-aliases {name präfix …}` for commands the child may call. `$s interp` returns the interpreter.

- The limits apply per evaluation. Exceeding them throws `{TCLUTILS TUTASK LIMIT COMMANDS}` or `{TCLUTILS TUTASK LIMIT TIME}`.
- Other errors keep the child's code, for example `{TCL LOOKUP COMMAND open}` for `open` in the safe interpreter.
- The script runs in the calling thread. `op` can be cancelled until it starts (next pass of the event loop); after that only the limits bound it. For real parallelism use the pool.
- An empty loop `while 1 {}` counts no commands. `-commands` alone does not stop it; the run hung until the outer cancellation. `-time 500` ended it after 500 ms with `LIMIT TIME`. Therefore always set `-time` as well.

21.10 Leaks and forgotten errors

`tasks` returns the living task objects, `stats` a dict with `tasks running awaits pipes pools sandboxes`. After a test run all values are expected to be 0:

```
stats
# -> tasks 0 running 0 awaits 0 pipes 0 pools 0 sandboxes 0
```

An error that neither `result`, `wait`, `ondone` nor a group has collected is reported by the task object on `destroy` via `bgerror`, with the task's `-errorcode`:

```
tutask: error in task <obj> was never collected: <Meldung>
```

This also applies to tasks with `-detach 1`.

21.11 In Tk programs

In the event handler, only call `spawn` and return; the task does the waiting. No `$t wait` outside a task in a handler: that starts a nested `vwait`, the same trap as `vwait` in a callback. Inside a task, `wait` and `await` are safe.

`examples/demo-task.tcl` in the tclutils tree shows both: under `tclsh` a console part, under `wish` additionally a progress bar with a cancel button. The handler only starts the task; `ondone` destroys it. After cancellation the `finally` branch resets the buttons; `stats` is 0 everywhere afterwards. Widgets themselves: book *tkutils*.

21.12 Error codes

All codes begin with `TCLUTILS TUTASK`:

Reason	Case	Message
CANCELLED	task cancelled, pool destroyed	task cancelled
TIMEOUT	-timeout expired	timeout after <ms> ms
NOTASK	await outside a coroutine	await outside a task or coroutine
NOTDONE	result of a running task	task <obj> is still running

Reason	Case	Message
OPTION	wrong argument (spawn without command, timer -1, empty pipe, -timeout x)	per case; for -timeout: -timeout must be a non-negative integer (ms)
NOTHREAD	pool without package Thread	the Thread package is not available
EMPTY	wait -mode first on an empty group	group is empty
INIT	the pool's -init failed	pool -init failed: <Fehler>
INSIDE	destroy from within the task itself	a task cannot destroy itself while running; use cancel
PROTOCOL	coroutine resumed unexpectedly (internal)	per case
LIMIT COMMANDS	command limit of the sandbox	message from Tcl
LIMIT TIME	time limit of the sandbox	message from Tcl

Unknown options go through `common::parseOptions` and throw `{TCLUTILS COMMON OPTION <opt>}`.

21.13 Summary

- `tutask` 0.2: tasks on coroutines, in the umbrella.
- An operation is a starter: callback in, cancel command out.
- `cancel` lets `finally` run and cancels the awaited operation.
- Groups (`all`, `first`), pool (package `Thread`), sandbox (limits).
- Under Unix, `pipe` does not kill grandchild processes on cancellation.
- `-commands` alone does not stop an empty loop; set `-time` as well.
- From 0.66.0 on, `await`, `run`, `$t wait` and `$g wait` check `-timeout` the same way; a wrong value throws `{TCLUTILS TUTASK OPTION}`.

Next chapter: Tests and tutestrun

22 Tests and tutestrun

22.1 Suite

```
tclsh8.6 tests/all.tcl
tclsh8.6 tests/all.tcl -match 'tusettings-*'
```

Each `*.test` file runs in its own process; the runner passes through tcltest options such as `-match`. The banner shows where `tclutils` resolves to. Time limit per file: environment TCLUTILS_TESTGRENZE (default 120 s). At the end the runner lists "Files with problems" or reports `All test files passed`; exit code 1 on problems. If `all.tcl` does not find the runner, it aborts with exit code 2 and `all.tcl: tclutils::tutestrun not found (set TCLUTILS_TM)`.

Result for tclutils 0.69.0 under Linux (Xvfb), macOS and Windows:

Run	Files	Tests	passed	skipped	failed
Tcl 8.6.14 with tls 1.7.22, Thread 2.8.9, tDOM 0.9.3	156	2642	2617	25	0
Tcl 9.0.4 (built from source) with tls 2.0 (OpenSSL 3.0.13), Thread 3.0.6, tDOM 0.9.7	156	2642	2625	17	0
macOS (Darwin 27, arm64), Tcl/Tk 9.0.4 (Aqua), state before the write-error tests	154	2582	2567	15	0
Windows, Tcl 8.6.17 (system encoding cp1252), state before the write-error tests	154	2582	2545	37	0
Windows, Tcl 9.0.4, state before the write-error tests	154	2582	2549	33	0

No test fails. Skipped tests depend on missing packages or on the platform (constraints such as `tdbc`, `haveTk`, `net`, `windows`).

22.2 tutestrun

tutestrun **0.2** is the shared runner for tclutils, tkutils and ctrlutils. **Not** in the umbrella — `tests/all.tcl` sources the `.tm` from the tree under test (its own `lib/tm`, `$env(TCLUTILS_TM)` or the sibling checkout); otherwise an old installation wins.

```
exit [::tclutils::tutestrun::run $here {tclutils} \
    -limitvar TCLUTILS_TESTGRENZE -argv $argv]
```

What `run` does:

1. **Own trees first.** Only the own trees (`lib/tm` of the repo and the sibling checkouts of the named libs) are removed from `TCL8_6_TM_PATH` / `TCL9_0_TM_PATH` and inserted at the front in the runner. Foreign entries stay in place. Reason: if the tree already stands behind a `site-tcl` with the same version number, `tcl::tm::path add` has no effect, and the installed copy would be tested.
2. **Banner.** Where each lib resolves to — looked up without loading anything —, other versions alongside as `(also: <version>)`, set variables (`TCL8_6_TM_PATH`, `TCL9_0_TM_PATH`, `TCLLIBPATH`, `<LIB>_TM`) as `note: … is set: …`.
3. **Limit per file.** `-limit` seconds (default 120) or the value of the environment variable from `-limitvar`, if it is a positive integer; a hanging file is terminated (Unix: `kill -TERM`, then `-KILL`; Windows: `taskkill /T /F`).
4. **Invalid bytes.** Tcl 9 reads channels strictly: a byte that is invalid in the system encoding made the run stall until the limit. tutestrun replaces such bytes (`-profile replace`); Tcl 8.6 does not need this.
5. **Test files in UTF-8.** Every file starts as `tclsh -encoding utf-8 file.test`. Tcl 8.6 would otherwise read it in the system encoding, cp1252 on Windows, and every expected

value with an umlaut would be wrong; with tutestrun 0.1, 19 tests failed that way.

6. **Summary.** Failed tests, a missing tcltest total, a nonzero exit or hangs end up under "Files with problems". tcltest ends a file with 0 even on errors — the total line is what counts.

Options of `run`: `-argv`, `-limit`, `-limitvar`. Unknown options: `{TCLUTILS TUTESTRUN OPTION <opt>}`. A test file started directly without the runner remains exposed to the `site-tcl` problem.

22.3 Checking the origin: tupkgfinder 0.2

The package database knows only one `ifneeded` per version. If the same version lies twice on the paths, the second copy is not visible there. tupkgfinder 0.2 therefore searches by itself:

- `candidates` — every copy of a package on module paths and `auto_path`, as a dict `{package version kind path}`.
- `shadows` — per package with more than one copy: which one is active, and for each copy a `status` (`active`, `same version`, `other version`). `duplicate` is 1 if a version is present twice.

```
package require tclutils::tupkgfinder
foreach d [::tclutils::tupkgfinder::candidates -pattern
    tclutils::tunum] {
    puts "[dict get $d version] [dict get $d kind] [dict get $d path]"
}
foreach d [::tclutils::tupkgfinder::shadows -pattern tclutils::*] {
    if {[dict get $d duplicate]} { puts "doppelt: [dict get $d
    package]" }
}
```

Nothing is loaded in the process. The output is `0.4 tm …/lib/tm/tclutils/tunum-0.4.tm` ; `shadows` found no duplicate copy, and `tclutils::tunum` was not loaded afterwards. For packages with unregistered copies, `shadows` runs an unsatisfiable `package require`; in doing so Tcl reads the `pkgIndex.tcl` files but loads nothing.

`kind` is `tm` (path of the `.tm` file) or `pkgIndex` (path of the `pkgIndex.tcl`; only literal `package ifneeded NAME VERSION` lines are recognized). Wrong options throw `{TCLUTILS TUPKGFINDER OPTION}`, for example `unknown option "-x" (-pattern, -tmpaths, -autopath)`.

22.4 Checking Rules: tulint

tulint checks Tcl sources and returns the findings as data: a list of dicts with `file`, `line`, `rule` , `severity` and `message`. The module prints nothing and never exits; the command line is `bin/tulint.tcl`. The rule names are the identifiers of the book *Tcl/Tk Quality*:

Rule	Severity	Finds
ENC-1	warning	non-ASCII characters in a *.test file
ENC-2	error	source without -encoding in pkgIndex.tcl
ENC-3	warning	non-ASCII characters outside comments in a *.tcl script

Rule	Severity	Finds
TST-3	error	the same test name twice in a file
DAT-2	info	open with w, wb, w+ or TRUNC outside test files

```
package require tclutils::tulint
set f [::tclutils::tulint::checkText \
    "test a {} -result {}\ntest a {} -result {}\n" -file x.test]
::tclutils::tulint::format $f
;# -> x.test:2: error TST-3: test name "a" already used in line 1
::tclutils::tulint::summary $f        ;# -> error 1 warning 0 info 0
```

TST-3 reads complete commands (`info complete`), not lines, and descends into braced bodies. Two branches of the same `if` or `switch` count as alternatives. `check paths ?-rules names? ?-exclude patterns?` walks directories. Own rules are added with `register name severity description cmd`; `cmd` gets the file name and the text and returns pairs `{line message}`. A comment `# tulint: ignore` RULE applies to the next line,`;# tulint: ignore` RULE at the end of a line to that line.

`bin/tulint.tcl` exits with 1 as soon as a finding of severity `error` is among them. `--severity` filters only the output, not the exit code. An unknown option or rule is an error `{TCLUTILS TULINT OPTION}` or `RULE`.

22.5 Windows

According to the CHANGELOG, the suite also runs under Windows since 0.64.0: no `/dev/null` or Unix roots in fixtures; tests that need a CA bundle are skipped without one (constraint `ca0K`). In 0.65.0 the Windows tests `tutask` 2.8w/4.4w for `pipe` were added.

22.6 Summary

- `tests/all.tcl` → tutestrun: own trees first, banner, limit.
- Source the runner from the tree, do not require it blindly.
- `tupkgfinder::shadows` finds duplicate copies.
- `tulint` checks error classes from the quality book.

Next chapter: Limits

23 Limits

- No Tk — widgets and dialogs belong in tkutils.
- No complete GNU coreutils; only the filters shown.
- SFTP and TDBC: their own `package require`, external packages if needed.
- `tuflow` loads `tudiagram` immediately. The other diagram modules only on the first call of the respective kind. `parse` reads flowcharts; `pie` and the other kinds are drawn by `toSvg`. `validate` does not throw. See Diagrams.
- `tustr::graphemes` is a simplified UAX #29 (no Hangul syllables, no Indic conjuncts).

`slugify` returns ASCII; scripts without Latin letters are dropped.
- `tupdf` counts pages by `/Type /Page`. `zugferd` looks for markers; it does not check the invoice. `object` throws `NOTFOUND` if the number is missing after inflating. See ODF and PDF Structure.
- `tuodf` reads paragraphs from `text:p`. It skips tables. `text` joins with `\n`, so an empty final paragraph makes the text end with it.
- `tupdftext` does not read encryption or text from images, does not keep columns apart and does not rejoin words hyphenated at the end of a line.
- `tuuniq` removes only adjacent duplicates. `tucut` is not CSV.
- `tujson::parse` turns `true` into `1` and `null` into empty.
- `tuhash` is pure Tcl. Text is converted to UTF-8 before the digest.
- `tumd` does not know nested lists, tables or Setext headings. The anchor keeps letters of every script, digits, `_` and `-`.
- `tuical` and `tuvcard` leave property values raw. `toIcs` and `toVcf` write CR/LF.
- `tuholiday` knows only `-region de` and no state-specific days.
- `tudict::getOr` puts the default before the path. `flatten` takes an even-length list as a dict.
- `tudiff` builds the comparison table in memory, limit `-maxcells`.
- `tusqlite` does not load `sqlite3`. A cell written as NULL comes back empty from `rows`.
- `tutdbc` loads no TDBC driver and is not in the umbrella. SQL NULL stays a column with an empty value. See TDBC and MaxMind.
- `tummdb` keeps the file in memory. `open` and `close` are not exported. An address without a record yields the empty string.
- `tudate::diff -unit days` divides seconds by 86400. `relative` answers in English.
- `tupath::relative` requires two absolute or two relative paths.
- A ZIP provider has no `put`. The message has no `-errorcode`.
- `tufetch` without a server yields `URL` and `CAFILE`. `socketCmd` does not open a socket. Under tls 2 it appends no CA bundle.
- `tutask`: the sandbox runs in the calling thread; only the pool (package `Thread`) is parallel. On abort under Unix, `pipe` terminates only the direct processes, not grandchildren. `-commands` alone does not stop an empty loop. See Tasks with tutask.
- Module docs and man pages are the API truth; this book is the path.

Stack, file view, explorer: book *Utils-Stack*.

23.1 Windows notes

The table reflects code and CHANGELOG, not own tests under Windows.

Topic	Behavior under Windows
tunum::format	rounds as under Linux (since 0.4), not per the C runtime
tupath::clean	.. stops at the drive root C:/
tusettings::dir	%APPDATA%, %LOCALAPPDATA%, %TEMP%
Recent list	same file recognized regardless of case

Topic	Behavior under Windows
HTTPS with tls 1.x	no system CA store: SSL_CERT_FILE or -cafile
tufetch without CA bundle	falls back to curl/wget, otherwise NOMETHOD
Test suite	runs according to the CHANGELOG; CA tests skipped without a bundle
tutask::pipe	abort via taskkill /T /F, child processes included

---

23.2 Summary

- Engines yes, GUI and app glue no.
- Load optional backends explicitly.
- Under Windows: provide a CA bundle for tls 1.x.
- `tutask` has documented limits for processes and sandbox limits.

Next chapter: Pitfalls

24 Pitfalls

- `lappend auto_path` instead of `tcl::tm::path add` for `.tm` trees.
- Starting the suite without the runner while loading `site-tcl` with the same version ahead of the workspace.
- `package require tclutils::tutestrun` against an old install — `source` the runner from the tree.
- Registering the HTTPS policy per module yourself and never undoing it — use `tuhttps`.
- `tusettings` values with a line break or leading spaces — an INI limit, error `TUSETTINGS VALUE`.
- Calling `tusettings::load …` without the full namespace — `invalid command name`; correct is `::tclutils::tusettings::load`.
- Reading `TUFETCH CAFILE` as "no CA" — it means: `-cafile` is not readable. If no CA at all is available under tls 1.x: `TUHTTPS NOCA` or `TUFETCH NOMETHOD`.
- Trying to write to a ZIP provider — the caps are `get list stat`.
- Rounding amounts with `format %.2f` — the result depends on the C runtime; use `tunum::format`.
- `tuuniq` on an unsorted list — it removes only neighbors, `a b b a` stays `a b a`.
- `tucut` for CSV with quotes — it cuts at the delimiter; fields containing a comma belong to `tucsv`.
- `tujson::parse` and then checking for `null` or `false` — `null` is empty, `true` is `1`, `false` is `0`. `parseTyped` keeps the kind.
- Writing to a ZIP provider and waiting for an `-errorcode` — the message is `does not support 'put'`, without a code.
- Taking `string length` as the visible character count — `tustr::graphemes`. `truncate` and `padLeft` still calculate with `string length`.
- Taking `tustr::slugify` and the `tumd` anchor for the same thing — `slugify` turns `Größe` into `groesse` (ASCII), `tumd` into the anchor `größe`.

- Reading an indented Markdown list as nested — `tumd` turns it into a flat list.
- Passing the stored value to `tunotes::create` — the change requires the variable name. `get` and `search` receive the value.
- Reading `tunotes::search` as a regular expression — it is `string match`, an asterisk in the query is a pattern character.
- Expecting state-specific public holidays from `tuholiday` — only the nationwide set `de`, plus Easter Sunday and Whit Sunday (without them: `-sundays 0`).
- `tuflow::parse` on a `pie` — `{TCLUTILS TUFLOW UNSUPPORTED}`. Drawing goes through `toSvg`.
- `tudiagram::addEdge` on unknown ids and expecting an error — the edge is attached to the model. `validate` reports `NONODE` and does not throw.
- Reading `flowchart TD` as direction `TD` in the model — it becomes `TB`.
- `tunum::parse -locale de-strict` and a period as decimal separator — `956.06` becomes `95606.0`.
- Reading `tudate::diff -unit days` across daylight saving time as calendar days — seconds divided by 86400; March 15 to April 15, 2024 gives 30.
- Reading `02/03/2026` as 3 February — `tudate::parse` tries `d/m/Y` before `m/d/Y`, so that is 2 March. `m/d/Y` only applies when `d/m/Y` gives no date (`10/29/2026`).
- A string with an umlaut passed to `tubase64::encode` or `tucrc::crc32` without `-encoding utf-8` — ö becomes the single byte `f6`, € is `BYTES`. `tuhash` takes UTF-8 instead.
- Expecting German from `tudate::relative` — the texts are `today`, `tomorrow`, `yesterday`.
- Reading `tudict::getOr` like `dict get` — the default comes before the path: `getOr $d vorgabe schlüssel …`.
- `tudict::flatten` on a value like `a b` — an even length counts as a dict, `{name {a b}}` becomes `name.a b`.
- `tuhash` on file bytes with the text default — `sha256` encodes UTF-8; `sha256File` hashes the bytes of the file.
- Expecting invalid Base64 to be an error — `decode` of `!!!` is empty, without `-errorcode`.
- Recognizing SQL NULL from `tusqlite::rows` by the sentinel — the cell comes back empty. `insert` sets NULL with `null`.
- Expecting `tusqlite::rows` to take a placeholder `:x` from a variable `x` — since 0.2 the value comes only from the parameter dict; without an entry it is NULL.
- With TDBC `allrows -as dicts`, reading a missing column as empty — NULL columns drop out of the dict. `tutdbc::rows` puts them back in as empty. `0` stays `0`.
- Expecting `tummdb::close` on an unknown handle to be an error — the call returns the empty string. `metadata` throws `HANDLE`.
- `tupdftext` on a scan — `{TCLUTILS TUPDFTEXT NOTEXT}`, not a silent empty string. `-notext empty` returns the empty page.
- `tupdf::object` on a number that is missing after inflating — `{TCLUTILS TUPDF NOTFOUND}`. The message names an object stream.
- Expecting `tuodf::text` without a line break after an empty final paragraph — the paragraphs are joined with `\n`, the text ends with it.
- Overlooking a second copy of the same version in the module path — `tupkgfinder::shadows`.
- Calling `$t wait` outside a task in a Tk handler — nested `vwait`; in the handler only `spawn`.

- `await` outside a coroutine — `TUTASK NOTASK`; wrap it in `spawn` or `run`.
- Limiting a sandbox with `-commands` only — `while 1 {}` keeps running; set `-time` as well.
- Expecting under Unix that `tutask::pipe` also ends grandchild processes on cancel — only the direct processes of the pipeline.
- Never collecting a task error — on `destroy` it arrives through `bgerror`.
- Expecting tkutils code in tclutils.

24.1 Fixed in 0.66.0

These pitfalls apply up to 0.65.0. They are fixed in 0.66.0:

- `$g wait -mode first` outside a task and destroying the group immediately — in 0.65.0 a later `bgerror` "invalid command name ...::my"; needed up to 0.65.0: `after idle [list $g destroy]`.
- `pool` fails in the constructor (`NOTHREAD`, `-size 0`) — in 0.65.0 an additional `bgerror can't read "jobs": no such variable`.
- `-init` of the pool fails — in 0.65.0 the job stays in the queue (`queued 1 jobs 1`).
- `tclutils::common` older than 0.2 on the path — `tutask` 0.1 loads anyway, `group` and `pool` fail only when called. `tutask` 0.2 requires `common` 0.2.
- `run -timeout`, `$t wait -timeout` or `$g wait -timeout` outside a task with `abc` or `-5` — up to 0.65.0 silently accepted, there was no timeout. From 0.66.0 on `TUTASK OPTION` as with `await`.

24.2 Summary

- Most pitfalls concern module paths, HTTPS policy and rounding.
- New in 0.65.0: pitfalls around `tutask` (Tk handlers, groups, sandbox, processes). Four of them and the silent acceptance of invalid `-timeout` values are fixed in 0.66.0.

Next chapter: Modules (Orientation)

25 Modules (Orientation)

Not a complete list — for state and status see `docs/guide/module-status.md` and `README.md` in the tclutils tree. Hygiene count with `tools/check-modules.tcl -require test,doc,man`: **146** modules in the 0.69.0 tree, none without a test, doc or man page; 25 of them not directly in the umbrella.

25.1 Categories

Category	Examples
Text filters	tucat, tuhead, tugrep, tusort, tucut, tused (chapter Text and Files); tunl, tufold, tufmt, tuexpand, tucolumn, tutable, tutac, turev, tushuf, tuseq, tutr, tuwc, tupr (chapter More Line Filters)

Category	Examples
Joining / pipelines	tupaste, tujoin, tucomm, tucsplit, tusplit, tutee, tuxargs, tutsort, tuawk, tufuzzy, tuagrep (chapter Joining, Splitting, Pipelines)
Data	tucsv, tujson, tuini, tuxml, tusqlite, tuldif, tubookmark (chapter Data Formats); tutdbc, tummdb (chapter TDBC and MaxMind)
Encoding	tubase64, tubase32, tucrc, tuhash, tuxxhash, tuiconv, tuuuid, tuurl, tudict, tudiff, tupatch (chapter Encoding, Hashing and Comparing)
Binary data	tuhexdump, tuod, tustrings, tuhexedit, tucmp, tusize, tustat, tucode (chapter Binary Data and File Information)
Notes	tumd, tuical, turrule, tuvcard, tunotes, tunotesdb, tucal, tuholiday (chapter Notes and Calendars)
Documents	tupdftext, tupdf, tuodf (chapters Text from PDF Files, ODF and PDF Structure)
Diagrams	tudiagram, tuflow and the kinds from toSvg (chapter Diagrams)
Images	tucolor, tupng, tupngdraw, tutablepng, tucodepng, tumonthpng, tupngpad, tusvg, tuimage, tuico, tudhash (chapter Images without Tk)
FS / archive	tufile, tufind, tuzip
Provider	tuprovider, ...::zip, ...::dav
Application	tulog, tuevent, turegistry, tuvalidate, tulist, tumath, tunumany, tuappinfo, tudeploy, tuexe, tuopen, tuzipfs, tuterm, tupagespec, tulayout (chapter Building Blocks for Applications)
Network	tufetch, tudav, tuhttps, tusparql
Settings / test	tusettings, tutestrun, tulint
Tasks / async	tutask
Strings / dates / numbers	tustr, tudate, tunum, tupath (chapter Characters, Numbers, Dates and Paths)
Diagnostics	tupkgfinder, tuappinfo, tusysinfo, tuedid

25.2 New in 0.69.0

Module	Version	Change
tucsv	0.2	-profile excel-de or rfc4180, text -bom 1, -encoding for file and writeFile (default UTF-8); writeFile keeps the line ends of -newline
tusqlite	0.2	rows and value with a parameter dict, the caller's variables no longer reach the query (behaviour); new update, delete
tusysinfo	0.1	new: operating system, CPU, memory, model, disks, monitors; in the umbrella; bin/tusysinfo.tcl
tuedid	0.1	new: read a monitor's EDID block; in the umbrella
tuholiday	0.2	-sundays 0 for the nine nationwide days; default 1 as before
tuzip	0.2	names in UTF-8 with the UTF-8 flag (behaviour); reading with -encoding auto
tudiff	0.2	-eol ignore|strict, -encoding for files; text ignores CR in strings too (behaviour)
tubookmark	0.3	order kept, key folders, HREF in single quotes (behaviour); <DD> description and toolbar folder are kept
tuical, tuvcard	0.2	toIcs and toVcf end the last line with CR/LF too (behaviour)
tuini	0.2	setValue, addSection, toIni refuse what INI cannot hold, such as a line break in a value (behaviour)
tupatch	0.3	the final \n stays; \ No newline at end of file as with diff -u (behaviour)
tusplit	0.3	line parts keep their line ends; file -lines splits bytes (behaviour)
tutee	0.3	writeFile copies bytes, returns the byte count (behaviour)
tumd	0.2	anchors as on GitHub, with umlauts and -1, -2 (behaviour)

Module	Version	Change
tuiconv	0.2	strict under 8.6 and 9.0, -replace 1 (behaviour)
tucat	0.2	no number after the last \n, files numbers through (behaviour)
tuldif	0.3	binary attributes keep their bytes, -binary (behaviour)
tucsplit	0.2	parts keep their line ends, -encoding (behaviour)
tuhead, tutail	0.2	line ends as head/tail, text = file, COUNT (behaviour)
tugrep	0.2	no empty line after the last \n (behaviour)
tustr	0.3	slugify spells umlauts out, toCamel/ toSnake with Unicode (behaviour)
tuer	0.4	entity names with umlauts; SYNTAX (behaviour)
tugantt	0.3	unknown unit DURATION; ids with umlauts; writeSvg/writePng through a temporary file (behaviour)
tuxychart	0.2	COUNT when values and categories do not match; writing through a temporary file (behaviour)
tusankey	0.2	bad lines VALUE instead of dropped silently; writing through a temporary file (behaviour)
tudate	0.3	days and months that do not exist are PARSE, also on 8.6; m/d/Y after d/m/Y; more formats (behaviour)
tunumany	0.3	decimal comma before the unit, 1,5K is 1500 (behaviour)
tubase64, tucrc	0.2	-encoding binary|utf-8; a character above 255 is BYTES (behaviour)
tudav	0.3	Basic authentication in UTF-8; . and .. in an href resolved (behaviour)
tuflow	0.4	node ids with umlauts (behaviour)
tuc4	0.2	ids with umlauts (behaviour)

Module	Version	Change
tugit	0.3	duplicate commit id is DUPID (behaviour)
turadar	0.3	exactly one value per axis, else COUNT or AXIS; writing through a temporary file (behaviour)
tusort, tuuniq, tucut, tujoin, tucolumn, tufmt	0.2	trailing \n as in the input (behaviour)
tupaste, tushuf, tufold	0.3	trailing \n as in the input (behaviour)
tutestrun	0.2	test files with -encoding utf-8 (behaviour)
tuarchitecture	0.3	ids with umlauts (behaviour)
tustate, tublock	0.2	ids with umlauts (behaviour)
tunotes	0.2	file always UTF-8; older files in the system encoding are still read (behaviour)
tumindmap	0.2	ids with umlauts before a shape (behaviour)
tulint	0.1	new: checking rules for Tcl sources, not in the umbrella
tudocx, tuxlsx	0.1	new: write and read Word and Excel files, like tuodf; in the umbrella
common	0.3	new replaceFile: writing through a temporary file
tused, tusettings	0.2	writing through replaceFile; if it fails, the old file stays and no channel is left open (behaviour)
tuprovider	0.3	local put through replaceFile (behaviour)
tutreemap	0.3	:::class with umlauts removed whole; writing through a temporary file (behaviour)
tuclass	0.3	; inside quotes does not split members (behaviour)
tuimage	0.2	;base64 inside a quoted parameter does not switch on Base64 (behaviour)

Module	Version	Change
tupacket, tujourney, tutimeline, tupie, tusequence	0.3	writeSvg/writePng through replaceFile (behaviour)
tukanban, tuquadrant	0.2	writeSvg/writePng through replaceFile (behaviour)
tudiagram	0.6	writePng through replaceFile (behaviour)
tusvg	0.4	write through replaceFile (behaviour)
tupngdraw	0.14	write through replaceFile (behaviour)
tupng	0.6	write* through replaceFile (behaviour)
tuappinfo	0.3	Tk line with patch level; writeLog UTF-8 through replaceFile (behaviour)
tuodf	0.2	paragraphs also returns headings (behaviour)

When copying over an older tree, delete `tucsv-0.1.tm`, `tusqlite-0.1.tm`, `tuholiday-0.1.tm`, `tuzip-0.1.tm`, `tudiff-0.1.tm`, `tubookmark-0.2.tm`, `tuical-0.1.tm`, `tuvcard-0.1.tm`, `tuini-0.1.tm`, `tupatch-0.2.tm`, `tusplit-0.2.tm`, `tutee-0.2.tm`, `tumd-0.1.tm`, `tuiconv-0.1.tm`, `tucat-0.1.tm`, `tuldif-0.2.tm`, `tucsplit-0.1.tm`, `tuhead-0.1.tm`, `tutail-0.1.tm`, `tugrep-0.1.3.tm`, `tustr-0.2.tm`, `tuer-0.3.tm`, `tugantt-0.2.tm`, `tuxychart-0.1.tm`, `tusankey-0.1.tm`, `tudate-0.2.tm`, `tunumany-0.2.tm`, `tubase64-0.1.tm`, `tucrc-0.1.tm`, `tudav-0.2.tm`, `tuflow-0.3.tm`, `tuc4-0.1.tm`, `tugit-0.2.tm`, `turadar-0.2.tm`, `tusort-0.1.tm`, `tuuniq-0.1.tm`, `tucut-0.1.tm`, `tupaste-0.2.tm`, `tujoin-0.1.2.tm`, `tushuf-0.2.tm`, `tucolumn-0.1.tm`, `tufold-0.2.tm`, `tufmt-0.1.tm`, `tutestrun-0.1.tm`, `tuarchitecture-0.2.tm`, `tustate-0.1.tm`, `tublock-0.1.tm`, `tunotes-0.1.tm`, `tumindmap-0.1.tm`, `common-0.2.tm`, `tused-0.1.3.tm`, `tusettings-0.1.tm`, `tuimage-0.1.tm`, `tuprovider-0.2.tm`, `tutreemap-0.2.tm`, `tuclass-0.2.tm`, `tupacket-0.2.tm`, `tujourney-0.2.tm`, `tukanban-0.1.tm`, `tutimeline-0.2.tm`, `tupie-0.2.tm`, `tuquadrant-0.1.tm`, `tusequence-0.2.tm`, `tudiagram-0.5.tm`, `tusvg-0.3.tm`, `tupngdraw-0.13.tm`, `tupng-0.5.tm`, `tuappinfo-0.2.tm`, `tuodf-0.1.tm` and `tclutils-0.68.0.tm`. The umbrella is at 0.69.0.

25.3 New in 0.68.0

Corrections from the review of this book; each with a regression test. Rows marked "behavior" change previous results.

Module	Version	Change
tuagrep	0.2	-maxdist checked; search without empty final line; error codes (behavior); number-like text kept

Module	Version	Change
tuappinfo	0.2	-anonymize hides all paths; unknown options are errors
tuarchitecture	0.2	label falling back to the id
tuawk	0.2	rules in their own frame (8.6 = 9.0); NR/NF initially 0; option without value is an error
tubase32	0.2	decode checks length and padding bits (LENGTH, BITS) (behavior)
tubookmark	0.2	escaping of "; each <A> one entry
tucal	0.2	year row by row like cal; title centered; no spaces at end of line (behavior)
tuclass	0.2	single-line class block; edges with cardinalities stay
tucmp	0.2	compares bytes; characters above 255 are BYTES (behavior)
tucode	0.2	signs with options; lookup U+00E4; matrix aligned
tucolor	0.2	triples decimal (8.6 = 9.0); fromHsv checks ranges (HSV) (behavior)
tucomm	0.2	-sort 1 sorts like the comparison (behavior)
tudeploy	0.2	addModulePaths without rejected roots; options checked first
tudhash	0.2	width and height checked
tudiagram	0.5	title is drawn; unknown theme is THEME; number-like text kept
tuer	0.3	quotes around labels removed
tuexe	0.2	option without value is an error
tuexpand	0.2	unexpand like GNU; expand rejects -all (behavior)
tufile	0.2	type, mime unchanged
tufind	0.2	-tails name unchanged
tuflow	0.3	header line case-insensitive; A & B --> C; sankey recognized
tufold	0.2	-words 1 without spaces at end of line
tufuzzy	0.2	error codes; -nocase checked
tugantt	0.2	without start date DATE; axis without duplicate dates; height automatic

Module	Version	Change
tugit	0.2	commit id as label
tugrep	0.1.3	file name with -fileswithmatches
tuhexdump	0.2	options checked; error codes; BYTES
tuhexedit	0.2	OFFSET, RANGE (no writing past the end), BYTES, ENCODING (behavior)
tuico	0.2	directory read unsigned
tujoin	0.1.2	options through parseOptions; error codes
tujourney	0.2	ratings with decimal places; otherwise VALUE
tulayout	0.2	ensureBlocks accepts definitions; options checked (OPTION)
tuldif	0.2	Base64 over UTF-8; CHANGETYPE (behavior)
tulog	0.2	options checked before creation
tummdb	0.2	IPv6 expanded correctly at ::
tumonthpng	0.5	header follows -firstweekday; -title for quarter and year (behavior); number-like text kept
tunl	0.2	start, step, width checked
tunotesdb	0.2	fromStore keeps children; child before parent allowed
tunumany	0.2	option without value is an error
tunumfmt	0.2	KiB etc. as IEC (2KiB = 2048)
tuod	0.2	-format/-address checked in advance; BYTES
tuopen	0.2	all commands exported; URLs under Windows through rundll32
tupacket	0.2	field label
tupagespec	0.2	numbers always decimal; number-like text kept
tupaste	0.2	options through parseOptions
tupatch	0.2	-reverse/-strict checked
tupie	0.2	1e3 valid; negative values VALUE
tupng	0.5	tRNS with color type 2; CRC checked (CRC); number-like text kept
tupngdraw	0.13	paste blends alpha correctly

Module	Version	Change
tupngpad	0.2	options checked; unknown -align is an error (behavior)
tuprovider	0.2	head returns content unchanged
tuprovider::sftp	0.2	host in the sftp target
turadar	0.2	legend only for named series; number-like text kept
turrule	0.2	YEARLY with BYMONTH/BYDAY/BYMONTHDAY; UNTIL as a whole day; UNSUPPORTED (behavior)
tuseq	0.2	options in any position; large numbers under 8.6
tusequence	0.2	<<->>; title is drawn; number-like text kept
tushuf	0.2	large -seed under 8.6
tusize	0.2	links with their own size; human up to YiB (behavior)
tusparql	0.2	endpoint with ?; Accept only once; all variables; number-like text kept
tusplit	0.2	splitBytes with BYTES; missing value is an error
tustat	0.2	links with lstat and target; modeOctal (behavior)
tustrings	0.2	-minlength checked; -encoding dropped; BYTES (behavior)
tusvg	0.3	method icon valid; 149 symbols
tutable	0.2	COLUMNS, HEADERS; minimum width 3
tutablepng	0.2	cell text unchanged; -padding/-spacing checked
tutdbc	0.2	column values in rows unchanged (01234)
tutee	0.2	open all targets first, clean up on error
tuterm	0.2	names reset; enable checked (behavior)
tutimeline	0.2	12:30 stays whole
tutr	0.2	RANGE, SET2; last mapping wins; squeeze-only possible
tutreemap	0.2	values ≤ 0 are dropped
tutsort	0.2	cycle message only with cycle nodes
tuvalidate	0.2	port decimal digits only; ipv4 without leading zeros
tuwc	0.2	file: bytes is the file size (behavior)
tuxxhash	0.2	text as UTF-8, option -encoding (behavior)

Module	Version	Change
tuzipfs	0.3	rcopy with dot files

When copying over an older tree, delete the module's old file (e.g. `tuawk-0.1.tm` next to `tuawk-0.2.tm`). The umbrella was at 0.68.0.

25.4 New in 0.67.0

Module	Version	Change
tupdftext	0.1	new: text of a PDF per page, pure Tcl; in the umbrella; tupdf.tcl text

Otherwise the module versions stayed the same; the umbrella was at 0.67.0.

25.5 New in 0.66.0

Module	Version	Change
tutask	0.2	requires common 0.2; pool and group errors fixed; -timeout checked everywhere

Otherwise the module versions stayed the same; the umbrella was at 0.66.0. When copying over an older tree, delete `tutask-0.1.tm`.

25.6 New in 0.65.0

Module	Version	Change
tutask	0.1	new: tasks on coroutines, await, cancel, groups, pool, sandbox; in the umbrella

Otherwise the module versions stayed the same; the umbrella was at 0.65.0.

25.7 New and Changed in 0.64.0

Module	Version	Change
tustr	0.2	graphemes, isPrintable (Unicode)
tudate	0.2	zones
tunum	0.4	format rounds half away from zero, the same on every platform
tupath	0.1	clean keeps the root (C:/ under Windows)
tusettings	0.1	new: INI settings, dir, recent list
tutestrun	0.1	new: shared runner, not in the umbrella
tupkgfinder	0.2	candidates, shadows

Still relevant from 0.63.0: `tuhttps` 0.1 (new), `tufetch` 0.4, `tudav` 0.2, `tupostgrest` 0.2, `tuprovider::ftp` 0.2 (tcllib only on `open`), `common` 0.2 (`ensureOneOf`).

25.8 Summary

- 148 modules in the 0.69.0 tree, all with a test, doc and man page.
- New in 0.69.0: `tucsv`, `tusqlite`, `tuholiday`, `tuzip`, `tudiff`, `tuical`, `tuvcard`, `tuini`, `tumd`, `tuiconv`, `tucat`, `tucsplit`, `tuhead`, `tutail`, `tugrep`, `tuxychart`, `tusankey` 0.2, `tupatch`, `tusplit`, `tutee`, `tuldif`, `tustr`, `tugantt` 0.3, `tuer` 0.4, `tubookmark` 0.3, `tudate`, `tunumany`, `tubase64`, `tucrc`, `tudav`, `tuflow`, `tuc4`, `tugit`, `turadar` and nine line filters, new `tusysinfo`, `tuedid`, `tulint`, `tudocx` and `tuxlsx` 0.1.
- In 0.68.0, 76 modules got fixes and a new version.
- New in 0.67.0: `tupdftext` 0.1.
- In 0.66.0 only `tutask` changed (0.2); new in 0.65.0 was `tutask` 0.1.

Next chapter: Glossary

26 Glossary

---

umbrella
: Package `tclutils`, loads the core modules.

module path
: `tcl::tm::path` — where `.tm` files are found.

provider
: Uniform storage API (`tuprovider` + backends).

caps
: List of the operations a provider can actually perform (e.g. ZIP: `get list stat`).

tusettings
: Per-user INI and recent lists.

tutestrun
: Shared `tests/all.tcl` runner of the three libraries.

tuhttps
: Shared HTTPS policy: verifying `tls` sockets, `https` registered only temporarily.

tutask
: Tasks on coroutines with `await`, timeout, `cancel`, groups, pool and sandbox (new in 0.65.0, version 0.2 in 0.66.0).

task
: Coroutine owned by an object (`spawn`); state `running`, `ok`, `error` or `cancelled`.

starter
: Command prefix of an operation: receives a callback, returns a cancel command, reports `ok` or `error` once.

sandbox
: Safe interpreter with a command and time limit (`tutask::sandbox`).

NOCA
: tls < 2.0 and no CA bundle found.

grapheme
: Visible character, possibly made of several code points (`tustr::graphemes`).

tudiagram
Node-edge model, layout in pixels, SVG and PNG in pure Tcl.

tuflow
Flowchart text to `tudiagram`. `toSvg` also draws the other kinds.

tumd
Small CommonMark subset to HTML, without nested lists.

tuholiday
Easter and the nationwide public holidays, region `de`.

tutdbc
NULL-safe helpers on an existing TDBC connection. Not a driver, not in the umbrella.

tummdb
Reader for MaxMind DB 2.0. `open` and `close` only qualified.

tuhash
SHA-256, SHA-1 and MD5 in pure Tcl. The text default is UTF-8.

tuurl
Percent encoding and query strings. For the query `-plus` is on.

tupdftext
Text of a PDF page by page, pure Tcl, without poppler (0.1, new in 0.67.0).

tupdf
PDF structure: version, objects, info, ZUGFeRD hint. The page text is handled by `tupdftext`.

tuodf
Plain OpenDocument from paragraphs. `mimetype` is the first entry.

shadow copy
Second copy of a package on the search paths that `package require` does not pick (`tupkgfinder::shadows`).

26.1 Summary

- Terms from paths, providers, HTTPS, tests, `tutask`, notes, PDF, ODF and TDBC.

Next chapter: Error Messages

27 Error Messages

Typical `-errorcode` patterns (excerpt). Complete: the respective `docs/<modul>.md`.

Code	Meaning
{TCLUTILS COMMON OPTION ...}	unknown option (parseOptions)
{TCLUTILS COMMON VALUE ...}	value not in the allowed set
{TCLUTILS COMMON BOOLEAN ...}	option requires a boolean
{TCLUTILS COMMON INTEGER ...}	option requires a positive integer
{TCLUTILS TUSETTINGS APP}	app name empty or containing path characters
{TCLUTILS TUSETTINGS KIND}	unknown kind for dir

Code	Meaning
{TCLUTILS TUSETTINGS SYNTAX}	INI not parsable
{TCLUTILS TUSETTINGS VALUE}	value or key not suitable for INI
{TCLUTILS TUHTTPS NOTLS}	package tls missing
{TCLUTILS TUHTTPS CAFILE}	-cafile is not a readable file
{TCLUTILS TUHTTPS NOCA}	tls < 2.0 and no CA bundle found
{TCLUTILS TUFETCH CAFILE}	-cafile is not a readable file (only on the native path with tls)
{TCLUTILS TUFETCH NOMETHOD}	no transport (also: tls 1.x without CA bundle, no curl/wget)
{TCLUTILS TUFETCH CONNECT}	connection/DNS/certificate failed
{TCLUTILS TUFETCH HTTP <Klasse> <Status>}	HTTP status outside 2xx, e.g. HTTP 4XX 404
{TCLUTILS TUFETCH URL}	URL neither http:// nor https://
{TCLUTILS TUFETCH TIMEOUT}	timeout
{TCLUTILS TUFETCH REDIRECT}	too many redirects
{TCLUTILS TUCSV QUOTE}	CSV field with an open quote (-strict 1)
{TCLUTILS TUCSV PROFILE <name>}	-profile neither excel-de nor rfc4180
{TCLUTILS TUJSON PARSE offset}	JSON invalid at this position
{TCLUTILS TUJSON TYPE}	toJson with an unknown kind
{TCLUTILS TUINI SYNTAX}	INI not parsable, including an open section
{TCLUTILS TUBASE64 MAXLEN}	-maxlen not an integer from 0 up
{TCLUTILS TUBASE64 BYTES}	encode without -encoding utf-8 and a character above 255
{TCLUTILS TUCRC BYTES}	crc32, adler32 without -encoding utf-8 and a character above 255
{TCLUTILS TUHASH ENCODING}	-encoding neither utf-8 nor binary
{TCLUTILS TUUUID INVALID}	version without UUID form
{TCLUTILS TUUUID VERSION}	generate -version neither 4 nor 7
{TCLUTILS TUDICT ARG}	getOr without a key
{TCLUTILS TUDIFF OPTION MAXCELLS}	-maxcells not a positive integer
{TCLUTILS TUDIFF TOO_LARGE}	old lines times new lines above -maxcells

Code	Meaning
{TCLUTILS TUXML NAME <name>}	element name invalid
{TCLUTILS TUXML ATTR <name>}	attribute name invalid
{TCLUTILS TUSQLITE EMPTY}	insert without a column, update without a new value
{TCLUTILS TUSQLITE WHERE}	update or delete with an empty condition dict
{TCLUTILS TUSQLITE PARAM <name>}	invalid placeholder name in rows/value
{TCLUTILS TUSYSINFO OPTION}, SECTION, PLATFORM	collect: unknown option, unknown section, unknown platform
{TCLUTILS TUEDID SHORT}, HEADER, HEX	EDID shorter than 128 bytes, no EDID header, not hex
{TCLUTILS TUTDBC EMPTY}	tutdbc::insert without a column
{TCLUTILS TUMMDB FORMAT}	file is not a MaxMind DB
{TCLUTILS TUMMDB HANDLE}	unknown handle for metadata or lookup
{TCLUTILS TUICAL SYNTAX}	iCalendar line without :, or END without BEGIN
{TCLUTILS TUICAL ARG}	newComponent with an odd property list
{TCLUTILS TUVCARD SYNTAX}	vCard line without :
{TCLUTILS TUNOTES NOTFOUND}	note or parent missing
{TCLUTILS TUHOLIDAY YEAR}	easter without a year
{TCLUTILS TUHOLIDAY REGION}	-region is not de
{TCLUTILS TUDIAGRAM DUPID}	node id already exists
{TCLUTILS TUDIAGRAM DIR}	direction neither LR, TB, RL nor BT
{TCLUTILS TUDIAGRAM ARG}	node shape outside the eight names
{TCLUTILS TUFLOW BADID}	invalid node id, e.g. with an appended -...
{TCLUTILS TUFLOW EMPTY}	flowchart without nodes
{TCLUTILS TUFLOW UNSUPPORTED}	parse on a kind that only toSvg draws
{TCLUTILS TUGIT DUPID}	a commit id occurs twice
{TCLUTILS TUICAL VALUE}, {TCLUTILS TUVCARD VALUE}	a value with a line break for setProperty or addProperty
{TCLUTILS TUSQLITE OPTION}	value with an unknown option or an option without a value
{TCLUTILS TURADAR COUNT}	number of values does not match the axes

Code	Meaning
{TCLUTILS TURADAR AXIS}	series without a value for an axis, or a key without an axis
{TCLUTILS TUPIE EMPTY}	pie chart without a sector
{TCLUTILS TUPIE VALUE}	negative value, or sectors do not sum to a positive value
{TCLUTILS TUC4 EMPTY}	C4 without elements
{TCLUTILS TUSTR ARG}	isPrintable not with exactly one character; truncate with a negative length
{TCLUTILS TUPATH RELATIVE}	relative: one path absolute, the other relative
{TCLUTILS TUPATH ARGS}	commonPath without a path
{TCLUTILS TUPATH NOTLINK}	readlink on something that is not a symbolic link
{TCLUTILS TUDATE PARSE}	tudate::parse cannot read the date, also a day that does not exist (2026-02-29)
{TCLUTILS TUDATE UNIT}	tudate::add with an unknown unit
{TCLUTILS TUNUM DECIMALS}	format -decimals not an integer from 0 up
{TCLUTILS TUNUM LOCALE}	parse locale not auto/de-strict, or format locale not de
{TCLUTILS TUPDFTEXT FILE}	PDF file not readable
{TCLUTILS TUPDFTEXT FORMAT}	no %PDF- header, no objects or no pages
{TCLUTILS TUPDFTEXT ENCRYPTED}	PDF encrypted
{TCLUTILS TUPDFTEXT NOTEXT}	no page has text
{TCLUTILS TUPDFTEXT OPTION}	unknown option or missing value
{TCLUTILS TUTESTRUN OPTION ...}	unknown runner option
{TCLUTILS TUTASK CANCELLED}	task cancelled or pool destroyed
{TCLUTILS TUTASK TIMEOUT}	-timeout of await/wait expired
{TCLUTILS TUTASK NOTASK}	await outside a coroutine
{TCLUTILS TUTASK NOTDONE}	result of a running task
{TCLUTILS TUTASK OPTION}	wrong argument (spawn, timer, pipe, -timeout)
{TCLUTILS TUTASK NOTHREAD}	pool without package Thread
{TCLUTILS TUTASK EMPTY}	wait -mode first on an empty group

Code	Meaning
{TCLUTILS TUTASK INIT}	-init of the pool failed
{TCLUTILS TUTASK INSIDE}	destroy from within the task itself
{TCLUTILS TUTASK PROTOCOL}	coroutine resumed unexpectedly
{TCLUTILS TUTASK LIMIT COMMANDS}	command limit of the sandbox
{TCLUTILS TUTASK LIMIT TIME}	time limit of the sandbox
{TCLUTILS TUPKGFINDER OPTION}	unknown option / wrong count
{TCLUTILS TUPDF FORMAT}	no %PDF- header, or endobj missing
{TCLUTILS TUPDF ARG}	object number is not an integer
{TCLUTILS TUPDF NOTFOUND}	object number not in the text after inflating
{TCLUTILS TUNL START|INCREMENT|WIDTH}	tunl: start or step not an integer, width not an integer from 1 up
{TCLUTILS TUTR RANGE}	tutr: descending range such as c-a
{TCLUTILS TUTR SET2}	tutr: empty second set without -delete
{TCLUTILS TUTABLE COLUMNS}	tutable: row with more cells than headers
{TCLUTILS TUTABLE HEADERS}	tutable: empty header list
{TCLUTILS TUAWK OPTION}	tuawk: unknown option or option without value
{TCLUTILS TUAGREP MAXDIST}	tuagrep: -maxdist not an integer from 0 up
{TCLUTILS TUSPLIT BYTES}	tusplit::splitBytes: characters above \xFF
{TCLUTILS TUSPLIT ENCODING name}	tusplit::splitLines: unknown -encoding
{TCLUTILS TUICONV UNMAPPABLE enc}	tuiconv: character cannot be written in enc (without -replace 1)
{TCLUTILS TUICONV INVALID enc}	tuiconv: bytes invalid in enc (without -replace 1)
{TCLUTILS TULDIF BINARY}	tuldif::toLdif: binary value with characters above \xFF
{TCLUTILS TULDIF OPTION}	tuldif: an option other than -binary
{TCLUTILS TUINI VALUE}	tuini: value, key or section INI cannot hold
{TCLUTILS TUCSPLIT ENCODING name}	tucsplit: unknown -encoding
{TCLUTILS TUHEAD COUNT}, {TCLUTILS TUTAIL COUNT}	count not an integer from 0

Code	Meaning
{TCLUTILS TUER SYNTAX}	tuer: line neither block nor entity nor relationship
{TCLUTILS TUGANTT DURATION}	tugantt: duration with an unknown unit
{TCLUTILS TUXYCHART COUNT}	tuxychart: number of values does not match the categories
{TCLUTILS TUSANKEY VALUE}	tusankey: line not source,target,value with a number from 0
{TCLUTILS <MODUL> BYTES}	bytes expected, characters above 255 found (tuhexdump, tuod, tustrings, tucmp, tuhexedit, tuxxhash)
{TCLUTILS TUHEXEDIT OFFSET|RANGE|ENCODING}	invalid offset; writing past the end of the file; character not encodable
{TCLUTILS TUHEXDUMP OFFSET|LENGTH}	-offset or -length invalid
{TCLUTILS TUCODE LOOKUP}	tucode::lookup: empty or outside the table
{TCLUTILS TUPNG CRC}	PNG chunk with a wrong or missing CRC
{TCLUTILS TUCOLOR HSV}	fromHsv: h not 0..360 or s/v not 0..100
{TCLUTILS TUTABLEPNG PADDING|SPACING}	not an integer from 0 up
{TCLUTILS TUMONTHPNG YEAR|FIRSTWEEKDAY}	year not 1..9999; first weekday not 1..7
{TCLUTILS TUPNGPAD TRIM|SQUARE|SIZE}	invalid value of the option
{TCLUTILS TULAYOUT OPTION}	tulayout: unknown option or option without value
{TCLUTILS TUAPPINFO OPTION|VALUE}	tuappinfo: unknown option; invalid value
{TCLUTILS TUDEPLOY VALUE}	-tmadd or -fail not a boolean
{TCLUTILS TUXXHASH OPTION|ENCODING}	tuxxhash: unknown option; -encoding not utf-8/binary
{TCLUTILS TULDIF CHANGETYPE}	LDIF change record (changetype:) is not read
{TCLUTILS TURRULE UNSUPPORTED}	RRULE part that turrule does not evaluate
{TCLUTILS TURRULE BYDAY|VALUE}	unknown weekday abbreviation; invalid value
{TCLUTILS TUBASE32 LENGTH|BITS}	Base32: impossible length; padding bits not 0
{TCLUTILS TUDIAGRAM THEME}	unknown diagram theme
{TCLUTILS TUJOURNEY VALUE}	rating not a number 1..5
{TCLUTILS TUPROVIDER FTP NOPKG}	FTP backend without tcllib ftp

`can't find package tclutils::…` — check the module path / `TCLUTILS_TM` (chapter Installation). Wording of the `tutask` messages: chapter Tasks with tutask.

27.1 Summary

- Error codes have the form `{TCLUTILS <MODUL> <GRUND>}`.
- Prefix traps (`trap {TCLUTILS TUFETCH}`) catch whole groups.

28 API Catalog: tclutils Modules

This catalog was derived from the shipped `.tm` source files. It shows **procedure signatures**, not necessarily complete contracts. For semantics, options, error codes and side effects, the respective implementation is authoritative.

The commands are written with their full namespace. An `args` signature means variable arguments; the allowed options have to be looked up in the respective module implementation.

Modules with procedures covered: **143**.

28.1 common

Source: `lib/tm/tclutils/common-0.3.tm`. Limited to `namespace export`.

```
::tclutils::common::readFile {path args}
::tclutils::common::readBinaryFile {path}
::tclutils::common::writeFile {path data args}
::tclutils::common::replaceFile {path data args}
::tclutils::common::splitLines {text}
::tclutils::common::splitDelimited {line delimiter}
::tclutils::common::parseOptions {defaults args}
::tclutils::common::ensureBoolean {value optionName}
::tclutils::common::ensurePositiveInteger {value what}
::tclutils::common::ensureOneOf {value allowed what}
```

28.2 tuagrep

Source: `lib/tm/tclutils/tuagrep-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuagrep::match {line pattern args}
::tclutils::tuagrep::search {text pattern args}
::tclutils::tuagrep::file {filename pattern args}
::tclutils::tuagrep::files {fileList pattern args}
```

28.3 tuappinfo

Source: `lib/tm/tclutils/tuappinfo-0.3.tm`. No export list found with the simple source pattern; visible procedures without a leading underscore are listed.

28.4 tuarchitecture

Source: `lib/tm/tclutils/tuarchitecture-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tuarchitecture::parse {text}
```

28.5 tuawk

Source: `lib/tm/tclutils/tuawk-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuawk::run {text rules args}
::tclutils::tuawk::file {path rules args}
```

28.6 tubase32

Source: `lib/tm/tclutils/tubase32-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tubase32::encode {data args}
::tclutils::tubase32::decode {text args}
::tclutils::tubase32::encodeFile {path args}
::tclutils::tubase32::decodeFile {path args}
```

28.7 tubase64

Source: `lib/tm/tclutils/tubase64-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tubase64::encode {data args}
::tclutils::tubase64::decode {text args}
::tclutils::tubase64::encodeFile {path args}
::tclutils::tubase64::decodeFile {path}
```

28.8 tubin

Source: `lib/tm/tclutils/tubin-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tubin::bytesToHex {bytes args}
::tclutils::tubin::hexToBytes {hex}
::tclutils::tubin::bytesToUnsignedList {bytes {offset 0} {length -1}}
::tclutils::tubin::u8 {bytes {offset 0}}
::tclutils::tubin::u16le {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 2 s 0xffff 0 }
::tclutils::tubin::u32le {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 4 i 0xffffffff 0 }
::tclutils::tubin::u64le {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 8 w "" [expr {1 << 64}] }
::tclutils::tubin::u16be {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 2 S 0xffff 0 }
::tclutils::tubin::u32be {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 4 I 0xffffffff 0 }
::tclutils::tubin::u64be {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 8 W "" [expr {1 << 64}] }
::tclutils::tubin::uintle {bytes {offset 0} {length 0}}
::tclutils::tubin::i8 {bytes {offset 0}} { Signed [u8    $bytes
    $offset]  8 }
::tclutils::tubin::i16le {bytes {offset 0}} { Signed [u16le $bytes
    $offset] 16 }
::tclutils::tubin::i16be {bytes {offset 0}} { Signed [u16be $bytes
```

```
    $offset] 16 }
::tclutils::tubin::i32le {bytes {offset 0}} { Signed [u32le $bytes
    $offset] 32 }
::tclutils::tubin::i32be {bytes {offset 0}} { Signed [u32be $bytes
    $offset] 32 }
::tclutils::tubin::i64le {bytes {offset 0}} { Signed [u64le $bytes
    $offset] 64 }
::tclutils::tubin::i64be {bytes {offset 0}} { Signed [u64be $bytes
    $offset] 64 }
::tclutils::tubin::f32le {bytes {offset 0}}
::tclutils::tubin::f64le {bytes {offset 0}}
::tclutils::tubin::uintbe {bytes {offset 0} {length 0}}
::tclutils::tubin::f32be {bytes {offset 0}}
::tclutils::tubin::f64be {bytes {offset 0}}
::tclutils::tubin::packU8 {value} { binary format c [CheckRange $value
    0xff u8] }
::tclutils::tubin::packU16LE {value} { binary format s [CheckRange
    $value 0xffff u16] }
::tclutils::tubin::packU32LE {value} { binary format i [CheckRange
    $value 0xffffffff u32] }
::tclutils::tubin::packU64LE {value} { binary format w [CheckRange
    $value [expr {(1 << 64) - 1}] u64] }
::tclutils::tubin::packU16BE {value} { binary format S [CheckRange
    $value 0xffff u16] }
::tclutils::tubin::packU32BE {value} { binary format I [CheckRange
    $value 0xffffffff u32] }
::tclutils::tubin::packU64BE {value} { binary format W [CheckRange
    $value [expr {(1 << 64) - 1}] u64] }
::tclutils::tubin::ascii {bytes}
::tclutils::tubin::reader {subcmd args}
```

28.9 tublock

Source: `lib/tm/tclutils/tublock-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tublock::parse {text}
```

28.10 tubookmark

Source: `lib/tm/tclutils/tubookmark-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tubookmark::parse {html}
::tclutils::tubookmark::serialize {bookmarks args}
```

28.11 tuc4

Source: `lib/tm/tclutils/tuc4-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuc4::parse {text}
```

28.12 tucal

Source: lib/tm/tclutils/tucal-0.2.tm. Limited to namespace export.

```
::tclutils::tucal::render {year month args}
::tclutils::tucal::month {args}
::tclutils::tucal::now {args}
::tclutils::tucal::year {year args}
::tclutils::tucal::three {args}
```

28.13 tucat

Source: lib/tm/tclutils/tucat-0.2.tm. Limited to namespace export.

```
::tclutils::tucat::text {text args}
::tclutils::tucat::file {path args}
::tclutils::tucat::files {paths args}
```

28.14 tuclass

Source: lib/tm/tclutils/tuclass-0.3.tm. Limited to namespace export.

```
::tclutils::tuclass::parse {text}
```

28.15 tucmp

Source: lib/tm/tclutils/tucmp-0.2.tm. Limited to namespace export.

```
::tclutils::tucmp::data {a b}
::tclutils::tucmp::files {file1 file2}
::tclutils::tucmp::equal {file1 file2}
::tclutils::tucmp::firstDifference {file1 file2}
```

28.16 tucode

Source: lib/tm/tclutils/tucode-0.2.tm. Limited to namespace export.

```
::tclutils::tucode::lookup {value}
::tclutils::tucode::lookupChar {char}
::tclutils::tucode::render {from to args}
::tclutils::tucode::table {from to args}
::tclutils::tucode::ascii {args}
::tclutils::tucode::latin1 {args}
::tclutils::tucode::ansi {args}
::tclutils::tucode::groups {}
::tclutils::tucode::signs {args}
```

28.17 tucodepng

Source: lib/tm/tclutils/tucodepng-0.1.tm. Limited to namespace export.

```
::tclutils::tucodepng::render {from to args}
::tclutils::tucodepng::write {file from to args}
::tclutils::tucodepng::ascii {args} { return [render 0 127 {*}$args] }
::tclutils::tucodepng::latin1 {args} { return [render 128 255
    {*}$args] }
::tclutils::tucodepng::all {args} { return [render 0 255 {*}$args] }
```

28.18 tucolor

Source: `lib/tm/tclutils/tucolor-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tucolor::rgb {color}
::tclutils::tucolor::hex {color}
::tclutils::tucolor::names {}
::tclutils::tucolor::exists {name}
::tclutils::tucolor::nearest {color}
::tclutils::tucolor::toHsv {color}
::tclutils::tucolor::fromHsv {hsv}
```

28.19 tucolumn

Source: `lib/tm/tclutils/tucolumn-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tucolumn::table {text args}
::tclutils::tucolumn::fill {items args}
::tclutils::tucolumn::text {text args}
::tclutils::tucolumn::file {path args}
```

28.20 tucomm

Source: `lib/tm/tclutils/tucomm-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tucomm::compareLines {aLines bLines args}
::tclutils::tucomm::text {aText bText args}
::tclutils::tucomm::files {aFile bFile args}
::tclutils::tucomm::formatColumns {comparison args}
```

28.21 tucrc

Source: `lib/tm/tclutils/tucrc-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tucrc::hex32 {value}
::tclutils::tucrc::crc32 {data args}
::tclutils::tucrc::adler32 {data args}
::tclutils::tucrc::crc32File {path}
::tclutils::tucrc::adler32File {path}
```

28.22 tucsplit

Source: `lib/tm/tclutils/tucsplit-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tucsplit::splitText {text pattern args}
::tclutils::tucsplit::file {path pattern args}
```

28.23 tucsv

Source: `lib/tm/tclutils/tucsv-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tucsv::parseLine {line args}
::tclutils::tucsv::joinLine {fields args}
::tclutils::tucsv::parse {data args}
::tclutils::tucsv::text {rows args}
::tclutils::tucsv::file {path args}
::tclutils::tucsv::dicts {rows args}
::tclutils::tucsv::writeFile {path rows args}
```

28.24 tucut

Source: `lib/tm/tclutils/tucut-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tucut::field {line delimiter index}
::tclutils::tucut::fields {line delimiter fieldSpec {joiner {}}}
::tclutils::tucut::chars {line charSpec}
::tclutils::tucut::line {line args}
::tclutils::tucut::text {text args}
::tclutils::tucut::file {path args}
```

28.25 tudate

Source: `lib/tm/tclutils/tudate-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tudate::parse {str args}
::tclutils::tudate::iso {seconds args}
::tclutils::tudate::add {seconds count unit}
::tclutils::tudate::diff {a b args}
::tclutils::tudate::relative {seconds args}
::tclutils::tudate::today {}
::tclutils::tudate::zones {}
```

28.26 tudav

Source: `lib/tm/tclutils/tudav-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tudav::client {url args}
::tclutils::tudav::configure {c args}
::tclutils::tudav::lastStatus {c}
::tclutils::tudav::destroy {c}
::tclutils::tudav::propfind {c args}
::tclutils::tudav::report {c path body args}
::tclutils::tudav::listResources {c args}
::tclutils::tudav::get {c path}
::tclutils::tudav::put {c path data args}
```

```
::tclutils::tudav::delete {c path args}
::tclutils::tudav::calendarQuery {c path fromIso toIso}
::tclutils::tudav::addressbookMultiget {c path hrefs}
```

28.27 tudeploy

Source: `lib/tm/tclutils/tudeploy-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tudeploy::baseDirs {}
::tclutils::tudeploy::platformTag {}
::tclutils::tudeploy::roots {args}
::tclutils::tudeploy::addModulePaths {args}
::tclutils::tudeploy::require {packages args}
::tclutils::tudeploy::resourceDirs {name args}
::tclutils::tudeploy::sourceModule {pkg args}
```

28.28 tudhash

Source: `lib/tm/tclutils/tudhash-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tudhash::fromGray {w h grayList}
::tclutils::tudhash::fromRGB {w h rgbList}
::tclutils::tudhash::distance {a b}
::tclutils::tudhash::similar {a b {maxDist 10}}
```

28.29 tudiagram

Source: `lib/tm/tclutils/tudiagram-0.6.tm`. Limited to `namespace export`.

```
::tclutils::tudiagram::create {args}
::tclutils::tudiagram::setMeta {d args}
::tclutils::tudiagram::addNode {d id args}
::tclutils::tudiagram::addEdge {d from to args}
::tclutils::tudiagram::addGroup {d id args}
::tclutils::tudiagram::validate {d}
::tclutils::tudiagram::layout {d}
```

28.30 tudict

Source: `lib/tm/tclutils/tudict-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tudict::getOr {dict default args}
::tclutils::tudict::paths {dict {prefix {}}}
::tclutils::tudict::flatten {dict {sep .}}
::tclutils::tudict::mergeDeep {args}
::tclutils::tudict::invert {dict}
```

28.31 tudiff

Source: `lib/tm/tclutils/tudiff-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tudiff::text {oldText newText args}
::tclutils::tudiff::files {oldFile newFile args}
::tclutils::tudiff::unifiedText {oldText newText args}
::tclutils::tudiff::unified {oldFile newFile args}
::tclutils::tudiff::contextText {oldText newText args}
::tclutils::tudiff::context {oldFile newFile args}
::tclutils::tudiff::directory {oldDir newDir args}
```

28.32 tudocx

Source: `lib/tm/tclutils/tudocx-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tudocx::createText {docxFile paragraphs args}
::tclutils::tudocx::part {docxFile name}
::tclutils::tudocx::parts {docxFile}
::tclutils::tudocx::paragraphs {docxFile}
::tclutils::tudocx::text {docxFile}
```

28.33 tuedid

Source: `lib/tm/tclutils/tuedid-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tuedid::vendor {code}
::tclutils::tuedid::parseHex {text}
::tclutils::tuedid::parse {bytes}
```

28.34 tuer

Source: `lib/tm/tclutils/tuer-0.4.tm`. Limited to `namespace export`.

```
::tclutils::tuer::parse {text}
```

28.35 tuevent

Source: `lib/tm/tclutils/tuevent-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tuevent::create {}
::tclutils::tuevent::subscribe {bus event handler}
::tclutils::tuevent::unsubscribe {bus event handler}
::tclutils::tuevent::emit {bus event args}
::tclutils::tuevent::handlers {bus event}
::tclutils::tuevent::events {bus}
::tclutils::tuevent::clear {bus {event ""}}
::tclutils::tuevent::destroy {bus}
```

28.36 tuexe

Source: `lib/tm/tclutils/tuexe-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuexe::find {names args}
::tclutils::tuexe::all {names args}
::tclutils::tuexe::exists {names args}
```

28.37 tuexpand

Source: lib/tm/tclutils/tuexpand-0.2.tm. Limited to namespace export.

```
::tclutils::tuexpand::expand {text args}
::tclutils::tuexpand::unexpand {text args}
::tclutils::tuexpand::file {path args}
```

28.38 tufetch

Source: lib/tm/tclutils/tufetch-0.4.tm. Limited to namespace export.

```
::tclutils::tufetch::get {url args}
::tclutils::tufetch::download {url path args}
```

28.39 tufile

Source: lib/tm/tclutils/tufile-0.2.tm. Limited to namespace export.

```
::tclutils::tufile::detect {path}
::tclutils::tufile::type {path}
::tclutils::tufile::mime {path}
::tclutils::tufile::describe {path}
::tclutils::tufile::checkExtension {path}
```

28.40 tufind

Source: lib/tm/tclutils/tufind-0.2.tm. Limited to namespace export.

```
::tclutils::tufind::all {args}
::tclutils::tufind::files {args}
::tclutils::tufind::directories {args}
::tclutils::tufind::walk {root patterns callback args}
```

28.41 tuflow

Source: lib/tm/tclutils/tuflow-0.4.tm. Limited to namespace export.

```
::tclutils::tuflow::parse {text args}
::tclutils::tuflow::toSvg {text args}
::tclutils::tuflow::toPng {text args}
::tclutils::tuflow::writeSvg {text file args}
::tclutils::tuflow::writePng {text file args}
```

28.42 tufmt

Source: lib/tm/tclutils/tufmt-0.2.tm. Limited to namespace export.

```
::tclutils::tufmt::reflow {text args}
::tclutils::tufmt::reflowFile {path args}
```

28.43 tufold

Source: lib/tm/tclutils/tufold-0.3.tm. Limited to namespace export.

```
::tclutils::tufold::foldLine {line args}
::tclutils::tufold::text {input args}
::tclutils::tufold::file {path args}
```

28.44 tufuzzy

Source: `lib/tm/tclutils/tufuzzy-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tufuzzy::distance {a b args}
::tclutils::tufuzzy::searchDistance {pattern text args}
::tclutils::tufuzzy::similarity {a b args}
::tclutils::tufuzzy::subsequence {pattern text args}
::tclutils::tufuzzy::bestMatch {pattern candidates args}
```

28.45 tugantt

Source: `lib/tm/tclutils/tugantt-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tugantt::parse {text}
::tclutils::tugantt::toSvg {model args}
::tclutils::tugantt::toPng {model args}
::tclutils::tugantt::writeSvg {model file args}
::tclutils::tugantt::writePng {model file args}
```

28.46 tugit

Source: `lib/tm/tclutils/tugit-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tugit::parse {text}
```

28.47 tugrep

Source: `lib/tm/tclutils/tugrep-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tugrep::match {line pattern args}
::tclutils::tugrep::search {text pattern args}
::tclutils::tugrep::file {filename pattern args}
::tclutils::tugrep::files {fileList pattern args}
```

28.48 tuhash

Source: `lib/tm/tclutils/tuhash-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tuhash::sha256 {data args}
::tclutils::tuhash::sha1 {data args}
::tclutils::tuhash::md5 {data args}
::tclutils::tuhash::sha256File {path} { return [_sha256
    [::tclutils::common::readBinaryFile $path]] }
::tclutils::tuhash::sha1File {path} { return [_sha1
    [::tclutils::common::readBinaryFile $path]] }
::tclutils::tuhash::md5File {path} { return [_md5
    [::tclutils::common::readBinaryFile $path]] }
```

28.49 tuhead

Source: lib/tm/tclutils/tuhead-0.2.tm. Limited to namespace export.

```
::tclutils::tuhead::text {text {count 10}}
::tclutils::tuhead::file {path {count 10} args}
```

28.50 tuhexdump

Source: lib/tm/tclutils/tuhexdump-0.2.tm. Limited to namespace export.

```
::tclutils::tuhexdump::data {bytesData args}
::tclutils::tuhexdump::file {filename args}
```

28.51 tuhexedit

Source: lib/tm/tclutils/tuhexedit-0.2.tm. Limited to namespace export.

```
::tclutils::tuhexedit::readBytes {filename offset length}
::tclutils::tuhexedit::writeBytes {filename offset bytes {createBackup
    1}}
::tclutils::tuhexedit::backup {filename {backupName ""}}
::tclutils::tuhexedit::findHex {filename hex}
::tclutils::tuhexedit::findString {filename text {encoding utf-8}}
::tclutils::tuhexedit::patch {filename offset hex args}
::tclutils::tuhexedit::dump {filename args}
```

28.52 tuholiday

Source: lib/tm/tclutils/tuholiday-0.2.tm. Limited to namespace export.

```
::tclutils::tuholiday::easter {year}
::tclutils::tuholiday::holidays {year args}
::tclutils::tuholiday::isHoliday {iso args}
```

28.53 tuhttps

Source: lib/tm/tclutils/tuhttps-0.1.tm. Limited to namespace export.

```
::tclutils::tuhttps::caFile {}
::tclutils::tuhttps::socketCmd {args}
::tclutils::tuhttps::with {prefix script}
```

28.54 tuical

Source: lib/tm/tclutils/tuical-0.2.tm. Limited to namespace export.

```
::tclutils::tuical::parse {ics}
::tclutils::tuical::toIcs {comps}
::tclutils::tuical::components {comp {type ""}}
::tclutils::tuical::properties {comp {name ""}}
::tclutils::tuical::property {comp name}
::tclutils::tuical::find {comps type}
::tclutils::tuical::events {comps}
```

```
::tclutils::tuical::escapeText {s}
::tclutils::tuical::unescapeText {s}
::tclutils::tuical::setProperty {comp name value {params {}}}
::tclutils::tuical::addProperty {comp name value {params {}}}
::tclutils::tuical::removeProperty {comp name}
::tclutils::tuical::todos {comps}
::tclutils::tuical::journals {comps}
::tclutils::tuical::eventInfo {comp}
::tclutils::tuical::todoInfo {comp}
::tclutils::tuical::journalInfo {comp}
::tclutils::tuical::newComponent {type {props {}}}
```

28.55 tuico

Source: `lib/tm/tclutils/tuico-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuico::write {outFile entries}
::tclutils::tuico::info {icoFile}
::tclutils::tuico::extract {icoFile size {outFile ""}}
```

28.56 tuiconv

Source: `lib/tm/tclutils/tuiconv-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuiconv::encodings {}
::tclutils::tuiconv::convert {data from to args}
::tclutils::tuiconv::readFile {filename encodingName args}
::tclutils::tuiconv::writeFile {filename text encodingName args}
::tclutils::tuiconv::convertFile {infile outfile from to args}
```

28.57 tuimage

Source: `lib/tm/tclutils/tuimage-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuimage::type {bytes}
::tclutils::tuimage::mime {bytes}
::tclutils::tuimage::dimensions {bytes}
::tclutils::tuimage::inspect {bytes}
::tclutils::tuimage::dataUri {mimetype bytes}
::tclutils::tuimage::fromDataUri {uri}
```

28.58 tuini

Source: `lib/tm/tclutils/tuini-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuini::parse {ini}
::tclutils::tuini::toIni {data}
::tclutils::tuini::sections {data}
::tclutils::tuini::keys {data {section ""}}
::tclutils::tuini::get {data section key {default ""}}
::tclutils::tuini::has {data section key}
::tclutils::tuini::setValue {data section key value}
```

28.59 tujoin

Source: `lib/tm/tclutils/tujoin-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tujoin::lines {leftLines rightLines args}
::tclutils::tujoin::texts {leftText rightText args}
::tclutils::tujoin::files {leftPath rightPath args}
```

28.60 tujourney

Source: `lib/tm/tclutils/tujourney-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tujourney::parse {text}
::tclutils::tujourney::toSvg {model args}
::tclutils::tujourney::toPng {model args}
::tclutils::tujourney::writeSvg {model file args}
::tclutils::tujourney::writePng {model file args}
```

28.61 tujson

Source: `lib/tm/tclutils/tujson-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tujson::escape {text}
::tclutils::tujson::quote {text}
::tclutils::tujson::minify {json}
::tclutils::tujson::validate {json}
::tclutils::tujson::pretty {json args}
::tclutils::tujson::parse {json}
::tclutils::tujson::fromJson {json}
::tclutils::tujson::parseTyped {json}
::tclutils::tujson::filePretty {infile outfile args}
::tclutils::tujson::fileMinify {infile outfile}
::tclutils::tujson::toJson {typed args}
::tclutils::tujson::str {s} { return [list string $s] }
::tclutils::tujson::num {n}
::tclutils::tujson::bool {b}
::tclutils::tujson::null {} { return [list null {}] }
::tclutils::tujson::obj {pairs} { return [list object $pairs] }
::tclutils::tujson::arr {elems} { return [list array $elems] }
```

28.62 tukanban

Source: `lib/tm/tclutils/tukanban-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tukanban::parse {text}
::tclutils::tukanban::toSvg {model args}
::tclutils::tukanban::toPng {model args}
::tclutils::tukanban::writeSvg {model file args}
::tclutils::tukanban::writePng {model file args}
```

28.63 tulayout

Source: `lib/tm/tclutils/tulayout-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tulayout::pageSize {paper {orientation portrait}}
::tclutils::tulayout::mmToPx {mm scale}
::tclutils::tulayout::pxToMm {px scale}
::tclutils::tulayout::snap {mm args}
::tclutils::tulayout::mergeBlocks {base preset args}
::tclutils::tulayout::mergeLayout {target preset args}
```

28.64 tuldif

Source: `lib/tm/tclutils/tuldif-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tuldif::parse {ldif args}
::tclutils::tuldif::toLdif {entries args}
::tclutils::tuldif::dn {entry}
::tclutils::tuldif::get {entry attr}
::tclutils::tuldif::attributes {entry}
::tclutils::tuldif::toDict {entry}
```

28.65 tulint

Source: `lib/tm/tclutils/tulint-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tulint::register {name severity description cmd}
::tclutils::tulint::unregister {name}
::tclutils::tulint::rules {}
::tclutils::tulint::checkText {text args}
::tclutils::tulint::checkFile {path args}
::tclutils::tulint::check {paths args}
::tclutils::tulint::summary {findings}
::tclutils::tulint::format {findings args}
```

28.66 tulist

Source: `lib/tm/tclutils/tulist-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tulist::unique {list}
::tclutils::tulist::flatten {list {depth 1}}
::tclutils::tulist::chunk {list size}
::tclutils::tulist::zip {args}
::tclutils::tulist::sum {list}
::tclutils::tulist::avg {list}
::tclutils::tulist::min {list}
::tclutils::tulist::max {list}
```

28.67 tulog

Source: `lib/tm/tclutils/tulog-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tulog::new {args}
::tclutils::tulog::assert {exprStr {msg ""}}
```

28.68 tumath

Source: `lib/tm/tclutils/tumath-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tumath::clamp {x lo hi}
::tclutils::tumath::inRange {x lo hi}
::tclutils::tumath::percent {part whole}
::tclutils::tumath::sign {x}
::tclutils::tumath::gcd {a b}
::tclutils::tumath::lcm {a b}
::tclutils::tumath::factorial {n}
::tclutils::tumath::roundTo {x ndigits}
```

28.69 tumd

Source: `lib/tm/tclutils/tumd-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tumd::frontmatter {markdown}
::tclutils::tumd::headings {markdown}
::tclutils::tumd::toc {markdown}
::tclutils::tumd::toHtml {markdown}
```

28.70 tumindmap

Source: `lib/tm/tclutils/tumindmap-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tumindmap::parse {text}
```

28.71 tummdb

Source: `lib/tm/tclutils/tummdb-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tummdb::metadata {h}
::tclutils::tummdb::lookup {h ip}
```

28.72 tumonthpng

Source: lib/tm/tclutils/tumonthpng-0.5.tm. Limited to namespace export.

```
::tclutils::tumonthpng::render {year month args}
::tclutils::tumonthpng::write {file year month args}
::tclutils::tumonthpng::renderQuarter {year month args}
::tclutils::tumonthpng::renderYear {year args}
::tclutils::tumonthpng::writeQuarter {file year month args}
::tclutils::tumonthpng::writeYear {file year args}
```

28.73 tunl

Source: lib/tm/tclutils/tunl-0.2.tm. Limited to namespace export.

```
::tclutils::tunl::text {text args}
::tclutils::tunl::file {path args}
```

28.74 tunotes

Source: lib/tm/tclutils/tunotes-0.2.tm. Limited to namespace export.

```
::tclutils::tunotes::create {storeVar title content {tags {}}
    {parentId ""}}
::tclutils::tunotes::update {storeVar id title content {tags {}}
    {parentId KEEP}}
::tclutils::tunotes::delete {storeVar id {cascade 1}}
::tclutils::tunotes::move {storeVar id newParentId}
::tclutils::tunotes::get {store id}
::tclutils::tunotes::exists {store id}
::tclutils::tunotes::roots {store}
::tclutils::tunotes::children {store parentId}
```

28.75 tunotesdb

Source: lib/tm/tclutils/tunotesdb-0.2.tm. Limited to namespace export.

```
::tclutils::tunotesdb::init {db}
::tclutils::tunotesdb::create {db title content {tagList {}} {parentId
    ""}}
::tclutils::tunotesdb::update {db id title content {tagList {}}
    {parentId KEEP}}
::tclutils::tunotesdb::get {db id}
::tclutils::tunotesdb::exists {db id}
::tclutils::tunotesdb::delete {db id {cascade 1}}
::tclutils::tunotesdb::move {db id newParentId}
```

28.76 tunum

Source: lib/tm/tclutils/tunum-0.4.tm. Limited to namespace export.

```
::tclutils::tunum::parse {s args}
::tclutils::tunum::isNumber {s}
::tclutils::tunum::format {x args}
::tclutils::tunum::sum {values args}
```

28.77 tunumany

Source: `lib/tm/tclutils/tunumany-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tunumany::parse {str args}
```

28.78 tunumfmt

Source: `lib/tm/tclutils/tunumfmt-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tunumfmt::toHuman {number args}
::tclutils::tunumfmt::fromHuman {str args}
::tclutils::tunumfmt::text {text args}
::tclutils::tunumfmt::file {path args}
```

28.79 tuod

Source: `lib/tm/tclutils/tuod-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuod::bytes {bytesData args}
::tclutils::tuod::data {bytesData args}
::tclutils::tuod::file {filename args}
```

28.80 tuodf

Source: `lib/tm/tclutils/tuodf-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuodf::createText {odtFile paragraphs args}
::tclutils::tuodf::part {odtFile name}
::tclutils::tuodf::parts {odtFile}
::tclutils::tuodf::paragraphs {odtFile}
::tclutils::tuodf::text {odtFile}
```

28.81 tuopen

Source: `lib/tm/tclutils/tuopen-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuopen::command {target args}
::tclutils::tuopen::launch {target args}
::tclutils::tuopen::editCommand {target args}
::tclutils::tuopen::edit {target args}
::tclutils::tuopen::openDir {target args}
```

28.82 tupacket

Source: `lib/tm/tclutils/tupacket-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tupacket::parse {text}
::tclutils::tupacket::toSvg {model args}
::tclutils::tupacket::toPng {model args}
::tclutils::tupacket::writeSvg {model file args}
::tclutils::tupacket::writePng {model file args}
```

28.83 tupagespec

Source: `lib/tm/tclutils/tupagespec-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tupagespec::parse {spec total args}
::tclutils::tupagespec::count {spec total}
::tclutils::tupagespec::compact {pages}
```

28.84 tupaste

Source: `lib/tm/tclutils/tupaste-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tupaste::lines {lineLists args}
::tclutils::tupaste::texts {textList args}
::tclutils::tupaste::files {paths args}
```

28.85 tupatch

Source: `lib/tm/tclutils/tupatch-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tupatch::parse {patchText}
::tclutils::tupatch::text {sourceText patchText args}
::tclutils::tupatch::fromFile {sourceFile patchText args}
```

28.86 tupath

Source: `lib/tm/tclutils/tupath-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tupath::normalize {path}
::tclutils::tupath::clean {path}
::tclutils::tupath::isAbsolute {path}
::tclutils::tupath::components {path}
::tclutils::tupath::join {args}
```

28.87 tupdf

Source: `lib/tm/tclutils/tupdf-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tupdf::version {pdfFile}
::tclutils::tupdf::objects {pdfFile}
::tclutils::tupdf::object {pdfFile id}
::tclutils::tupdf::trailer {pdfFile}
::tclutils::tupdf::zugferd {pdfFile}
::tclutils::tupdf::metadata {pdfFile}
::tclutils::tupdf::summary {pdfFile}
```

28.88 tupdftext

Source: lib/tm/tclutils/tupdftext-0.1.tm. Limited to namespace export.

```
::tclutils::tupdftext::pages {file args}
::tclutils::tupdftext::text {file args}
::tclutils::tupdftext::summary {file}
```

28.89 tupie

Source: lib/tm/tclutils/tupie-0.3.tm. Limited to namespace export.

```
::tclutils::tupie::parse {text}
::tclutils::tupie::toSvg {model args}
::tclutils::tupie::toPng {model args}
::tclutils::tupie::writeSvg {model file args}
::tclutils::tupie::writePng {model file args}
```

28.90 tupkgfinder

Source: lib/tm/tclutils/tupkgfinder-0.2.tm. Limited to namespace export.

```
::tclutils::tupkgfinder::paths {}
::tclutils::tupkgfinder::versions {packageName}
::tclutils::tupkgfinder::pkgInfo {packageName}
::tclutils::tupkgfinder::which {packageName}
::tclutils::tupkgfinder::findFileSystem {pattern args}
::tclutils::tupkgfinder::report {packageName}
```

28.91 tupng

Source: lib/tm/tclutils/tupng-0.6.tm. Limited to namespace export.

```
::tclutils::tupng::encodeRGB {image args}
::tclutils::tupng::encodeRGBA {image args}
::tclutils::tupng::encodeGray {image args}
::tclutils::tupng::encodeIndexed {palette image args}
```

28.92 tupngdraw

Source: lib/tm/tclutils/tupngdraw-0.14.tm. Limited to namespace export.

```
::tclutils::tupngdraw::new {args}
```

28.93 tupngpad

Source: lib/tm/tclutils/tupngpad-0.2.tm. Limited to namespace export.

```
::tclutils::tupngpad::bbox {rgba w h}
::tclutils::tupngpad::trim {rgba w h}
::tclutils::tupngpad::padTo {rgba w h tw th args}
::tclutils::tupngpad::batch {files outdir args}
```

28.94 tupostgrest

Source: `lib/tm/tclutils/tupostgrest-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tupostgrest::new {baseUrl args}
::tclutils::tupostgrest::token {client jwt}
::tclutils::tupostgrest::request {client method path args}
::tclutils::tupostgrest::get {client table args}
::tclutils::tupostgrest::insert {client table row args}
::tclutils::tupostgrest::update {client table patch args}
::tclutils::tupostgrest::delete {client table args}
::tclutils::tupostgrest::rpc {client fn {args_ {}}}
```

28.95 tupr

Source: `lib/tm/tclutils/tupr-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tupr::text {text args}
::tclutils::tupr::file {path args}
```

28.96 tuprovider

Source: `lib/tm/tclutils/tuprovider-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tuprovider::register {scheme class}
::tclutils::tuprovider::schemes {}
::tclutils::tuprovider::open {scheme args}
```

28.97 tuquadrant

Source: `lib/tm/tclutils/tuquadrant-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuquadrant::parse {text}
::tclutils::tuquadrant::toSvg {model args}
::tclutils::tuquadrant::toPng {model args}
::tclutils::tuquadrant::writeSvg {model file args}
::tclutils::tuquadrant::writePng {model file args}
```

28.98 turadar

Source: `lib/tm/tclutils/turadar-0.3.tm`. Limited to `namespace export`.

```
::tclutils::turadar::parse {text}
::tclutils::turadar::toSvg {model args}
::tclutils::turadar::toPng {model args}
::tclutils::turadar::writeSvg {model file args}
::tclutils::turadar::writePng {model file args}
```

28.99 turegistry

Source: `lib/tm/tclutils/turegistry-0.1.tm`. Limited to `namespace export`.

```
::tclutils::turegistry::create {}
::tclutils::turegistry::put {reg key value}
::tclutils::turegistry::get {reg key args}
::tclutils::turegistry::has {reg key}
::tclutils::turegistry::keys {reg {pattern *}}
::tclutils::turegistry::remove {reg key}
::tclutils::turegistry::destroy {reg}
```

28.100 turequirement

Source: `lib/tm/tclutils/turequirement-0.1.tm`. Limited to `namespace export`.

```
::tclutils::turequirement::parse {text}
```

28.101 turev

Source: `lib/tm/tclutils/turev-0.1.tm`. Limited to `namespace export`.

```
::tclutils::turev::line {str}
::tclutils::turev::text {text}
::tclutils::turev::file {path}
```

28.102 turrule

Source: `lib/tm/tclutils/turrule-0.2.tm`. Limited to `namespace export`.

```
::tclutils::turrule::parse {rr}
::tclutils::turrule::occurrences {args}
::tclutils::turrule::eventsInRange {ics fromIso toIso}
```

28.103 tusankey

Source: `lib/tm/tclutils/tusankey-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tusankey::parse {text}
::tclutils::tusankey::toSvg {model args}
::tclutils::tusankey::toPng {model args}
::tclutils::tusankey::writeSvg {model file args}
::tclutils::tusankey::writePng {model file args}
```

28.104 tused

Source: `lib/tm/tclutils/tused-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tused::replace {text pattern replacement args}
::tclutils::tused::substitute {line pattern replacement args}
::tclutils::tused::delete {text pattern args}
::tclutils::tused::process {text rules}
::tclutils::tused::processFile {infile outfile rules args}
::tclutils::tused::script {text scriptText}
```

28.105 tuseq

Source: lib/tm/tclutils/tuseq-0.2.tm. Limited to namespace export.

```
::tclutils::tuseq::range {args}
::tclutils::tuseq::text {args}
```

28.106 tusequence

Source: lib/tm/tclutils/tusequence-0.3.tm. Limited to namespace export.

```
::tclutils::tusequence::parse {text}
::tclutils::tusequence::toSvg {model args}
::tclutils::tusequence::toPng {model args}
::tclutils::tusequence::writeSvg {model file args}
::tclutils::tusequence::writePng {model file args}
```

28.107 tusettings

Source: lib/tm/tclutils/tusettings-0.2.tm. Limited to namespace export.

```
::tclutils::tusettings::dir {kind app args}
::tclutils::tusettings::file {app {name settings}}
::tclutils::tusettings::load {app args}
::tclutils::tusettings::save {app settings args}
::tclutils::tusettings::value {settings section key {default ""}}
```

28.108 tushuf

Source: lib/tm/tclutils/tushuf-0.3.tm. Limited to namespace export.

```
::tclutils::tushuf::lines {lineList args}
::tclutils::tushuf::text {text args}
::tclutils::tushuf::file {path args}
```

28.109 tusize

Source: lib/tm/tclutils/tusize-0.2.tm. Limited to namespace export.

```
::tclutils::tusize::bytes {path}
::tclutils::tusize::human {n args}
::tclutils::tusize::entries {dir}
```

28.110 tusort

Source: lib/tm/tclutils/tusort-0.2.tm. Limited to namespace export.

```
::tclutils::tusort::sortList {items args}
::tclutils::tusort::text {text args}
::tclutils::tusort::file {path args}
```

28.111 tusparql

Source: lib/tm/tclutils/tusparql-0.2.tm. Limited to namespace export.

```
::tclutils::tusparql::query {endpoint sparql args}
::tclutils::tusparql::ask {endpoint sparql args}
```

28.112 tusplit

Source: `lib/tm/tclutils/tusplit-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tusplit::splitLines {text args}
::tclutils::tusplit::splitBytes {data args}
::tclutils::tusplit::file {path args}
::tclutils::tusplit::lines {path n args}
::tclutils::tusplit::bytes {path n args}
```

28.113 tusqlite

Source: `lib/tm/tclutils/tusqlite-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tusqlite::null {}
::tclutils::tusqlite::quoteId {name}
::tclutils::tusqlite::insert {db table data}
::tclutils::tusqlite::rows {db sql {params {}}}
::tclutils::tusqlite::value {db sql args}
::tclutils::tusqlite::update {db table set where}
::tclutils::tusqlite::delete {db table where}
```

28.114 tustat

Source: `lib/tm/tclutils/tustat-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tustat::file {path}
::tclutils::tustat::files {paths}
::tclutils::tustat::render {statDict args}
```

28.115 tustate

Source: `lib/tm/tclutils/tustate-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tustate::parse {text}
```

28.116 tustr

Source: `lib/tm/tclutils/tustr-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tustr::isEmpty {s} { return [expr {$s eq ""}] }
::tclutils::tustr::truncate {s maxlen {ellipsis "..."}}
::tclutils::tustr::padLeft {s width {char " "}}
::tclutils::tustr::padRight {s width {char " "}}
::tclutils::tustr::center {s width {char " "}}
```

28.117 tustrings

Source: `lib/tm/tclutils/tustrings-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tustrings::extract {data args}
::tclutils::tustrings::file {filename args}
```

28.118 tusvg

Source: `lib/tm/tclutils/tusvg-0.4.tm`. Limited to `namespace export`.

```
::tclutils::tusvg::new {args}
```

28.119 tusysinfo

Source: `lib/tm/tclutils/tusysinfo-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tusysinfo::collect {args}
::tclutils::tusysinfo::report {info args}
::tclutils::tusysinfo::toJson {info args}
```

28.120 tutable

Source: `lib/tm/tclutils/tutable-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tutable::render {headers rows args}
```

28.121 tutablepng

Source: `lib/tm/tclutils/tutablepng-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tutablepng::render {rows args}
::tclutils::tutablepng::write {file rows args}
```

28.122 tutac

Source: `lib/tm/tclutils/tutac-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tutac::lines {lineList}
::tclutils::tutac::text {text}
::tclutils::tutac::file {path}
```

28.123 tutail

Source: `lib/tm/tclutils/tutail-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tutail::text {text {count 10}}
::tclutils::tutail::file {path {count 10} args}
```

28.124 tutask

Source: `lib/tm/tclutils/tutask-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tutask::await {starter args}
::tclutils::tutask::timer {ms {value ""}}
::tclutils::tutask::sleep {ms}
::tclutils::tutask::onwrite {varName}
::tclutils::tutask::pipe {cmdlist args}
::tclutils::tutask::spawn {args}
::tclutils::tutask::current {}
::tclutils::tutask::run {args}
```

28.125 tutdbc

Source: `lib/tm/tclutils/tutdbc-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tutdbc::null {}
::tclutils::tutdbc::quoteId {name}
::tclutils::tutdbc::rows {conn sql {binds {}}}
::tclutils::tutdbc::value {conn sql {binds {}} {default ""}}
::tclutils::tutdbc::execute {conn sql {binds {}}}
::tclutils::tutdbc::insert {conn table data}
::tclutils::tutdbc::transaction {conn script}
```

28.126 tutee

Source: `lib/tm/tclutils/tutee-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tutee::write {data paths args}
::tclutils::tutee::writeFile {inputFile paths args}
::tclutils::tutee::copyChannel {inChan outChans args}
```

28.127 tuterm

Source: `lib/tm/tclutils/tuterm-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuterm::style {args}
::tclutils::tuterm::wrap {text args}
::tclutils::tuterm::off {attr}
::tclutils::tuterm::names {{what all}}
::tclutils::tuterm::strip {text}
::tclutils::tuterm::enable {args}
::tclutils::tuterm::auto {}
::tclutils::tuterm::enableVT {}
::tclutils::tuterm::disableVT {}
```

28.128 tutestrun

Source: `lib/tm/tclutils/tutestrun-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tutestrun::run {dir libs args}
```

28.129 tutimeline

Source: `lib/tm/tclutils/tutimeline-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tutimeline::parse {text}
::tclutils::tutimeline::toSvg {model args}
::tclutils::tutimeline::toPng {model args}
::tclutils::tutimeline::writeSvg {model file args}
::tclutils::tutimeline::writePng {model file args}
```

28.130 tutr

Source: `lib/tm/tclutils/tutr-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tutr::translate {text set1 set2}
::tclutils::tutr::delete {text set1}
::tclutils::tutr::squeeze {text set1}
::tclutils::tutr::file {path set1 set2 args}
```

28.131 tutreemap

Source: `lib/tm/tclutils/tutreemap-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tutreemap::parse {text}
::tclutils::tutreemap::toSvg {model args}
::tclutils::tutreemap::toPng {model args}
::tclutils::tutreemap::writeSvg {model file args}
::tclutils::tutreemap::writePng {model file args}
```

28.132 tutsort

Source: `lib/tm/tclutils/tutsort-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tutsort::pairs {tokens}
::tclutils::tutsort::text {text}
::tclutils::tutsort::file {path}
```

28.133 tuuniq

Source: `lib/tm/tclutils/tuuniq-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuuniq::uniqList {items args}
::tclutils::tuuniq::adjacentCount {items args}
::tclutils::tuuniq::count {items args}
::tclutils::tuuniq::text {text args}
::tclutils::tuuniq::file {path args}
::tclutils::tuuniq::countText {text args}
::tclutils::tuuniq::countFile {path args}
```

28.134 tuurl

Source: `lib/tm/tclutils/tuurl-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tuurl::encode {str args}
::tclutils::tuurl::decode {str args}
::tclutils::tuurl::buildQuery {d args}
::tclutils::tuurl::parseQuery {q args}
```

28.135 tuuuid

Source: `lib/tm/tclutils/tuuuid-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tuuuid::v4 {}
::tclutils::tuuuid::v7 {}
::tclutils::tuuuid::nil {}
::tclutils::tuuuid::generate {args}
::tclutils::tuuuid::validate {uuid}
::tclutils::tuuuid::version {uuid}
```

28.136 tuvalidate

Source: `lib/tm/tclutils/tuvalidate-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuvalidate::email {s}
::tclutils::tuvalidate::url {s}
::tclutils::tuvalidate::ipv4 {s}
::tclutils::tuvalidate::port {s}
::tclutils::tuvalidate::alpha {s} { return [regexp {^[A-Za-z]+$} $s] }
::tclutils::tuvalidate::alnum {s} { return [regexp {^[A-Za-z0-9]+$}
    $s] }
::tclutils::tuvalidate::numeric {s} { return [string is double -strict
    $s] }
::tclutils::tuvalidate::integer {s} { return [string is integer
    -strict $s] }
```

28.137 tuvcard

Source: `lib/tm/tclutils/tuvcard-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuvcard::parse {vcf}
::tclutils::tuvcard::toVcf {cards}
::tclutils::tuvcard::properties {card {name ""}}
::tclutils::tuvcard::property {card name}
::tclutils::tuvcard::get {card name}
::tclutils::tuvcard::names {card}
::tclutils::tuvcard::fullName {card}
::tclutils::tuvcard::addProperty {card name value {params {}}}
::tclutils::tuvcard::removeProperty {card index}
::tclutils::tuvcard::setProperty {card index name value {params {}}}
```

28.138 tuwc

Source: `lib/tm/tclutils/tuwc-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuwc::text {data}
::tclutils::tuwc::file {filename}
```

28.139 tuxargs

Source: `lib/tm/tclutils/tuxargs-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tuxargs::batches {items args}
::tclutils::tuxargs::apply {items commandPrefix args}
::tclutils::tuxargs::command {items commandPrefix args}
```

28.140 tuxlsx

Source: `lib/tm/tclutils/tuxlsx-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tuxlsx::writeRows {xlsxFile rows args}
::tclutils::tuxlsx::part {xlsxFile name}
::tclutils::tuxlsx::parts {xlsxFile}
::tclutils::tuxlsx::sheets {xlsxFile}
::tclutils::tuxlsx::rows {xlsxFile args}
::tclutils::tuxlsx::text {xlsxFile}
```

28.141 tuxml

Source: `lib/tm/tclutils/tuxml-0.1.tm`. Limited to `namespace export`.

```
::tclutils::tuxml::escape {text args}
::tclutils::tuxml::unescape {text}
::tclutils::tuxml::declaration {{encoding UTF-8}}
::tclutils::tuxml::attrs {attributes}
::tclutils::tuxml::tag {name {attributes {}}}
::tclutils::tuxml::element {name attributes content}
::tclutils::tuxml::textElement {name attributes text}
```

28.142 tuxxhash

Source: `lib/tm/tclutils/tuxxhash-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuxxhash::xxh32 {data args}
::tclutils::tuxxhash::xxh32file {path {seed 0}}
```

28.143 tuxychart

Source: `lib/tm/tclutils/tuxychart-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuxychart::parse {text}
::tclutils::tuxychart::toSvg {model args}
::tclutils::tuxychart::toPng {model args}
::tclutils::tuxychart::writeSvg {model file args}
::tclutils::tuxychart::writePng {model file args}
```

28.144 tuzip

Source: `lib/tm/tclutils/tuzip-0.2.tm`. Limited to `namespace export`.

```
::tclutils::tuzip::entries {zipfile args}
::tclutils::tuzip::names {zipfile args}
::tclutils::tuzip::readMember {zipfile member args}
::tclutils::tuzip::extract {zipfile member outfile args}
::tclutils::tuzip::create {zipfile files args}
::tclutils::tuzip::createMembers {zipfile members args}
```

28.145 tuzipfs

Source: `lib/tm/tclutils/tuzipfs-0.3.tm`. Limited to `namespace export`.

```
::tclutils::tuzipfs::available {}
::tclutils::tuzipfs::requireAvailable {}
::tclutils::tuzipfs::root {}
::tclutils::tuzipfs::mounts {}
::tclutils::tuzipfs::mount {archive {mountName ""}}
::tclutils::tuzipfs::unmount {mountpoint}
```

29 Practice and Failure Cases

29.1 Processing CSV Lines

`tucsv` provides the functions `parseLine`, `joinLine`, `parse`, `text`, `file`, `dicts` and `writeFile`. The parser options `-delimiter`, `-quote`, `-trim` and `-strict` are defined in the source of `tucsv-0.2.tm`; `text`, `file` and `writeFile` also know `-profile`, `-bom` and `-encoding`. The output of `joinLine` can be controlled with `-alwaysQuote`.

```
package require tclutils::tucsv

set record [list {Müller, GmbH} {Er sagte "Hallo"} 48683]
set csvLine [::tclutils::tucsv::joinLine $record]
set parsed [::tclutils::tucsv::parseLine $csvLine]
if {$parsed ne $record} {
    error "CSV roundtrip failed"
}
puts $csvLine
```

The values are built as a Tcl list. This keeps embedded commas and quotes within a field. The test checks the round trip, not byte equality with a foreign CSV file.

29.2 Handling Error Cases Specifically

With `-strict` enabled, an unclosed quote produces the error code `{TCLUTILS TUCSV QUOTE}`. This lets the caller handle parser errors separately from file or runtime errors.

```
package require tclutils::tucsv

try {
    ::tclutils::tucsv::parseLine {"nicht geschlossen}
```

```
} trap {TCLUTILS TUCSV QUOTE} {message options} {
    puts "CSV input rejected: $message"
}
```

The `trap` clause responds only to the given error code. Other errors still need their own handling or have to be passed on.

29.3 Dictionaries with `tudict`

The implementation of `tudict-0.1.tm` exports `getOr`, `paths`, `flatten`, `mergeDeep` and `invert`. For nested data, pay particular attention to the argument order and the handling of missing keys.

```
package require tclutils::tudict

set defaults [dict create ui [dict create language de theme light]]
set overrides [dict create ui [dict create theme dark]]
set settings [::tclutils::tudict::mergeDeep $defaults $overrides]
puts [dict get $settings ui language]
puts [dict get $settings ui theme]
```

`mergeDeep` is meant for nested dictionaries. Before production use, check whether individual values can themselves be valid dictionaries and how the application interprets them.

29.4 File Paths and Portability

`tupath` exports `normalize`, `clean`, `isAbsolute`, `components`, `join`, `relative`, `commonPath` and `readlink`. `file normalize` and `tupath::clean` are not equivalent: normalization can depend on the file system and the platform; purely textual cleanup is a different thing.

```
package require tclutils::tupath
set relativePath [::tclutils::tupath::join docs manual index.md]
puts $relativePath
```

Limits: The API catalog lists signatures; the examples above are derived from the sources. This does not claim that the examples were run in full on every platform.

30 License


```
Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Attribution-ShareAlike 4.0 International
(CC BY-SA 4.0)
The author's code examples: MIT License (below)

---

Text

This book is licensed under the Creative Commons license
```

"Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0).

You may copy and redistribute the material in any medium or format and remix, transform and build upon it, for any purpose, even commercially, under the following terms:

* Attribution: You must give appropriate credit, provide a link to the license, and indicate if changes were made.

* ShareAlike: If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

The full license text is binding:
https://creativecommons.org/licenses/by-sa/4.0/legalcode

Code examples

The author's code examples (in the code blocks of the book and as scripts next to it) are additionally released under the MIT License. Code examples and quoted text taken from other projects (Tcl/Tk, pdf4tcl and others) keep the license of their original; the source is named where they appear.

MIT License

Copyright (c) 2026 Gregor Ebbing

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE

```
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Index

-

-fontfile 122

C

capabilities 144
caps 200
coroutines 177

G

grapheme 200

M

module path 12, 200

N

NOCA 200

P

provider 200

S

sandbox 181, 200
shadow copy 201
starter 178, 200

T

task 200
tclutils 11
tical 85
tls 172
tuagrep 41
tuappinfo 153
tuarchitecture 105
tuawk 38
tubase32 58
tublock 104
tubookmark 54
tuc4 103
tucal 83
tuclass 99
tucmp 70
tucode 74
tucodepng 132
tucolor 124
tucolumn 24
tucomm 32
tucsplit 33
tucsv 48

tudate 43
tudeploy 158
tudhash 143
tudiagram 92, 201
tudict 63
tudocx 88
tuedid 157
tuer 100
tuevent 148
tuexe 160
tuexpand 23
tufetch 172
tuflow 92, 201
tufmt 22
tufold 22
tufuzzy 40
tugantt 108
tugit 105
tuhash 201
tuhexdump 66
tuhexedit 69
tuholiday 201
tuhttps 171, 200
tuico 141
tuiconv 61
tuimage 140
tuini 50
tujoin 30
tujourney 115
tujson 49
tukanban 121
tulayout 169
tuldif 53
tulint 185
tulist 151
tulog 146
tumath 152
tumd 76, 201
tumindmap 102
tummdb 56, 201
tumonthpng 133
tunl 21
tunotesdb 81
tunum 43
tunumany 152
tuod 67
tuodf 88, 201

tuopen 161
tupacket 120
tupagespec 167
tupaste 30
tupatch 64
tupath 43
tupdf 88, 201
tupdftext 86, 201
tupie 111
tupkgfinder 185
tupng 124
tupngdraw 127
tupngpad 136
tupr 29
tuprovider 144
tuquadrant 113
turadar 118
turegistry 149
turequirement 101
turev 25
turrule 78
tusankey 117
tuseq 26
tusequence 106
tusettings 175, 200
tushuf 25
tusize 71
tusparql 174
tusplit 34
tusqlite 52
tustat 73
tustate 99
tustr 43
tustrings 68
tusvg 137
tusysinfo 155
tutable 24
tutablepng 131
tutac 25
tutask 177, 200
tutdbc 56, 201
tutee 35
tuterm 166
tutestrun 184, 200
tutimeline 116
tutr 27
tutreemap 119

tutsort 37
tuurl 201
tuvalidate 149
tuwc 28
tuxargs 36
tuxlsx 88
tuxml 51
tuxxhash 60
tuxychart 112
tuzipfs 163

U

umbrella 200